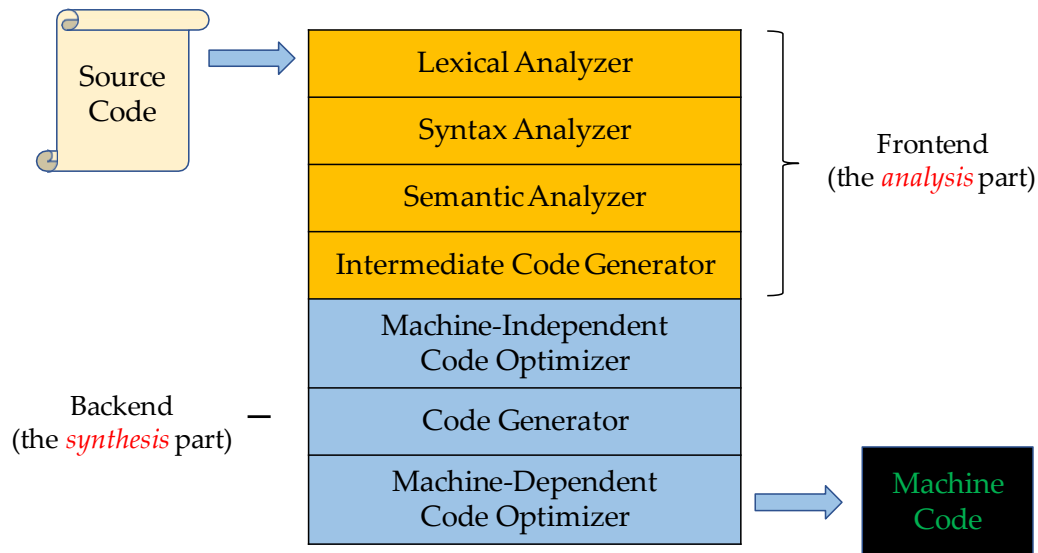


The Structure of a Compiler



编译器的结构

输入：源代码

前端：（分析部分）

词法分析器

语法分析器

语义分析器

中间代码生成器

后端：（合成部分）

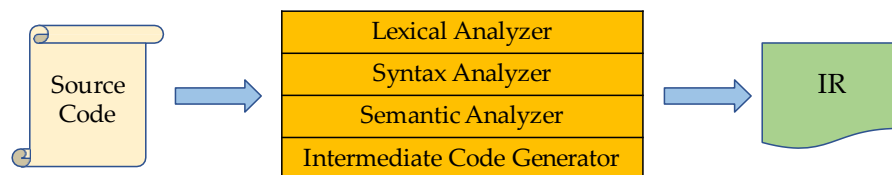
机器无关代码优化器

代码生成器

机器相关代码优化器

输出：机器码

The Frontend (前端) of a Compiler



- Breaks up the source program into **constituent pieces** and imposes a **grammatical structure** on them
- Uses the grammatical structure to create an **intermediate representation (IR)** of the source program
- Collect the information about the source program and stores it in a data structure called **symbol table** (will be passed to backend with IR)

编译器的前端

词法分析器

语法分析器

语义分析器

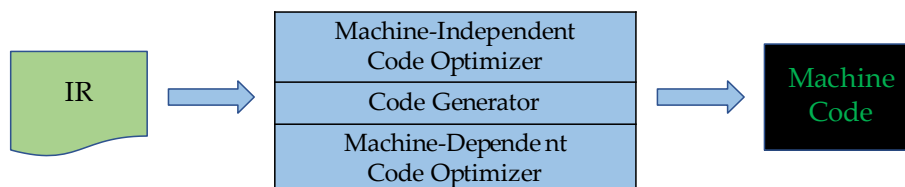
中间代码生成器

将源程序分解为多个组成部分，并在其上强加语法结构

使用语法结构创建源程序的中间表示（IR）

收集有关源程序的信息，并将其存储在称为符号表的数据结构中（将与 IR 一起传给后端）

The Backend (后端) of a Compiler



- Constructs the target program (typically, in machine language) from the IR and the information in the symbol table
- Performs code optimizations during the process

编译器的后端

机器无关代码优化器

代码生成器

机器相关代码优化器

根据 IR 和符号表中的信息构建目标程序（通常使用机器语言）

在此过程中执行代码优化

Lexical Analysis (Scanning, 词法分析)

Character stream $\longrightarrow$ **Lexical Analyzer** $\longrightarrow$ Tokenstream

- The lexical analyzer (lexer/tokenizer/scanner) breaks down the source code into a sequence of “lexemes” (词素) or “words”
- For each lexeme, produce a “token” (词法单元) in the form:

$\langle$ token-name, attribute-value $\rangle$

An **abstract symbol** that is used during syntax analysis

Points to an entry in the symbol table.
Info in the table entry is for semantic analysis and code generation.

字符流 $\rightarrow$ 词法分析 $\rightarrow$ token 流

词法分析器将源代码分解为一系列 词素 或 “words”

对于每个词素，生成一个形式上为 $\langle$ token 名, 属性值 $\rangle$ 的词法单元

词法单元中单元名称：语法分析过程中使用的抽象符号

属性值：指向符号表中的一个条目。表条目中的信息用于语义分析和代码生成。

Lexemes vs. Tokens

- A **lexeme** is a string of characters that is a lowest-level syntactic unit in the programming language
 - "words" and punctuation of the programming language (**instance**)
- A **token** is a syntactic category representing a class of lexemes
 - **In English:** Noun, Verb, Adjective...
 - **In programming language:** Identifier, Keyword, Whitespace... (**pattern**)

<https://courses.cs.vt.edu/~cs1104/Compilers/Compilers.070.html>

词素 vs token

词素是一个字符串，是程序设计语言中最低级别的句法单元

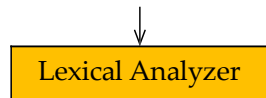
- “单词”和程序设计语言的标点符号记号

token 是一种语法类别，代表一类词汇

- 英语：名词，动词，形容词...
- 在编程语言中：标识符，关键字，空白... (模式)

Lexical Analysis (Example)

position = initial + rate * 60



<id, 1>

<=>

<id, 2>

<+>

<id, 3>

<*>

<60>

SYMBOL TABLE

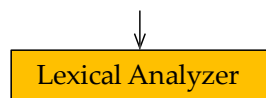
1	position	...
2	initial	...
3	rate	...

Note: <=>, <+>, <*>, <60> are not in the defined form. This is for notational convenience. <=> could have been <assign, -> and <60> could have been <number, 4>.

注意: <=>, <+>, <*>, <60>不是定义的形式。这是为了便于注释。: <=>可能是 <assign, -> and <60>可能是<number, 4>.

Lexical Analysis (Analogy)

position = initial + rate * 60



<id, 1>

<=>

<id, 2>

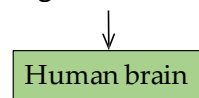
<+>

<id, 3>

<*>

<60>

BUPT is a good university



<noun, "BUPT">

<verb, "is">

<article, "a">

<adjective, "good">

<noun, "university">

Syntax Analysis (Parsing, 语法分析)

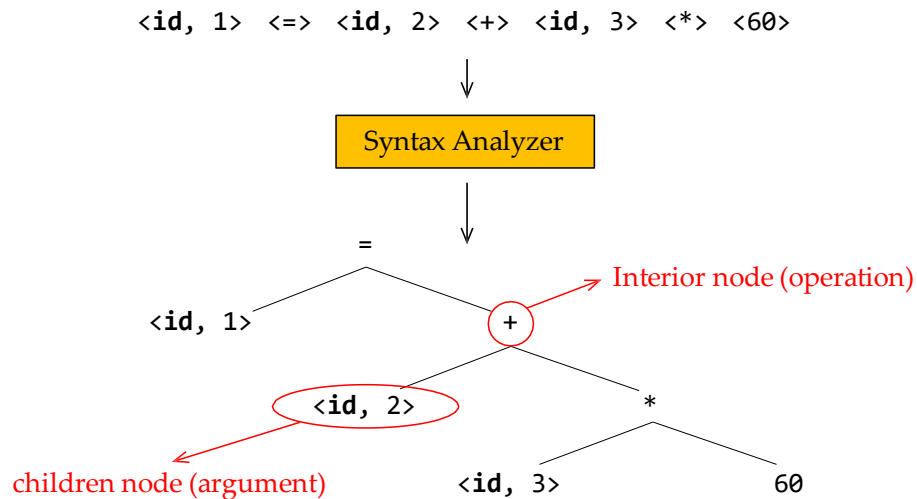


- The syntax analyzer (parser) uses the **token names** produced by the lexer to create an intermediate representation that depicts the grammar structure of the token stream, typically a *syntax tree*
- Each interior node represents an **operation** and the children of the node represent the **arguments** of the operation

语法分析器（解析器）使用 lexer 生成的 token 的名称来创建一个中间表示，该中间表示描述 token 流的语法结构，通常用语法树来表示

每个内部节点表示一个操作，内部节点的子节点表示该操作的参数

Syntax Analysis (Example)

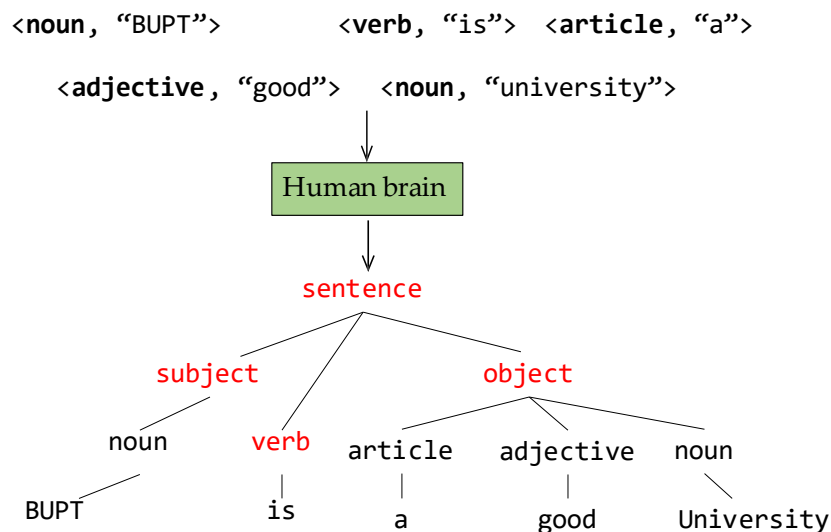


Fall 2024

BUPT Compilers

26

Syntax Analysis in English



Fall 2024

BUPT Compilers

27

BUPT 为名词, is 为动词, a 为冠词, good 为形容词, university 为名词

Semantic Analysis (语义分析)



- The semantic analyzer uses the syntax tree and the information in the symbol table to check the source program for **semantic consistency** with the language definition
- Also gathers **type information** for type checking, type conversion, and intermediate code generation

语义分析器使用语法树和符号表中的信息来检查源程序与语言定义的语义一致性
还收集用于类型检查、类型转换和中间代码生成的类型信息

What is Semantics?

- The **syntax** of a programming language describes the **proper form** of its programs
- The **semantics** of a programming language describes the **meaning** of its programs, i.e., what each program does when it executes

什么是语义?

编程语言的语法描述程序的正确形式

编程语言的语义描述程序的含义，即每个程序在执行时做什么

Semantic Analysis in English

Jack said Jerry left his assignment at home.

What does “his” refer to? Jack’s or Jerry’s?

Jack said Jack left his assignment at home.

How many Jacks? Which one left the assignment?

Examples are from Aiken’s notes (Stanford CS143)

英语中的语义分析

杰克说杰瑞把他的作业落在家里了。“他的”指的是什么？杰克的还是杰瑞的？

杰克说杰克把他的作业落在家里了。有多少个杰克？哪一个把作业落在家里了？

Semantic Analysis in Programming

- Understanding the meaning of a program is very hard 😞
- Compilers perform only very limited analysis to catch semantic inconsistencies.

1. {	<i>Which value will be printed?</i>
2. int Jack = 3;	
3. {	Programming languages define
4. int Jack = 4;	strict rules to avoid ambiguities.
5. print Jack;	Compiler will bind Jack at line 5 to
6. }	its inner definition at line 4.
7. }	

程序中的语义分析

理解程序的含义是非常困难的。

编译器只执行非常有限的分析来捕捉语义不一致。

编程语言定义了严格的规则来避免歧义。

编译器将把第 5 行的 Jack 绑定到第 4 行的内部定义。

Type Checking (类型检查)

- An important part of semantic analysis is type checking
- Compilers check that each operator has matching operands (of correct types)

Example: Many language definitions require an array index to be an integer.

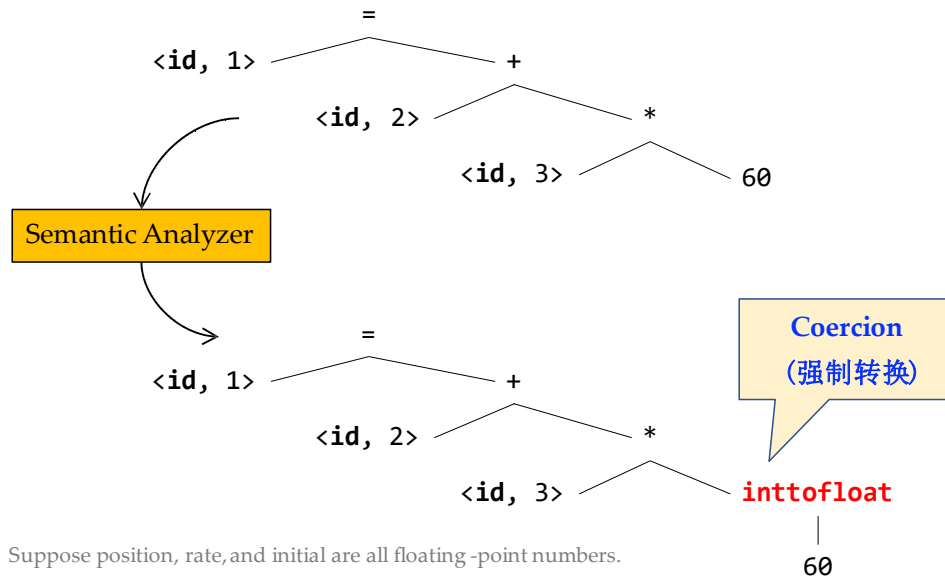
Compilers should report an error if the argument of an array element access operation is a floating-point number.

类型检查是语义分析的一个重要部分

编译器检查每个运算符是否具有匹配的操作数（正确类型）

示例：许多语言定义要求数组索引为整数。如果数组元素访问操作的参数是浮点数，则编译器应报告错误。

Semantic Analysis (Example)



Intermediate Code Generation

(中间代码生成)

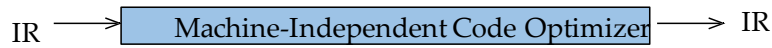


- After semantic analysis, compilers generate an intermediate representation, typically *three-address code* (三地址码)
 - *Assembly-like instructions* with three operands per instruction
 - Each operand acts like a register
 - Each assignment instruction has at most one operator on the RHS
 - Easy to translate into machine instructions of the target machine

语义分析后，编译器生成一个中间表示，通常为三地址码

- 每个指令有三个操作数的类汇编指令
- 每个操作数的作用就像一个寄存器
- 每个赋值指令在 RHS 上最多有一个运算符
- 易于翻译成目标机器的机器指令

Machine-Independent Code Optimization (机器无关的代码优化)



- Akin to article editing/revising in English
- Improve the intermediate code for better target code
 - Run faster
 - Use less memory
 - Shorter code
 - Consume less power ...

使用英语编辑/修改文章改进中间代码

以获得更好的目标代码

- 跑得更快
- 使用更少的内存
- 较短代码
- 耗电更少...

Code Optimization (Example)

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

1. **60 is a constant integer value.** Its conversion to floating-point can be done once and for all at compile time
2. **t3** is only used for value transmitting

Optimization

```
t1 = id3 * 60.0
id1 = id2 + t1
```

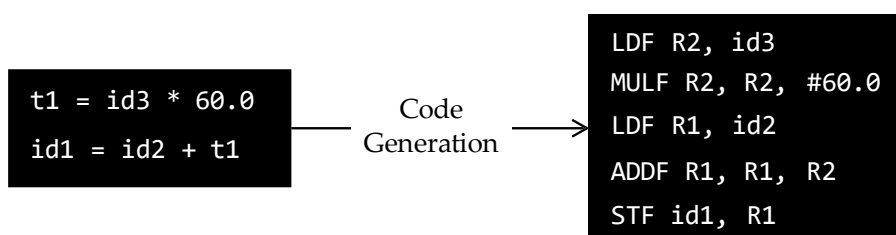
60 是一个恒定的整数值。它到浮点的转换可以在编译时一次性完成。

t3 仅用于值传输

Code Generation (代码生成)

IR $\longrightarrow$ Code Generator $\longrightarrow$ Target-machine code

- Map IR to target language, analogous to human translation
- It is crucial to **allocate register and memory** to hold values



将 IR 映射到目标语言，类似于人工翻译

分配寄存器和内存以保存值至关重要

Symbol Table Management

- **Performed by the frontend**, symbol table is passed along with the intermediate code to the backend
- Record the **variable names** and various **attributes**
 - storage allocated, type, scope
- Record the **procedure names** and various **attributes**
 - the number and type of arguments
 - the way of passing arguments (by value or by reference)
 - the return type

符号表管理

由前端执行，符号表与中间代码一起传递到后端

记录变量名称和各种属性

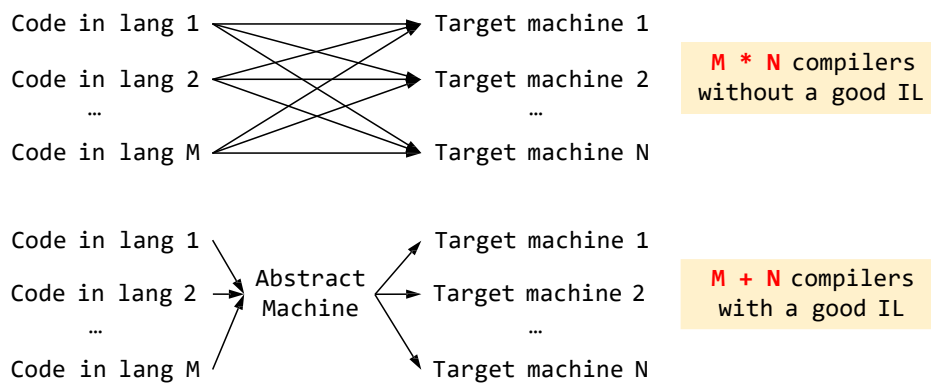
- 分配的存储、类型、范围

记录过程名称和各种属性

- 参数的数量和类型
- 传递参数的方式（通过值或引用）
- 返回类型

Intermediate Language (IL)

- Intermediate code is in IL (e.g., three-address code)
- A good IL eases compiler implementation



中间语言 (IL)

中间代码在 IL 中 (例如, 三个地址代码)

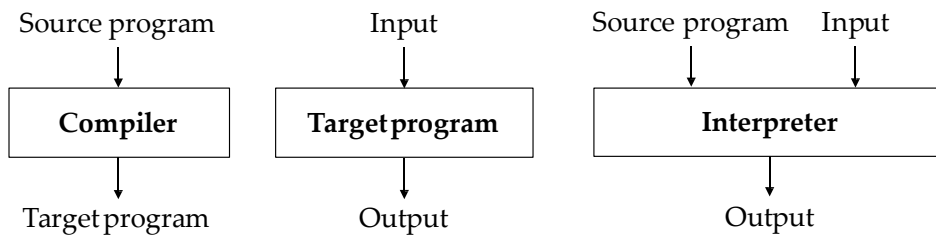
一个好的 IL 简化了编译器的实现

Compilers vs. Interpreters

1

A **compiler** translates **source programs** written in high-level languages into **machine codes** that can run directly on the target computer.

An **interpreter** **directly executes** each statement in the source code, without requiring the program to have been compiled into machine codes.



编译器将用高级语言编写的源程序翻译成可以直接在目标计算机上运行的机器代码。

解释器直接执行源代码中的每个语句，而不需要将程序编译成机器代码。

Compilers vs. Interpreters

2

Interpreters often take less time to analyze the source code: they simply parse each statement and execute it (e.g., Python code).

In comparison, compilers typically analyze the relationships among statements (e.g., control and data flows) to enable optimizations.

3

Interpreters continue executing a program until the first error is met, in which case they stop.

For compiled languages, programs are executable only after they are successfully compiled.

解释器通常花更少的时间来分析源代码：他们只需解析每个语句并执行它（例如 Python 代码）。

相比之下，编译器通常会分析语句（例如，控制流和数据流）之间的关系，以实现优化。

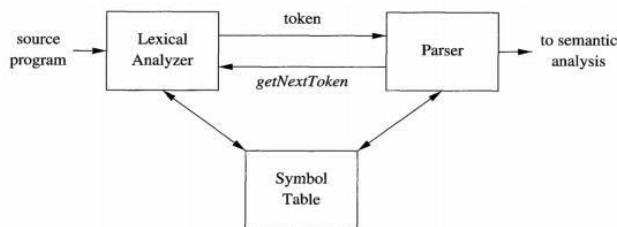
解释器持续执行程序，直到遇到第一个错误才停止。

对于编译语言，程序只有在成功编译后才能执行。

Chapter 2: Lexical Analysis

The Role of Lexical Analyzer

- Read the input characters of the source program, group them into lexemes, and produces a sequence of tokens
- Add lexemes into the symbol table when necessary
- Strip out comments and whitespace (blank, newline, tab etc.)
- Associate error messages with a line number (tracking newlines)



作为编译器的第一阶段，词法分析器的主要任务是读取源程序的输入字符，将它们组成为词素，生成并输出一个词法单元序列，每个词法单元对应一个词素。这个词法单元序列被输出到语法分析器进行语法分析。这些相互作用如图 3.1 所示。通常，交互是通过让语法分析器调用词法分析器来实现的。 `getNextToken` 命令所指示的调用使词法分析器从其输入中读取字符，直到它可以识别下一个词素为止。词法分析器根据这个词素生成下一个词法单元并将其返回给语法分析器。

- 词法分析器与符号表交互也很常见。当词法分析器发现构成标识符的词素时，它需要将该词素输入到符号表中。在某些情况下，词法分析器可以从符号表中读取有关标识符种类的信息，以帮助它确定必须传递哪个词法单元。

由于词法分析器是编译器中读取源文本的部分，因此除了词素识别之外，它还可以执行某些其他任务。

- 其中一项任务是去除注释和空格（空白、换行符、制表符以及可能用于分隔输入中词法单元的其他字符）。

- 另一项任务是将编译器生成的错误消息与源程序的位置联系起来。例如，词法分析器可以跟踪所遇到的换行符的数量，因此它可以给每个错误消息赋予一个行号。在某些编译器中，词法分析器会制作源程序的副本，并将错误消息插入到适当的位置。如果源程序使用宏预处理器，则宏的扩展也可以由词法分析器完成。

Why Separated Lexical Analysis?

- **Simplicity of compiler design.** The lexical analyzer can perform simple tasks such as dealing with comments and whitespace
- **Improved compiler efficiency.** Lexical analysis is much simpler comparing to syntax analysis
- **Higher portability of compilers.** Only lexical analyzer needs to deal with input-device-specific peculiarities (e.g., line separator)

出于多种原因，编译器的分析部分通常分为词法分析和解析（语法分析）阶段。

1.设计的简单性是最重要的考虑因素。词法分析和语法分析的分离通常使我们能够简化这些任务中的至少一项。例如，如果一个语法分析器必须将注释和空格作为语法单元进行处理，那么他就会比那些假设空白和注释已被词法分析器删除的语法分析器要复杂得多。如果我们正在设计一种新语言，分离词汇和语法问题有助于我们得到一个更加清晰的语言设计方案。

2.编译效率提高。把词法分析器独立出来使得我们能够使用专用于词法分析器、不进行语法分析的技术。此外，我们可以使用专门的用于读取输入字符的缓冲技术可以显著加快编译速度。

3.编译器可移植性增强。输入设备相关的特殊性可以被限制在词法分析器中。（如：行分隔）

Tokens, Patterns, and Lexemes

- A **token** is a pair `<token name, attribute value>`
 - **Token name** is an abstract symbol representing a kind of **lexical unit**
 - **Attribute value** (optional) points to the symbol table
- A **pattern** is a description of the form that the lexemes of a token may take.
- A **lexeme** is a sequence of characters in the source program that matches the pattern for a token.
 - It is identified by the lexical analyzer as **an instance of the token**.

在讨论词法分析时，我们使用三个相关但不同的术语：

- **词法单元**是由词法单元名称和可选属性值组成的对。
 - 词法单元名称是表示某种词汇单元的抽象符号，例如，特定的关键字，或表示标识符的输入字符序列。词法单元名称是语法分析器处理的输入符号。后面的内容中，我们一般会用黑体字给出词法单元的名称。我们将使用词法单元的名字来引用一个词法单元
 - 属性值（可选）指向符号表
- **模式**描述了一个词法单元的词素可能具有的形式。当词法单元是一个关键字的情况下，它的模式就是组成这个关键字的字符序列。对于标识符和其他一些词法单元，模式是一个更复杂的结构，它可以和很多符号串匹配。
- **词素**是源程序中与某个词法单元模式匹配的一个字符序列，并被词法分析器识别为该词法单元的一个实例。

Examples

TOKEN	INFORMAL DESCRIPTION	SAMPLE LEXEMES
if	characters i, f	if
else	characters e, l, s, e	else
comparison	< or > or <= or >= or == or !=	<=, !=
id	letter followed by letters and digits	pi, score, D2
number	any numeric constant	3.14159, 0, 6.02e23
literal	anything but ", surrounded by "'s	"core dumped"

Consider the C statement: `printf("Total = %d\n", score);`

Lexeme	printf	score	"Total = %d\n"	(	...
Token	id	id	literal	left_parenthesis	...

例 3.1：图 3.2 给出了一些常见的词法单元、非正式描述的词法单元的模式，并给出了一些示例词素。下面说明上述概念在实践中是如何应用的。在 C 语句中

```
printf("Total = %d\n", score);
```

printf 和 score 都是和词法单元 id 的模式匹配的词素，而 “Total = %d\n” 则是一个与 literal 匹配的词素。

在许多编程语言中，以下类涵盖了大部分或全部词法单元：

1. 每个关键字有一个词法单元。关键字的模式就是该关键字本身。
2. 表示运算符的词法单元，它可以表示单个运算符，也如图 3.2 中的 comparison 那样，表示一类运算符。
3. 一个表示所有标识符的词法单元。
4. 一个或多个表示常量的词法单元，例如数字和字面值字符串。
5. 每个标点符号有一个词法单元，例如左括号、右括号、逗号和分号。

Attributes for Tokens

- When more than one lexeme match a pattern, the lexical analyzer must provide additional information, named *attribute values*, to the subsequent compiler phases
 - *Token names* influence parsing decisions
 - *Attribute values* influence semantic analysis, code generation etc.
- For example, an **id** token is often associated with: (1) its lexeme, (2) type, and (3) the location at which it is first found. Token attributes are stored in the *symbol table*.

`A = B * 2` $\longrightarrow$

- <id, pointer to symbol-table entry for A>
- <assign_op>
- <id, pointer to symbol-table entry for B> <mult_op>
- <number, integer value 2>

Fall 2024

BUPT Compilers

7

当多个词素可以匹配某个模式时，词法分析器必须向编译器的后续阶段提供有关被匹配词素的附加信息。例如，词法单元 `number` 的模式与 `0` 和 `1` 都能匹配，但是对于代码生成器来说，至关重要的是知道在源程序中找到了哪个词素。因此，在许多情况下，词法分析器不仅返回给语法分析器一个词法单元名字，还返回一个描述该词法单元所表示的词素的属性值；

- 词法单元名字将影响语法分析过程中的决策，
- 而这个属性值会影响语法分析后对这个词法单元的翻译。

我们将假设词法单元至多具有一个相关的属性值，当然该属性值可能是一个组合了多种信息的结构化数据。最重要的例子是词法单元 `id`，我们需要将大量信息与它关联。通常，和一个标识符有关的信息——例如，它的词素、类型以及它第一次出现的位置（在发出一个有关该标识符的错误消息时需要使用这个信息）——都保存在符号表中。因此，一个标识符的属性值是一个指向符号表中该标识符对应条目的指针。

示例 3.2: Fortran 语句 `A=B*2` 的词法单元名称和相关属性值在下面写为名字-属性对的序列。

请注意，在某些对中，特别是运算符、标点符号和关键字的对中，不需要属性值。在此示例中，词法单元 `number` 有一个整数属性值。实际上，编译器会保存一个代表该常量的字符串，并将指向该字符串的指针用作 `number` 的属性值。

Lexical Errors

- **The lexical analyzer is unable to proceed:** none of the patterns for tokens match any prefix of the remaining input
- Example: `int 3a = a * 3;`

如果没有其他组件的帮助，词法分析器很难发现源代码中的错误。比如，当词法分析器在 `c` 语言程序片段 `fi (a == f(x))` 中第一次遇到 `fi` 时，它无法指出 `fi` 究竟是关键字 `if` 的拼写错误还是一个未声明的函数标识符。由于 `fi` 是词法单元 `id` 的一个合法词素，因此词法分析器必须将词法单元 `id` 返回给语法分析器，并让编译器的某些其他阶段（在这种情况下是语法分析器）去处理这个由于字母换位而导致的错误。

然而，假设出现所有词法单元模式都无法与剩余输入的某个前缀匹配的情况，词法分析器就不能继续处理输入。最简单的恢复策略是“恐慌”恢复。我们从剩余的输入中不断删除字符，直到词法分析器能够，直到词法分析器可以在剩余输入的开头找到一个正确的词法单元为止。这种恢复技术可能会给语法分析器带来混乱，但是在交互式计算环境中，这可能已经足够了。

Specification of Tokens

- **Regular expression** (正则表达式, **regex** for short) is an important notation for specifying lexeme patterns
- Content of this part
 - Strings and Languages (串和语言)
 - Operations on Languages (语言上的运算)
 - Regular Expressions
 - Regular Definitions (正则定义)
 - Extensions of Regular Expressions

正则表达式是指定词素模式的重要符号。虽然它们不能表达所有可能的模式，但它们可以高效地描述在处理词法单元时要用到的模式类型。在本节中，我们将研究正则表达式的形式表示法本部分内容：

- 字符串和语言
- 语言上的运算
- 正则表达式
- 常规定义（正则定义）
- 正则表达式的扩展

Strings and Languages

- **Alphabet (字母表)**: any finite set of symbols
 - Examples of symbols: **letters**, **digits**, and **punctuations**
 - Examples of alphabets: **{1, 0}**, **ASCII**, **Unicode**
- A **string (串)** over an alphabet is a finite sequence of symbols drawn from the alphabet
 - The length of a string s , denoted $|s|$, is the number of occurrences of symbols in s (i.e., cardinality)
 - **Empty string (空串)**: the string of length 0, ϵ

字母表是一个有限的符号集合。

- 符号的典型示例是字母、数字和标点符号。
- 集合 $\{0, 1\}$ 是二进制字母表。ASCII 是字母表的一个重要例子；它被用于许多软件系统中。Unicode 包含了大约 100,000 个来自世界各地字母的字符，是字母表的另一个重要示例。

字母表上的字符串（串）是从字母表中抽取的符号的有限序列。字符串 s 的长度（用 $|s|$ 表示）是指有多少个

Strings and Languages Cont.

- String-related terms (using **university** for illustration)
 - **Prefix (前缀)** of string s : any string obtained by removing 0 or more symbols from the end of s (ϵ , **uni**, **univer**, **university**)
 - **Proper prefix (真前缀)**: a prefix that is not ϵ and not equal to s itself (**uni**, **univer**)
 - **Suffix (后缀)**: any string obtained by removing 0 or more symbols from the beginning of s (ϵ , **sity**, **versity**, **university**).
 - **Proper suffix (真后缀)**: a suffix that is not ϵ and not equal to s itself (**sity**, **versity**)

- 字符串相关术语（以 banana 为例进行说明）
 - 字符串 s 的前缀（前缀）：从 s 的末尾去掉 0 个或多个符号后得到的任何字符串
 - 适当的前缀（真前缀）：不是空且不为整个字符串
 - 后缀（Suffix）：从 s 开头去掉 0 个或多个符号后得到的字符串
 - 正后缀（真后缀）：不是空且不为字符串本身

Strings and Languages Cont.

- String-related terms (using **university** for illustration)
 - **Substring (子串)** of s: any string obtained by removing any prefix and any suffix from s (**ver**, **sity**, **uni**, **iv**, **ε**, **university**)
 - **Proper substring (真子串)**: a substring that is not ϵ and not equal to s itself (**ver**)
 - **Subsequence (子序列)**: any string formed by removing 0 or more not necessarily consecutive symbols from s (**uvrit**, **uiri**)



How many substrings does **napkin** have?

子串：从 s 中去掉任何前缀和后缀后得到的任何字符串,s 的所有前缀、后缀也都是它的子串。对任意的符号串 s, s 自身、 ϵ 都是 s 的前缀、后缀, 也是 s 的子串。

真子串：不为空且不等于字符串

子序列：从 s 中删除 0 个或多个不一定连续的符号后形成的字符串

Let the length of the string be n, so looking at the example above, we understand that to find all possible numbers of substrings, we need to add substrings of length n, (n-1), (n-2), (n-3), (n-4),.....2, 1.

Total number of substrings = $n + (n - 1) + (n - 2) + (n - 3) + (n - 4) + \cdots + 2 + 1$.

= $n * (n + 1) / 2 = 21$

So now we have a formula for evaluating the number of substrings where n is the length of a given string.

22 个 (考虑空串)

Strings and Languages Cont.

- String-related operations (串的运算)
 - **Concatenation (连接)**: the concatenation of two strings x and y , denoted xy , is the string formed by appending y to x
 - $x = \text{dog}, y = \text{house}, xy = \text{doghouse}$
 - **Exponentiation (幂/指数运算)**: $s^0 = \epsilon, s^1 = s, s^i = s^{i-1}s$
 - $x = \text{dog}, x^0 = \epsilon, x^1 = \text{dog}, x^3 = \text{dogdogdog}$

如果 x 和 y 是字符串, 则 x 和 y 的连接 (表示为 xy) 是把 y 附加到 x 后面而形成的字符串。例如, 如果 $x = \text{dog}$ 且 $y = \text{house}$, 则 $xy = \text{doghouse}$ 。空串是连接运算的单位元; 也就是说, 对于任何字符串 s 都有, $s = s = s$ 。

如果我们将两个串的连接视为这两个串的乘积, 我们可以如下定义字符串的“指数”运算如下: 定义 s^0 定义为 ϵ , 并且对于所有 $i > 0$, 将 s^i 为 $s^{i-1}s$ 。因为 $s = s$, 由此可知 $s^1 = s$ 。 $s^2 = ss$, $s^3 = sss$, 依此类推。

Strings and Languages Cont.

- A **language** (语言) is any **countable set**¹ of strings over some fixed alphabets
 - The set containing only the empty string, that is $\{\epsilon\}$, is a language, denoted $\emptyset$
 - The set of all **grammatically correct English sentences**
 - The set of all **syntactically well-formed C programs**

¹In mathematics, a countable set is a set with the same cardinality (number of elements) as some subset of the set of natural numbers. A countable set is either a finite set or a countably infinite set.

- 语言 (语言) 是在某个固定字母表上的字符串的任何可数集合。

只包含空字符串的集合，即 $\{\epsilon\}$ ，即为空

所有语法正确的英语句子的集合

所有语法规范的 C 程序的集合

可数集 (Countable set)，是指每个元素都能与自然数集 N 的每个元素之间能建立一一对应的集合。如果将可数集的每个元素标上与它对应的那个自然数记号，那么可数集的元素就可以按自然数的顺序排成一个无穷序列 $a_1, a_2, a_3, \dots, a_n, \dots$ 。比如全体正偶数的集合是一个可数集，全体正奇数的集合也是可数集，它们与自然数集可以建立如下的一一对应。

Strings and Languages Cont.



- Operations on Languages (语言的运算)
 - 并，连接，Kleene闭包，正闭包

OPERATION	DEFINITION AND NOTATION
<i>Union of L and M</i>	$L \cup M = \{s \mid s \text{ is in } L \text{ or } s \text{ is in } M\}$
<i>Concatenation of L and M</i>	$LM = \{st \mid s \text{ is in } L \text{ and } t \text{ is in } M\}$
<i>Kleene closure of L</i>	$L^* = \bigcup_{i=0}^{\infty} L^i$
<i>Positive closure of L</i>	$L^+ = \bigcup_{i=1}^{\infty} L^i$

https://en.wikipedia.org/wiki/Stephen_Cole_Kleene

在词法分析中，最重要的语言上运算是并、连接和闭包。

并是常见的集合运算。语言的连接就是以各种可能的方式从第一种语言中任取一个字符串，再从第二语言任取一个串，然后将它们连接后得到的所有字符串的集合。

一个语言 L 的 (Kleene) 闭包，表示为 L^* ，就是将 L 连接 0 次或多次得到的字符串集合。注意 L^0 ，即“将 L 连接零次得到集合”被定义为 $\{g\}$ ，并且， L^i 被归纳地定义为 $L^{i-1}L$ 。

最后， L 的正闭包，表示为 L^+ ，与 Kleene 闭包基本相同，但没有项 L^0 。也就是说，除非属于 L ，否则不属于 L^+ 中。

Strings and Languages Cont.

- Operations on Languages (Example)

- $L = \{A, B, \dots, Z, a, b, \dots, z\}$
- $D = \{0, 1, \dots, 9\}$
- $L \cup D = \{A, B, \dots, Z, a, b, \dots, z, 0, 1, \dots, 9\}$
- LD : the set of 520 strings of length two, each consisting of one letter followed by one digit
- L^4 : the set of all 4-letter string
- L^* : the set of all strings of letters, including ϵ
- $L(L \cup D)^*$: ?
- D^+ : ?

例 3.3: 令 L 为字母的集合 $\{A, B, \dots, Z; a, b, \dots, z\}$ 并令 D 为数字集合 $\{0, 1, \dots, 9\}$ 。我们可以用两种本质上相同的方式来看待 L 和 D 。一种方式是 L 和 D 分别是大写字母和小写字母以及数字组成的字母表。第二种方式是 L 和 D 是语言, 其所有字符串的长度都恰好为 1。以下是可以使用图 3.6 中的运算符从语言 L 和 D 构造的一些新语言:

1. $L \cup D$ 是字母和数字的集——严格地说, 该语言有 62 个长度为 1 的字符串, 每个字符串要么是一个字母, 要么是一个数字。

2. LD 是 520 个长度为 2 的字符串的集合, 每个字符串由一个字母后跟一个数字组成。

3. L^4 是所有 4 个字母字符串的集合。

4. L^* 是所有由字母组成的串的集合, 包括空串。

5. $L(L \cup D)^*$ 是所有以字母开头的字母和数字字符串的集合。

6. D^+ 是由一个或多个数位构成的字符串的集合。

$L(L \cup D)^*$: 字母数字组合包括空 (字母在前)

D^+ : 数字组合, 不含空

Regular Expressions

Rules that define regexps over an alphabet Σ :

- BASIS: two rules form the basis:
 - ϵ is a regexp, $L(\epsilon) = \{\epsilon\}$
 - If a is a symbol in Σ , then a is a regexp, and $L(a) = \{a\}$
- INDUCTION: Suppose r and s are regexps denoting the languages $L(r)$ and $L(s)$
 - $(r) | (s)$ is a regexp denoting the language $L(r) \cup L(s)$
 - $(r)(s)$ is a regexp denoting the language $L(r)L(s)$
 - $(r)^*$ is a regexp denoting $(L(r))^*$
 - (r) is a regexp denoting $L(r)$. Additional parentheses do not change the language an expression denotes.

在例 3.3 中，我们可以首先给字母和数位集合命名，并使用语言运算符并、连接和闭包这些运算符来描述标识符。这个处理方法非常有用，以至于，人们常常使用一种称为正则表达式的表示方法来描述语言。正则表达式可以描述所有通过对某个字母表上的符号应用这些运算符而得到的语言。在这种表示法中，如果使用 `letter_` 代表任一字母或下划线，并且 `digit_` 来表示数位，那么我们可以通过以下方式描述 C 标识符的语言：

`Letter_ (letter_ | digit) *`

上面的竖线表示并运算，括号用于对子表达式进行分组，星号表示“零个或多个”括号中表达式的连接，将 `letter_` 和表达式其余部分并列表示连接。

正则表达式是使用下面描述的规则，从较小的正则表达式递归构建的。每个正则表达式 r 表示一种语言 $L(r)$ ，它也是根据 r 的子表达式所表示的语言递归定义的。以下是定义某些字母表上的正则表达式以及这些表达式表示的语言的规则。

归纳基础：如下两条规则构成归纳基础：

1. 是正则表达式， $L()$ 是 $\{g\}$ ，即该语言只包含空字符串。

2. 如果 a 是 Σ 中的一个符号, 则 a 是一个正则表达式, 且 $L(a) = \{a\}$, 即这个语言仅包含一个长度为 1 的字符串。请注意, 按照惯例, 我们使用斜体表示符号, 使用粗体表示相应的正则表达式。

归纳步骤: 由较小的正则表达式构建较大的正则表达式的归纳步骤分四个部分。假设 r 和 s 是分别正则表达式, 表示语言 $L(r)$ 和 $L(s)$ 的。

1. $(r)|(s)$ 是正则表达式, 表示语言 $L(r) \cup L(s)$ 的。

2. $(r)(s)$ 是正则表达式, 表示语言 $L(r)L(s)$ 的。

3. $(r)^*$ 是正则表达式, 表示 $(L(r))^*$ 的。

4. (r) 是表示 $L(r)$ 的正则表达式。最后一条规则表明我们可以在表达式两边添加额外的括号对并不影响表达式所表示的语言。

Regular Expressions Cont.

- Following the rules, regexps often contain **unnecessary pairs of parentheses**. We may drop some if we adopt the conventions:
 - **Precedence**: **closure**^{*} > **concatenation** > **union** |
 - **Associativity**: All three operators are **left associative**, meaning that operations are grouped from the left, e.g.,
 $a | b | c$ would be interpreted as $(a | b) | c$
- Example: $(a) | ((b)^*(c)) = a | b^*c$

正如所定义的, 正则表达式通常包含一些不必要的括号对。如果我们采用以下约定, 则可以删除某些括号对:

- 一元运算符 $*$ 具有最高优先级并且是左结合的。
- 连接具有第二高优先级并且是左结合的
- $|$ 具有最低优先级并且是左结合的。

例如，根据这些约定，我们可以将正则表达式 $(a)|((b)^*(c))$ 替换为 $a|b^*c$ 。两个表达式都表示同一组字符串，这些字符串要么是单个 a ，要么是零个或多个 b 后跟一个 c 。

Regular Expressions Cont.

- Examples: Let $\Sigma = \{a, b\}$
 - $a|b$ denotes the language $\{a, b\}$
 - $(a|b)(a|b)$ denotes $\{aa, ab, ba, bb\}$
 - a^* denotes $\{\epsilon, a, aa, aaa, \dots\}$
 - $(a|b)^*$ denotes the set of all strings consisting of 0 or more a 's or b 's: $\{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$
 - $a|a^*b$ denotes the string a and all strings consisting of 0 or more a 's and ending in b : $\{a, b, ab, aab, aaab, \dots\}$

举例说明：假设 $\Sigma = \{a, b\}$

- $a|b$ 表示语言 $\{a, b\}$
- $(a|b)(a|b)$ 表示语言 $\{aa, ab, ba, bb\}$ ，字母表上长度为 2 的所有字符串的集合。同一语言的另一个正则表达式是 $aa|ab|ba|bb$ 。
- a^* 表示由零个或多个 a 或 b 的实例构成的串的集合，即 $\{\epsilon, a, aa, aaa, \dots\}$ 。
- $(a|b)^*$ 表示由零个或多个 a 或 b 实例组成的所有字符串的集合，即 a 和 b 的构成的所有字符串的集合 $\{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$ 。同一语言的另一个正则表达式是 $(a|b)^*$ 。
- $a|a^*b$ 表示语言 $\{a, b, ab, aab, aaab, \dots\}$ ，即字符串 a 以及所有由零个或多个 a 组成并以 b 结尾的字符串集合。

Regular Expressions Cont.

- A **regular language** (正则语言) is a language that can be defined by a regexp
- If two regexps r and s denote the same language, they are **equivalent**, written as $r = s$
- Each **algebraic law** below asserts that expressions of two different forms are equivalent

LAW	DESCRIPTION
$r s = s r$	$ $ is commutative
$r (s t) = (r s) t$	$ $ is associative
$r(st) = (rs)t$	Concatenation is associative
$r(s t) = rs rt; (s t)r = sr tr$	Concatenation distributes over $ $
$\epsilon r = r\epsilon = r$	ϵ is the identity for concatenation
$r^* = (r \epsilon)^*$	ϵ is guaranteed in a closure
$r^{**} = r^*$	$*$ is idempotent

$(a|b)(a|b)$
 $=$
 $aa|ab|ba|bb$
 Is it true???

可以用正则表达式定义的语言称为正则集。

如果两个正则表达式 r 和 s 表示同一个正则集，我们说它们是等价的，写成 $r = s$ 。例

如， $(a|b) = (b|a)$ 。

正则表达式遵循一些代数定律；每条定律都断言两种不同形式的表达是等价的。图

3.7 显示了一些适用于任意正则表达式 r 、 s 和 t 的代数定律。

$(a|b)(a|b) = aa|ab|ba|bb$ Is it true??? 是正确的

Regular Definitions

- For **notational convenience**, we can give names to certain regexps and use those names in subsequent expressions
- If Σ is an alphabet of basic symbols, then a **regular definition** is a sequence of definitions of the form:

$$\begin{aligned}d_1 &\rightarrow r_1 \\d_2 &\rightarrow r_2 \\&\dots \\d_n &\rightarrow r_n\end{aligned}$$

where:

- Each d_i is a new symbol not in Σ and not the same as the other d 's
- Each r_i is a regexp over the alphabet $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$

为了方便表示，我们可能希望给某些正则表达式命名，并在后续表达式中使用这些名称，就好像这些名称本身就是符号一样。如果 Σ 是基本符号的字母表，则正则定义是具有如下形式的定义序列：

$$\begin{aligned}d_1 &\rightarrow r_1 \\d_2 &\rightarrow r_2 \\&\dots \\d_n &\rightarrow r_n\end{aligned}$$

其中：

1. 每个 d_i 是一个新符号，它们都不在 Σ 中，并且各不相同；
2. 每个 r_i 是字母表 $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$ 上的正则表达式。

我们限制每个 r_i 中只含有 Σ 中的符号和在它之前定义的所有 d ，可以避免递归定义问题，并且我们可以为每个 r_i 单独构造出只包含 Σ 中符号的正则表达式。为此，我们首先将 r_2 （它不能使用 d_1 之外的任何 d ）中的 d_1 替换为 r_1 ，然后再将 r_3 中的 d_1 和 d_2 替换为 r_1 和（替换之后的） r_2 ，依此类推。最后，将 r_n 中的 d_i （ $i = 1; 2; \dots; n-1$ ）替换为 r_i 的经替换后的版本，在这些版本中都只包含 Σ 中的符号。

Regular Definitions - Examples

- **Regexp** for **C identifiers**

letter_ → A | B | ... | Z | a | b | ... | z | _
digit → 0 | 1 | ... | 9
id → *letter_* (*letter_* | *digit*)*

_hello valid?
3times valid?

- **Regexp** for **C identifiers**

$(A|B|...|Z|a|b|...|z|_)((A|B|...|Z|a|b|...|z|_)|(0|1|...|9))^*$

例 3.5: C 标识符是由字母、数字和下划线组成的字符串。这是 C 标识符语言的常规定义。我们将按照惯例使用斜体来表示常规定义中定义的符号。

Extensions of Regular Expressions

- **Basic operators**: union |, concatenation, and Kleene closure * (proposed by Kleene in 1950s)

- A few **notational extensions**:

- **One of more instances**: the unary, postfix operator +
 - $r^+ = rr^*$, $r^* = r^+ | \epsilon$
- **Zero or one instance**: the unary postfix operator ?
 - $r? = r | \epsilon$
- **Character classes**: shorthand for a logical sequence
 - $[a_1a_2...a_n] = a_1 | a_2 | ... | a_n$
 - $[a-e] = a | b | c | d | e$

- The extensions are **only for notational convenience**, they do not change the descriptive power of regexps

自从 Kleene 在 20 世纪 50 年代引入了具有基本运算符并、连接和 Kleene 闭包的正则表达式以来，正则表达式中添加了许多扩展，以增强正则表达式描述字符串模式的能力。在这里，我们介绍一些最早出现在像 lex 这样的 Unix 实用程序中的扩展表示法。这些扩展标识法在词法分析器的规约中特别有用。本章的参考文献包含对当今仍在使用的一些正则表达式变体的讨论。

1. 一个或多个实例。单目后缀运算符 $+$ 表示一个正则表达式及其语言的正闭包。也就是说，如果 r 是正则表达式，则 $(r)^+$ 表示语言 $(L(r))^+$ 。运算符 $+$ 与运算符 $*$ 具有相同的优先级和结合性。两个有用的代数定律 $r^* = r + r^*$ 和 $r + r^* = r^*$ 说明了克林闭包和正闭包之间的关系。

2. 零个或一个实例。单目后缀运算符 $?$ 表示 “零或一次出现。” 也就是说， $r?$ 等价于 $r| \epsilon$ ，或者换句话说， $L(r?) = L(r) \cup \{\epsilon\}$ 。 $?$ 运算符与 $+$ 运算符具有相同的优先级和结合性。

3. 字符类。正则表达式 $a_1|a_2|\dots|a_n$ ，其中 a_i 是字母表的各个符号，可以用简写形式 $[a_1 a_2 a_n]$ 替换。更重要的是，当 $a_1; a_2; \dots; a_n$ 构成逻辑上连续的序列时，例如连续的大写字母、小写字母或数字，我们可以用 $a_1 - a_n$ 来替换它们，即只写出第一个和最后一个符号，中间用连词符分隔。因此， $[abc]$ 是 $a|b|c$ 的简写，而 $[a-z]$ 是 $a|b|z$ 的简写。

Recognition of Tokens

- Lexical analyzer examines the input string and finds a prefix that matches one of the tokens
- The first thing when building a lexical analyzer is to define the patterns of tokens using regular definitions
- **A special token:** $ws \rightarrow (blank | tab | newline)^+$
 - When the lexical analyzer recognizes the **whitespace token**, it does not return it to the parser, but rather restart the lexical analysis from the character that follows the whitespace (the next token gets returned to the parser)

在上一节中，我们学习了如何使用正则表达式来表达模式。现在，我们必须研究如何根据各个需要识别的词法单元的模式来构建一段代码。这段代码能够检查输入字符串，并在输入的前缀中找出一个和某个模式匹配的词素。我们的讨论将利用以下运行示例。

此外，我们还可以让词法分析器负责消除空白符，方法是让它识别如下定义词法单元"ws"：

```
ws ! ( Blank | tab | newline ) +
```

这里，blank、tab 和 newline 是用于表示具有同样名字的 ASCII 字符的抽象符号。词法单元 ws 同其他词法单元的不同之处在于，当我们识别到 ws 时，我们并不会将其返回给语法分析器，而是从这个空白之后的字符开始继续进行词法分析。返回给语法分析器的是下一个词法单元。

Example: Patterns and Tokens

<i>digit</i>	→	[0-9]
<i>digits</i>	→	<i>digit</i> ⁺
<i>number</i>	→	<i>digits</i> (. <i>digits</i>)? (E [+-]? <i>digits</i>)?
<i>letter</i>	→	[A-Za-z]
<i>id</i>	→	<i>letter</i> (<i>letter</i> <i>digit</i>)*
<i>if</i>	→	if
<i>then</i>	→	then
<i>else</i>	→	else
<i>relop</i>	→	< > <= >= = <>

Patterns for tokens

LEXEMES	TOKEN NAME	ATTRIBUTE VALUE
Any <i>ws</i>	—	—
if	if	—
then	then	—
else	else	—
Any <i>id</i>	id	Pointer to table entry
Any <i>number</i>	number	Pointer to table entry
<	relop	LT
<=	relop	LE
=	relop	EQ
<>	relop	NE
>	relop	GT
>=	relop	GE

Tokens, their patterns, and attribute values

例 3.8: 图 3.10 的语法片段描述了分支语句和条件表达式的简单形式。此语法类似于 Pascal 语言的语法, 因为 **then** 关键字显式地出现在条件表达式的后面。对于 **relop**, 我们使用 Pascal 或 SQL 等语言的比较运算符, 其中 **=** 表示“等于”, **<>** 表示“不等于”, 因为它呈现了一个有趣的词素结构。

文法的终结符包括 **if**、**then**、**else**、**relop**、**id** 和 **number**, 对于词法分析器来说是词法单元的名称。这些词法单元的模式使用正则定义进行描述, 如图 3.11 所示。**id** 和 **number** 的模式与示例 3.7 中看到的类似。

对于这种语言, 词法分析器将识别关键字 **if**、**then** 和 **else**, 以及与 **relop**、**id** 和 **number** 模式匹配的词素。为了简化问题, 我们做出一个如下常见的假设: 关键字也是保留字: 也就是说, 它们不是标识符, 虽然它们的词素与标识符的模式匹配。

我们对词法分析器的目标总结在图 3.12 中。该表显示, 对于各个词素或词素的集合, 应该将哪个词法单元名返回到语法分析器以及应该返回什么属性值, 如第 3.1.3 节中讨论的。请注意, 对于其中的六个关系运算符, 符号常量 LT、LE 等被当作属性值返回, 其目的是指明我们发现的是词法单元 **relop** 的哪个实例。找到的运算符将影响编译器输出的代码。

Transition Diagrams (状态转换图)

- An important step in constructing a lexical analyzer is to convert patterns into “**transition diagrams**”
- Transition diagrams have a collection of nodes, called **states** (状态) and **edges** (边) directed from one node to another

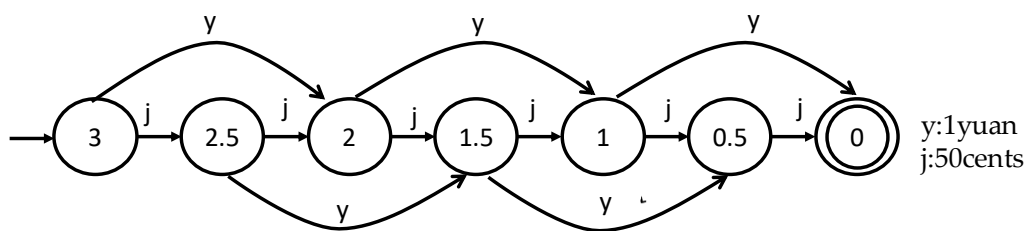
作为构建词法分析器的中间步骤，我们首先将模式转换为特定风格的流图，称为“状态转换图”。在本节中，我们手动执行从正则表达式模式到转换图的转换，但是在第 3.6 节中，我们将看到有一种自动化的方法可以从正则表达式的集合构造状态转换图。

状态转移图有一组被称为“状态”的结点或圆圈。词法分析器在扫描输入串的过程中寻找和某个模式匹配的词素，而转换图中的每个状态代表一个可能在该过程中出现的情况。我们可以将一个状态视为对我们已经看到的位于 `lexemeBegin` 指针和 `forward` 指针之间的字符的总结，它包含了我们在进行词法分析时需要的全部信息（如在图 3.3 的情况下）。

转换图的边从图的一种状态指向另一种状态。每条边的标号都包含了一个符号或多个符号。如果我们处于某种状态 `s`，并且下一个输入符号是 `a`，我们会寻找一条从 `s` 离开且标号为 `a` 的边（该边的标号中可能还包括其他符号）。如果我们找到这样的一条边，我们就将 `forward` 指针前移并进入转移图中该边所指的状态。我们将假设我们所有的转换图都是确定性的，这意味着在对于任何一个给定的状态和任何一个给定的符号，最多只有一个边从该状态离开的边的标号包含该符号。从第 3.5 节开始，我们将放宽对确定性的要求，使词法分析器的设计者更加容易完成任务，但同时提高了对实现者的技巧要求。

What is the process of operating a vending machine?

- Suppose a vending machine can only accept 50 cents and 1 yuan coins. How many states are there for the vending machine to purchase a \$3 bottle of Coke?



有限状态系统

状态：将事物进行区分的标识

具有离散状态的系统：数字电路(0 和 1)，交通信号灯（离散状态系统的状态是有限的）

具有连续状态的系统：水库水位变化，温度变化，具有无穷多个状态

有限状态系统是离散状态的系统

自动售货机的操作流程？如购买一瓶价格为 3 元的可乐需要做哪些步骤？

（虽然现在是用支付宝或者微信支付。。。）

假设某一台售货机只可接受 5 角和 1 元的硬币

购买一瓶 3 元的可乐，售货机的状态有几种？(状态可理解为购物的步骤)

假设已投入 2 元，为了买到可乐，还需投入 1 个 1 元硬币或 2 个 5 角硬币

购买一瓶 3 元的可乐，售货机的状态有几种？

需要 3 元: 还没有投入任何硬币的机器状态

需要 2.5 元: 已投入 1 个 5 角硬币后的状态

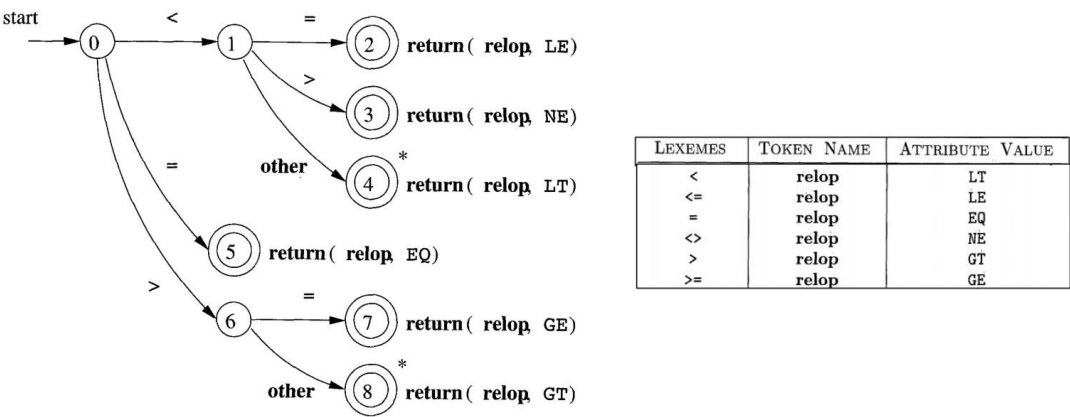
需要 2.0 元: 已投入 2 个 5 角或 1 个 1 元的硬币后的状态

需要 1.5 元: 已投入 3 个 5 角或 1 个 1 元和 1 个 5 角硬币后的状态

需要 1 元: 已投入 4 个 5 角或 1 个 1 元和 2 个 5 角的硬币后的状态

需要 0.5 元: 已投入 5 个 5 角或 3 个 5 角和 1 个 1 元或 1 个 5 角和 2 个 1 元硬币后的状态

需要 0 元: 表示已投入至少 3 元的硬币，可以购买可乐了

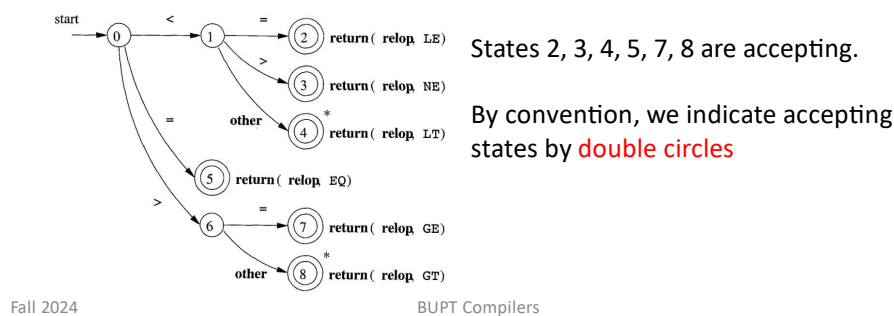


The transition diagram in the left is for recognizing **relop** tokens

例 3.9：图 3.13 是识别所有与词法单元 relop 匹配的词素的状态转换图。

States

- Represent conditions that could occur during the process of scanning the input for recognizing lexemes of tokens (what characters we have seen)
- The **start state** (开始状态), or initial state, is indicated by an edge labeled “start”, which enters from nowhere
- Certain states are said to be **accepting** (接受状态), or final, indicating that a lexeme has been found



Fall 2024

BUPT Compilers

31

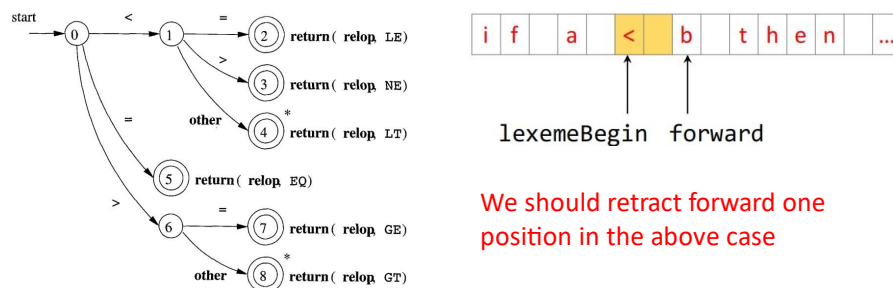
1. 某些状态称为接受状态或最终状态。这些状态表明已找到了一个词素，尽管实际的词素可能并不包含 lexemeBegin 指针和 forward 指针之间的所有字符。我们一直用双圆圈表示一个接受状态，并且如果该状态要执行一个动作的话——通常会向语法分析器返回一个词法单元和相关的属性值——我们将把该行动附加到该接受状态上。

2. 另外，如果需要将 forward 指针回退一个位置（即相应的词素并不包含那个在最后一步使我们到达接受状态的符号），那么我们将在该接受状态的附近加上*。在我们的示例中，不需要将 forward 指针回退多个位置，但如果需要，我们可以将接受状态附加相应数目的*。

3. 指定一个状态为开始状态，或初始状态；该状态由一条没有出发结点的、标号为“start”，的边指明。在读入任何输入符号之前，状态转换图总是从开始状态开始。

States Cont.

- At certain accepting states, the found lexeme may not contain all characters that we have seen from the start state
- Such states are annotated with *
- When entering * states, it is necessary to **retract (回退)** the forward pointer, which points to the next char in the input string)

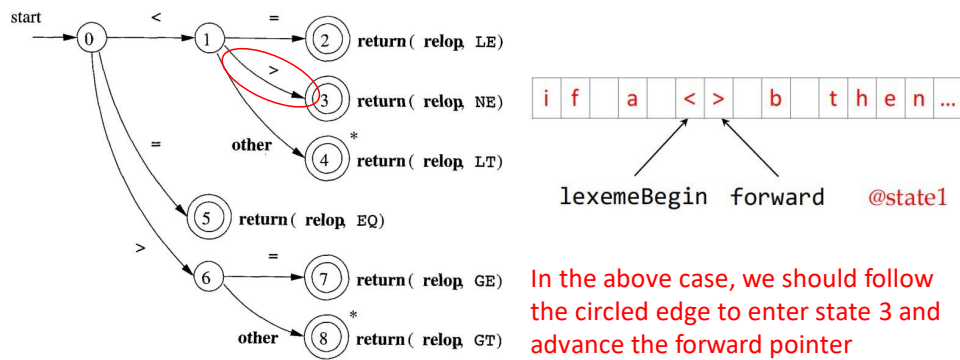


我们从初始状态 0 开始，即起始状态。如果我们看到的第一个输入符号是 <，那么在所有与 relop 模式匹配的词典中，我们只能选择 <、<> 或 <=。因此，我们进入状态 1，并查看下一个字符。如果是=，那么我们识别出词素<=，进入状态 2，并返回带有属性值为 LE 的 relop 词法单元。其中符号常量 LE 代表了这个具体的比较运算符。如果在状态 1 中下一个字符是 >，则我们得到词素 <>，从而进入状态 3 并返回一个词法单元，表明已经找到一个不等于运算符。对于任何其他字符，识别得到的词素为 <，我们进入状态 4 并向语法分析器返回这个信息。但请注意，状态 4 具有一个 *号，说明我们必须将输入回退一个位置。

另一方面，如果在状态 0 中我们看到的第一个字符是 =，那么这个字符一定是要识别的词素。我们立即从状态 5 返回该信息。其余的可能性是第一个字符是 >的情况。那么，我们应该进入状态 6，并根据下一个字符决定词素是 >=（如果我们接下来看到的符号为=），还是只是 >（对于任何其他字符）。请注意，在状态 0 中，如果我们看到的是不同于 <、=、或>的任何字符，我们就不可能看到一个 relop 词素，因此这个状态转换图将不会被使用。

Edges

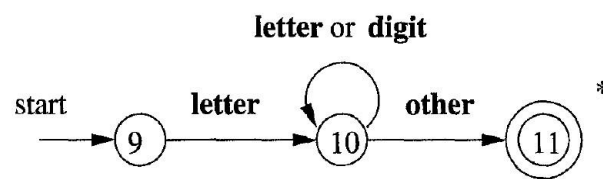
- **Edges** are directed from one state to another
- Each edge is labeled by a symbol or set of symbols



- 边从一个状态指向另一个状态
- 每条边都由一个或一组符号标记

Recognition of Reserved Words and Identifiers (保留字和标识符的识别)

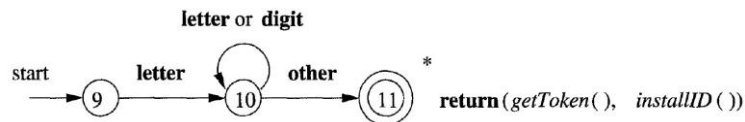
- In many languages, **reserved** words or **keywords** (e.g., `then`) also match the pattern of identifiers
- **Problem:** the transition diagram that searches for identifiers can also recognize reserved words



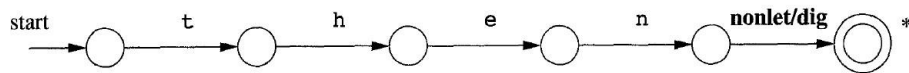
识别关键字和标识符时有一个问题要解决。通常，像 `if` 或 `then` 这样的关键字是被保留的（正如它们在我们运行的示例中一样），因此它们不是标识符，即使它们看起来像标识符。因此，虽然我们通常使用如图 3.14 所示的转换图来搜索标识符词素，但该图也可以将识别出连续使用的例子中的关键字 `if`、`then` 和 `else`。

Handling Reserved Words

- **Strategy 1:** Preinstall the reserved words in the symbol table. Put a field in the symbol-table entries to indicate that these strings are not ordinary identifiers



- **Strategy 2:** Create a separate transition diagram with a high priority for each keyword



有两种方法可以处理看起来像标识符的保留字：

1. 首先将保留字安装到符号表中。符号表条目的字段指示这些字符串并不是普通标识符，并指出它们所代表的词法单元。我们假设图 3.14 中使用的是这种方法。当我们找到一个标识符时，对 `installID` 的调用会将其放入符号表（如果符号表尚不存在），并返回一个指针，指向这个刚找到的词素所对应的符号表条目。当然，任何在词法分析时不在符号表中的标识符都不可能是一个保留字，因此它的词法单元是 `id`。函数 `getToken` 检查对应于刚找到的词素的符号表条目，并根据符号表中的信息返回该词素所代表的词法单元名称——要么是 `id`，要么是一个在初始化时就被加入到符号表中的关键字词法单元。

2. 为每个关键词创建单独的转换图；关键字 `then` 的示例如图 3.15 所示。请注意，这样的转换图包含的状态表示看到该关键字的各个后续字母后的情况，最后是一个“非字母或数字”的测试，也就是检查后面是否为某个不可能成为标识符一部分的字符。有必要检查标识符是否已经结束，否则在碰到词素像 `thenextvalue` 这样以 `then` 为前缀的 `id` 词法单元时，我们可能会错误地返回词法单元 `then`。如果我们采用这种方法，我们必须设定词法单元之间的优先级，使得当一个词素同时匹配 `id` 的模式和关键字的模式时，优先识别保留字

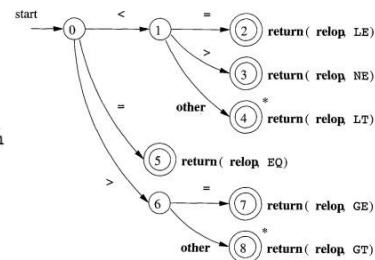
词法单元，而不是 id 词法单元。我们在示例中没有使用这种方法，这也是我们没有对图 3.15 中的状态进行编号的原因。

Building a Lexical Analyzer from Transition Diagrams

```

TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
               or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}

```

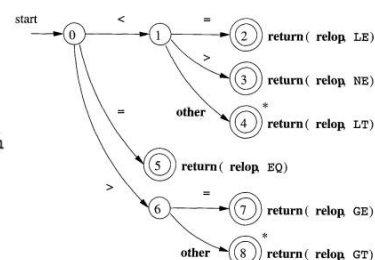


Building a Lexical Analyzer from Transition Diagrams

```

TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
               or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}

```

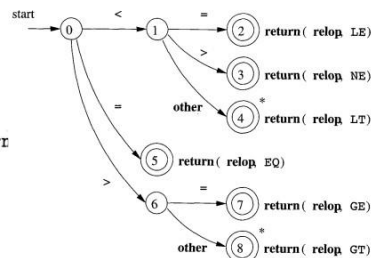


Use a variable state to record the current state

Building a Lexical Analyzer from Transition Diagrams

```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
               or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```

Sketch of implementation of relop transition diagram



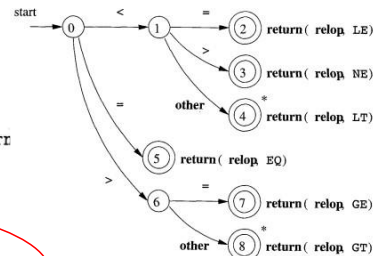
A switch statement based on the value of state takes us to code for each of the possible states

有一个 switch 语句根据 state 的值将我们转到对应于各个可能状态的相应代码段，我们可以在那里找到该状态需要执行的动作。一个状态的代码本身通常也是一个 switch 语句或多路分支，这个语句读取和检查下一个输入字符，由此确定下一个状态。

Building a Lexical Analyzer from Transition Diagrams

```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
               or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```

Sketch of implementation of relop transition diagram



The code of a normal state:

1. Read the next character
2. Determine the next state
3. If step 2 fails, do error recovery

示例 3.10: 在图 3.18 中, 我们可以看到 `getRelop()` 方法的一个概述, 它是一个 C++ 函数, 其作用是模拟图 3.13 的转换图并返回一个 `TOKEN` 类型的对象, 该对象由词法单元名称 (在这种情况下必定是 `relop`) 和一个属性值 (在这种情况下是六个比较运算符之一的编码)。 `getRelop()` 首先创建一个新对象 `retToken` 并将该对象的第一个分量初始化为 `RELOP`, 即词法单元 `relop` 的编码。

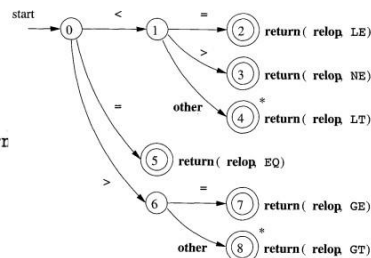
我们看到 `case 0` 中一个典型的状态行为。函数 `nextChar()` 从输入中获取下一个字符并将其分配给局部变量 `c`。然后, 我们检查 `c` 中是否有我们期望找到的三个字符, 并在每种情况下使根据图 3.13 所示的状态转换图完成状态转换。例如, 如果下一个输入字符是 `=`, 则转换到状态 5。

如果下一个输入字符不是某个比较运算符的首字符, 则 `getRelop()` 会调用函数 `fail()`。 `fail()` 的具体操作依赖于词法分析器的全局错误恢复策略。它应该将 `forward` 指针重置为 `lexemeBegin` 的值, 使得我们可以使用另一个转换图从尚未处理的输入部分的真正开始位置开始识别。然后, 它可能还会将变量 `state` 值更改为另一个转换图的初始状态, 该转换图将搜索另一个词法单元。在另一种情况下, 如果所有的转换图都已经用过, 则 `fail()` 可以启动一个错误纠错步骤, 如第 3.1.4 节中所述的方法来纠正输入并找到一个词素。

Building a Lexical Analyzer from Transition Diagrams

```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
                or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```

Sketch of implementation of relop transition diagram



The code of an accepting state:

1. Perform retraction if the state has *
2. Set token attribute values
3. Return the token to parser

我们还在图 3.18 中展示了状态 8 的行为。因为状态 8 带有一个 *号，所以我们必须将输入指针回退一个位置（即，将 c 放回输入流）。该任务是由函数 `retract()` 完成的。由于状态 8 代表了对词素 `>` 的识别，我们把返回对象中的第二个分量（我们假设这个分量的名字是 `attribute`）设置为 `GT`，即该运算符的编码。

Building the Entire Lexical Analyzer

- **Strategy 1:** Try the transition diagram for each token sequentially
 - fail() resets the pointer forward and starts the next diagram
- **Strategy 2:** Run transition diagrams in parallel
 - Need to resolve the case where one diagram finds a lexeme and others are still able to process input.
 - **Solution:** take the longest prefix of the input that matches any pattern
- **Strategy 3: Combining all transition diagrams into one (preferred)**
 - Allow the transition diagram to read input until there is no possible next state
 - Take the longest lexeme that matched any pattern

为了在适当的地方模拟适当的状态转换图，我们考虑几种将如图 3.18 所示的代码集成到整个词法分析器中的方法。

1.我们可以让词法分析器顺序尝试各个词法单元的转换图。然后，在每次调用例 3.10 中的函数 fail()时，它重置 forward 指针并启动下一个转换图。这种方法使我们可以像图 3-15 中建议的那样，为各个关键字使用各自的状态转换图。我们只需在使用 id 的状态转换图之前使用这些关键字的转换图，就可以使得关键字被识别为保留字。

2.我们可以并行运行各种转换图，“将下一个输入字符提供给所有转换图，并允许每个转换图做出它应执行的转换。如果我们采用这一策略，我们必须小心解决如下问题：一个状态转换图已经找到了一个与它的模式相匹配的词素，但另外的一个或多个状态转换图仍然可以继续处理输入。解决这个问题的常见策略是取最长的和某个模式相匹配的输入前缀。该规则允许我们识别出 thennext 而不是关键字 then，识别出 -> 而不是 -，例如。

3. 有一个更好的方法，也是我们将在以下各节中采用的方法，是将所有转换图合并为一个。我们允许合并后的转换图尽量读取输入，直到不存在下一个状态为止；然后像上面的第 (2) 项中讨论的那样取最长的和某个模式匹配的最长词素。在我们的运行示例中，进

行这种合并很简单，因为没有两个词法单元以相同的字符开头；即，根据第一个字符就可以知道我们正在寻找的是哪个词法单元。因此，我们可以直接将状态 0、9、12 和 22 合并成一个开始状态，而保持其他转换不变。然而，一般来说，正如我们不久将看到的那样，合并几个词法单元的转换图的问题更为复杂。

The Lexical-Analyzer Generator Lex

- Lex, or a more recent tool Flex, allows one to specify a lexical analyzer by specifying regexps to describe patterns for tokens
- Often used with Yacc/Bison to create the frontend of compiler

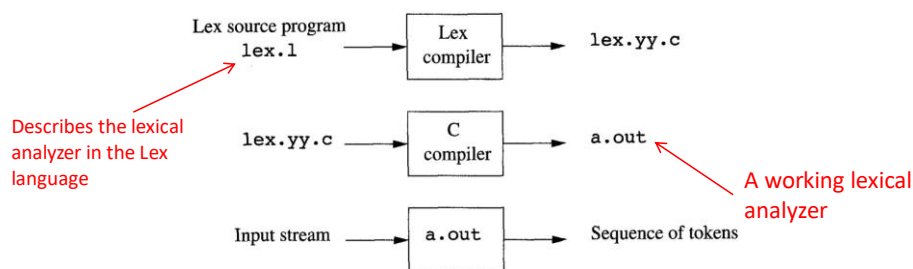


Figure 3.22: Creating a lexical analyzer with Lex

Lex 在本节中，我们将介绍一个名为 Lex 的工具，在最近的实现中它也称为 Flex。它支持使用正则表达式来描述词法单元的模式，由此给出一个词法分析器的归约。Lex 工具的输入表示方法为 Lex 语言，而工具本身则称为 Lex 编译器。在它的核心部分，Lex 编译器将输入的模式转换为一个状态转换图，并生成相应的实现代码，并存放到名为 lex.yy.c 的文件。这些代码模拟了状态转换图。

图 3.22 显示了 Lex 的使用方式。首先，用 Lex 语言编写一个输入文件（我们称之为 lex.l），描述要生成的词法分析器。然后，Lex 编译器将 lex.l 转换为 C 程序，该文件始终命名为 lex.yy.c。最后，文件 lex.yy.c 总是被 C 编译器编译为一个名为 a.out 的文件。C 编译器的输出是一个读取输入字符流并生成词法单元流工作的可允许的词法分析器。

编译后的 C 程序（在图 3.22 中称为 a.out）的通常是一个被语法分析器调用的子例程。这个子例程返回一个整数值，即可能出现的某个词法单元名的编码。而词法单元的属

性值，无论是一个数字编码、还是一个指向符号表的指针，或者什么都没有，都被放置在全局变量 `yylval` 中，该变量由词法分析器和语法分析器共享，从而可以轻松返回一个词法单元名字和一个属性值。

Structure of Lex Programs

- **A Lex program has three sections separated by `%%`**
 - **Declaration (声明)**
 - Variables, manifest constants (e.g., token names)
 - Regular definitions
 - **Translation rules (转换规则) in the form “Pattern {Action}”**
 - Each **pattern (模式)** is a regexp (may use the regular definitions of the declaration section)
 - **Actions (动作)** are fragments of code, typically in C, which are executed when the pattern is matched
 - **Auxiliary functions section (辅助函数)**
 - Additional functions that can be used in the actions

Fall 2024

BUPT Compilers

44

- 声明部分包括变量声明、明示常量（manifest constant，被声明的表示一个常数的标识符，例如，一个词法单元的名字）的声明和 3.3.4 节中描述的正则定义。

- Lex 程序的每个翻译规则的形式为模式{ 动作 }

其中，每个模式都是一个正则表达式，它可以使用声明部分中给出的正则定义。这些动作是代码片段，虽然人们已经创建了许多使用其他语言的 Lex 变体，但这些代码片段通常用 C 语言编写。

- 第三部分包含各个动作需要使用的所有辅助函数。还有一种方法是将这些函数可以单独编译，并与词法分析器的代码一起加载。

Lex Program Example

```
%{
    /* definitions of manifest constants
       LT, LE, EQ, NE, GT, GE,
       IF, THEN, ELSE, ID, NUMBER, RELOP */
}%

/* regular definitions */
delim    [ \t\n]
ws       {delim}+
letter   [A-Za-z]
digit    [0-9]
id       {letter}{(letter){digit}}*
number   {digit}+(\.{digit})+(E[+-]?{digit})?

%%
```

Anything in between %{ and }% is copied directly to lex.yy.c.
In the example, there is only a comment, not real C code to define manifest constants

Regular definitions that can be used in translation rules

Section separator

Fall 2020

CS323 Compilers

45

例 3.11：图 3.23 是一个 Lex 程序，它能够识别图 3.12 的各个词法单元并返回找到的词法单元。观察这段代码可以发现 Lex 的许多重要特点。

在声明部分，我们看到一对特殊的括号：%{ 和 }%。这些括号内的所有内容都会直接复制到文件 lex.yy.c，它们不会被视作正则定义。通常将明示常量的定义放在该括号里，并使用 C 语言的#define 语句给每个明示常量赋予一个唯一的整数编码。在我们的示例中，我们在注释中列出了 LT、IF 等明示常量，但没有显示它们被赋予哪些特定的整数。

声明部分中还包含一个正则定义的序列。这些定义使用了第 3.3.5 节中描述的正则表达式的扩展表示法。那些将在后面的定义中或某个翻译规则的模式中使用的正则定义用大括号括起来。因此，例如，delim 被定义为表示一个包含了空格、制表符和换行符的字符类的简写；与所有 UNIX 命令一样，后两个字符分别用反斜杠再跟上 t 或 n 表示。这个表示法和 Unix 命令使用的方法相同。于是，ws 通过正则表达式 {delim}+ 定义为一个或多个分隔符组成的序列。

请注意，在 id 和 number 的定义中，圆括号是用于分组的元符号，并不代表其圆括号本身。相反，number 定义中的 E 代表其自身。如果我们希望使用 Lex 的某个元符号（例如何括号、+、* 或? 等）来代表其自身，我们可以在它们前面加上反斜杠。例

如，我们在 `number` 的定义中看到 `\`，就表示小数点本身。在它前面加上反斜线的原因是，和在 UNIX 正则表达式中一样，该字符在 `lex` 是一个代表“任一字符”的元符号。

Lex Program Example Cont.

```
{ws}      /* no action and no return */
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yylval = (int) installID(); return(ID);}
{number}  {yylval = (int) installNum(); return(NUMBER);}
"<"      {yylval = LT; return(RELOP);}
"<="     {yylval = LE; return(RELOP);}
"="       {yylval = EQ; return(RELOP);}
"<>"     {yylval = NE; return(RELOP);}
">"      {yylval = GT; return(RELOP);}
">="     {yylval = GE; return(RELOP);}

%%
```

Continue to recognize other tokens

Return token name to the parser

Place the lexeme found in the symbol table

A global variable that stores a pointer to the symbol table entry for the lexeme. Can be used by the parser or a later component of the compiler.

首先，`ws`（在第一部分中声明的标识符）有一个相关的空动作。如果我们发现了一个空白符，我们并不把它返回给语法分析器，而是继续寻找另一个词素。第二个词法单元具有简单的正则表达式模式 `if`。如果我们在输入中看到两个字母 `if`，并且 `if` 后面没有跟随其他字母或数位（如果有的话，词法分析器会去寻找一个和 `id` 模式匹配的最长输入前缀），那么词法分析器从输入中读入这两个字符，并返回词法单元名 `IF`，即明示常量 `IF` 所代表的整数值。关键字 `then` 和 `else` 的处理方式类似。

第五个词法单元的模式由 `id` 定义的模式。请注意，虽然像 `if` 这样的关键字既匹配此模式也匹配之前的一个模式，但在最长匹配前缀与两个或多个模式匹配的情况下 `Lex` 会选择最先被列出的那个模式。当 `id` 被匹配时，相应的处理动作分为三步：

1. 调用函数 `installID()` 将找到的词素放入符号表中。
2. 该函数返回一个指向符号表的指针，该指针被放置在全局变量 `yylval` 中，可供语法分析器或编译器的后续组件使用。请注意，`installID()` 可以使用以下两个由 `Lex` 生成的、

由词法分析器自动赋值的变量： (a) `ytext` 是指向词素开头的指针，类似于图 3.3 中的 `lexemeBegin`。 (b) `yyleng` 存放刚找到的词素的长度。

3. 词法单元名 ID 返回给语法分析器。

当一个词素与模式 `number` 匹配时，执行的处理与此类似，它使用辅助函数 `installNum()` 完成处理。

Lex Program Example Cont.

- Everything in the auxiliary function section is copied directly to the file `lex.yy.c`
- Auxiliary functions may be used in actions in the translation rules

```
int installID() { /* function to install the lexeme, whose
                  first character is pointed to by ytext,
                  and whose length is yyleng, into the
                  symbol table and return a pointer
                  thereto */
}

int installNum() { /* similar to installID, but puts numer-
                    ical constants into a separate table */
}
```

Variables defined and set automatically by the lexical analyzer Lex generates

在辅助函数部分，我们看到两个这样的函数： `installID()` 和 `installNum()`。和位于 `%{...%}` 中的声明部分一样，出现在辅助部分中的所有内容都被直接复制到文件 `lex.yy.c` 中，虽然它们位于转换规则部分之后，但这些函数可以在规则部分的动作定义中使用。

Conflict Resolution

- When the generated lexical analyzer runs, it analyzes the input looking for **prefixes that match any of its patterns**.^{*}
- **Rule 1:** If it finds multiple such prefixes, it takes the **longest** one
 - The analyzer will treat `<=` as a single lexeme, rather than `<` as one lexeme and `=` as the next
- **Rule 2:** If it finds a prefix matching different patterns, **the pattern listed first** in the Lex program is chosen.
 - If the keyword patterns are listed before identifier pattern, the lexical analyzer will not recognize keywords as identifiers

^{*}See Flex manual for details (Chapter 8: How the input is matched) at <http://dinosaur.compilertools.net/flex/>

我们已经提到了当输入的多个前缀与一种或多种模式匹配时，Lex 用来决定要选择
正确词素的两条规则：

1. 总是优先选择较长的前缀而不是较短的前缀。
2. 如果最长的可能前缀与两个或多个模式匹配，则优先选择 Lex 程序中首先列出的模式。

Finite Automata (有穷自动机)

- Finite automata are the simplest machines to recognize patterns
- They are essentially graphs like transition diagrams. They simply say “yes” or “no” about each possible input string.
 - Nondeterministic finite automata (NFA, 非确定有穷自动机): A symbol can label several edges out of the same state (allowing multiple target states), and the empty string ϵ is a possible label.
 - Deterministic finite automata (DFA, 确定有穷自动机): For each state and for each symbol of its input alphabet, there is exactly one edge with that symbol leaving that state.
- NFA and DFA recognize the same languages, regular languages, that regexps can describe.

现在我们将了解 Lex 如何将其输入程序转变为词法分析器。转变的核心是被称为有穷自动机的表示方法。这些自动机本质上是与状态转换图类似的图，但有一些区别：

1. 有限自动机是识别器；它们只能对每个可能的输入字符串回答“是”或“否”。
2. 有限自动机有两种形式：

(a) 非确定性有限自动机 (NFA) 对其边的标号没有任何限制。一个符号标记离开同一状态的多条边，并且空字符串，也可以作为标号。

(b) 确定性有限自动机 (DFA) 对于每个状态以及自动机输入字母表中的每个符号都有且只有一条离开该状态，以该符号为标号的边。

确定性和非确定的有穷自动机能识别的语言的集合是相同的。事实上，这些语言正好是能够用正则表达式描述的语言的集合，这个集合中的语言称为正则语言。

Nondeterministic Finite Automata

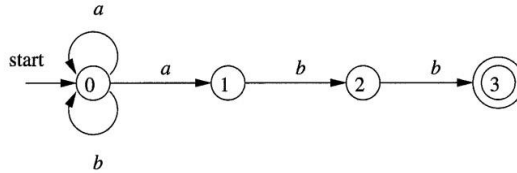
- A nondeterministic finite automaton (NFA) consists of:
 - A finite set of states S
 - A set of input symbols Σ , the **input alphabet**. We assume that the empty string ϵ is never a member of Σ .
 - A **transition function** that gives, for each state, and for each symbol in $\Sigma \cup \{\epsilon\}$ a set of next states.
 - A **start state** (or initial state) s_0 from S
 - A set of **accepting states** (or final states) F , a subset of S

不确定性有限自动机 (NFA) 包括：

1. 有限状态集 S 。
2. 一组输入符号集合，即输入字母表。我们假设代表空字符串的 ϵ 永远不是 Σ 的成员。
3. 一个转换函数，它为每个状态和 $\Sigma \cup \{\epsilon\}$ 中的每个符号都给出了相应的后继状态的集合。
4. S 中的一个状态 s_0 ，被指定为开始状态（或初始状态）。
5. S 的一个子集 F 被指定为接受状态（或者说终止状态）。

NFA Example

- $S = \{0, 1, 2, 3\}$
- Start state: 0
- Accepting states: $\{3\}$
- Transition function



- $(0, a) \rightarrow \{0, 1\}$ $(0, b) \rightarrow \{0\}$
- $(1, b) \rightarrow \{2\}$ $(2, b) \rightarrow \{3\}$

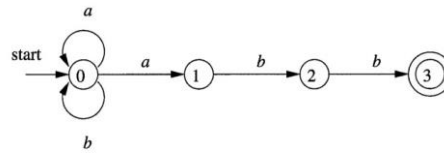
我们可以通过转移图来表示 NFA 或 DFA，图中的结点是状态，带标号的边表示自动机的转移函数。从状态 s 到状态 t 存在一条标号为 a 的边当且仅当状态 t 是状态 s 在输入 a 上的后继状态之一。该图非常类似于转换图，除了：

- 相同的符号可以标记从同一状态出发到达多个目标状态的多条边
- 一条边的标号不仅可以是输入字母表中的符号，也可以是空字符串。

例 3.14：识别正则表达式 $(a|b)^*abb$ 语言的 NFA 转换图如图 3.24 所示。这个抽象示例描述了所有由 a 和 b 组成的，以字符串 abb 结尾的字符串。这个例子将贯穿本节。虽然它很抽象，但是书籍上它与一些具有实际意义的语言的正则表达式相似。例如，描述所有其名字以 $.o$ 结尾的文件的表达式是 $\text{any}^*.o$ ，其中 any 代表任何可打印字符。

按照我们的转换图约定，状态 3 的双圈表示该状态是接受状态。请注意，从起始状态 0 到接受状态的所有路径都是先在状态 0 上运行一段时间，然后从输入读取 abb ，分别进入状态 1、2 和 3。因此，能够到达接受状态的所有字符串都是以 abb 结尾的字符串。

Transition Table



- Another representation of an NFA
 - **Rows** correspond to states
 - **Columns** correspond to the input symbols or ϵ
 - **The table entry** for a state-input pair lists the set of next states
 - $\emptyset$: the transition function has no info about the state -input pair

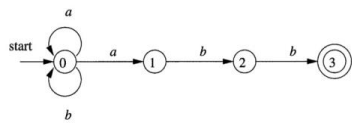
STATE	a	b	ϵ
0	$\{0, 1\}$	$\{0\}$	$\emptyset$
1	$\emptyset$	$\{2\}$	$\emptyset$
2	$\emptyset$	$\{3\}$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

我们还可以通过转换表来表示 NFA，表的各行对应于状态，各列对应于输入符号和 ϵ 。对应于一个给定状态和给定输入的条目是将 NFA 的转换函数应用于这些参数后得到的值。如果转换函数没有给出对应于某个状态-输入对的信息，我们就把 $\emptyset$ 放入相应的表项中。

例 3.15：图 3.24 的 NFA 的转换表如图 3.25 所示。转换表的优点是我们可以轻松确定一个给定状态和一个输入符号相对应的转换。它的缺点是，当输入字母表很大时，且大多数状态在大多数输入字符上没有转换的时候，转换表需要占用大量空间。

Acceptance of Input Strings

- An NFA **accepts** an input string **x** **if and only if**
 - There is a path in the transition graph from the start state to one accepting state, such that the symbols along the path form x (ϵ labels are ignored).



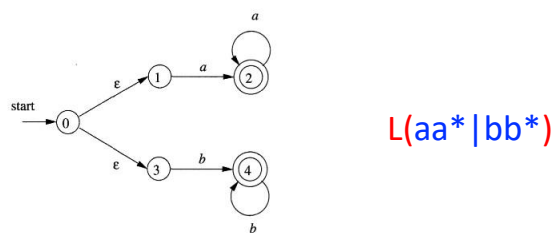
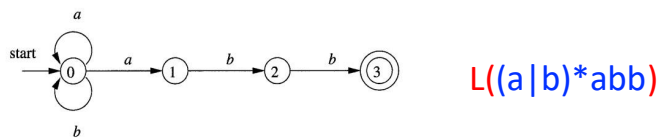
accepts the string *aabb*

- The **language** defined or accepted by an NFA
 - The set of strings labelling some path from the start state to an accepting state

请注意，可能还存在多条具有相同标号序列、但是到达不同状态的路径。例如下面的路径是另一条从状态 0 出发、标号序列同样为 *aabb* 的路径。这条路径最后仍回到状态 0，但状态 0 不是接受状态。然而，请记住，只要存在某条其标号序列为某字符串的路径能够从开始状态达到某个接受状态，NFA 就会接受该字符串。存在某些到达非接受状态的路径，并不会影响这个结论。

由一个 NFA 定义（或接受）的语言是从开始到某个接受状态的所有路径上的标号串的集合。

NFA and Regular Languages



正如前面提到的，图 3.24 的 NFA 定义的语言和正则表达式 $(a | b)^* abb$ 定义的语言相同，即所有来自字母表 $\{a, b\}$ 且以 abb 结尾的串的集合。我们可以用 $L(A)$ 来代表自动机 A 接受的语言。

例 3.17: 图 3.26 是一个接受 $L(aa^* | bb^*)$ 的 NFA。

字符串 aaa 被这个 NFA 接受。请注意，路径中的空标号在连接时消失了，因此这条路径的标签是 aaa 。

Deterministic Finite Automata (DFA)

- A deterministic finite automaton is a special NFA where:
 - There are no moves on input ϵ
 - For each state s and input symbol a , there is exactly one edge out of s labeled a
- DFA can efficiently accept/reject strings (recognize patterns)
- Every regexp and every NFA can be converted to a DFA accepting the same language

确定性有限自动机 (DFA) 是 NFA 的特例，其中：

1. 没有输入 ϵ 上的转换
2. 对于每个状态 s 和输入符号 a ，由且只有一条标号为 a 的边离开 s 。

如果我们使用转换表来表示一个 DFA，那么表中的每个表项就是一个状态。因此，我们可以不使用花括号，直接写出这个状态，因为花括号只是用来说明表项的内容的一个集合。

NFA 抽象地表示了用来识别某种语言中字符串的算法，而相应的 DFA 则是一个简单的具体的识别字符串的算法。

- DFA 可以高效地接受/剔除字符串（识别模式）
- 每个 regexp 和每个 NFA 都可以转换成接受相同语言的 DFA

Simulating a DFA

- **Input:**

- String x terminated by an end-of-file character eof.
- DFA D with start state s_0 , accepting states F , and transition function $move$

- **Output:** Answer “yes” if D accepts x ; “no” otherwise

```
 $s \leftarrow s_0;$   
 $c \leftarrow nextChar();$   
while (  $c \neq eof$  ) {  
     $s \leftarrow move(s, c);$   
     $c \leftarrow nextChar();$   
}  
if (  $s$  is in  $F$  ) return "yes";  
else return "no";
```

算法 3.18: 模拟 DFA。

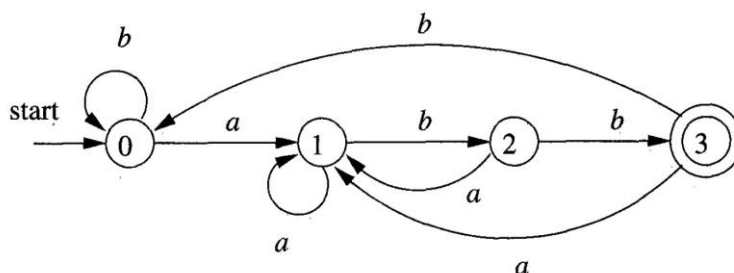
INPUT: 一个以文件结束符 eof 结尾的输入字符串 x 。DFA D 具有起始状态 s_0 、接受状态集 F 和转换函数 $move$ 。

输出: 如果 D 接受 x , 则回答 “是” ; 否则回答 “否” 。

方法: 将图 3.27 中的算法应用于输入字符串 x 。函数 $move(s; c)$ 给出了从状态 s 出发, 标号为 c 的边所到达的状态。函数 $nextChar$ 返回输入字符串 x 的下一个字符。

DFA Example

- Given the input string `ababb`, the DFA below enters the sequence of states `0, 1, 2, 1, 2, 3` and returns "yes"



What's the language defined by this DFA?

示例 3.19: 在图 3.28 中显示的一个 DFA 的转换图。该 DFA 接受的语言与与图 3.24 的 NFA 所接受的语言相同, 都是 $(a \mid b)abb$ 。给定输入字符串 `ababb`, 该 DFA 顺序进入状态序列 `0; 1; 2; 1; 2; 3` 并返回 `\yes`。”

From Regular Expressions to Automata

- Regexps concisely & precisely describe the patterns of tokens
- DFA can efficiently recognize patterns (comparatively, the simulation of NFA is less straightforward*)
- When implementing lexical analyzers, regexps are often converted to DFA:
 - $\text{Regexp} \rightarrow \text{NFA} \rightarrow \text{DFA}$

* There may be multiple transitions at a state when seeing a symbol

正则表达式非常适合描述词法分析器和其他模式处理软件。如第 3.5 节所述。然而，该软件的实现需要模拟 DFA（如算法 3.18 所示），或者模拟 NFA 的执行。由于 NFA 对于一个输入符号可以选择不同的转换（如图 3.24 所示状态 0 上输入为 a 时）它可以执行输入 上的转换（如图 3.26 中的状态 0 上时），甚至可以选择是对 或是对真实的输入符号执行转换，因此对 NFA 的模拟不如 对 DFA 的模拟简单。因此，我们需要将一个 NFA 转换为一个识别相同语言的 DFA。

在本节中，我们将首先介绍如何将 NFA 转换为 DFA。然后，我们使用这种称为“子集构造”的技术给出一个直接模拟 NFA 的算法。这个算法可用于那些将 NFA 转化到 DFA 比直接模拟 MFA 更加耗时的（非词法分析的）情形。接着，我们将说明如何把正则表达式转换为 NFA，在必要时可以根据这个 NFA 构造出一个 DFA。最后讨论了不同的正则表达式实现技术之间的时空权衡问题，并说明如何为具体的应用程序选择适当的方法。

Conversion of an NFA to a DFA

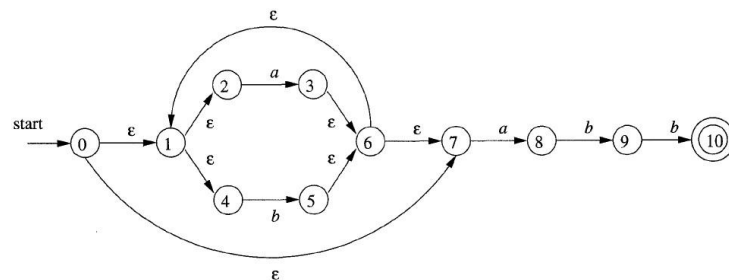
- The subset construction algorithm (子集构造法)
 - **Insight**: Each state of the constructed DFA corresponds to a set of NFA states
 - After reading the input $a_1a_2\dots a_n$, the DFA is in the state which corresponds to the set of states that the NFA can reach, from its start state, following paths labeled $a_1a_2\dots a_n$
 - The algorithm simulates “in parallel” all possible moves an NFA can make on a given input string

子集构造背后的总体思想是让构造得到的 DFA 的每个状态对应于 NFA 的一个状态集合。DFA 在读取输入 $a_1 a_2 a_1 a_2 a_n$ 之后到达的状态对应于相应 NFA 从其起始状态出发，沿着 $a_1 a_2 a_n$ 为标号的路径能够到达的状态集合。

- 算法 “并行” 模拟 NFA 在给定输入字符串上可能做出的所有动作

Example for Algorithm Illustration

- The NFA below accepts the string *babb*
 - There exists a path from the start state 0 to the accepting state 10, on which the labels on the edges form the string *babb*



算法示例

- 下面的 NFA 接受字符串 *babb*

存在一条从起始状态 0 到接受状态 10 的路径，在这条路径上，边上的标签构成了一个字符串 *babb*

Subset Construction Technique

- It is possible that # DFA states is exponential in # NFA states (**worst case**)
 - Each DFA state corresponds to a subset of NFA states
- However, for real languages, the NFA and DFA have **approximately the same number of states**, and the exponential behavior is not seen

DFA 状态数有可能是 NFA 状态数的指数，在这种情况下，我们在试图实现这个 DFA 时会遇到困难。然而，基于自动机的词法分析方法的处理能力部分源于以下事实：对于一个真实语言，它的 NFA 和 DFA 的状态数量大约相同，状态数量呈指数关系的情形尚未在实践中出现过。

Subset Construction Technique Cont.

- Operations used in the algorithm:
 - **ϵ -closure(s)**: Set of NFA states reachable from NFA state s on ϵ -transitions alone
 - **ϵ -closure(T)**: Set of NFA states reachable from some NFA state s in set T on ϵ -transitions alone
 - That is, $\bigcup_{s \in T} \epsilon\text{-closure}(s)$
 - **move(T, a)**: Set of NFA states to which there is a transition on input symbol a from some state s in T

我们的第一个问题是正确处理 N 的空转换。在图 3.31 中，我们看到了几个函数的定义，这些函数描述了一些需要在这个算法中执行的 N 的状态集上的基本计算。请注意， s 表示 N 的单个状态，而 T 代表 N 的一个状态集。

- $\epsilon\text{-closure}(s)$: 仅在 ϵ -transitions 上可从 NFA 状态 s 到达的 NFA 状态集合
 - $\epsilon\text{-closure}(T)$: 仅在 ϵ -transitions 上可从集合 T 中的某个 NFA 状态 s 到达的 NFA 状态的集合
- move (T , a): 输入符号 a 可以从 T 中的某个状态 s 过渡到的 NFA 状态集合

Subset Construction Technique Cont.

- Computing ϵ -closure(T)

- It is a graph traversal process (only consider ϵ edges)

- Computing ϵ -closure(s) is essentially the same

```
push all states of  $T$  onto  $stack$ ;  
initialize  $\epsilon$ -closure( $T$ ) to  $T$ ;  
while (  $stack$  is not empty ) {  
    pop  $t$ , the top element, off  $stack$ ;  
    for ( each state  $u$  with an edge from  $t$  to  $u$  labeled  $\epsilon$  )  
        if (  $u$  is not in  $\epsilon$ -closure( $T$ ) ) {  
            add  $u$  to  $\epsilon$ -closure( $T$ );  
            push  $u$  onto  $stack$ ;  
        }  
}
```

我们必须找出当 N 读入了某个输入串之后可能位于的所有状态集合。作为基础，在读取第一个输入符号之前， N 可以处于 ϵ -closure(s_0) 中的任何状态上，其中 s_0 是 N 的起始状态。下面进行归纳，假设 N 在读取输入字符串 x 后可以处于状态集 T 中的状态上。如果接下来读取输入 a ，则 N 可以立即进入 $move(T; a)$ 中的任何状态。然而， N 可以在读入 a 后再执行几次 ϵ -转换；因此 N 在读入 xa 之后，可以处于 ϵ -closure ($move(T; a)$) 中的任何状态上。

Subset Construction Technique Cont.

- The construction of the set of D 's states, $Dstates$, and the transition function $Dtran$ is also a search process

- Initially, the only state in $Dstates$ is $\epsilon\text{-closure}(s_0)$ and it is unmarked
- Unmarked state means that its next states have not been explored

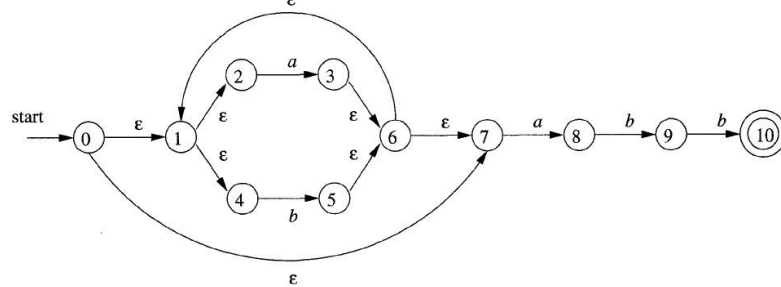
```
while ( there is an unmarked state  $T$  in  $Dstates$  ) {  
    mark  $T$ ;  
    for ( each input symbol  $a$  ) {  
         $U = \epsilon\text{-closure}(\text{move}(T, a))$ ;  
        if (  $U$  is not in  $Dstates$  )  
            add  $U$  as an unmarked state to  $Dstates$ ;  
         $Dtran[T, a] = U$ ;  
    }  
}
```

根据这些想法，我们可以得到如图 3.32 中显示的方法，该方法构造了 D 的状态集 $Dstates$ 和 D 的转换函数 $Dtran$ 。

D 的起始状态是 $\epsilon\text{-closure}(s_0)$ ， D 的接受状态是所有至少包含了 N 的一个接受状态的状态集合。我们只需要说明如何对 NFA 的任何状态集合 T 计算 $\epsilon\text{-closure}(s_0)$ ，就可以完整地描述子集构造法。这个计算过程如图 3.33 所示，它是从一个状态集合开始的一次简单的图搜索过程，不过此时假设这个图只存在标号为空的边。

Example

- A: $\epsilon\text{-closure}(0) = \{0, 1, 2, 4, 7\}$
- B: $Dtran[A, a] = \epsilon\text{-closure}(\{3, 8\}) = \{1, 2, 3, 4, 6, 7, 8\}$
- C: $Dtran[A, b] = \epsilon\text{-closure}(\{5\}) = \{1, 2, 4, 5, 6, 7\}$
- D: $Dtran[B, b] = \epsilon\text{-closure}(\{5, 9\}) = \{1, 2, 4, 5, 6, 7, 9\}$
- ...



例 3.21: 图 3.34 显示了另一个接受 $(a \mid b)^*abb$ 的 NFA; 它恰好是我们将第 3.7 节中根据这个正则表达式直接构造得到的 NFA。让我们将算法 3.20 应用于图 3.34。

等价 DFA 的起始状态 A 为 $\epsilon\text{-closure}(0)$, 即 $A = \{0; 1; 2; 4; 7\}$ 。A 中的状态就是能够从状态 0 出发, 只经过标号为空的路径达到的所有状态。请注意, 因为路径可以包含边, 所以状态 0 也是可从它自身出发经过标号为空的路径到达的状态。

NFA 的输入字母表是 $\{a; b\}$ 。因此, 我们的第一步是标记 A, 并计算 $Dtran[A; a] = \epsilon\text{-closure}(\text{move}(A; a))$ 和 $Dtran[A; b] = \epsilon\text{-closure}(\text{move}(A; b))$ 。在状态 0、1、2、4 和 7 中, 只有 2 和 7 分别到达状态 3 和 8。因此, $\text{move}(A; a) = \{3; 8\}$ 。同时, $\epsilon\text{-closure}(\{3; 8\}) = \{1; 2; 3; 4; 6; 7; 8\}$, 所以我们得出结论

$$Dtran[A; a] = \epsilon\text{-closure}(\text{move}(A; a)) = \epsilon\text{-closure}(\{3; 8\}) = \{1; 2; 3; 4; 6; 7; 8\}$$

我们称这个集合为 B, 得到 $Dtran[A; a] = B$ 。

现在, 我们要计算 $Dtran[A; b]$ 。在 A 中的状态中, 只有 4 有一个输入 b 上的转换, 它到达状态 5。因此,

$$Dtran[A; b] = \epsilon\text{-closure}(\text{move}(A; b)) = \epsilon\text{-closure}(\{5\}) = \{1; 2; 4; 5; 6; 7\}$$

我们称这个集合为 C, 因此 $Dtran[A; b] = C$ 。

如果我们对未加标记的集合 B 和 C 继续这个处理过程，最终会使得这个 DFA 的所有状态都被加上标记。这个结论一定正确，因为 11 个 NFA 状态的集合“仅”有 211 个子集。我们实际构造出 5 个不同的 DFA 状态、它们对应的 NFA 状态集以及 DFA D 的转换表如图所示图 3.35 所示，D 的转换图如图 3.36 所示。状态 A 是 D 的起始状态，而包含 NFA 的状态 10 的状态 E，是唯一的接受状态。

请注意， 相比 图 3.28 中接受同一语言的 DFA，这个 DFA D 多了一个状态。状态 A 和 C 具有相同的转换函数，因此可以合并。我们将在 3.9.6 节中讨论最小化 DFA 的状态数的问题。

T	T_a	T_b	
$\{0,1,2,4,7\}$ A	$\{1,2,3,4,6,7,8\}$ B	$\{1,2,4,5,6,7\}$	C
$\{1,2,3,4,6,7,8\}$ B	$\{1,2,3,4,6,7,8\}$ B	$\{1,2,4,5,6,7,9\}$	D
$\{1,2,4,5,6,7\}$ C	$\{1,2,3,4,6,7,8\}$ B	$\{1,2,4,5,6,7\}$	C
$\{1,2,4,5,6,7,9\}$ D	$\{1,2,3,4,6,7,8\}$ B	$\{1,2,4,5,6,7,10\}$	E
$\{1,2,4,5,6,7,10\}$ E	$\{1,2,3,4,6,7,8\}$ B	$\{1,2,4,5,6,7\}$	C

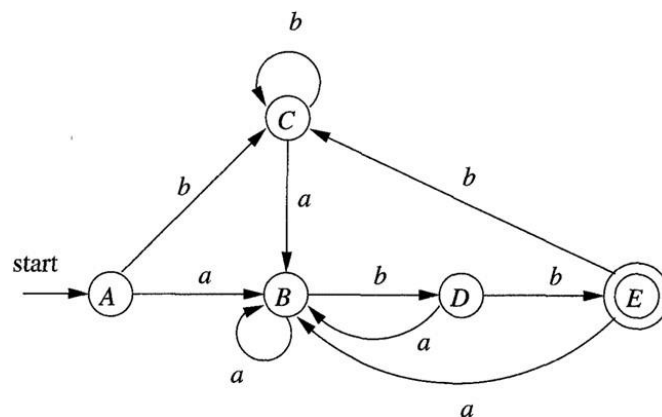
Transition Table of the DFA

- **Start state:** A; **Accepting states:** {E}

NFA STATE	DFA STATE	<i>a</i>	<i>b</i>
{0, 1, 2, 4, 7}	<i>A</i>	<i>B</i>	<i>C</i>
{1, 2, 3, 4, 6, 7, 8}	<i>B</i>	<i>B</i>	<i>D</i>
{1, 2, 4, 5, 6, 7}	<i>C</i>	<i>B</i>	<i>C</i>
{1, 2, 4, 5, 6, 7, 9}	<i>D</i>	<i>B</i>	<i>E</i>
{1, 2, 3, 5, 6, 7, 10}	<i>E</i>	<i>B</i>	<i>C</i>

This DFA can be further minimized: A and C have the same moves on all symbols and can be merged.

Transition Graph of the DFA



此 DFA D 识别的语言同样为 $L(D)=\{(a|b)^*abb\}$ ，由此可知构造出来的 DFA D 与 NFA M 是等价的

Regular Expression to NFA

- **Thompson's construction algorithm** (Thompson构造法)
- The algorithm works **recursively** by splitting an expression into its constituent subexpressions, from which the NFA will be constructed using a set of rules
- The rules for constructing an NFA:
 - **Two basis rules (基本规则)**: handle subexpressions with no operators
 - **Three inductive rules (归纳规则)**: construct larger NFA's from the NFA's for subexpressions

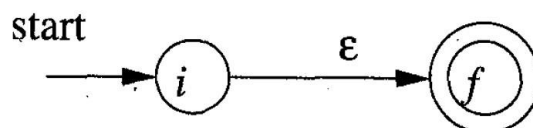
我们现在给出一种算法，用于将任何正则表达式转换为接受相同语言的 NFA。该算法是语法制导的，即它沿着正则表达式的语法分析树自底向上递归地进行处理。对于每个子表达式，算法都会构造一个只有一个接受状态的 NFA。

- 基本规则用于处理不带运算符的子表达式
- 归纳规则则根据一个给定表达式的直接子表达式的 NFA 构造出这个表达式的 NFA 的。

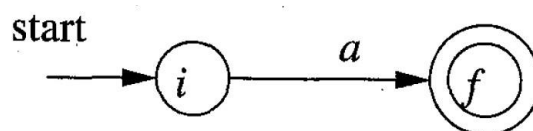
Thompson's Construction Algorithm

Two basis rules:

1. The **empty expression** ϵ is converted to



2. Any subexpression a (a **single symbol** in input alphabet) is converted to



BASIS:

- 对于表达式空构造下面的 NFA

这里, i 是一个新状态, 即该 NFA 的起始状态, f 是另一个新状态, 即该 NFA 的接受状态。

- 对于字母表中的子表达式 a , 构造 下面的 NFA

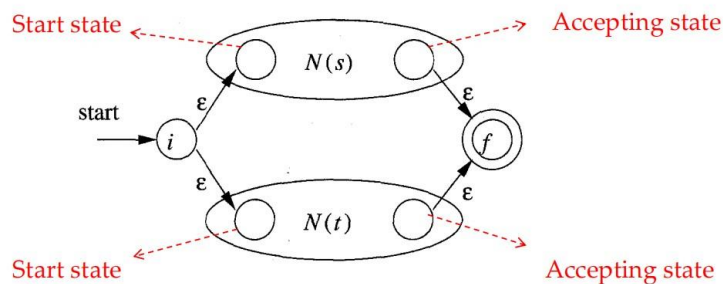
同样 i 和 f 都是新状态, 分别是这个 NFA 的开始状态和接受状态。

请注意, 在这两种基础构造规则中, 对于空或某个 a 的作为 r 的子表达式的每次出现, 我们都会使用新状态分别构造出一个独立的 NFA。

Thompson's Construction Algorithm

The inductive rules: the union case

- $s \mid t$: $N(s)$ and $N(t)$ are NFA's for subexpressions s and t



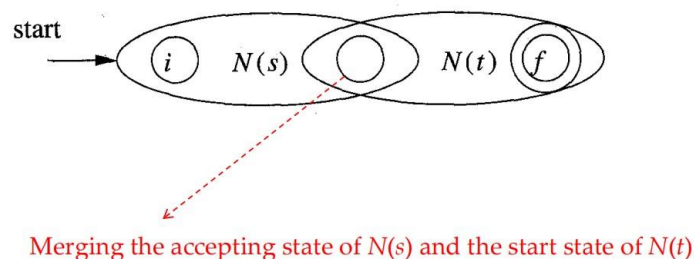
归纳：假设正则表达式 s 和 t 的 NFA 分别为 $N(s)$ 和 $N(t)$ 。

a) 假设 $r = s \mid t$, r 的 NFA, 即 $N(r)$, 如图 3.40 所示构建。这里, i 和 f 是新状态, 分别是 $N(r)$ 的开始状态和接受状态。从 i 到 $N(s)$ 和 $N(t)$ 的起始状态各有一个空转换, 从 $N(s)$ 和 $N(t)$ 到接受状态也各有一个空转换。请注意, $N(s)$ 和 $N(t)$ 的接受状态在 $N(r)$ 中不是接受状态。由于从 i 到 f 的任何路径要么只通过 $N(s)$, 要么只通过 $N(t)$, 并且离开 i 或进入 f 的空转换都不会改变路径上的标号, 因此我们可判定 $N(r)$ 识别 $L(s) \mid L(t)$, 也就是 $L(r)$ 相同。也就是说, 图 3.40 中的 NFA 是一个正确的处理并运算符的构造。

Thompson's Construction Algorithm

The inductive rules: the concatenation case

- **st**: $N(s)$ and $N(t)$ are NFA's for subexpressions s and t

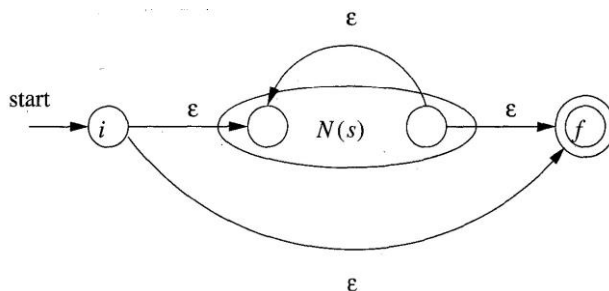


b) 假设 $r = st$ 。然后如图 3.41 所示构造 $N(r)$ 。 $N(s)$ 的起始状态成为 $N(r)$ 的起始状态， $N(t)$ 的接受状态是 $N(r)$ 的唯一接受状态。 $N(s)$ 的接受状态和 $N(t)$ 的开始状态合并为一个状态，合并后的状态拥有原来进入或离开合并前的两个状态的全部转换。图 3.41 中一条从 i 到 f 的路径必须首先经过 $N(s)$ ，因此这条路径的标号以 $L(s)$ 中的某个字符串开始。然后该路径继续通过 $N(t)$ ，因此该路径的标号以 $L(t)$ 中的某个字符串结束。正如我们要论证的，没有转换离开构造得到的接受状态，也没有转换进入开始状态，因此一个路径不可能在离开 $N(s)$ 后重新进入 $N(s)$ 。因此， $N(r)$ 恰好接受 $L(s)L(t)$ ，并且它是 $r = st$ 的一个正确的 NFA。

Thompson's Construction Algorithm

The inductive rules: the Kleene star case

- s^* : $N(s)$ is the NFA for subexpression s

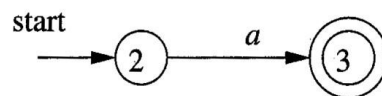


假设 $r = s^*$ 。然后对于 r ，我们构造如图 3.42 所示的 NFA $N(r)$ 。这里， i 和 f 是两个新状态，分别是 $N(r)$ 的起始状态和唯一的接受状态。要从 i 到 f ，我们可以沿着新引入的保号为空的路径前进，这个路径对应于 $L(s)^0$ 中的一个字符串。我们也可以到达 $N(s)$ 的开始状态，然后经过该 NFA，再零次或多次从它的接受状态回到它的开始状态。这些选项允许 $N(r)$ 接受 $L(s)^1$ 、 $L(s)^2$ 等集合中的所有字符串，因此 $N(r)$ 识别的所有串的集合就是 $L(s)^*$ 。

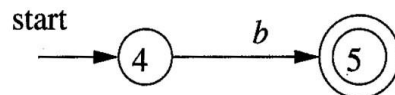
Example

- Use Thompson's algorithm to construct an NFA for the regexp
 $r = (a|b)^*abb$

NFA for the first a (apply basis rule #1)



NFA for the first b (apply basis rule #1)

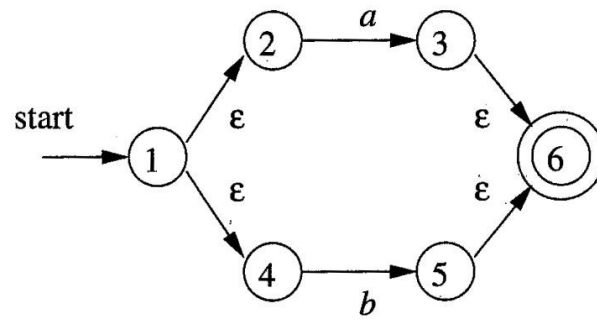


例 3.24: 让我们使用算法 3.23 为 $r = (a|b)^*abb$ 构造一个 NFA。图 3.43 显示了 r 的一颗语法分析树，它类似于第 2.2.3 节中为算术表达式构建的语法分析树。对于子表达式 r_1 ，即第一个 a ，我们构造 NFA:

我们在选择这个 NFA 中的状态编号时考虑了和接下来生成的 NFA 的状态编号之间的一致性。对于 r_2 ，我们构造:

Example Cont.

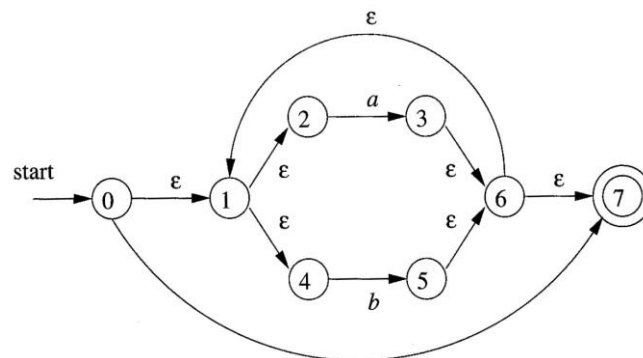
- NFA for $(a|b)$ (apply inductive rule #1)



现在，我们可以使用图 3.40 的构造方法，将 $N(r_1)$ 和 $N(r_2)$ 合并，获得 $r_3 = r_1 | r_2$ 的 NFA；该 NFA 如图 3.44 所示。

Example Cont.

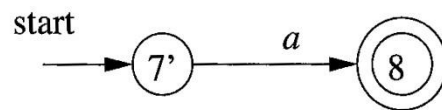
- NFA for $(a|b)^*$ (apply inductive rule #3)



$r4 = (r3)$ 的 NFA 与 $r3$ 的 NFA 相同。子表达式 $r5 = (r3)^*$ 的 NFA 构造如图 3.45 所示。我们使用图 3.42 所示的方法根据图 3.44 中的 NFA 构造出这个 NFA。

Example Cont.

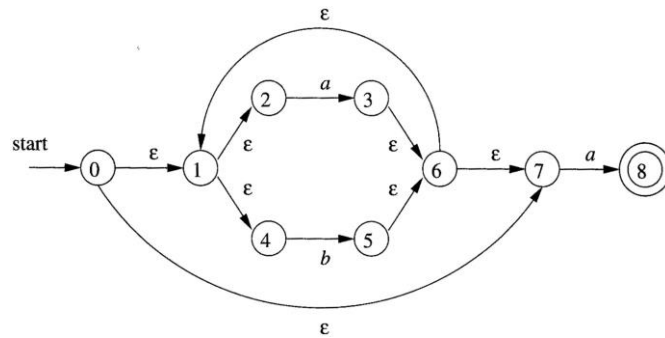
- NFA for the second **a** (apply basic rule #1)



现在，考虑子表达式 $r6$ ，它是表达式中的另一个 a 。我们再次对 a 使用基本构造法，但是我们必须使用新的状态。虽然 $r1$ 和 $r6$ 是相同的表达式，但这个构造方法不允许重复使用我们为 $r1$ 构建的 NFA。 $r6$ 的 NFA 为：

Example Cont.

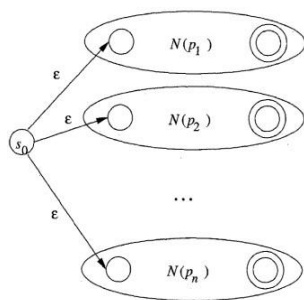
- NFA for $(a|b)^*a$ (apply inductive rule #2) ...



为了获得 $r_7 = r_5 r_6$ 的 NFA，我们应用图 3.41 的构造方法。将状态 7 和 7' 合并，产生图 3.46 的 NFA。继续以这种方式构造出两个分别名为 r_8 和 r_{10} 、对应于子表达式 b 的新 NFA，最终构造出如图 3.34 所示的 $(a|b)^*abb$ 的 NFA。

Combining NFA's

- **Why?** In the lexical analyzer, we need a single automaton to recognize lexemes matching any pattern (in the lex program)
- **How?** Introduce a new start state with ϵ -transitions to each of the start states of the NFA's N_i for pattern p_i



- The language that can be accepted by the big NFA is the union of the languages that can be accepted by the small NFA's

- Different accepting states correspond to different patterns

- 为什么？在词法分析器中，我们需要一个自动机来识别与任何模式（在 lex 程序中）匹配的词目
- 怎么做？为 NFA 的 N 个 i 的每个起始状态引入一个新的起始状态，每个起始状态都有空转换
- 大 NFA 所能接受的语言是小 NFA 所能接受的语言的总和。
- 不同的接受状态对应不同的模式

DFA's for Lexical Analyzers

- Convert the NFA for all the patterns into an equivalent DFA, using the subset construction algorithm
- An accepting state of the DFA corresponds to a subset of the NFA states, in which at least one is an accepting NFA state
 - If there are more than one accepting NFA state, this means that conflicts arise (the prefix of the input string matches multiple patterns)
 - Upon conflicts, find the first pattern whose accepting state is in the set and make that pattern the output of the DFA state

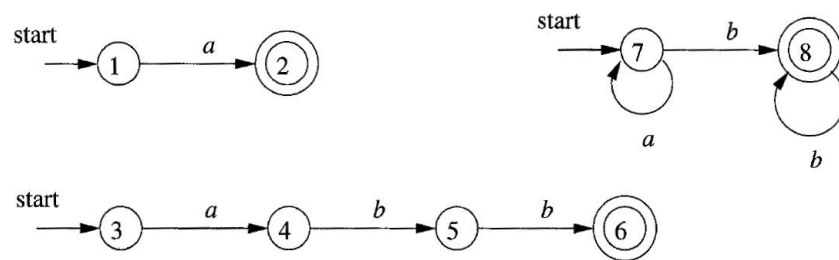
请注意，这三种模式呈现了第 3.5.3 节中讨论过的冲突。更明确地说，字符串 `abb` 同时满足第二个和第三个模式，但是我们将把它看作模式 `p2` 的词素，因为该模式在上面的 `Lex` 程序中首先列出。然后，输入字符串（例如 `aabbbb`）有许多前缀都满足第三个模式，`Lex` 规则是取最长的，所以我们继续读取 `b`，直到遇到另一个 `a`，然后我们报告识别的词素就是从第一个 `a` 开始的、包含了其后所有 `b` 的符号串。

词法分析器的 DFA

- 使用子集构建算法，将所有模式的 NFA 转换为等价的 DFA
- DFA 的接受状态对应于 NFA 状态的子集，该子集中至少有一个是 NFA 的接受状态。但是，如果该子集中包含一个以上的 NFA 接受状态，就意味着会出现冲突（输入字符串的前缀匹配多个模式）。此时，解决冲突的方式：按照该接受状态所对应的模式在所有模式中的排序，选择排序最靠前的模式的接受状态作为 DFA 状态的输出

Example

- Suppose we have three patterns: p_1 , p_2 , and p_3
 - a {action A_1 for pattern p_1 }
 - abb {action A_2 for pattern p_2 }
 - a^*b^+ {action A_3 for pattern p_3 }
- We first construct an NFA for each pattern



- 假设我们有三种模式: p , p_2 , 和 p_3
 - a { 行动 A_1 for pattern p }1
 - abb {action A_2 for pattern p }2
 - $A B^+ \{$ 行动 A_3 for pattern p }3
- 我们首先为每个模式构建一个 NFA

Example Cont.

- Combining the three NFA's

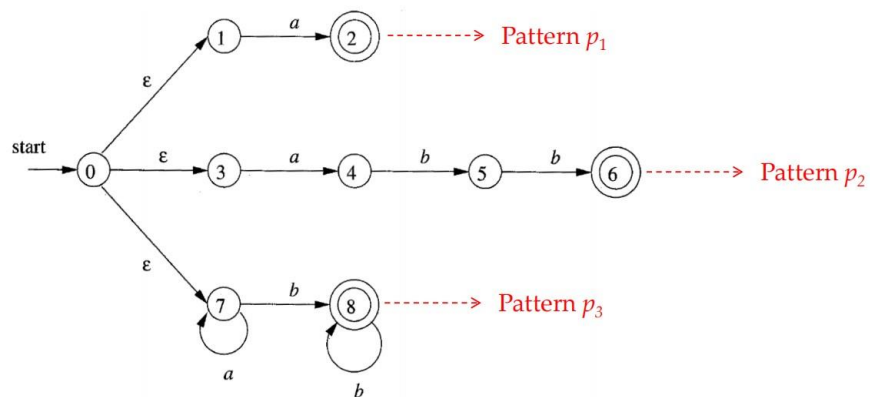


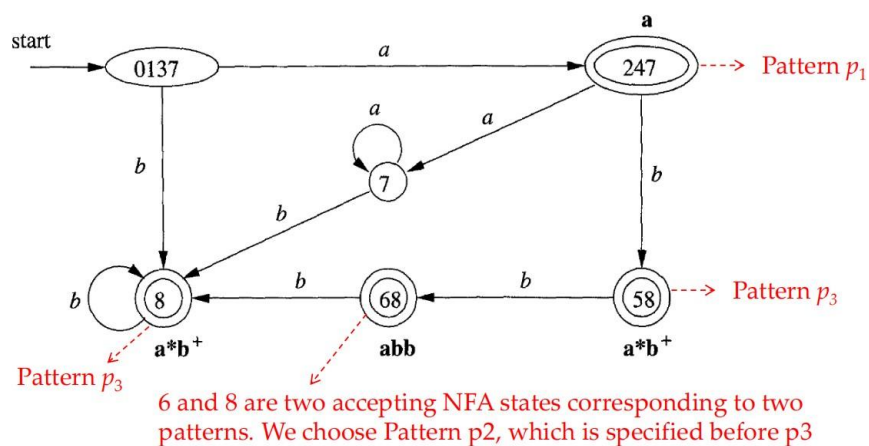
图 3.51 列出了识别这三种模式的三个 NFA。其中第三个是算法 3.23 的转换结果经简化得到的。然后，图 3.52 显示了通过加入一个新开始状态 0 和 3 个空转换将这三个 NFA 合并得到的单个 NFA。

Example Cont.

T	T_a	T_b
$\{0,1,3,7\}$	$\{2,4,7\}$	$\{8\}$
$\{2,4,7\}$	$\{7\}$	$\{5,8\}$
$\{7\}$	$\{7\}$	$\{8\}$
$\{5,8\}$	$\emptyset$	$\{6,8\}$
$\{8\}$	$\emptyset$	$\{8\}$
$\{6,8\}$	$\{1,2,3,6,f\}$	$\{8\}$

Example Cont.

- Converting the big NFA to a DFA



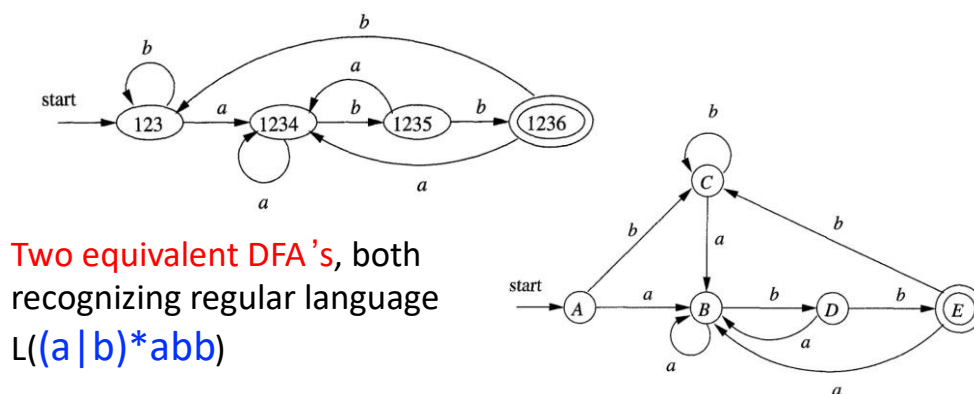
例 3.28: 图 3.54 显示了基于 DFA 的转换图, 该 DFA 是通过图 3.52 中 NFA 的子集构造而构造的。图中的接受状态由该状态所识别的模式作为标号。例如状态 {6; 8} 有两个接受状态, 分别对应于模式 abb 和 a^*b^+ 。由于前者先被列出, 因此该模式就是与状态 {6, 8} 所关联的模式;。

我们在词法分析器中使用 DFA, 就像我们使用 NFA 一样。我们模拟这个 DFA 的运行, 直到某一点上没有后继状态为止 (或者严格来说, 下一个状态是空, 即对应于空的 NFA 状态集合的死状态)。此时, 我们回头查找我们进入过的状态序列, 一旦找到接受状态就执行与该状态相关联的动作。

例 3.29: 假设图 3.54 的 DFA 的输入为 $abba$ 。处理输入时的状态序列为 0137; 247; 58; 68, 并且在读入最后一个 a 时, 没有离开状态 68 的相应转换。因此, 我们从后向前考察这个状态序列, 在这种情况下, 68 本身是一个接受状态, 对应于模式 $p2 = abb$ 。

Minimizing # States of a DFA

- There can be many DFA's recognizing the same language



可以有多个识别相同语言的 DFA。例如, 请注意图 1 和 2 的 DFA。3.36 和 3.63 都识别语言 $L((a|b)^*abb)$ 。这两个自动机不但各个状态的名字不同, 就连它们的的状态

个数也不一样。如果我们使用 DFA 来实现词法分析器，我们总是希望使用的 DFA 状态数量尽可能少，因为描述词法分析器的转换表需要为每个状态分配条目。

状态名字的问题是次要的。我们可以说，如果只需改变状态名字就可以将一个自动机转换成为另一个自动机，我们就说这两个自动机是同构的。图 3.36 和 3.63 中的两个自动机不是同构的。然而，这两个自动机之间却有着密切的关系。

Minimizing # States of a DFA

- There can be many DFA's recognizing the same language

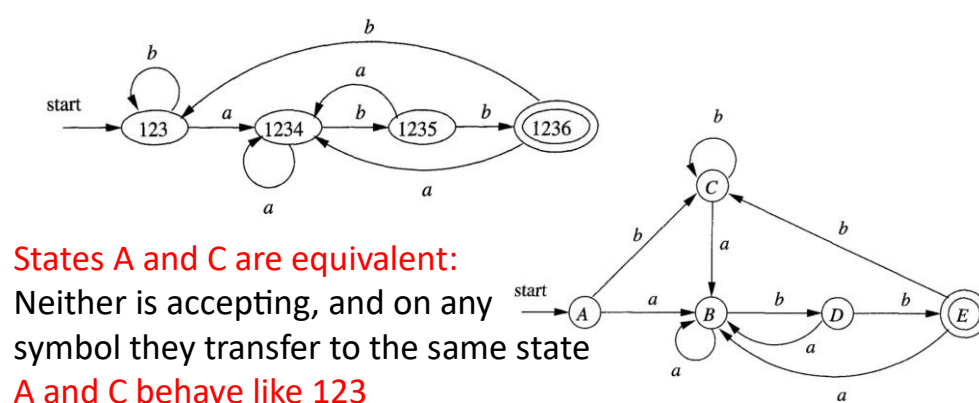


图 3.36 的状态 A 和 C 实际上是等价的，因为它们都不是接受状态，并且对任意输入，它们都会转移到相同的状态——在输入 a 上转到 B，在输入 b 上转到 C。此外，状态 A 和 C 的行为都会图 3.63 中的状态 123 相似。同样，图 3.36 中的状态 B 的行为和图 3.63 中的状态 1234 的行为类似，状态 D 的行为和状态 1235 的行为相似，状态 E 的行为和状态 1236 的行为相似。

Minimizing # States of a DFA Cont.

- There is always a unique minimum-state DFA for any regular language (state name does not matter)
- The minimum-state DFA can be constructed from any DFA for the same language by grouping sets of equivalent states

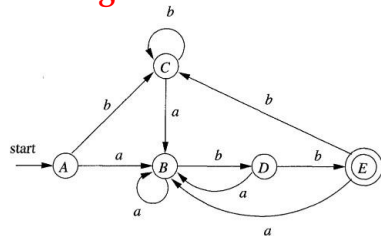
可以得出一个重要的结论，任何正则语言都有一个唯一的（不计同构）状态数目最少的 DFA。此外，从任意一个接受相同语言的 DFA 出发，通过分组合并等价状态，我们总是可以构建得到这个状态数最少的 DFA。

Distinguishing States

- **Distinguishable states**

- We say that string x distinguishes state s from state t if **exactly one** of the states reached from s and t by following the path with label x is an accepting state
- States s and t are **distinguishable** if there exists some string that distinguishes them

- **Indistinguishable states** are equivalent and can be merged



- The empty string ϵ distinguishes any accepting state from any nonaccepting state
- The string bb distinguishes state A from B , since bb takes A to a nonaccepting state C , but takes B to accepting state E

我们将给出一个将任意 DFA 转换为其最小状态等效 DFA 的算法。该算法首先创建输入 DFA 的状态集合的划分。为理解这个算法，我们要了解输入串是如何区分各个状态的。如果分别从状态 s 和 t 出发，沿着标号为 x 的路径到达的两个状态中只有一个是接受状态，则字符串 x 区分状态 s 和状态 t 。如果存在某个能够区分状态 s 和状态 t 的串，那么它们就是可区分的。

- 状态 s 和 t 可区分：

- 存在一个串 x ，使得从 s 和 t 出发的所经历的状态路径中，恰好有一个状态是接受状态，我们就说字符串 x 将状态 s 与状态 t 区分开来。

- 如果存在某种字符串可将状态 s 和 t 区分开来，则这两个状态是可区分的

- 不可区分的状态是等价的，可以合并

空字符串去跟任何接受状态和非接收状态

字符串 bb 将状态 A 与状态 B 区分开来，因为 bb 将 A 带入非接受状态 C ，而将 B 带入接受状态 E 。

DFA State-Minimization Algorithm

Works by partitioning the states of a DFA into groups of states that cannot be distinguished (an iterative process)

- The algorithm maintains a partition (划分), whose groups are sets of states that have not yet been distinguished
- Any two states from different groups are known to be distinguishable
- When the partition cannot be refined further by breaking any group into smaller groups, we have the minimum n -state DFA

状态最小化算法的工作原理是将一个 DFA 的状态集合划分为多个组，每个组中的各个状态之间相互不可区分。然后，将每个组中的状态合并为状态最少 DFA 的一个状态。算法在执行过程中了状态集合的一个划分，分划中的每个组内的各个状态尚不能区分，但是来自不同组的任意两个状态是可区分的。当任意一个组都不能再被分解为更小的组时，这个分划就不能进一步精化，此时我们就有了最小状态 DFA。

最初，该分区由两个组组成：接受状态和不接受状态。算法的基本步骤是从当前分区中取一个状态组，例如 $A = \{s_1; s_2; \dots; s_k\}$ ，并选定某个输入符号 a ，查看 a 是否可以用于区分 A 中的某些状态。我们检查输入 a 上每个 $s_1; s_2; \dots; s_k$ 的转换，如果这些转换达到的状态落入当前分区的两个或多个组，我们就将 A 分成多个组，使得 s_i 和 s_j 在同一组中当且仅当它们在输入 a 上的转换都到达同一个组的状态。我们重复这个分割过程，直到无法根据某个输入符号对任意个组进行分割为止。该思想体现在下面的算法中。

The Partitioning Part

1. Start with an initial partition Π with two groups, F and $S-F$, the accepting and nonaccepting states of D

2. Apply the procedure below to construct a new partition Π_{new}

```
initially, let  $\Pi_{new} = \Pi$ ;  
for ( each group  $G$  of  $\Pi$  ) {  
    partition  $G$  into subgroups such that two states  $s$  and  $t$   
        are in the same subgroup if and only if for all  
        input symbols  $a$ , states  $s$  and  $t$  have transitions on  $a$   
        to states in the same group of  $\Pi$ ;  
    /* at worst, a state will be in a subgroup by itself */  
    replace  $G$  in  $\Pi_{new}$  by the set of all subgroups formed;  
}
```

3. If $\Pi_{new} == \Pi$, let $\Pi_{final} = \Pi$ and the partitioning finishes; Otherwise, $\Pi = \Pi_{new}$ and repeat step 2

1. 首先构造包含两个组 F 和 $S-F$ 的初始划分，这两个组分别是 D 的接受状态组和不接受状态组。

2. 应用图 3.64 的过程构建一个新分区 Π_{new} 。

最初，让 $\Pi_{new} = \Pi$ ； for (的每个组 G) 将 G 划分为子组，使得两个状态 s 和 t 位于同一子组中当且仅当对于所有输入符号 a ，状态 s 和 t 在 a 上有到同一组中状态的转换；/* 在最坏的情况下，状态本身将成为一个子群 */ 将 Π_{new} 中的 G 替换为形成的所有子群的集合； g 图 3.64：构造 Π_{new}

3. 如果 $\Pi_{new} = \Pi$ ，令 $\Pi_{final} = \Pi$ 并继续步骤 (4)。否则，用 Π_{new} 替换 Π 并重复步骤 (2)。

The Construction Part

Choose one state in each group of Π_{final} as the **representative** for that group. The representatives will be the states of the minimum state DFA D'

- The start state of D' is the representative of the group containing the start state of D
- The accepting states of D' are the representatives of those groups that contain an accepting state of D
- Establish transitions:
 - Let s be the representative of group G in Π_{final} ; Let the transition of D from s on input a be to state t ; Let r be the representative of t 's group H
 - Then in D' , there is a transition from s to r on input a

4. 在 Π_{final} 的每组中选择一个状态做为该组的代表。这些代表构成了状态最少 DFA D' 的状态。 D' 的其他部分按照如下步骤构建：

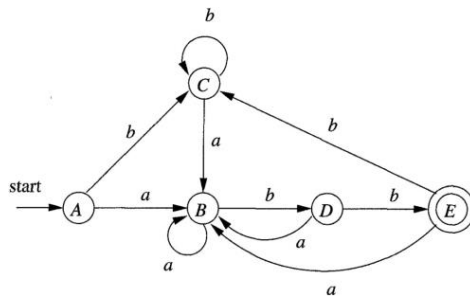
D' 的起始状态是包含 D 的起始状态的组的代表。

D' 的接受状态是那些包含 D 的接受状态的组的代表。请注意，每个组要么只包含接受状态，要么只包含不接受状态，因为我们一开始就将这两类状态分开了，而图 3.64 中的过程总是通过分解已经构造得到的组来得到新的组。

设 s 为 Π_{final} 的某个组 G 的代表，并设 DFA D 从输入 a 上离开状态 s 的转换到达状态 t 。令 r 为 t 所在组 H 的代表。然后在 D' 中存在一个从 s 到 r 在输入 a 上的转换。请注意，在 D 中，组 G 中的每个状态都必须在输入 a 上进入组 H 的某个状态，否则，组 G 将根据图 3.64 的过程分割成更小的组。

Example

- Initial partition: {A, B, C, D}, {E}
- Handling group {A, B, C, D}: b splits it to two subgroups {A, B, C} and {D}
- Handling group {A, B, C}: b splits it to two subgroups {A, C} and {B}
- Picking A, B, D, E as representatives to construct the minimum -state DFA



STATE	a	b
A	B	A
B	B	D
D	B	E
E	B	A

- 初始分区: {a、b、c、d}, {e}
- 处理组 {A、B、C、D}: b 将其分成两个子组 {A、B、C} 和 {D} ——A,B,C 遇到 b 分别转换到 C,D,C,还在{A, B, C, D}这个集合中, 但 D 遇到转换到 E, E 不在{A, B, C, D}中。
- 处理组 {A、B、C}: b 将其分成两个子组 {A、C} 和 {B}—— A,C 遇到 b 分别转换到 C,C,还在{A, B, C}这个集合中, 但 B 遇到 b 转换到 D, D 不在{A, B, C}中。
- 选取 A、B、D、E 作为代表, 构建最小状态 DFA

State Minimization in Lexical Analyzers

The basic idea is the same as the state-minimization algorithm for DFA.

- Differences are:
 - Each accepting state in the lexical analyzer's DFA corresponds to a different pattern. These states are not equivalent.
 - So the initial partition should be: one group of non-accepting states + groups of accepting states for different patterns

词法分析器中的状态最小化

其基本思想与 DFA 的状态最小化算法相同。

- 不同之处在于

词法分析器 DFA 中的每个接受状态都对应一个

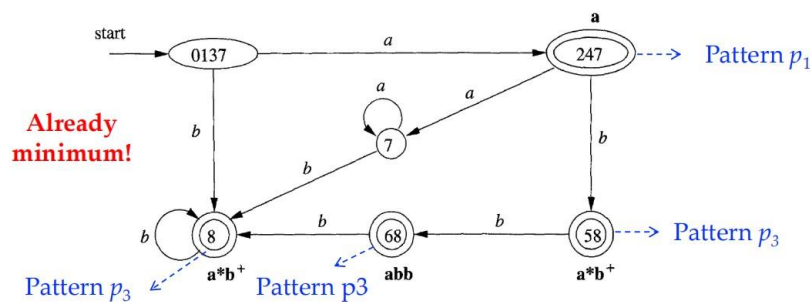
不同的模式。这些状态并不等同。

因此，初始分区应为：一组不接受状态 + 不同模式的接受状态组

Example

Initial partition: $\{0137, 7\}, \{247\}, \{68\}, \{8, 58\}, \{\emptyset\}$

- We add a dead state $\emptyset$: we suppose has transitions to itself on inputs a and b. It is also the target of missing transitions on a from states 8, 58, and 68.



初始分区: $\{0137, 7\}, \{247\}, \{68\}, \{8, 58\}, \{\emptyset\}$

我们添加一个死态 $\emptyset$: 我们假设它在输入 a 和 b 上有向自身的转换。所有输入符号上都转向自己的非接受状态。在一个 DFA 中, 每个状态出发每个输入符号上都会有一个转换, 发现正确词素的时候 DFA 结束。

Describing Syntax

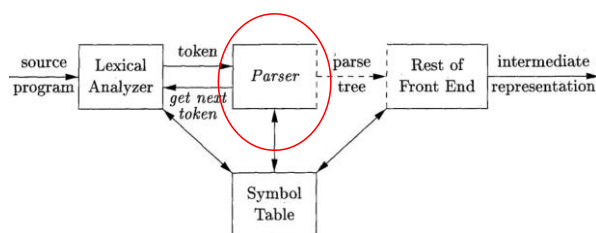
- The syntax of programming language constructs can be specified by **context-free grammars** or BNF (Backus-Naur Form) notation
 - A grammar gives a precise, yet easy-to-understand, **syntactic specification** of a programming language
 - For certain grammars, we can **automatically construct an efficient parser**
 - A properly designed grammar **defines the structure of a language** and helps translate source programs into correct object code and detect errors
 - A grammar allows a language to be **evolved or developed iteratively**, by adding new constructs to perform new tasks

编程语言结构的语法可以通过上下文无关文法或 BNF（巴科斯范式）表示法来指定，在 2.2 节中介绍。文法为语言设计者和编译器编写者提供了巨大的好处。

- 文法给出了一个程序设计语言的精确且易于理解的语法规范。
- 对于某些类型的文法，我们可以自动构造一个有效的语法分析器，它能够确定一个源程序的语法结构。语法分析器的构造过程可以揭示语法的二义性，同时还可能发现一些容易在语言的初始设计阶段被忽略的问题。
- 一个正确设计的文法给出了一个语言的结构。该结构有助于把源程序翻译成正确的目标代码，也有助于检测错误。
- 一个文法支持逐步加入可以完成新任务的新语言构造从而迭代地演化和开发语言。如果对语言的实现遵循语言的文法结构，那么在实现中加入这些新构造的工作就变得更加容易。

The Role of the Parser

- The parser obtains a string of tokens from the lexical analyzer and verifies that the string of token names can be generated by the grammar for the source language
- Report syntax errors in an intelligent fashion
- For well-formed programs, the parser constructs a parse tree
 - The parse tree need not be constructed explicitly



语法分析器的角色

语法分析器从词法分析器获得 token 串，并验证 token 名称是否可以由源语言的语法生成

以智能的方式报告语法错误

对于格式良好的程序，语法分析器构建一个语法分析树

语法分析树无需显式构建

Classification of Parsers

- **Universal parsers (通用语法分析器)**
 - Some methods (e.g., Earley's algorithm¹) can parse any grammar
 - However, they are too inefficient to be used in practice
- **Top-down parsers (自顶向下语法分析器)**
 - Construct parse trees from the top (root) to the bottom (leaves)
- **Bottom-up parsers (自底向上语法分析器)**
 - Construct parse trees from the bottom (leaves) to the top (root)

Note: Top-down and bottom-up parsing both scan the input from left to right, one symbol at a time. They work only for certain grammars, which are expressive enough.

¹ <http://loup-vaillant.fr/tutorials/earley-parsing/>

语法分析器的分类

- 通用语法分析器
 - 一些方法（例如，Earley 算法 1）可以解析任何语法
 - 然而，它们效率太低，无法在实践中使用
- 自顶向下语法分析器
 - 构建从顶部（根）到底部（叶）的语法分析树
- 自底向上语法分析器
 - 从底部（叶子）到顶部（根）构建语法分析树

注意：自上而下和自下而上的解析都从左到右扫描输入，一次扫描一个符号。它们只适用于某些语法，这些语法具有足够的表达能力。

Context-Free Grammar (上下文无关文法)

- A context-free grammar (CFG) consists of four parts:
 - **Terminals (终结符号):** Basic symbols from which strings are formed (token names)
 - **Nonterminals (非终结符号):** Syntactic variables that denote sets of strings
 - Usually correspond to a language construct, such as *stmt* (statements)
 - One nonterminal is distinguished as the **start symbol (开始符号)**
 - The set of strings denoted by the start symbol is the language generated by the CFG
 - **Productions (产生式):** Specify the manner in which the terminals and nonterminals can be combined to form strings
 - **Format:** head (left side) $\rightarrow$ body (right side)
 - The **head** is a nonterminal; the **body** consists of zero or more terminals/nonterminals
 - **Example:** *expression* $\rightarrow$ *expression* + *term*

上下文无关语法（CFG）由四个部分组成：

- 终结符号：形成字符串的基本符号（token 名称）
- 非终结符号：表示字符串集的语法变量 通常对应于一个语言结构，如 *stmt* (statements)
- 我们指定一个非终结符作为开始符号 由开始符号表示的字符串集是 CFG 生成的语言
- 产生式：指定终结符号和非终结符号组合形成字符串的方式
- 格式：head（左侧） $\rightarrow$ body（右侧）
- head 是非终结符号；body 由零个或多个终结符号或非终结符号组成

例如： *expression* $\rightarrow$ *expression* + *term*

CFG Example

- The grammar below defines simple arithmetic expressions
 - **Terminal symbols:** `id`, `+`, `-`, `*`, `/`, `(`, `)`
 - **Nonterminals:** `expression`, `term`, `factor`
 - **Start symbol:** `expression`
 - **Productions:**
 - `expression` $\rightarrow$ `expression` `+` `term`
 - `expression` $\rightarrow$ `expression` `-` `term`
 - `expression` $\rightarrow$ `term`
 - `term` $\rightarrow$ `term` `*` `factor`
 - `term` $\rightarrow$ `term` `/` `factor`
 - `term` $\rightarrow$ `factor`
 - `factor` $\rightarrow$ `(` `expression` `)`
 - `factor` $\rightarrow$ `id`

CFG 示例

下面的语法定义了简单的算术表达式

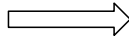
终结符: `id`, `+`, `-`, `*`, `/`, `(`, `)`

非终结符: `expression`, `term`, `factor`

开始符: `expression`

Notational Simplification

```
expression → expression + term
expression → expression - term
expression → term
term → term * factor
term → term / factor
term → factor
factor → ( expression )
factor → id
```



```
E → E + T | E - T | T
T → T * F | T / F | F
F → ( E ) | id
```

- **|** is a **meta symbol** to specify alternatives
- **(** and **)** are not meta symbols, they are terminal symbols

符号简化

“|”是指定备选方案的元符号

“(”和“)”不是元符号，而是终结符号

Derivations

- **Derivation (推导):** Starting with the start symbol, nonterminals are rewritten using productions until only terminals remain
- Example:
 - CFG: $E \rightarrow - E \mid E + E \mid E * E \mid (E) \mid \text{id}$
 - A derivation (a sequence of rewrites) of $-(\text{id})$ from E
 - $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(\text{id})$

推导: 从开始符号开始, 使用产生式重写非终结符, 直到只剩下终结符为止

示例:

从 E 导出 $-(\text{id})$ (一系列重写)

Notations

- $\Rightarrow$ means “derives in one step”
- $\overset{*}{\Rightarrow}$ means “derives in zero or more steps”
 - $\alpha \overset{*}{\Rightarrow} \alpha$ holds for any string α
 - If $\alpha \overset{*}{\Rightarrow} \beta$ and $\beta \Rightarrow \gamma$, then $\alpha \overset{*}{\Rightarrow} \gamma$
 - Example: $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(\mathbf{id})$ can be written as $E \overset{*}{\Rightarrow} -(\mathbf{id})$
- $\overset{+}{\Rightarrow}$ means “derives in one or more steps”

符号

$\Rightarrow$ 意思是单步推导

$\Rightarrow^*$ 意思是“以零个或多个步骤推导” $\alpha \Rightarrow^* \alpha$ 适用于任何字符串 α

$\Rightarrow^+$ 意思是“以一个或多个步骤推导”

Terminologies

- If $S \Rightarrow^* \alpha$, where S is the start symbol of a grammar G , we say that α is *sentential form* of G (文法的句型)
 - May contain both terminals and nonterminals, and may be empty
 - **Example:** $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\mathbf{id} + E) \Rightarrow -(\mathbf{id} + \mathbf{id})$, here all strings of grammar symbols are sentential forms
- A *sentence* (句子) of G is a sentential form with no nonterminals
 - In the above example, only the last string $-(\mathbf{id} + \mathbf{id})$ is a sentence
- The *language generated* by a grammar is its set of sentences

术语

- 如果 $S \Rightarrow^* \alpha$, 其中 S 是语法 G 的开始符号, 我们说 α 是 G 文法的句型
 - 文法的句型可以同时包含终结符号和非终结符号, 并且可以为空
 - 示例 $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\mathbf{id} + E) \Rightarrow -(\mathbf{id} + \mathbf{id})$ 这里所有的语法符号串都是文法的句型
- 一个 G 的句子是一个没有非终结符的文法句型
 - 在上面的例子中, 只有最后一个字符串 $-(\mathbf{id} + \mathbf{id})$ 是一个句子
- 语法产生的语言是它产生句子的集合

Leftmost/Rightmost Derivations

- At each step of a derivation, we need to choose which nonterminal to replace
- In **leftmost derivations (最左推导)**, the leftmost nonterminal in each sentential form is always chosen to be replaced

$$\blacksquare E \xRightarrow{lm} - E \xRightarrow{lm} - (E) \xRightarrow{lm} - (E + E) \xRightarrow{lm} - (\mathbf{id} + E) \xRightarrow{lm} - (\mathbf{id} + \mathbf{id})$$

- In **rightmost derivations (最右推导)**, the rightmost nonterminal is always chosen to be replaced

$$\blacksquare E \xRightarrow{rm} - E \xRightarrow{rm} - (E) \xRightarrow{rm} - (E + E) \xRightarrow{rm} - (E + \mathbf{id}) \xRightarrow{rm} - (\mathbf{id} + \mathbf{id})$$

最左/最右推导

在推导的每一步，我们都需要选择要替换的非终结符

在最左推导中,总是选择每个句型中最左边的非终结符来替换

在最右推导中,总是选择最右边的非终结符来替换

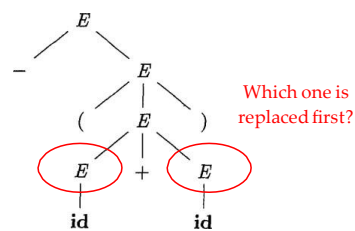
Parse Trees (语法分析树)

- A **parse tree** is a graphical representation of a derivation
 - The **root node** (根结点) is the start symbol of the grammar
 - Each **leaf node** (叶子结点) is labeled by a terminal symbol or ϵ
 - Each **interior node** (内部结点) is labeled by a nonterminal symbol
 - Each interior node represents the application of a production
 - The interior node is labeled with the nonterminal in the head of the production; the children nodes are labeled, from left to right, by the symbols in the body of the production

CFG: $E \rightarrow - E \mid E + E \mid E * E \mid (E) \mid \text{id}$

$E \xRightarrow{lm} - E \xRightarrow{lm} - (E) \xRightarrow{lm} - (E + E) \xRightarrow{lm} - (\text{id} + E) \xRightarrow{lm} - (\text{id} + \text{id})$

$E \xRightarrow{rm} - E \xRightarrow{rm} - (E) \xRightarrow{rm} - (E + E) \xRightarrow{rm} - (E + \text{id}) \xRightarrow{rm} - (\text{id} + \text{id})$



语法分析树是一种推导的图形表示

- 根结点是文法的开始符号
- 每个叶子结点由终结符或 ϵ 标记
- 每个内部结点由非终结符标记
- 每个内部结点代表产生式的应用
 - 内部节点用产生式的头部中的非终结符进行标记;
 - 子节点由产生式主体中的符号从左到右进行标记

问题：语法分析树中红色虚线圈起来的 E 哪一个会先被替换？

答案：在最左推导中，左孩子先被替换；在最右推导中，右孩子先被替换。

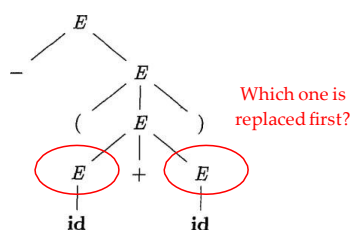
Parse Trees (语法分析树) Cont.

- The leaves, from left to right, constitute a **sentential form** of the grammar, which is called the *yield* or *frontier* of the tree
- There is a **many-to-one** relationship between **derivations** and **parse trees**
- There is a **one-to-one** relationship between **leftmost/rightmost derivations** and **parse trees**

CFG: $E \rightarrow - E \mid E + E \mid E * E \mid (E) \mid \text{id}$

$E \xRightarrow{lm} - E \xRightarrow{lm} - (E) \xRightarrow{lm} - (E + E) \xRightarrow{lm} - (\text{id} + E) \xRightarrow{lm} - (\text{id} + \text{id})$

$E \xRightarrow{rm} - E \xRightarrow{rm} - (E) \xRightarrow{rm} - (E + E) \xRightarrow{rm} - (E + \text{id}) \xRightarrow{rm} - (\text{id} + \text{id})$



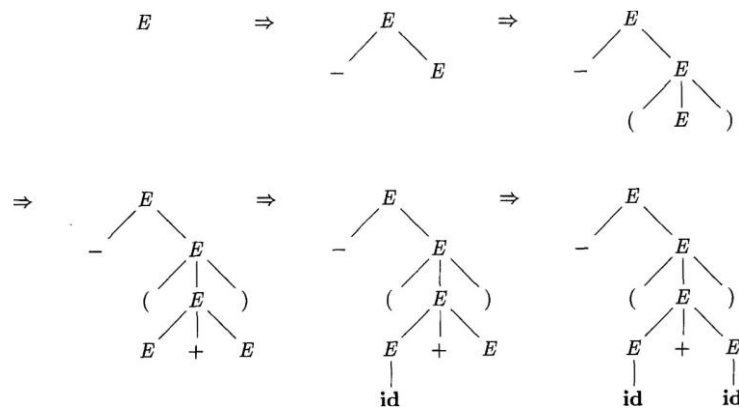
从左到右，叶子结点构成了语法的一种句型，称为树的产出或边缘

推导和语法分析树之间存在多对一关系

最左/最右推导和语法分析树之间存在一对一关系

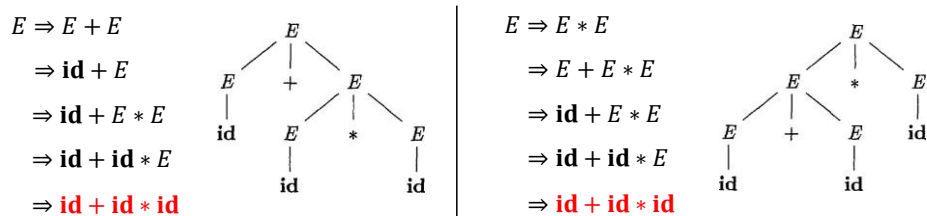
Constructing Parse Trees (Example)

Derivation: $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\text{id} + E) \Rightarrow -(\text{id} + \text{id})$



Ambiguity (二义性)

- If a grammar produces more than one parse tree for some sentence, it is ambiguous
- Example CFG: $E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$



Both are leftmost derivations

如果一个语法为某个句子生成了多个语法分析树，则它具有二义性

示例：该文法为 $\text{id}+\text{id}*\text{id}$ 生成了两种语法分析树。

Ambiguity (二义性) Cont.

- The grammar of a programming language typically needs to be unambiguous
 - Otherwise, there will be multiple ways to interpret a program
 - Given $E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$, how to interpret $a + b * c$?
- In some cases, it is convenient to use carefully chosen ambiguous grammars, together with disambiguating rules to discard undesirable parse trees
 - For example: multiplication before addition

程序设计语言的语法通常需要明确

- 否则，将有多种方法来解释程序
- 问题：给定 $E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$ ，如何解读 $a + b * c$?
- 答案：该文法是上一张 PPT 的示例，根据两种语法分析树，第一种解释：应先

处理 $b*c$ ，然后与 a 相加；第二种解释：应先处理 $a+b$ ，然后与 c 相乘

在某些情况下，使用精心选择的歧义语法以及消歧规则来丢弃不需要的语法分析树是很方便的

CFG vs. Regular Expressions

- **CFGs are more expressive than regular expressions**
 - Every construct that can be described by a regular expression can be described by a grammar, but not vice-versa
 - Every regular language is a context-free language, but not vice-versa
- Example: $L = \{a^n b^n \mid n > 0\}$
 - The language L can be described by CFG $S \rightarrow aSb \mid ab$
 - L cannot be described by regular expressions. In other words, we cannot construct a DFA to accept L

CFG 与正则表达式比较

CFG 比正则表达式更具表达能力

- 可以用正则表达式描述的每个构造都可以用语法描述，但反之不成立
- 每个正则语言都是上下文无关的语言，但反之不成立

示例: $L = \{a^n b^n \mid n > 0\}$

语言 L 可以用 CFG: $S \rightarrow aSb \mid ab$ 来描述

L 不能用正则表达式来描述。换句话说，我们不能构造一个 DFA 来接受 L

Proof by Contradiction

- Suppose there is a DFA D that accepts L and D has k states
- When processing a^{k+1} ..., D must enter a state s more than once (D enters one state after processing a symbol)¹
- Assume that D enters the state s after reading the i th and j th a ($i \neq j, i \leq k+1, j \leq k+1$)
- Since D accepts L , $a^i b^j$ must reach an accepting state. There must exist a path labeled b^+ from s to an accepting state
- Since a^i reaches the state s and there is a path labeled b^+ from s to an accepting state, D will accept $a^i b^j$. Contradiction!!!

¹ $a^{k+1}b^{k+1}$ is a string in L so D must accept it

反证法（了解即可）

假设存在 DFA D 接受 L , D 有 k 个状态

处理 $a^{k+1} \dots$ 时, D 必须进入状态 s , 且不止一次 (D 每处理一个符号都会进入一种状态)

假设 D 进入状态 s 在读入第 i 个 a 和第 j 个 a 后 ($i \neq j, i \leq k+1, j \leq k+1$)

按照上一张 PPT 的示例, 因为 D 接受文法 L , 所以 $a^i b^j$ 必须达到接受状态。必须存在一个标记为 b^j 从 s 到接受状态的路径

因为 a^i 到达了状态 s , 所以必有一条标记为 b^j 从 s 到接受状态的; 路径, D 将接受 $a^i b^j$ 。矛盾!!!

Any Regular Language Can Be Described by a CFG

- **(Proof by Construction)** Each regular language can be accepted by an NFA. We can construct a CFG to describe the language:
 - For each state i of the NFA, create a nonterminal symbol A_i
 - If state i has a transition to state j on input a , add the production $A_i \rightarrow aA_j$
 - If state i goes to state j on input ϵ , add the production $A_i \rightarrow A_j$
 - If i is an accepting state, add $A_i \rightarrow \epsilon$
 - If i is the start state, make A_i be the start symbol of the grammar

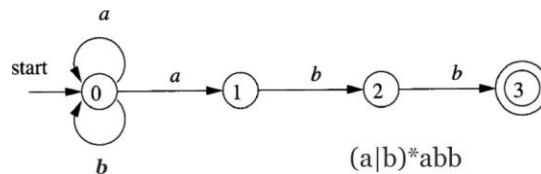
任何正则语言都可以用 CFG 描述

(结构证明) NFA 可以接受每种常规语言。我们可以构造一个 CFG 来描述语言:

- 对于每个在 NFA 中的状态 i , 创建一个非终结符 A_i
- 如果状态 i 在碰到输入 a 转为状态 j 时, 添加产生式 $A_i \rightarrow A_j$
- 如果状态 i 是一个接受状态, 添加产生式 $A_i \rightarrow \epsilon$
- 如果状态 i 是开始状态, 将 A_i 作为文法的开始符号

Example: NFA to CFG

- $A_0 \rightarrow aA_0 \mid bA_0 \mid aA_1$
- $A_1 \rightarrow bA_2$
- $A_2 \rightarrow bA_3$
- $A_3 \rightarrow \epsilon$



Consider the string **baabb**: The process of the NFA accepting the sentence corresponds exactly to the derivation of the sentence from the grammar

示例：NFA 到 CFG

考虑字符串 baabb: NFA 接受句子的过程与从语法中推导出句子的过程完全一致

Grammar Design

- CFGs are capable of describing most, but not all, of the syntax of programming languages
 - “Identifiers should be declared before use” cannot be described by a CFG
 - Subsequent phases must analyze the output of the parser to ensure compliance with such rules
- Before parsing, we typically apply several transformations to a grammar to make it more suitable for parsing
 - Eliminating ambiguity (消除二义性)
 - Eliminating left recursion (消除左递归)
 - Left factoring (提取左公因子)

CFG 能够描述编程语言的大部分（但不是全部）语法

- CFG 不能描述 “使用前应声明标识符”
- 后续阶段必须分析解析器的输出，以确保符合此类规则

在解析之前，我们通常对语法进行几种转换，使其更适合解析

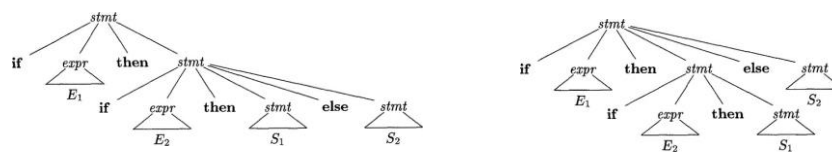
- 消除二义性
- 消除左递归
- 提取左公因子

Eliminating Ambiguity (1)

$stmt \rightarrow$

- `if expr then stmt`
- `if expr then stmt else stmt`
- `other`

Two parse trees for `if  $E_1$  then if  $E_2$  then  $S_1$  else  $S_2$`



Which parse tree is preferred in programming?
(i.e., `else` matches which `then`?)

消除二义性

问题：在编程中，哪个语法分析树是首选的？（即，`else` 匹配哪个 `then`？）

答案：第一种语法分析树

Eliminating Ambiguity (2)

- **Principle of proximity:** match each **else** with the closest unmatched **then**
 - **Idea of rewriting:** A statement appearing between a **then** and an **else** must be matched (must not end with an unmatched **then**)

```
stmt    →  matched_stmt  
         |  open_stmt  
matched_stmt →  if expr then matched_stmt else matched_stmt  
               |  other  
open_stmt  →  if expr then stmt  
               |  if expr then matched_stmt else open_stmt
```

Rewriting grammars to eliminate ambiguity is difficult.
There are no general rules to guide the process.



消除二义性

接近原则：用最接近的未匹配的 then 匹配 else

· 重写的想法：出现在 then 和 else 之间的语句必须匹配（不能以不匹配的 then 结束）

重写语法以消除歧义是困难的。没有指导这一过程的一般规则。

Eliminating Left Recursion

- A grammar is **left recursive** if it has a nonterminal A such that there is a derivation $A \Rightarrow^+ A\alpha$ for some string α
 - $S \rightarrow Aa \mid b$
 - $A \rightarrow Ac \mid Sd \mid \epsilon$
 - Because $S \Rightarrow Aa \Rightarrow Sda$
- **Immediate left recursion (立即左递归)**: the grammar has a production of the form $A \rightarrow A\alpha$
- Top-down parsing methods cannot handle left-recursive grammars (bottom-up parsing methods can handle...)

消除左递归

如果语法中存在一个非终结符 A ，使得某个字符 α 有相关的产生式 $A \Rightarrow^+ A\alpha$ ，则该语法为左递归语法

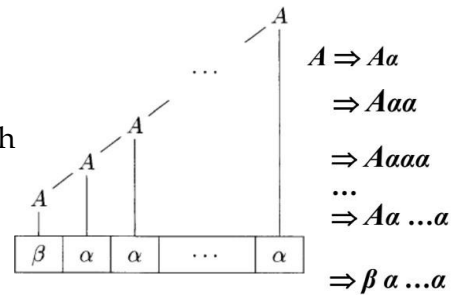
立即左递归: 语法有产生式 $A \rightarrow A\alpha$

自顶向下分析方法无法处理左递归语法（自底向上的分析方法可以处理...）

Eliminating Immediate Left Recursion

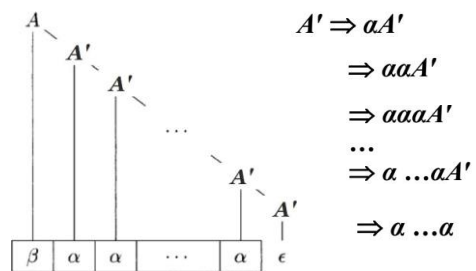
- Simple grammar: $A \rightarrow A\alpha \mid \beta$

- It generates sentences starting with the symbol β followed by zero or more α 's



- Replace the grammar by:

- $A \rightarrow \beta A'$
- $A' \rightarrow \alpha A' \mid \epsilon$
- It is right recursive now



消除立即左递归

简单语法: $A \rightarrow A\alpha \mid \beta$ 它生成以符号 β 开头的句子, 后面跟零个或更多的 α ' s

将语法替换为:

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \epsilon$

它现在是右递归了

Eliminating Immediate Left Recursion

- The general case: $A \rightarrow A\alpha_1 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \dots \mid \beta_n$
- Replace the grammar by:
 - $A \rightarrow \beta_1 A' \mid \dots \mid \beta_n A'$
 - $A' \rightarrow \alpha_1 A' \mid \dots \mid \alpha_m A' \mid \epsilon$

消除立即左递归

$$\bullet A \rightarrow A\alpha_1 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \dots \mid \beta_n$$

将语法替换为:

$$A \rightarrow \beta_1 A' \mid \dots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \dots \mid \alpha_m A' \mid \epsilon$$

Example

$$\begin{array}{lcl}
 E \rightarrow E + T \mid T & & E \rightarrow T E' \\
 \quad \quad \quad \underbrace{\quad}_\alpha \quad \underbrace{\quad}_\beta & & E' \rightarrow + T E' \mid \epsilon \\
 T \rightarrow T * F \mid F & \longrightarrow & T \rightarrow F T' \\
 \quad \quad \quad \underbrace{\quad}_\alpha \quad \underbrace{\quad}_\beta & & T' \rightarrow * F T' \mid \epsilon \\
 F \rightarrow (E) \mid \text{id} & & F \rightarrow (E) \mid \text{id}
 \end{array}$$

$$\begin{array}{lcl}
 \boxed{E \rightarrow E + T \mid T} & \Longrightarrow & \boxed{E \rightarrow T E'} \\
 \boxed{T \rightarrow T * F \mid F} & & \boxed{E' \rightarrow + T E' \mid \epsilon} \\
 F \rightarrow (E) \mid \text{id} & & \boxed{T \rightarrow F T'} \\
 & & \boxed{T' \rightarrow * F T' \mid \epsilon} \\
 & & F \rightarrow (E) \mid \text{id}
 \end{array}$$

Eliminating Left Recursion

- The technique for eliminating immediate left recursion does not work for the non-immediate left recursions
- The general left recursion eliminating algorithm (**iterative**)
 - Input:** Grammar G with no cycles or ϵ -productions
 - Output:** An equivalent grammar with no left recursion

arrange the nonterminals in some order $A_1, A_2, \dots, A_n$.

```

for ( each  $i$  from 1 to  $n$  ) {
  for ( each  $j$  from 1 to  $i - 1$  ) {
    replace each production of the form  $A_i \rightarrow A_j \gamma$  by the
    productions  $A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$ , where
     $A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$  are all current  $A_j$ -productions
  }
  eliminate the immediate left recursion among the  $A_i$ -productions
}
  
```

消除立即左递归的技术不适用于非立即左递归

虽然形式上产生式没有直接左递归，但由于形成了环，导致进行推导后后出现左递归。

通用左递归消除算法（迭代）

- 输入：没有循环或 ϵ -产生式的语法 G
- 输出：没有左递归的等价语法

以下是算法

对于每个非终结符 A_i ，检查是否有形如 $A_i \rightarrow A_j v$ 的产生式（ A_j 是其他的非终结符）

将 $A_i \rightarrow A_j v$ 修改为 $A_i \rightarrow \delta_1 v \mid \delta_2 v \mid \cdots \mid \delta_k v$

如果 $A_j \rightarrow \delta_1 \mid \delta_2 \mid \cdots$

接着消除 A_i 的所有直接左递归

Example

$S \rightarrow Aa \mid b \quad A \rightarrow Ac \mid Sd \mid \epsilon$

- Order the nonterminals: S, A
- $i = 1$:
 - The inner loop does not run; there is no immediate left recursion among S -productions
- $i = 2$:
 - $j = 1$, replace the production $A \rightarrow Sd$ by $A \rightarrow Aad \mid bd$
 - $A \rightarrow Aad \mid bd \mid Ac \mid \epsilon$
 - Eliminate immediate left recursion

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow bdA' \mid A' \\ A' &\rightarrow cA' \mid adA' \mid \epsilon \end{aligned}$$

The example grammar contains an ϵ -production, but it is harmless

Left Factoring (提取左公因子)

- If we have the following two productions

$$\begin{aligned} stmt &\rightarrow \text{if } expr \text{ then } stmt \text{ else } stmt \\ &| \text{ if } expr \text{ then } stmt \end{aligned}$$

- On seeing input **if**, we cannot immediately decide which production to choose
- In general, if $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ are two productions, and the input begins with a nonempty string derived from α . We may defer choosing productions by expanding A to $\alpha A'$ first

$\begin{aligned} A &\rightarrow \alpha A' \\ A' &\rightarrow \beta_1 \mid \beta_2 \end{aligned}$
--

如果我们有以下两个产生式

看到输入时**if**，我们无法立即决定选择哪个产生式

一般来说，如果 $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ 是两个产生式，并且输入以从 α 派生的非空字符串开始。

我们可以通过扩大 A 到 $\alpha A'$ 来推迟选择产生式

通过改写产生式来推迟决策，直到输入足够多的信息后再做出。

Algorithm: Left Factoring a Grammar

- **Input:** Grammar G
- **Output:** An equivalent left-factored grammar
- For each nonterminal A , find the longest prefix α common to two or more of its alternatives.
- If $\alpha \neq \epsilon$, replace all A -productions $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n \mid \gamma$, where γ represents all alternatives that do not begin with α , by

$$\begin{aligned} A &\rightarrow \alpha A' \mid \gamma \\ A' &\rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n \end{aligned}$$

- Repeatedly apply the above transformation until no two alternatives for a nonterminal have a common prefix

算法：语法的左公因子提取

输入：语法 G

输出：等效的左公因子提取语法

- 对于每个非终结符 A ，查找最长的前缀 α ，它是两个或多个替换项的公共前缀。
- 重复转换，直到非终结符的两个替代项都没有公共前缀为止

Top-Down Parsing

- **Problem definition:** Constructing a parse tree for the input string, starting from the root and creating the nodes of the parse tree in preorder (depth-first)
 - Equivalent to **finding a leftmost derivation for an input string**
- **Basic idea (two actions):**
 - **Predict:** At each step of parsing, determine the production to be applied for the leftmost nonterminal
 - **Match:** Match the terminals in the chosen production's body with the input string

自顶向下语法分析

问题定义：为输入字符串构建一个语法分析树，从根开始，按预序创建语法分析树的节点（深度优先）

- 相当于为输入字符串找到最左推导

基本思想（两个步骤）：

- 预测：在解析的每一步，确定最左边非终结符所用的产生式
- 匹配：将所选产生式主体中的终结符与输入字符串相匹配

Top-Down Parsing Example

- Grammar**

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id$$

- Input string**

id + id * id

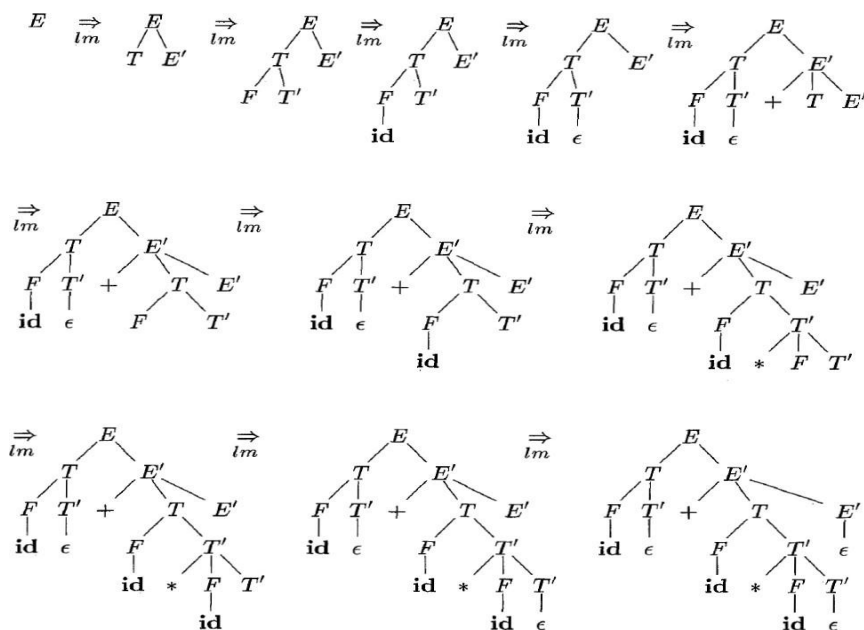
Is the input string a sentence of the grammar?



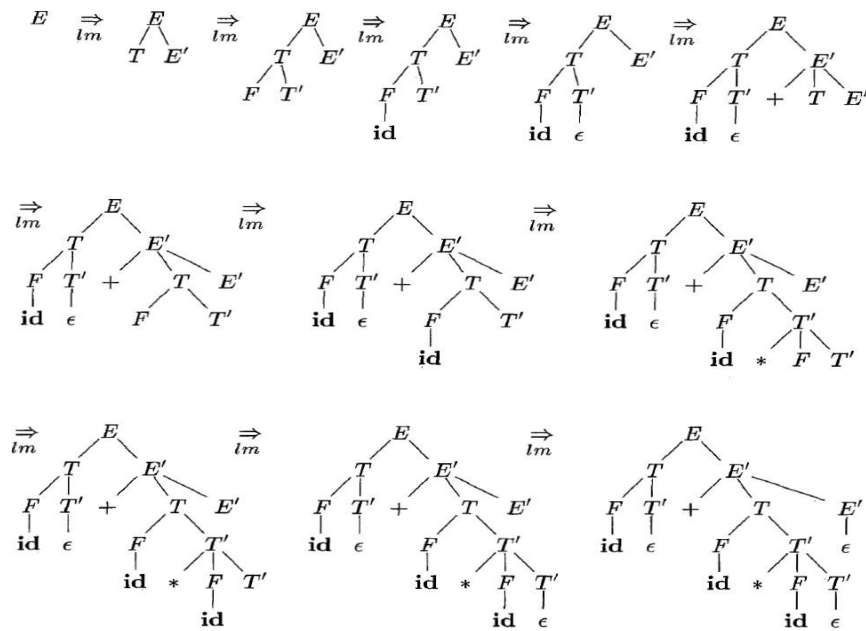
问题：输入串是该文法的句子吗？

答案：是，语法分析树见下一页 PPT

- Grammar:** $E \rightarrow TE'$ $E' \rightarrow +TE' \mid \epsilon$ $T \rightarrow FT'$ $T' \rightarrow *FT' \mid \epsilon$ $F \rightarrow (E) \mid id$
Input string: id + id * id



Key decision in top-down parsing: Which production to apply at each step?



Fall 2024

BUPT Compilers

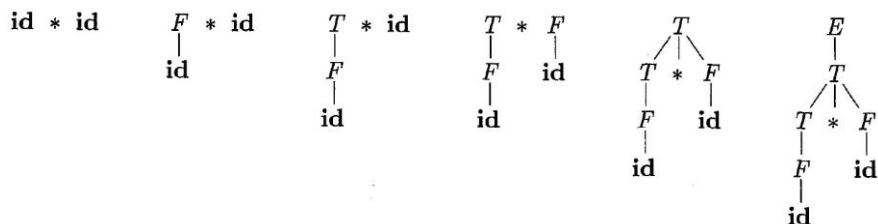
42

自顶向下解析中的关键决策：在每一步应用哪个产生式？

Bottom-Up Parsing

$$\begin{aligned}
 E &\rightarrow E + T \mid T \\
 T &\rightarrow T * F \mid F \\
 F &\rightarrow (E) \mid \text{id}
 \end{aligned}$$

- **Problem definition:** Constructing a parse tree for an input string beginning at the leaves (**terminals**) and working up towards the root (**start symbol of the grammar**)
 - Equivalent to **finding a rightmost derivation (in reverse) for an input string**
- **Shift-reduce parsing (移入-归约分析技术)** is a **general style** of bottom-up parsing (using a **stack** to hold grammar symbols)



Fall 2024

BUPT Compilers

43

自底向上语法分析

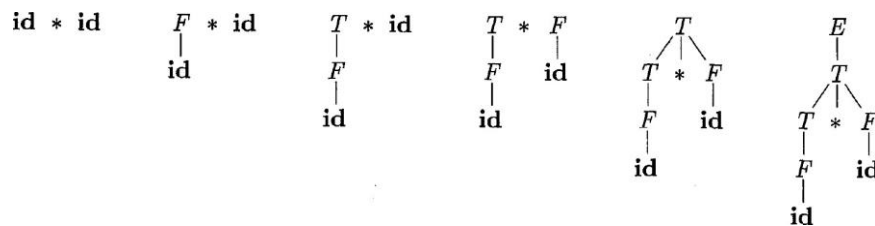
问题定义：为输入字符串构建一个从叶子（终结符）开始并向上延伸到根（语法的开始符号）的语法分析树

- 相当于为输入字符串找到最右推导（相反）

移入-归约分析技术是自底向上语法分析的通用方式（使用栈来保存语法符号）

Reductions (归约)

- Bottom-up parsing can be seen as a process of “reducing” a string ω to the start symbol of the grammar
- At each *reduction step*, a specific substring (at the top of the stack) matching the body of a production is replaced by the head of the production (the reverse of a step in derivation)



自底向上语法分析可以看作是一个将字符串 ω “归约”到开始符号的过程

在每一个归约步骤中，与产生式主体匹配的特定子串（位于堆栈顶部）都会被产生式的头部替换（与推导步骤相反）

Shift-Reduce Parsing Example

Parsing steps on input $\text{id}_1 * \text{id}_2$

STACK	INPUT	ACTION
\$	$\text{id}_1 * \text{id}_2$ \$	shift
\$ id_1	$* \text{id}_2$ \$	reduce by $F \rightarrow \text{id}$
\$ F	$* \text{id}_2$ \$	reduce by $T \rightarrow F$
\$ T	$* \text{id}_2$ \$	shift
\$ $T *$	id_2 \$	shift
\$ $T * \text{id}_2$	\$	reduce by $F \rightarrow \text{id}$
\$ $T * F$	\$	reduce by $T \rightarrow T * F$
\$ T	\$	reduce by $E \rightarrow T$
\$ E	\$	accept

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid \text{id}$

Key decisions:

1. When to reduce?
2. Which production to apply?

移入-归约分析技术示例

关键决策：

1. 什么时候归约？
2. 应用哪个产生式？

Recursive-Descent Parsing

(递归下降的语法分析)

- A recursive-descent parsing program has a set of procedures, one for each nonterminal
 - The procedure for a nonterminal scans the structure (a substring of the input) corresponding to the nonterminal
- Execution begins with the procedure for the start symbol
 - Announce success if the procedure scans the entire input string

```
void A() { // a typical procedure for a nonterminal
1)      Choose an A-production,  $A \rightarrow X_1X_2 \cdots X_k$ ;
2)      for (  $i = 1$  to  $k$  ) {
3)          if (  $X_i$  is a nonterminal )
4)              call procedure  $X_i()$ ;
5)          else if (  $X_i$  equals the current input symbol  $a$  )
6)              advance the input to the next symbol;
7)          else /* an error has occurred */;
      }
}
```

Fall 2024

BUPT Compilers

47

递归下降的语法分析程序对于每个非终结符都有一套程序

- 该程序会扫描与非终结符对应的结构（输入的子字符串）

从开始符号的程序开始执行

- 如果程序扫描整个输入字符串，则宣布成功

Backtracking (回溯)

```
void A() {  
1)    Choose an A-production,  $A \rightarrow X_1X_2 \cdots X_k$ ;  
2)    for (  $i = 1$  to  $k$  ) {  
3)        if (  $X_i$  is a nonterminal )  
4)            call procedure  $X_i()$ ;  
5)        else if (  $X_i$  equals the current input symbol  $a$  )  
6)            advance the input to the next symbol;  
7)    else /* an error has occurred */;  
    }  
}
```



If there is a failure at line 7, does this mean that there must be syntax errors?

问题：如果第 7 行出现故障，这是否意味着一定存在语法错误？

答案：不一定。见下一页 PPT

Backtracking (回溯)

```
void A() {  
1)    Choose an A-production,  $A \rightarrow X_1X_2 \cdots X_k$ ;  
2)    for (  $i = 1$  to  $k$  ) {  
3)        if (  $X_i$  is a nonterminal )  
4)            call procedure  $X_i()$ ;  
5)        else if (  $X_i$  equals the current input symbol  $a$  )  
6)            advance the input to the next symbol;  
7)    else /* an error has occurred */;  
    }  
}
```



The failure might be caused by a wrong choice of A-production at line 1 !!!

故障可能是由于在底=第 1 行选错了 A-产生式

Backtracking (回溯)

- General recursive-descent parsing may require **repeated scans** over the input (**backtracking**)
- To allow backtracking, we need to modify the algorithm

```
void A() {  
1) Choose an A-production,  $A \rightarrow X_1X_2 \cdots X_k$ ;  
2)   for (  $i = 1$  to  $k$  ) {  
3)       if (  $X_i$  is a nonterminal )  
4)           call procedure  $X_i()$ ;  
5)       else if (  $X_i$  equals the current input symbol  $a$  )  
6)           advance the input to the next symbol;  
7)       else /* an error has occurred */;  
   }  
}
```

Instead of exploring one A-production, we must try each possible production in some order.

一般的递归下降语法分析可能需要对输入进行重复扫描，以尝试不同的产生式（回溯）

为了允许回溯，我们需要修改算法

与考察 A-产生式不同，我们必须按照一定的顺序尝试每一种可能的产生式。

Backtracking (回溯)

- General recursive-descent parsing may require **repeated scans** over the input (**backtracking**)
- To allow backtracking, we need to modify the algorithm

```
void A() {  
1)    Choose an A-production,  $A \rightarrow X_1X_2 \cdots X_k$ ;  
2)    for (  $i = 1$  to  $k$  ) {  
3)        if (  $X_i$  is a nonterminal )  
4)            call procedure  $X_i()$ ;  
5)        else if (  $X_i$  equals the current input symbol  $a$  )  
6)            advance the input to the next symbol;  
7)        else /* an error has occurred */;  
    }  
}
```

When there is a failure at line 7, return to line 1 and try another A-production.

当第 7 行出现故障时，返回第 1 行并尝试另一个 A 产生式

Backtracking (回溯)

- General recursive-descent parsing may require **repeated scans** over the input (**backtracking**)
- To allow backtracking, we need to modify the algorithm

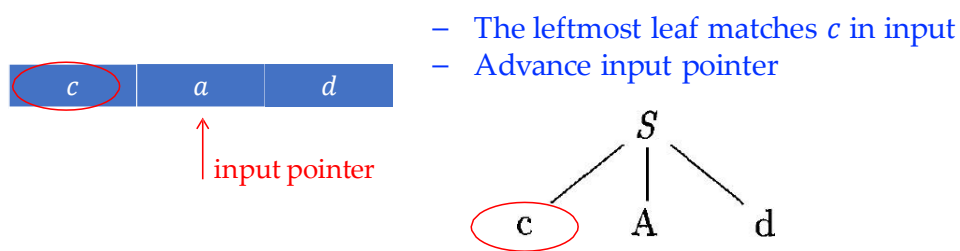
```
void A() {  
1)    Choose an A-production,  $A \rightarrow X_1X_2 \cdots X_k$ ;  
2)    for (  $i = 1$  to  $k$  ) {  
3)        if (  $X_i$  is a nonterminal )  
4)            call procedure  $X_i()$ ;  
5)        else if (  $X_i$  equals the current input symbol  $a$  )  
6)            advance the input to the next symbol;  
7)        else /* an error has occurred */;  
    }  
}
```

In order to try another A-production, we must reset the input pointer that points to the next symbol to scan (**failed trials consume symbols**)

为了尝试另一个 A-产生式时，我们必须重置指向下一个要扫描的符号的输入指针（失败尝试的消耗符号）

Backtracking Example

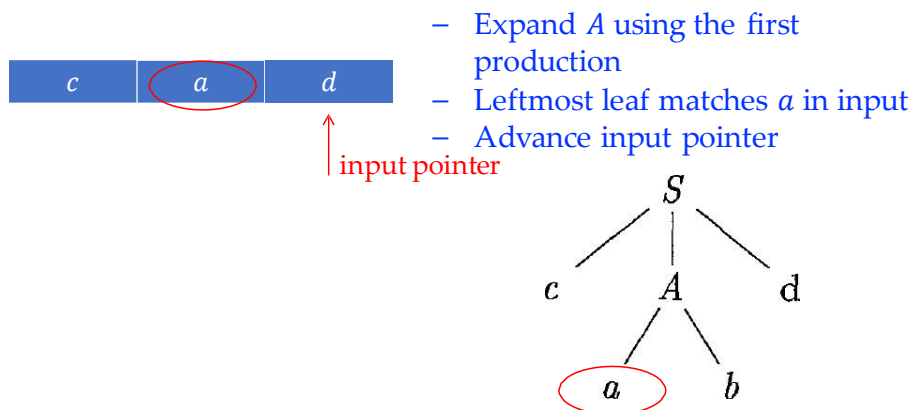
- **Grammar:** $S \rightarrow cAd$ $A \rightarrow ab \mid a$
- **Input string:** cad



S 结点最左边的叶子匹配输入中的 c
移进输入指针

Backtracking Example

- Grammar: $S \rightarrow cAd$ $A \rightarrow ab \mid a$
- Input string: cad



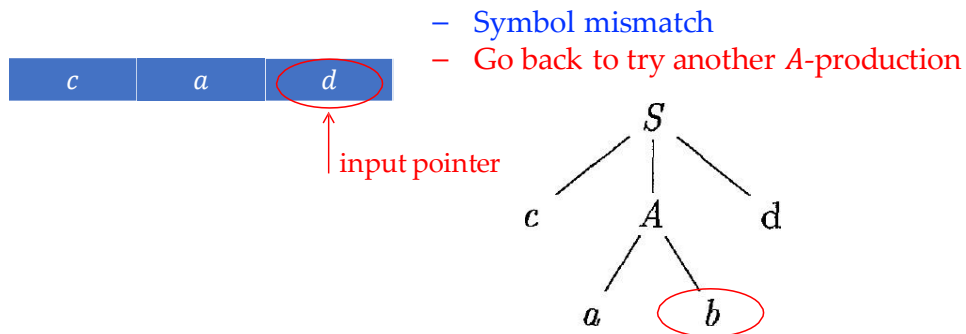
使用第 1 个产生式扩展 A

A 结点最左边的叶子匹配输入中的 a

移进输入指针

Backtracking Example

- Grammar: $S \rightarrow cAd$ $A \rightarrow ab \mid a$
- Input string: cad



Fall 2024

BUPT Compilers

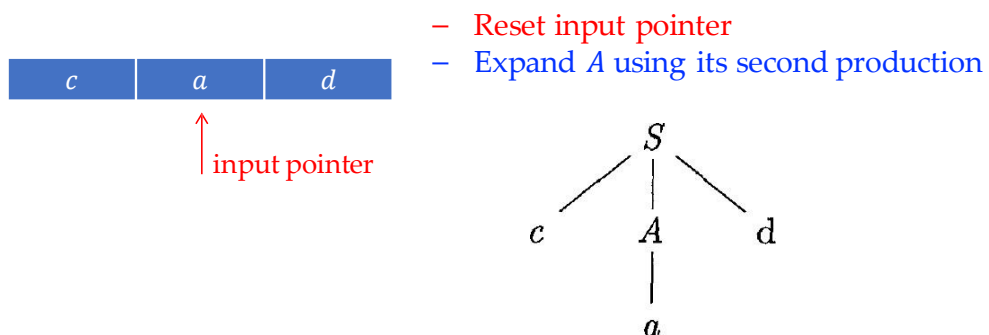
55

指针指向的 d 与符号 b 不匹配

返回尝试另一个 A -产生式

Backtracking Example

- Grammar: $S \rightarrow cAd$ $A \rightarrow ab \mid a$
- Input string: cad



Fall 2024

BUPT Compilers

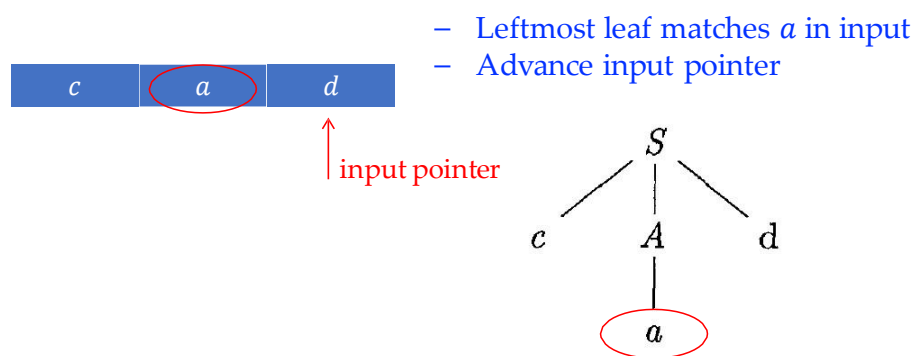
56

重置输入指针的位置

使用 A 的第二个产生式扩展 A

Backtracking Example

- Grammar: $S \rightarrow cAd$ $A \rightarrow ab \mid a$
- Input string: cad



A 结点最左边的叶子匹配输入中的 a

移进输入指针

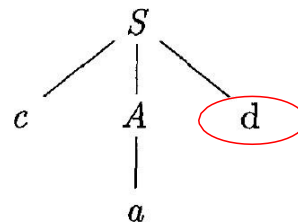
Backtracking Example

- Grammar: $S \rightarrow cAd$ $A \rightarrow ab \mid a$
- Input string: cad



- The last leaf node matches d in input
- Announce success!

Scanned entire input



最后的叶子匹配输入中的 d

成功

The Problem of Left Recursion

Suppose there is $A \rightarrow A\alpha \dots$

```
void A() {  
1)    Choose an A-production,  $A \rightarrow X_1X_2 \dots X_k$ ;  
2)    for (  $i = 1$  to  $k$  ) {  
3)        if (  $X_i$  is a nonterminal )  
4)            call procedure  $X_i()$ ;  
5)        else if (  $X_i$  equals the current input symbol  $a$  )  
6)            advance the input to the next symbol;  
7)        else /* an error has occurred */;  
    }  
}
```

If there is **left recursion** in a CFG, a recursive-descent parser may go into **an infinite loop**! Revise the CFG before parsing!!!

左递归带来的问题

如果 CFG 中有左递归，递归下降的语法分析器可能会进入无限循环，所以需要解析前修改 CFG

Can We Avoid Backtracking?

```
void A() {  
1)   Choose an  $A$ -production,  $A \rightarrow X_1X_2 \cdots X_k$ ;  
2)   for (  $i = 1$  to  $k$  ) {  
3)       if (  $X_i$  is a nonterminal )  
4)           call procedure  $X_i()$ ;  
5)       else if (  $X_i$  equals the current input symbol  $a$  )  
6)           advance the input to the next symbol;  
7)       else /* an error has occurred */;  
    }  
}
```

Key problem: At line 1, we make *random choices* (brute force search)

我们能避免回溯吗?

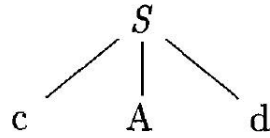
关键问题：在第 1 行，我们进行随机选择（暴力搜索）

Can We Avoid Backtracking?

- Grammar: $S \rightarrow cAd$ $A \rightarrow c \mid a$
- Input string: cad



↑ input pointer



When rewriting A , is it a good idea to choose $A \rightarrow c$?

No! If we look ahead, the next char in the input is a .

$A \rightarrow c$ is obviously a bad choice!!!

重写时 A , 选择产生式 $A \rightarrow c$ 好吗?

不, 如果我们向前看, 输入中的下一个字符是 a

$A \rightarrow c$ 显然是个糟糕的选择

Looking Ahead Helps!

- Suppose the input string is $x\mathbf{a}$
- Suppose the current sentential form is $x\mathbf{A}\beta$
 - $\mathbf{A}$ is a non-terminal; β may contain both terminals and non-terminals

If we know the following fact for the productions $\mathbf{A} \rightarrow \alpha \mid \gamma$:

- $a \in FIRST(\alpha)$: α derives strings that begin with a
- $a \notin FIRST(\gamma)$: γ derives strings that do not begin with a

**FIRST(α) denotes the set of terminals that begin strings derived from α*

After matching x , we should choose $\mathbf{A} \rightarrow \alpha$ to rewrite $\mathbf{A}$

向前看有用

假设输入字符串为 $x\mathbf{a}$

假设当前的句型是 $x\mathbf{A}\beta$

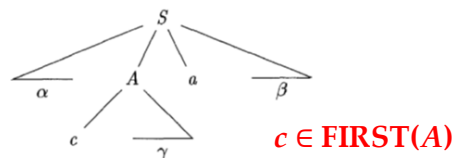
- $\mathbf{A}$ 是非终结符； β 可以同时包含终结符和非终结符

如果我们知道产生式 $\mathbf{A} \rightarrow \alpha \mid \gamma$ 的以下事实:

- $a \in FIRST(\alpha)$: α 可以推导以 a 开头的字符串
- $a \notin FIRST(\gamma)$: γ 不可以推导出以 a 开头的字符串

**FIRST(α)* 表示可从 α 推导得到的串的首符号的集合

FIRST



- **FIRST(α)**, where α is any string of grammar symbols

- The set of **terminals** that **begin strings derived from α**
- If $\alpha \Rightarrow^* \epsilon$, then ϵ is also in $\text{FIRST}(\alpha)$

▪ FIRST function is useful in parsing

- $A \rightarrow \alpha \mid \beta$; suppose $\text{FIRST}(\alpha)$ and $\text{FIRST}(\beta)$ are disjoint
- Choose $A \rightarrow \alpha$ if the next input symbol $a \in \text{FIRST}(\alpha)$
- Choose $A \rightarrow \beta$ if the next input symbol $a \in \text{FIRST}(\beta)$

$\text{FIRST}(\alpha)$, α 是语法符号串

如果 α 是终结符, FIRST 集为字符串首字母

如果 α 可以推导为空, 则空在 FIRST 集中

FIRST 集在语法分析是有用的

$A \rightarrow \alpha \mid \beta$, 假设 $\text{FIRST}(\alpha)$ and $\text{FIRST}(\beta)$ 交集为空

如果下一个 a 在 $\text{FIRST}(\alpha)$ 中, 选择产生式 $A \rightarrow \alpha$

如果下一个 a 在 $\text{FIRST}(\beta)$ 中, 选择产生式 $A \rightarrow \beta$

Computing FIRST

- **FIRST(X)**, where X is a grammar symbol
 - If X is a **terminal**, then $\text{FIRST}(X) = \{X\}$
 - If X is a **nonterminal** and $X \rightarrow Y_1 Y_2 \dots Y_k$ ($k \geq 1$) is a production
 - If for some i , a is in $\text{FIRST}(Y_i)$ and ϵ is in all of $\text{FIRST}(Y_1), \dots, \text{FIRST}(Y_{i-1})$, then add a to $\text{FIRST}(X)$
 - If ϵ is in all of $\text{FIRST}(Y_1), \dots, \text{FIRST}(Y_k)$, then add ϵ to $\text{FIRST}(X)$
 - If X is a **nonterminal** and $X \rightarrow \epsilon$, then add ϵ to $\text{FIRST}(X)$

计算 FIRST 集

计算 **FIRST(X)** X 是语法符号

- 如果 X 是一个终结符，那么 $\text{FIRST}(X) = \{X\}$
- 如果 X 是一个非终结符，且 $X \rightarrow Y_1 Y_2 \dots Y_k$ ($k \geq 1$) 是一个产生式
 - 如果对于某个 i ， a 在 $\text{FIRST}(Y_i)$ 中且 ϵ 在所有的 $\text{FIRST}(Y_1), \dots, \text{FIRST}(Y_{i-1})$

中，就把 a 加入到 $\text{FIRST}(X)$ 中

- 如果对于所有的 $\text{FIRST}(Y_1), \dots, \text{FIRST}(Y_k)$ ， ϵ 在其中，那么将 ϵ 加入到 $\text{FIRST}(X)$ 中

- 如果 X 是一个非终结符，并且存在产生式 $X \rightarrow \epsilon$ ，那么将 ϵ 加入到 $\text{FIRST}(X)$ 中

Computing FIRST Cont.

- **FIRST($X_1X_2 \dots X_n$)**, where $X_1X_2 \dots X_n$ is a string of grammar symbols
 - Add all **non- ϵ symbols** of FIRST(X_1) to FIRST($X_1X_2 \dots X_n$)
 - If **ϵ is in FIRST(X_1)**, add non- ϵ symbols of FIRST(X_2) to FIRST($X_1X_2 \dots X_n$); If **ϵ is in FIRST(X_1) and FIRST(X_2)**, add non- ϵ symbols of FIRST(X_3) to FIRST($X_1X_2 \dots X_n$); ...
 - If **ϵ is in FIRST(X_i) for all i** , add ϵ to FIRST($X_1X_2 \dots X_n$)

计算 FIRST 集

计算 **FIRST($X_1X_2 \dots X_n$)** $X_1X_2 \dots X_n$ 是语法符号的串

- 向 FIRST($X_1X_2 \dots X_n$)加入 FIRST(X_1)中所有的非 ϵ 符号
- 如果 ϵ 在 FIRST(X_1)中，再加入 FIRST(X_2)中所有的非 ϵ 符号；如果 ϵ 在 FIRST(X_1)和 FIRST(X_2)中，加入 FIRST(X_3)中的所有非 ϵ 符号，以此类推。
- 最后，如果对于所有的 i ， ϵ 都在 FIRST(X_i)中，那么将加入到 FIRST($X_1X_2 \dots X_n$)中。

FIRST Example

- Grammar

- $E \rightarrow TE'$ $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$ $T' \rightarrow *FT' \mid \epsilon$ $F \rightarrow (E) \mid \text{id}$

- FIRST sets

- $\text{FIRST}(F) = \{ (, \text{id} \}$
- $\text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}$
- $\text{FIRST}(E) = \text{FIRST}(T) = \{ (, \text{id} \}$
- $\text{FIRST}(E') = \{ +, \epsilon \}$ $\text{FIRST}(T') = \{ *, \epsilon \}$
- $\text{FIRST}(TE') = \text{FIRST}(T) = \{ (, \text{id} \}$
- ...

Looking Ahead Helps Cont.

- Suppose the input string is $x\mathbf{a}$
- Suppose the current sentential form is $x\mathbf{A}\beta$
 - $\mathbf{A}$ is a non-terminal; β may contain both terminals and non-terminals

If we know that for the production $\mathbf{A} \rightarrow \alpha$, $\epsilon \in \text{FIRST}(\alpha)$, can we choose the production to rewrite $\mathbf{A}$?

Yes, only if β can derive strings beginning with a , that is, $\mathbf{A}$ can be followed by a in some sentential forms (i.e., $a \in \text{FOLLOW}(\mathbf{A})$)

假设输入字符串为 xa

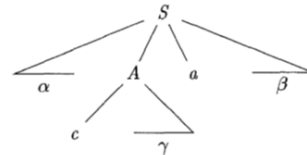
假设当前的句型是 $xA\beta$

- A 是非终结符； β 可以同时包含终结符和非终结符

如果我们知道生产 $A \rightarrow \alpha, \epsilon \in FIRST(\alpha)$, 我们能选择重写 A 的产生式吗?

可以, 只有在 β 可以推导以 a 开头的字符串, 即, 在一些句型中 A 后面可以跟着 a
(即, $a \in FOLLOW(A)$)

FOLLOW



- **FOLLOW(A)**, where A is a nonterminal $a \in FOLLOW(A)$

- The set of **terminals** a that can appear immediately to the right of A in some sentential form

- **FOLLOW function is also useful in parsing**

- Consider a production $A \rightarrow \alpha$; when $\alpha \xRightarrow{*} \epsilon$, FOLLOW(A) can help choose the right production
- **Example**: if the next input symbol is b , and $b \in FOLLOW(A)$, then we can choose $A \rightarrow \alpha$

FOLLOW(A), A 是非终结符

$a \in FOLLOW(A)$ 表示终结符在某些句型中可以出现在 A 的右边

对产生式 $A \rightarrow \alpha$, 当 $\alpha \Rightarrow^* \epsilon$, FOLLOW(A)可以帮助我们选择正确的产生式

Computing FOLLOW

- Computing FOLLOW(A) for all nonterminals A

- Add $\$$ in FOLLOW(S), where S is the start symbol and $\$$ is the input right endmarker
- Apply the rules below, until all FOLLOW sets do not change
 1. If there is a production $A \rightarrow \alpha B \beta$, then everything in FIRST(β) except ϵ is in FOLLOW(B)
 2. If there is a production $A \rightarrow \alpha B$ (or $A \rightarrow \alpha B \beta$ and FIRST(β) contains ϵ) then everything in FOLLOW(A) is in FOLLOW(B)

ϵ will not appear in any FOLLOW set

计算 FOLLOW 集

计算所有非终结符 A 的 FOLLOW(A)集合

- 将 $\$$ 添加到 FOLLOW (S) 中，其中 S 是开始符号， $\$$ 是输入右端的结束标记
- 不断应用下面的规则，直到所有 FOLLOW 集不再改变
- 如果存在一个产生式 $A \rightarrow \alpha B \beta$ ，那么 FIRST(β)中除 ϵ 之外的所有符号都在

FOLLOW(B)中

- 如果存在一个产生式 $A \rightarrow \alpha B$ ，或存在产生式 $A \rightarrow \alpha B \beta$ 且 FIRST(β)包含 ϵ ，

那么 FOLLOW(A)中的所有符号都在 FOLLOW(B)中

ϵ 不会在任何 FOLLOW 集中

FOLLOW Example

- Grammar

- $E \rightarrow TE'$ $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$ $T' \rightarrow *FT' \mid \epsilon$ $F \rightarrow (E) \mid \text{id}$

- FOLLOW sets

- $\text{FOLLOW}(E) = \{\$, \})\}$
- $\text{FOLLOW}(E') = \{\$, \})\}$
- $\text{FOLLOW}(T) = \{+, \$, \})\}$
- $\text{FOLLOW}(T') = \{+, \$, \})\}$
- $\text{FOLLOW}(F) = \{*, +, \$, \})\}$

- \$ is always in FOLLOW(E)
- Everything in FIRST()) except ϵ is in FOLLOW(E)
- Everything in FIRST(E') except ϵ is in FOLLOW(T)
- Since $E' \rightarrow \epsilon$, everything in FOLLOW(E) is in FOLLOW(T)

LL(1) Grammars

- Recursive-descent parsers needing no backtracking can be constructed for a class of grammars called **LL(1)**
 - **1st L**: scanning the input from left to right
 - **2nd L**: producing a leftmost derivation
 - **1**: using one input symbol of lookahead at each step to make parsing decision

我们可以为一类称为 LL(1)的语法构造不需要回溯的递归下降语法分析器

从左到右扫描输入

产生最左推导

在每一步使用一个前瞻输入符号来作出分析决策

LL(1) Grammars Cont.

A grammar G is LL(1) if and only if for any two distinct productions $A \rightarrow \alpha \mid \beta$, the following conditions hold:

1. There is no terminal a such that α and β derive strings beginning with a
2. At most one of α and β can derive the empty string
3. If $\beta \Rightarrow^* \epsilon$, then α does not derive any string beginning with a terminal in FOLLOW(A) and vice versa

* The three conditions basically rule out the possibility of applying both productions

More formally:

1. $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$ (conditions 1-2 above)
2. If $\epsilon \in \text{FIRST}(\beta)$, then $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$ and vice versa

语法 G 是 LL(1) 当且仅当对于任意两个不同的产生式 $A \rightarrow \alpha \mid \beta$, 以下条件成立:

1. 不存在终结符 a 使得 α 和 β 都可以推导以 a 开头的字符串
2. α 和 β 中最多只有一个可以推导空字符串
3. 如果 $\beta \Rightarrow^* \epsilon$, 那么 α 不能推导出任何以 FOLLOW(A) 中某个终结符开头的字符串, 反之亦然

*这三个条件基本上排除了应用两个产生式的可能性

更正式地说:

1. $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$ (上述条件 1-2)

如果 $\epsilon \in \text{FIRST}(\beta)$, 那么 $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$, 反之亦然

Parsing Table (预测分析表)

- Recursive-descent parsers (or **LL parsers**) are table-based parsers
- A predictive **parsing table** is a two-dimensional array that determines which production the parser should choose when it sees a nonterminal **A** and a symbol **a** on its input stream
- The parsing table of an LL(1) parser has **no entries with multiple productions**

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
<i>E</i>	$E \rightarrow TE'$			$E \rightarrow TE'$		
<i>E'</i>		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
<i>T</i>	$T \rightarrow FT'$			$T \rightarrow FT'$		
<i>T'</i>		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
<i>F</i>	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

递归下降语法分析器（或 LL 分析器）是基于表的解析器

预测分析表是一个二维数组，用于确定解析器在看到非终结符 A 和符号 a 时应该选择哪个产生式

LL(1)解析器的预测分析表没有包含多个产生式的条目

Parsing Table Construction

The following algorithm can be applied to any CFG

- **Input:** Grammar G **Output:** Parsing table M
- **Method:**
 - For each production $A \rightarrow \alpha$ of G , do the following:
 - For each terminal a in $\text{FIRST}(\alpha)$, add $A \rightarrow \alpha$ to $M[A, a]$
 - If ϵ is in $\text{FIRST}(\alpha)$, then for each terminal b (including the right endmarker $\$$) in $\text{FOLLOW}(A)$, add $A \rightarrow \alpha$ to $M[A, b]$
 - Set all empty entries in the table to **error**

构造预测分析表

以下算法可以应用于任何 CFG

输入：文法 G

输出：预测分析表 M

方法：对于每个产生式 $A \rightarrow \alpha$ ，进行以下处理：

执行以下操作：

- 对于 $\text{FIRST}(\alpha)$ 中每个终结符 a ，将 $A \rightarrow \alpha$ 加入到 $M[A, a]$
- 如果 ϵ 在 $\text{FIRST}(\alpha)$ 中，对于 $\text{FOLLOW}(A)$ 中每个终结符 b ，包括 $\$$ ，将 $A \rightarrow \alpha$ 加入到 $M[A, b]$

将表中的所有空条目设置为错误

Parsing Table Construction Example

- Grammar**

- $E \rightarrow TE' \quad E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT' \quad T' \rightarrow *FT' \mid \epsilon \quad F \rightarrow (E) \mid \text{id}$

- **FIRST sets:** $E, T, F: \{ (, \text{id} \} \quad E': \{ +, \epsilon \} \quad T': \{ *, \epsilon \}$

- **FOLLOW sets:** $E, E': \{ \$,) \} \quad T, T': \{ +, \$,) \} \quad F: \{ *, +, \$,) \}$

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

For $E \rightarrow TE'$:

FIRST(TE')
= FIRST(T)
= $\{ (, \text{id} \}$

Parsing Table Construction Example

- Grammar**

- $E \rightarrow TE' \quad E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT' \quad T' \rightarrow *FT' \mid \epsilon \quad F \rightarrow (E) \mid \text{id}$

- **FIRST sets:** $E, T, F: \{ (, \text{id} \} \quad E': \{ +, \epsilon \} \quad T': \{ *, \epsilon \}$

- **FOLLOW sets:** $E, E': \{ \$,) \} \quad T, T': \{ +, \$,) \} \quad F: \{ *, +, \$,) \}$

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

For $E' \rightarrow \epsilon$:

ϵ in FIRST(ϵ)
FOLLOW(E')
= $\{ \$,) \}$

Conflicts in Parsing Tables

- **Grammar:** $S \rightarrow iEtSS' \mid a \quad S' \rightarrow eS \mid \epsilon \quad E \rightarrow b$

- $FIRST(eS) = \{e\}$, so we add $S' \rightarrow eS$ to $M[S', e]$
- $FOLLOW(S') = \{\$, e\}$, so we add $S' \rightarrow \epsilon$ to $M[S', e]$

NON - TERMINAL	INPUT SYMBOL					
	a	b	e	i	t	$\$$
S	$S \rightarrow a$			$S \rightarrow iEtSS'$		
S'			$S' \rightarrow \epsilon$ $S' \rightarrow eS$			$S' \rightarrow \epsilon$
E		$E \rightarrow b$				

- LL(1) grammar is never ambiguous.
- This grammar is not LL(1). The language has no LL(1) grammar !!!

预测分析表中的冲突

LL(1)文法不存在二义性

Conflicts in Parsing Tables

- **Grammar:** $S \rightarrow iEtSS' \mid a \quad S' \rightarrow eS \mid \epsilon \quad E \rightarrow b$

- $FIRST(eS) = \{e\}$, so we add $S' \rightarrow eS$ to $M[S', e]$
- $FOLLOW(S') = \{\$, e\}$, so we add $S' \rightarrow \epsilon$ to $M[S', e]$

NON - TERMINAL	INPUT SYMBOL					
	a	b	e	i	t	$\$$
S	$S \rightarrow a$			$S \rightarrow iEtSS'$		
S'			$S' \rightarrow \epsilon$ $S' \rightarrow eS$			$S' \rightarrow \epsilon$
E		$E \rightarrow b$				

Solution: Choose $S' \rightarrow eS$ to resolve ambiguity (associating an **else** with the closest previous **then**)

解决方案：选择 $S' \rightarrow es$ 解决歧义（将 else 与最近的一个 then 关联）

Recursive-Descent Parsing for LL(1) Grammars

```
void A() {  
1)   Choose an A-production,  $A \rightarrow X_1X_2 \cdots X_k$ ;  
2)   for (  $i = 1$  to  $k$  ) {  
3)       if (  $X_i$  is a nonterminal )  
4)           call procedure  $X_i()$ ;  
5)       else if (  $X_i$  equals the current input symbol  $a$  )  
6)           advance the input to the next symbol;  
7)       else /* an error has occurred */;  
   }  
}
```

Replace line 1 with: Choose A-production according to the parse table
• Assume input symbol is a , then the choice is the production in $M[A, a]$

LL(1)文法的递归下降语法分析

将第 1 行替换为：根据预测分析表选择 A-产生式

假设输入符号为 a , 那么选择 $M[A, a]$ 中的产生式

Non-Recursive Predictive Parsing

```
void A() {  
  1) Choose an A-production,  $A \rightarrow X_1 X_2 \cdots X_k$ ;  
  2) for (  $i = 1$  to  $k$  ) {  
  3)   if (  $X_i$  is a nonterminal )  
  4)   call procedure  $X_i()$ ;  
  5)   else if (  $X_i$  equals the current input symbol  $a$  )  
  6)     advance the input to the next symbol;  
  7)   else /* an error has occurred */;  
}
```



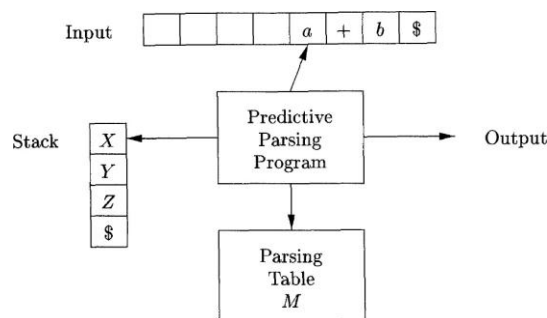
Recursive-descent parsing has recursive calls.
Can we design a non-recursive parser?

递归下降的语法分析具有递归调用。

我们能设计一个非递归解析器吗？

Non-Recursive Predictive Parsing

- A non-recursive predictive parser can be built by **explicitly maintaining a stack** (not implicitly via recursive calls)
 - **Input buffer** contains the string to be parsed, ending with \$
 - **Stack** holds a sequence of grammar symbols with \$ at the bottom. Initially, the stack contains only \$ and the start symbol S on top of \$



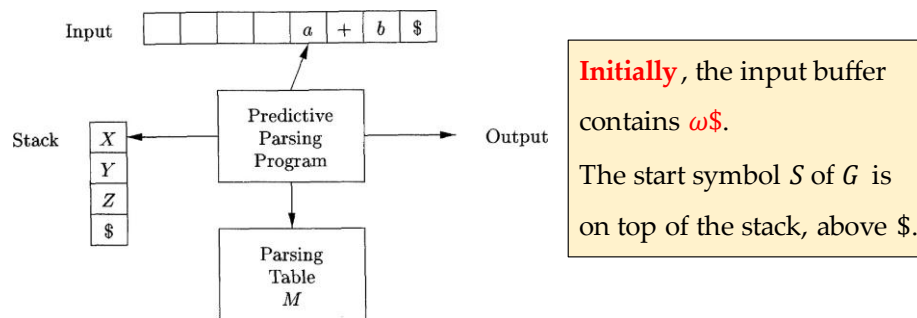
非递归的预测分析器可以通过显式维护堆栈（而不是通过递归调用隐式维护）来构建

- 输入缓冲区包含要解析的字符串，以\$结尾
- 堆栈包含底部为\$的语法符号序列。

最初，堆栈只包含\$和在\$顶部的开始符号 S

Table-Driven Predictive Parsing

- **Input:** A string ω and a parsing table M for grammar G
- **Output:** If ω is in $L(G)$, a leftmost derivation of ω ; otherwise, an error indication



表驱动的预测语法分析

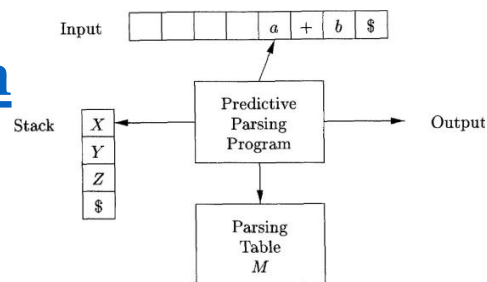
输入：字符串 ω 和文法 G 的预测分析表 M

输出：如果 ω 在 $L(G)$ 中，输出 ω 的一个最左边推导；否则，给出一个错误指示

最初，输入缓冲区包含 $\omega\$$

开始符号 S 位于堆栈顶部，在\$上方。

Parsing Algorithm



1. let a be the first symbol of ω ;
2. let X be the top stack symbol;
3. while ($X \neq \$$) { /* stack is not empty */
4. if ($X = a$) pop the stack and let a be the next symbol of ω ;
5. else if (X is a terminal) *error()*; /* X can only match a */
6. else if ($M[X, a]$ is an error entry) *error()*;
7. else if ($M[X, a] = X \rightarrow Y_1 Y_2 \dots Y_k$) {
8. output the production $X \rightarrow Y_1 Y_2 \dots Y_k$;
9. pop the stack;
10. push $Y_k, Y_{k-1}, \dots, Y_1$ onto the stack, with Y_1 on top; /* order is critical */
11. }
12. let X be the top stack symbol;
13. }

Example

$$\begin{aligned}
 E &\rightarrow TE' & E' &\rightarrow +TE' \mid \epsilon \\
 T &\rightarrow FT' & T' &\rightarrow *FT' \mid \epsilon & F &\rightarrow (E) \mid \text{id}
 \end{aligned}$$

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

Input:
id + id * id

MATCHED	STACK	INPUT	ACTION
	$E\$$	id + id * id\$	
	$TE'\$$	id + id * id\$	output $E \rightarrow TE'$
	$FT'E'\$$	id + id * id\$	output $T \rightarrow FT'$
	id $T'E'\$$	id + id * id\$	output $F \rightarrow \text{id}$
	$T'E'\$$	+ id * id\$	match id
	$E'\$$	+ id * id\$	output $T' \rightarrow \epsilon$
	+ $TE'\$$	+ id * id\$	output $E' \rightarrow + TE'$
	$TE'\$$	id * id\$	match +

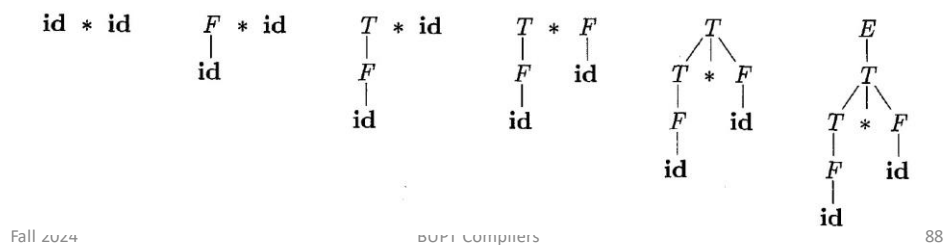
Matched part
+
Stack content
(from top to bottom)
=
A left-sentential form
总是最左句型

匹配部分+栈内容（从顶部到底部）=总是最左句型

Bottom-Up Parsing

$$\begin{array}{l} E \rightarrow E + T \mid T \\ T \rightarrow T * F \mid F \\ F \rightarrow (E) \mid \text{id} \end{array}$$

- **Problem definition:** Constructing a parse tree for an input string beginning at the leaves (**terminals**) and working up towards the root (**start symbol of the grammar**)
 - Equivalent to **finding a rightmost derivation (in reverse) for an input string**
- **Shift-reduce parsing (移入-归约分析技术)** is a **general style** of bottom-up parsing (using a **stack** to hold grammar symbols)



自底向上的语法分析

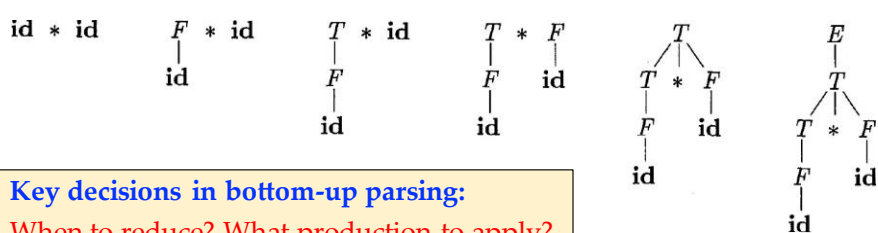
问题定义：为输入字符串构建一个从叶子（终结符）开始并向上延伸到根（语法的开始符号）的语法分析树

- 相当于为输入字符串找到最右边的推导（相反）

移入-归约分析技术是一种自底向上语法分析的通用方式（使用栈来保存语法符号）

Reductions (归约)

- Bottom-up parsing can be seen as a process of “reducing” a string ω to the start symbol of the grammar
- At each *reduction step*, a specific substring (at the top of the stack) matching the body of a production is replaced by the head of the production (the reverse of a step in derivation)

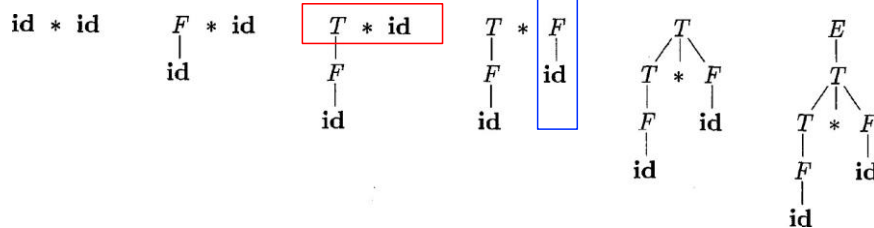


自底向上语法分析可以看作是一个将字符串 ω “归约”到开始符号的过程

在每一个归约步骤中，与产生式体匹配的特定子串（位于栈顶部）会被产生式的头部替换（与推导步骤相反）

Illustration of Challenges

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$



- For sentential form $T * \text{id}$, there are two applicable productions:
 - T is the body of $E \rightarrow T$
 - id is the body of $F \rightarrow \text{id}$
- Why reducing id to F , instead of reducing T to E ?
 - Reason:** If reducing T to E , $E * \text{id}$ is no longer a sentential form
 - Challenge:** how do we know this before making choices?



对于句型 $T * \text{id}$, 有两个适用的产生式:

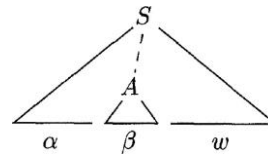
T 是 $E \rightarrow T$ 的体

id 是 $F \rightarrow \text{id}$ 的体

为什么把 id 归约为 F , 而不是将 T 归约为 E

- 原因: 如果将 T 归约为 E , $E * \text{id}$ 不再是句型
- 在这之前我们如何作决定?

Handles (句柄)



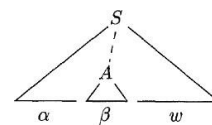
- Informally, a substring that matches the body of a production, and whose reduction represents one step along the reverse of a rightmost derivation
- Formally, if $S \xRightarrow{*}_{rm} \alpha A w \xRightarrow{}_{rm} \alpha \beta w$, then production $A \rightarrow \beta$ (or simply β) in the position following α is a handle of the sentential form $\alpha \beta w$

RIGHT SENTENTIAL FORM	HANDLE	REDUCING PRODUCTION
$\text{id}_1 * \text{id}_2$	id_1	$F \rightarrow \text{id}$
$F * \text{id}_2$	F	$T \rightarrow F$
$T * \text{id}_2$	id_2	$F \rightarrow \text{id}$
$T * F$	$T * F$	$E \rightarrow T * F$

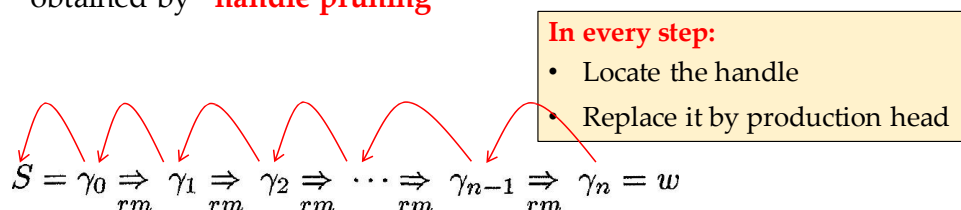
非正式地讲，句柄是和某个产生式体匹配的子串，对它的归约代表了相应的最右推导中的一个反向步骤

正式地讲，如果 $S \xRightarrow{*}_{rm} \alpha A w \xRightarrow{}_{rm} \alpha \beta w$ ，那么紧跟 α 的产生式 $A \rightarrow \beta$ （或者简单地 β ）是句型 $\alpha \beta w$ 的一个句柄

Handle Pruning (句柄剪枝)



- In a right-sentential form (最右句型), the string to the right of the handle must contain only terminal symbols
- If a grammar is unambiguous, every right-sentential form of the grammar has exactly one handle*
- A rightmost derivation in reverse (bottom-up parsing) can be obtained by “handle pruning”



* Given an unambiguous grammar, a right-sentential form can only be derived in one way

在最右句型中，句柄右侧的字符串必须仅包含终结符号

如果一个文法是无二义性的，那么该文法的每个右句型都有且只有一个句柄（如果文法二义，句柄可能不唯一）

通过句柄剪枝可以得到一个反向的最右推导

Shift-Reduce Parsing

- Shift-reduce parsing is a general style of bottom-up parsing in which:
 - A **stack** holds grammar symbols
 - An **input buffer** holds the rest of the string to be parsed
 - The **stack content (from bottom to top)** and **the input buffer content** form a **right-sentential form** (assuming no errors)

移入-归约语法分析技术是自底向上语法分析的一种通用方式，其中：

- 使用栈保存文法符号
- 使用输入缓冲区保存要进行语法分析的字符串的其余部分
- 栈内容（从底部到顶部）和输入缓冲区内容形成最右句型（假设没有错误）

Shift-Reduce Parsing

Initial status:

STACK	INPUT
\$	ω \$

Actions:

Shift
Reduce
Accept
Error

Shift-reduce process:

- The parser **shifts** zero or more input symbols onto the stack, until it is ready to reduce a string β (a **handle**) on top of the stack*
- **Reduce** β to the head of the appropriate production



The parser repeats the above cycle until it has detected an error or the stack contains the start symbol and input is empty

* During shift-reduce parsing, the handle will always eventually appear on top of the stack

初始状态:

移入-归约过程:

· 语法分析器将零个或多个输入符号转移到栈顶上, 直到它可以对栈顶的一个文法符号串 β 进行归约为止

- 将 β 归约为某个产生式的头

语法分析器重复上述循环, 直到检测到错误或栈包含开始符号且输入为空

*在移入-归约解析过程中, 句柄最终总是出现在栈的顶部

Example

Parsing steps on input $\text{id}_1 * \text{id}_2$

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid \text{id}$

STACK	INPUT	ACTION
\$	$\text{id}_1 * \text{id}_2$ \$	shift
\$ id_1	* id_2 \$	reduce by $F \rightarrow \text{id}$
\$ F	* id_2 \$	reduce by $T \rightarrow F$
\$ T	* id_2 \$	shift
\$ $T *$	id_2 \$	shift
\$ $T * \text{id}_2$	\$	reduce by $F \rightarrow \text{id}$
\$ $T * F$	\$	reduce by $T \rightarrow T * F$
\$ T	\$	reduce by $E \rightarrow T$
\$ E	\$	accept

Why Handles Always Appear at Stack Top?

(Analysis by enumeration) Consider the possible forms of two successive steps in any rightmost derivation*

$$(1) \quad S \xRightarrow{*}_{rm} \alpha A z \Rightarrow \alpha \beta B y z \Rightarrow \alpha \beta \gamma y z$$

- **Step 1:** the **rightmost nonterminal** A is replaced by **a string with nonterminals**. Suppose B is the rightmost nonterminal in the string.
- **Step 2:** B is replaced by a string γ

$$(2) \quad S \xRightarrow{*}_{rm} \alpha B x A z \Rightarrow \alpha B x y z \Rightarrow \alpha \gamma x y z$$

- **Step 1:** the **rightmost nonterminal** A is replaced by **a string y with terminals only**
- **Step 2:** the next rightmost nonterminal B (**somewhere to the left of y**) is replaced by a string γ

* The analysis is trivial when there is only one step of derivation

为什么句柄总是出现在栈顶?

(枚举分析) 考虑在任何最右推导中两个连续步骤的可能具有的形式

情况 1:

步骤 1: 最右边的非终结符 A 被一个带有非终结符的字符串替换。假设 B 是字符串中最右边的非终结符。

步骤 2: B 被替换为 γ

情况 2:

步骤 1: 最右边的非终结符 A 被替换为 y , y 中仅包含终结符。

步骤 2: 下一个最右边的非终结符 B 将位于 γ 左侧的某个地方

Why Handles Always Appear at Stack Top?

$$(1) \quad S \xRightarrow{*}_{rm} \alpha A z \Rightarrow \alpha \beta B y z \Rightarrow \alpha \beta \gamma y z$$

$$(2) \quad S \xRightarrow{*}_{rm} \alpha B x A z \Rightarrow \alpha B x y z \Rightarrow \alpha \gamma x y z$$

STACK	INPUT	ACTION
...	...	...
$\alpha \beta \gamma$	yz\$	Reduce
$\alpha \beta B$	yz\$	Shift
$\alpha \beta B y$	z\$	Reduce
αA	z\$	Shift
...	...	...

x, y, z are strings of terminals

STACK	INPUT	ACTION
...	...	...
$\alpha \gamma$	xyz\$	Reduce
αB	xyz\$	Shift
$\alpha B x$	yz\$	Shift
$\alpha B x y$	z\$	Reduce
$\alpha B x A$	z\$	Shift
...	...	...

在这两种情况下, 语法分析器在进行一次归约后, 都必须接着移入零个或多个符号才
能在栈顶找到下一个句柄。因此它不需要到栈中间去寻找句柄。

Conflicts During Shift-Reduce Parsing

- There are context-free grammars for which shift-reduce parsing cannot be used
- Every shift-reduce parser for such a grammar can reach a configuration in which the parser, knowing the entire stack and also the next k input symbols, cannot decide
 - Whether to shift or to reduce (shift/reduce conflicts, 移入/归约冲突)
 - Which of several possible reductions to make (reduce/reduce conflicts, 归约/归约冲突)

移入-归约语法分析中的冲突

有些上下文无关文法不能使用移入-归约语法分析技术

这种文法的每一个移入-归约语法分析器都会达到这样一种格局：即使移入-归约语法分析器知道整个栈的内容以及下一个输入符号 k ，仍然无法决定是进行移入还是归约（移入/归约冲突）或者无法在多个可能的归约方法中选择正确的归约动作（归约/归约冲突）

Shift/Reduce Conflict Example

$$\begin{array}{lcl} stmt & \rightarrow & \text{if } expr \text{ then } stmt \\ & | & \text{if } expr \text{ then } stmt \text{ else } stmt \\ & | & \text{other} \end{array}$$

STACK
 $\dots$ **if** *expr* **then** *stmt*

INPUT
 else $\dots$ \$

Reduce or shift? What if there is a *stmt* after **else**?



移入/归约冲突示例，是应该把栈内的进行规约还是继续移入？

Reduce/Reduce Conflict Example

- Parsing input **id(id, id)**

STACK	INPUT
\$id(id	, id\$

- (1) $stmt \rightarrow id (parameter_list)$
- (2) $stmt \rightarrow expr := expr$
- (3) $parameter_list \rightarrow parameter_list , parameter$
- (4) $parameter_list \rightarrow parameter$
- (5) $parameter \rightarrow id$
- (6) $expr \rightarrow id (expr_list)$
- (7) $expr \rightarrow id$
- (8) $expr_list \rightarrow expr_list , expr$
- (9) $expr_list \rightarrow expr$



Reduce by which production?

归约/归约冲突示例，不知用哪个产生式进行规约

LR Parsing (LR语法分析技术)

- **LR(k) parsers:** the most prevalent type of bottom-up parsers
 - **L:** left-to-right scan of the input
 - **R:** construct a rightmost derivation in reverse
 - **k :** use k input symbols of lookahead in making parsing decisions
- **LR(0) and LR(1) parsers** are of practical interest
 - When $k \geq 2$, the parser becomes too complex to construct (parsing table will be too huge to manage)

LR(k)语法分析器：最流行的自底向上的语法分析器类型

- **L:** 从左到右扫描输入
- **R:** 反向构造最右推导
- **k :** 在做出解析决策时使用 k 个前瞻性输入符号

LR(0)和 LR(1)语法分析器具有实际意义

当 $k \geq 2$ 时，语法分析器变得太复杂，无法构建（解析表太大，无法管理）

Advantages of LR Parsers

- **Table-driven** (like non-recursive LL parsers) and **powerful**
 - Although it is too much work to construct an LR parser by hand, there are parser generators to construct parsing tables automatically
 - Comparatively, LL parsers tend to be easier to write by hand, but less powerful (handle fewer grammars)
- LR-parsing is the most general **nonbacktracking shift-reduce parsing** method known
- LR parsers can be constructed to **recognize virtually all programming language constructs** for which CFGs can be written
- LR grammars can **describe more languages** than LL grammars
 - Recall the stringent conditions for a grammar to be LL(1)

LR 语法分析器的优势

- 表驱动（类似于非递归 LL 语法分析器）、强大
 - 尽管手工构建 LR 语法分析器工作量太大，但也有一些语法分析器生成器可以自动构建语法分析表
- 相比之下，LL 语法分析器往往更容易手工编写，但功能较弱（处理的语法较少）
- LR 语法分析技术是已知的最通用的非回溯移入-归约语法分析方法
- LR 语法分析器可以被构造为识别几乎所有可以为其编写 CFG 的编程语言结构
- LR 语法比 LL 语法可以描述更多的语言
 - 回想文法成为 LL(1)的严格条件

When to Shift/Reduce?

STACK	INPUT	ACTION
\$	id₁ * id₂ \$	shift
\$ id₁	* id₂ \$	reduce by $F \rightarrow \text{id}$
\$ F	* id₂ \$	reduce by $T \rightarrow F$
\$ T	* id₂ \$	shift
\$ T *	id₂ \$	shift
\$ T * id₂	\$	reduce by $F \rightarrow \text{id}$
\$ T * F	\$	reduce by $T \rightarrow T * F$
\$ T	\$	reduce by $E \rightarrow T$
\$ E	\$	accept

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid \text{id}$

Parsing input **id₁** * **id₂**

How does a shift/reduce parser know that T on stack top is not a handle (so the right action is to shift, not reducing T to E)?



移入-归约语法分析器如何知道 T 栈顶上没有句柄（所以正确的操作是移入，而不是归约 T 到 E)?

LR(0) Items (LR(0) 项)

- An LR parser makes shift-reduce decisions by **maintaining states** to keep track of **what have been seen** in a parse
- An **LR(0) item** (item for short) is a **production with a dot** at some position of the body, indicating how much we have seen at a given point in the parsing process
 - $A \rightarrow \cdot XYZ$ $A \rightarrow X \cdot YZ$ $A \rightarrow XY \cdot Z$ $A \rightarrow XYZ \cdot$
 - $A \rightarrow X \cdot YZ$: we have just seen on the input a string derivable from X and we hope to see a string derivable from YZ next
- States**: sets of LR(0) items (**LR(0)项集**)

The production $A \rightarrow \epsilon$ generates only one item $A \rightarrow \cdot$

一个 LR 语法分析器通过维护一些状态来跟踪在语法分析中所处的位置，从而来做出移入-归约的决策。

LR(0)项（简称 item）是产生式再加上一个位于它的体中某处的点，表示在语法分析过程中的给定点上，我们已经看到了一个产生式的哪些部分。

$A \rightarrow \cdot XYZA \rightarrow X \cdot YZA \rightarrow XY \cdot ZA \rightarrow XYZ \cdot$

$A \rightarrow X \cdot YZ$: 我们刚刚在输入中看到了一个可以由 X 推导得到的字符串，我们希望接下来看到一个能从 YZ 推导得到的字符串

状态：LR(0)项集

产生式 $A \rightarrow \epsilon$ 仅生成一个 item $A \rightarrow \cdot$

Canonical LR(0) Collection

- One collection of sets of LR(0) items, called the **canonical LR(0) collection** (规范LR(0)项集族), provides the basis for constructing a DFA that is used to make parsing decisions
- To construct canonical LR(0) collection for a grammar, we need to define:
 - An augmented grammar (增广文法)
 - Two functions:
 - (1) CLOSURE of item sets (项集闭包)
 - (2) GOTO

规范 LR(0)项集族

LR(0)项集合的一个集合，称为规范 LR(0)项集族，为构建用于做出语法分析决定的 DFA 提供了基础

为了构造文法的 LR(0)项集规范族，我们需要定义：

- 增广文法
- 两个函数：(1) 项集闭包的 CLOSURE 和 (2) GOTO

Augmented Grammar

- Augmenting a grammar G with start symbol S
 - Introduce a new start symbol S' to replace S
 - Add a new production $S' \rightarrow S$
- Obviously, $L(G) = L(G')$
- **Benefit:** With the augmentation, acceptance occurs when and only when the parser is about to reduce by $S' \rightarrow S$
 - Otherwise, acceptance could occur at many points since there may be multiple S -productions

增广文法

增广一个以 S 为开始符号的文法 G :

- 引入一个新开始符号 S' 代替 S
- 增加产生式 $S' \rightarrow S$

明显地 $L(G) = L(G')$

优点：通过增广，只有当语法分析器使用 $S' \rightarrow S$ 进行归约时输入符号串被接受。否则，输入符号串可能在多处被接受，因为可能存在多个 S -产生式

Closure of Item Sets

- If I is a set of items for a grammar G , then $\text{CLOSURE}(I)$ is the set of items constructed from I by the two rules
 1. Initially, add every item in I to $\text{CLOSURE}(I)$
 2. If $A \rightarrow \alpha \cdot B\beta$ is in $\text{CLOSURE}(I)$ and $B \rightarrow \gamma$ is a production, then add the item $B \rightarrow \cdot \gamma$ to $\text{CLOSURE}(I)$, if it is not already there. Apply this rule until no more new items can be added to $\text{CLOSURE}(I)$
- **Intuition:** $A \rightarrow \alpha \cdot B\beta$ indicates that we hope to see a substring derivable from $B\beta$. This substring will have a prefix derivable from B . Therefore, we add items for all B -productions.

项集闭包

如果 I 是文法 G 的一组项集, 那么 $\text{CLOSURE}(I)$ 就是按照下面两条规则构造的项集:

1. 一开始, 将 I 中的各个项加入到 $\text{CLOSURE}(I)$
2. 如果 $A \rightarrow \alpha \cdot B\beta$ 在 $\text{CLOSURE}(I)$ 中, $B \rightarrow \gamma$ 是一个产生式, 并且 $B \rightarrow \cdot \gamma$ 不在 $\text{CLOSURE}(I)$ 中, 那么将 $B \rightarrow \cdot \gamma$ 加入 $\text{CLOSURE}(I)$ 中。不断应用这条规则, 直到没有新项可以加入到

直观地讲: $A \rightarrow \alpha \cdot B\beta$ 表示我们希望看到从一个能够从 $B\beta$ 推导得到的子串。这个子串的某个前缀可以从 B 推导得到。因此, 我们将所有 B -产生式加入到项集。

Algorithm for CLOSURE(I)

// the earlier natural language description is already clear enough

```
SetOfItems CLOSURE( $I$ ) {  
     $J = I$ ;  
    repeat  
        for ( each item  $A \rightarrow \alpha \cdot B \beta$  in  $J$  )  
            for ( each production  $B \rightarrow \gamma$  of  $G$  )  
                if (  $B \rightarrow \cdot \gamma$  is not in  $J$  )  
                    add  $B \rightarrow \cdot \gamma$  to  $J$ ;  
    until no more items are added to  $J$  on one round;  
    return  $J$ ;  
}
```

Example

$E \rightarrow E + T \mid T$
$T \rightarrow T * F \mid F$
$F \rightarrow (E) \mid \text{id}$

- Augmented grammar

▪ $E' \rightarrow E$ $E \rightarrow E + T \mid T$ $T \rightarrow T * F \mid F$ $F \rightarrow (E) \mid \text{id}$

- Computing the closure of the item set $\{[E' \rightarrow \cdot E]\}$

- Initially, $[E' \rightarrow \cdot E]$ is in the closure
- Add $[E \rightarrow \cdot E + T]$ and $[E \rightarrow \cdot T]$ to the closure
- Add $[T \rightarrow \cdot T * F]$ and $[T \rightarrow \cdot F]$ to the closure
- Add $[F \rightarrow \cdot (E)]$ and $[F \rightarrow \cdot \text{id}]$ to the closure (reached fixed point)

The Function GOTO

$E \rightarrow E + T \mid T$ $T \rightarrow T * F \mid F$ $F \rightarrow (E) \mid \text{id}$
--

- **GOTO(I, X)**, where I is a set of items and X is a grammar symbol, is defined to be the closure of the set of all items $[A \rightarrow \alpha X \cdot \beta]$ such that $[A \rightarrow \alpha \cdot X \beta]$ is in I
 - $\text{CLOSURE}(\{[A \rightarrow \alpha X \cdot \beta] \mid [A \rightarrow \alpha \cdot X \beta] \in I\})$
- **Example:** Computing $\text{GOTO}(I, +)$ for $I = \{[E' \rightarrow E \cdot], [E \rightarrow E \cdot + T]\}$
 - There is only one item $[E \rightarrow E \cdot + T]$ in which $+$ follows $\cdot$
 - Then compute the $\text{CLOSURE}(\{[E \rightarrow E \cdot + T]\})$, which contains
 - $[E \rightarrow E + \cdot T]$
 - $[T \rightarrow \cdot T * F] \downarrow [T \rightarrow \cdot F]$
 - $[F \rightarrow \cdot (E)] \downarrow [F \rightarrow \cdot \text{id}]$

GOTO 函数

返回项目集 I 对应于文法符号 X 的后继项目集闭包

Constructing Canonical LR(0) Collection

```

void items( $G'$ ) {
     $C = \{\text{CLOSURE}(\{[S' \rightarrow \cdot S]\})\};$   $\longrightarrow$  Initial item set
    repeat
        for ( each set of items  $I$  in  $C$  )
            for ( each grammar symbol  $X$  )
                if (  $\text{GOTO}(I, X)$  is not empty and not in  $C$  )
                    add  $\text{GOTO}(I, X)$  to  $C$ ;
    until no new sets of items are added to  $C$  on a round;
}

```

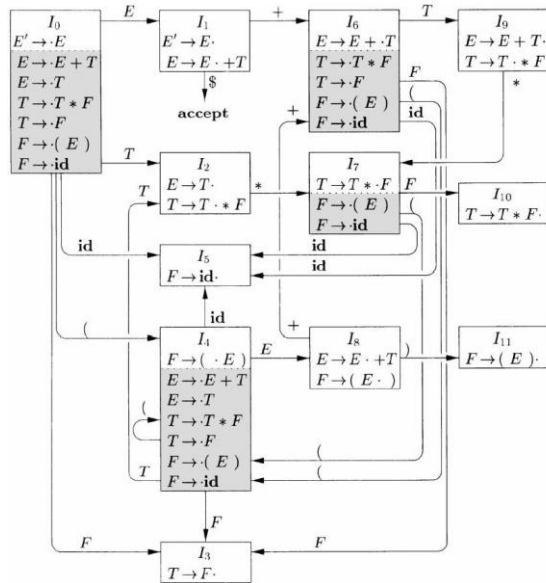
Iteratively find all possible GOTO targets (corresponding to states in the automaton for parsing)

构造规范 LR(0)项集族

迭代查找所有可能的 GOTO 目标（对应于用于语法分析的自动机中的状态）

Example

The canonical LR(0) collection for the grammar below is $\{I_0, I_1, \dots, I_{11}\}$



- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

LR(0) Automaton

- The central idea behind “Simple LR”, or SLR, is constructing the LR(0) automaton from the grammar
 - The states are the item sets in the canonical LR(0) collection
 - The transitions are given by the GOTO function
 - The start state is $CLOSURE(\{S' \rightarrow \cdot S\})$

LR(0)自动机

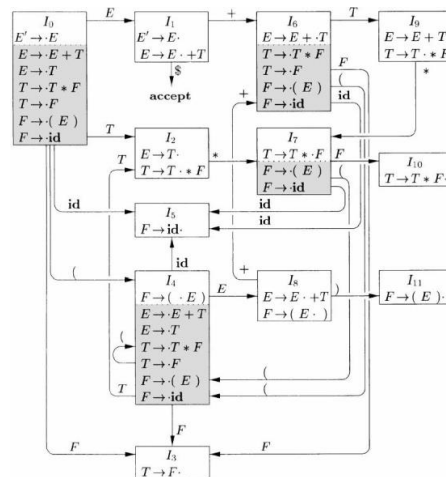
“简单 LR 语法分析技术”（即 SLR 分析技术）的中心思想是根据文法构造出 LR(0)自动机。

- 这个自动机的状态是规范 LR(0)项集族
- 它的转换由 GOTO 函数给出
- 开始状态是 $CLOSURE(\{S' \rightarrow \cdot S\})$

The Use of LR(0) Automaton

Helps make shift-reduce decisions:

- Suppose that the string γ of **grammar symbols** takes the automaton from the start **state 0** to some **state j**
- **Shift** on next **input symbol a** if **state j** has a **transition on a**
- Otherwise, **reduce**; the items in state $j will tell us which production to use$



LR(0)自动机的使用

帮助做出移入-归约决定：

- 假设文法符号 γ 使自动机从开始状态 0 运行到某个状态 j
- 如果下一个输入符号为 a 且状态 j 有一个在 a 上的转换，那就移入 a
- 否则，归约；状态 j 的项将会告诉我们使用哪个产生式进行归约

Example: Parsing **id * id** (1)

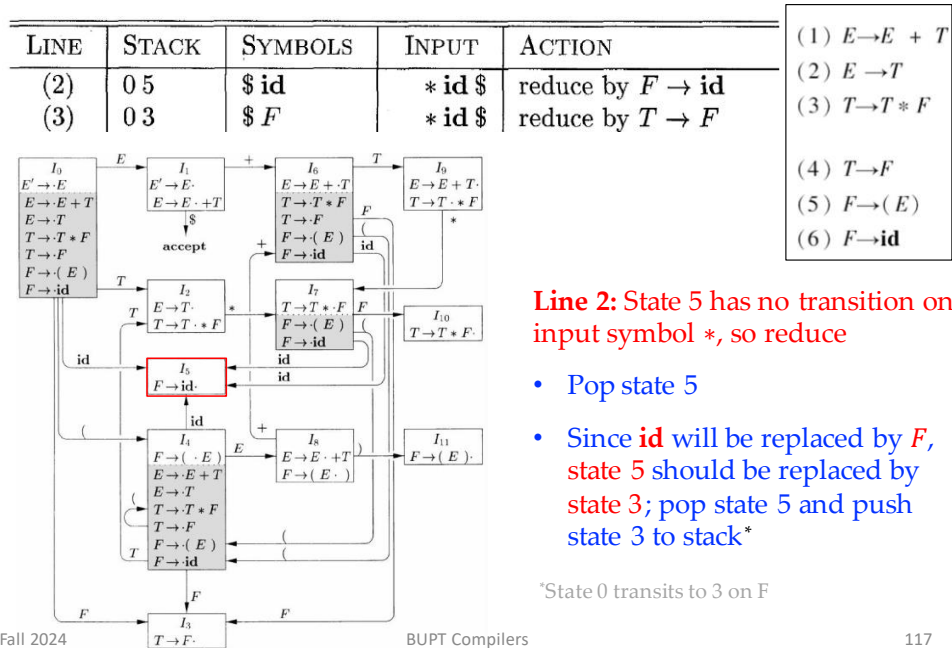
- We only keep states in the stack; grammar symbols can be recovered from the states

LINE	STACK	SYMBOLS	INPUT	ACTION
(1)	0	\$	id * id \$	shift to 5
(2)	0 5	\$ id	* id \$	reduce by $F \rightarrow \text{id}$
(3)	0 3	\$ F	* id \$	reduce by $T \rightarrow F$
(4)	0 2	\$ T	* id \$	shift to 7
(5)	0 2 7	\$ T *	id \$	shift to 5
(6)	0 2 7 5	\$ T * id	\$	reduce by $F \rightarrow \text{id}$
(7)	0 2 7 10	\$ T * F	\$	reduce by $T \rightarrow T * F$
(8)	0 2	\$ T	\$	reduce by $E \rightarrow T$
(9)	0 1	\$ E	\$	accept

栈只保存状态，文法符号可以从相应状态中获取

从状态 0 开始，移入 id，进入状态 5，id 出栈规约为 F，F 进栈，状态 5 出栈，状态 3 进栈；

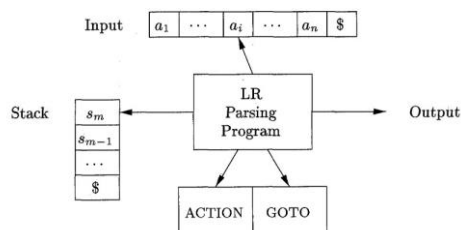
Example: Parsing $\text{id} * \text{id}$ (2)



在第 2 行，状态 5 已压入栈中，从状态 5 出发没有输入 $*$ 上的转换，因此选择归约。
因为 id 将会被 F 代替，状态 5 将会被状态 3 代替，所以弹出状态 5，压入状态 3。

LR Parser Structure

- An LR parser consists of an **input**, an **output**, a **stack**, a **driver program**, and a **parsing table** (ACTION + GOTO)
- The driver program is the same for all LR parsers; only the parsing table changes from one parser to another
- The stack holds a sequence of states
 - In SLR, the stack holds states from the LR(0) automaton
- The parser decides the next action based on (1) the state at the top of the stack and (2) the terminal read from the input buffer



LR 语法分析器结构

LR 语法分析器由一个输入、一个输出、一个栈、一个驱动程序和一个语法分析表 (ACTION+GOTO) 组成

所有 LR 语法分析器的驱动程序都是相同的；只有语法分析表随语法分析器的不同而变化

栈保存一个状态序列

在 SLR 中，栈保存来自 LR(0)自动机的状态

语法分析器根据 (1) 栈顶部的状态和 (2) 从输入缓冲区读取的终结符来决定下一个动作

Parsing Table: ACTION + GOTO

- The **ACTION** function takes two arguments: (1) a state i and (2) a terminal a (or \$)
- **ACTION** $[i, a]$ can have one of the four forms of values:
 - **Shift j** : shift input a to the stack, but uses state j to represent a
 - **Reduce $A \rightarrow \beta$** : reduce β on the top of the stack to head A
 - **Accept**: The parser accepts the input and finishes parsing
 - **Error**: syntax errors exist
- The **GOTO** function is extended from the one defined on sets of items to states: if $\text{GOTO}(\bar{l}_i, A) = \bar{l}_j$, then $\text{GOTO}(i, A) = j$

LR 语法分析表: ACTION + GOTO

ACTION 函数接受两个参数: (1) 状态 i 和 (2) 终结符 a (或\$)

ACTION $[i, a]$ 可以具有以下四种形式的值之一:

- 移入 j : 其中 j 是一个状态, 把输入符号 a 移入栈中, 但使用状态 j 代表 a
- 归约 $A \rightarrow \beta$: 将栈顶的 β 归约为产生式头 A
- 接受: 语法分析器接受输入并完成语法分析过程

- 报错：存在语法错误

将定义在项集上的 GOTO 函数扩展为定义在状态集上的函数：如果 $\text{GOTO}(I_i, A) = I_j$ ，那么 GOTO 也把状态 i 和一个非终结符 A 映射到状态 j

LR Parser Configurations (态势)

- “**Configuration**” is notation for representing the complete state of the parser. A *configuration* is a pair:

Stack contents (top on the right) $\leftarrow (s_0 s_1 \dots s_m, a_i a_{i+1} \dots a_n \$) \rightarrow$ Remaining input

- By construction, each state (except s_0) in an LR parser corresponds to a set of items and a grammar symbol (the symbol that leads to the state transition, i.e., the symbol on the incoming edges)
 - Suppose X_i is the grammar symbol for state s_i
 - Then $X_0 X_1 \dots X_m a_i a_{i+1} \dots a_n$ is a **right-sentential form** (assume no errors)

LR 语法分析器的态势

“态势”是表示语法分析器完整状态的符号。态势是一个形如： $(s_0 s_1 \dots s_m, a_i a_{i+1} \dots a_n \$)$ 的对

第一个分量是栈中的内容，第二个分量是余下的输入。

根据结构，每个状态（除 s_0 ）对应于一组项和文法符号（导致状态转换的符号，即传入边上的符号）

- 假设 X_i 是状态 s_i 的文法符号
- 那么 $X_0 X_1 \dots X_m a_i a_{i+1} \dots a_n$ 是最右句型（假设没有错误）

Behavior of the LR Parser

- For the configuration $(s_0 s_1 \dots s_m, a_i a_{i+1} \dots a_n \$)$, the LR parser checks $\text{ACTION}[s_m, a_i]$ in the parsing table to decide the parsing action
 - **shift s** : shift the next state s onto the stack, entering the configuration $(s_0 s_1 \dots s_m s, a_{i+1} \dots a_n \$)$
 - **reduce $A \rightarrow \beta$** : execute a reduce move, entering the configuration $(s_0 s_1 \dots s_{m-r} s, a_i a_{i+1} \dots a_n \$)$, where $r = \text{length of } \beta$, and $s = \text{GOTO}(s_{m-r}, A)$
 - **accept**: parsing is completed
 - **error**: the parser has found an error and calls an error recovery routine

LR-Parsing Algorithm

- **Input**: The parsing table for a grammar G and an input string ω
- **Output**: If ω is in $L(G)$, the reduction steps of a bottom-up parse for ω ; otherwise, an error indication
- **Initial configuration**: $(s_0, \omega \$)$

```
let  $a$  be the first symbol of  $w\$$ ;  
while(1) { /* repeat forever */  
    let  $s$  be the state on top of the stack;  
    if (  $\text{ACTION}[s, a] = \text{shift } t$  ) {  
        push  $t$  onto the stack;  
        let  $a$  be the next input symbol;  
    } else if (  $\text{ACTION}[s, a] = \text{reduce } A \rightarrow \beta$  ) {  
        pop  $|\beta|$  symbols off the stack;  
        let state  $t$  now be on top of the stack;  
        push  $\text{GOTO}[t, A]$  onto the stack;  
        output the production  $A \rightarrow \beta$ ;  
    } else if (  $\text{ACTION}[s, a] = \text{accept}$  ) break; /* parsing is done */  
    else call error-recovery routine;  
}
```

Parsing Table Example

STATE	ACTION						GOTO		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow \text{id}$

- s5: shift by pushing state 5 r3: reduce using production No. 3
- GOTO entries for terminals are not listed, can be checked in ACTION part

LR Parsing Example

- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow \text{id}$

- Input string: **id * id + id**

	STACK	SYMBOLS	INPUT	ACTION
(1)	0		id * id + id \$	shift
(2)	0 5	id	* id + id \$	reduce by $F \rightarrow \text{id}$
(3)	0 3	F	* id + id \$	reduce by $T \rightarrow F$
(4)	0 2	T	* id + id \$	shift
(5)	0 2 7	T *	id + id \$	shift
(6)	0 2 7 5	T * id	+ id \$	reduce by $F \rightarrow \text{id}$
(7)	0 2 7 10	T * F	+ id \$	reduce by $T \rightarrow T * F$
(8)	0 2	T	+ id \$	reduce by $E \rightarrow T$
(9)	0 1	E	+ id \$	shift
(10)	0 1 6	E +	id \$	shift
(11)	0 1 6 5	E + id	\$	reduce by $F \rightarrow \text{id}$
(12)	0 1 6 3	E + F	\$	reduce by $T \rightarrow F$
(13)	0 1 6 9	E + T	\$	reduce by $E \rightarrow E + T$
(14)	0 1	E	\$	accept

- Push state 5
- Remove id
- Pop three states 2, 7, 10
- Push state 2 (GOTO[0, T] = 2)

Constructing SLR-Parsing Tables (1)

- The SLR-parsing table for a grammar G can be constructed based on the LR(0) item sets and LR(0) automaton
 1. Construct the canonical LR(0) collection $\{I_0, I_1, \dots, I_n\}$ for the augmented grammar G'
 2. State i is constructed from I_i . ACTION can be determined as follows:
 - If $[A \rightarrow \alpha \cdot a\beta]$ is in I_i and $\text{GOTO}[I_i, a] = I_j$, then set $\text{ACTION}[i, a]$ to “shift j ” (here a must be a terminal)
 - If $[A \rightarrow \alpha \cdot]$ is in I_i , then set $\text{ACTION}[i, a]$ to “reduce $A \rightarrow \alpha$ ” for **all a in FOLLOW(A)**; here A may not be S'
 - If $[S' \rightarrow S \cdot]$ is in I_i , then set $\text{ACTION}[i, \$]$ to “accept”
 3. The goto transitions for state i are constructed for all nonterminals A using the rule: If $\text{GOTO}(I_i, A) = I_j$, then $\text{GOTO}(i, A) = j$

构造 SLR 语法分析表

SLR 语法分析表 G 以 LR(0) 项和 LR(0) 自动机为基础。

1. 构造增广文法 G' 的规范 LR(0) 项集族 $\{I_0, I_1, \dots, I_n\}$
2. 根据 I_i 构造得到状态 i ，状态 i 的语法分析动作按照下面的方法决定：
 - 如果 $[A \rightarrow \alpha \cdot a\beta]$ 在 I_i 中并且 $\text{GOTO}[I_i, a] = I_j$ ，那么将 $\text{ACTION}[i, a]$ 设置为“移入 j ”
 - 如果 $[A \rightarrow \alpha \cdot]$ 在 I_i 中，那么对于 **FOLLOW(A)** 中的所有 a ，将设置 $\text{ACTION}[i, a]$ 为“归约 $A \rightarrow \alpha$ ”。这里 A 不等于 S'
 - 如果 $[S' \rightarrow S \cdot]$ 在 I_i 中，那么将设置 $\text{ACTION}[i, \$]$ 为“接受”
3. 状态 i 对于各个非终结符号 A 的转换使用下面的规则构造得到：如果 $\text{GOTO}[I_i, A] = I_j$ ，那么 $\text{GOTO}[i, A] = j$

Constructing SLR-Parsing Tables (2)

4. All entries not defined in steps 2 and 3 are set to “**error**”
5. Initial state is the one constructed from the item set containing $[S' \rightarrow \cdot S]$

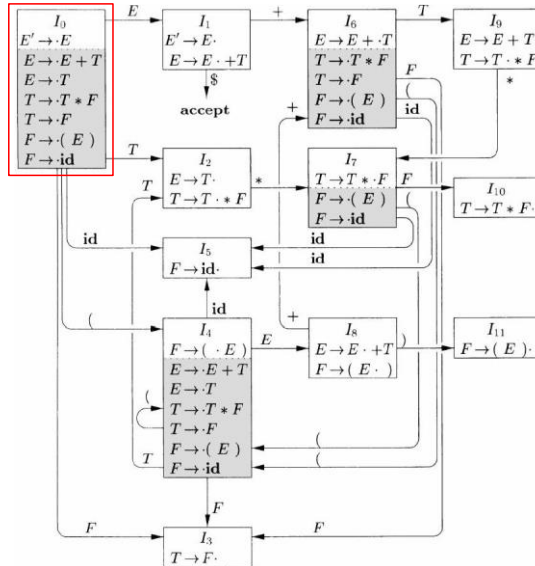
If there is no conflict during the parsing table construction (i.e., multiple actions for a table entry), the grammar is **SLR(1)**

4. 规则 2、3 没有定义的所有条目都设置为 “报错”
5. 语法分析器的初始状态就是根据 $[S' \rightarrow \cdot S]$ 所在项集构造得到的状态

如果在语法分析构造过程中没有冲突（即，一个表项的多个条目），则语法为 SLR

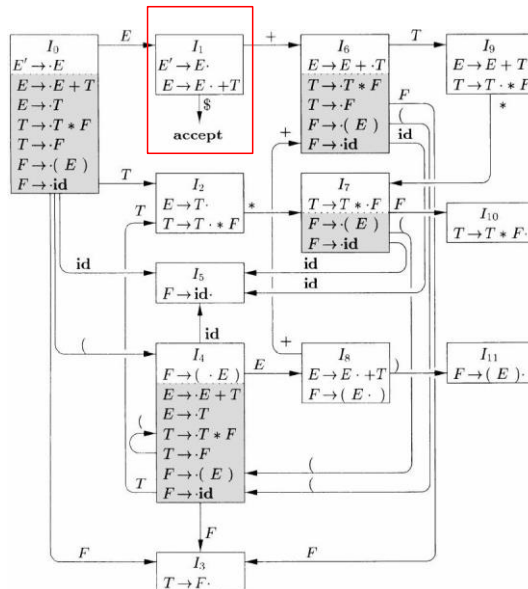
(1)

Example (1)



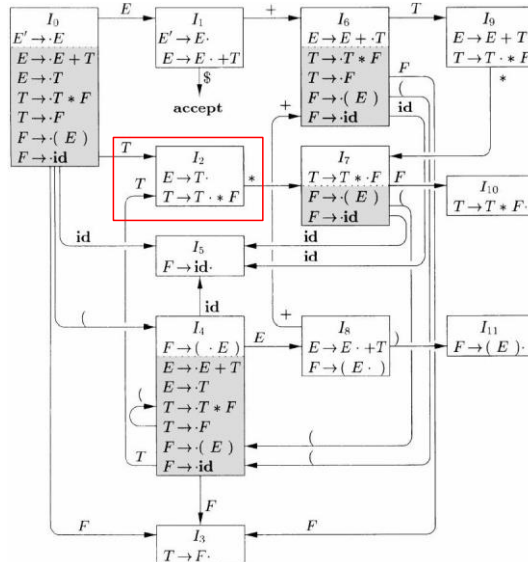
- $\text{ACTION}(0, \text{ }) = s4$ (shift 4)
- $\text{ACTION}(0, \text{id}) = s5$
- $\text{GOTO}[0, E] = 1$
- $\text{GOTO}[0, T] = 2$
- $\text{GOTO}[0, F] = 3$

Example (2)



- $\text{ACTION}(1, +) = s6$
- $\text{ACTION}(1, \$) = \text{accept}$

Example (3)



- ACTION(2,*) = s7
- ACTION(2,\$) = reduce $E \rightarrow T$
- ACTION(2,+) = reduce $E \rightarrow T$
- ACTION(2,)) = reduce $E \rightarrow T$

$\text{FOLLOW}(E) = \{\$, +,)\}$

Non-SLR Grammar

- Grammar
 - $S \rightarrow L = R \mid R$
 - $L \rightarrow * R \mid \text{id}$
 - $R \rightarrow L$
- For item set I_2 :
 - According to item #1:
ACTION[2,=] is "s6"
 - According to item #2:
ACTION[2,=] is "reduce $R \rightarrow L$ "
($\text{FOLLOW}(R)$ contains =)

$I_0: S' \rightarrow \cdot S$	$I_5: L \rightarrow \text{id} \cdot$
$S \rightarrow \cdot L = R$	
$S \rightarrow \cdot R$	$I_6: S \rightarrow L = \cdot R$
$L \rightarrow \cdot * R$	$R \rightarrow \cdot L$
$L \rightarrow \cdot \text{id}$	$L \rightarrow \cdot * R$
$R \rightarrow \cdot L$	$L \rightarrow \cdot \text{id}$
$I_1: S' \rightarrow S \cdot$	$I_7: L \rightarrow * R \cdot$
$I_2: S \rightarrow L \cdot = R$	$I_8: R \rightarrow L \cdot$
$R \rightarrow L \cdot$	$I_9: S \rightarrow L = R \cdot$
$I_3: S \rightarrow R \cdot$	
$I_4: L \rightarrow * \cdot R$	
$R \rightarrow \cdot L$	
$L \rightarrow \cdot * R$	
$L \rightarrow \cdot \text{id}$	

This grammar is not ambiguous

CLR and LALR will succeed on a larger collection of grammars, including the above one. However, there exist unambiguous grammars for which every LR parser construction method will encounter conflicts.

CLR 和 LALR 将在更大的文法集合上取得成功，包括上面的文法集合。然而，存在着明确的文法，对于这些文法，每个 LR 语法分析器构造方法都会遇到冲突。

Weakness of the SLR Method

- In SLR, the state i calls for reduction by $A \rightarrow \alpha$ if the item set I_i contains item $[A \rightarrow \alpha \cdot]$ and input symbol a is in $\text{FOLLOW}(A)$
- In some situations, after reduction, **the content $\beta\alpha$ on stack top would become βA that cannot be followed by a in any right-sentential form**

在 SLR 中，状态 i 要求归约 $A \rightarrow \alpha$ ，如果项集 I_i 包含项 $[A \rightarrow \alpha \cdot]$ ，同时输入符号 a 在 $\text{FOLLOW}(A)$ 中

在某些情况下，归约后 $\beta\alpha$ 在栈顶上会变成 βA ，后面跟着的不是 a 的任何最右句型

Example: Parsing $\text{id} = \text{id}$

- $S \rightarrow L = R \mid R$
- $L \rightarrow * R \mid \text{id}$
- $R \rightarrow L$

I_0 : $S' \rightarrow \cdot S$
 $S \rightarrow \cdot L = R$
 $S \rightarrow \cdot R$
 $L \rightarrow \cdot * R$
 $L \rightarrow \cdot \text{id}$
 $R \rightarrow \cdot L$

I_1 : $S' \rightarrow S \cdot$

I_2 : $S \rightarrow L \cdot = R$
 $R \rightarrow L \cdot$

I_3 : $S \rightarrow R \cdot$

I_4 : $L \rightarrow * \cdot R$
 $R \rightarrow \cdot L$
 $L \rightarrow \cdot * R$
 $L \rightarrow \cdot \text{id}$

I_5 : $L \rightarrow \text{id} \cdot$

I_6 : $S \rightarrow L = \cdot R$
 $R \rightarrow \cdot L$
 $L \rightarrow \cdot * R$
 $L \rightarrow \cdot \text{id}$

I_7 : $L \rightarrow * R \cdot$

I_8 : $R \rightarrow L \cdot$

I_9 : $S \rightarrow L = R \cdot$

Stack	Symbols	Input	Action
\$0		id = id	Shift 5
\$05	id	= id	Reduce $L \rightarrow \text{id}$
\$02	L	= id	Suppose reduce $R \rightarrow L$
\$03	R	= id	Error!

* Cannot shift, cannot reduce since $\text{FOLLOW}(S) = \{\$ \}$

Problem: SLR reduce too casually

How to know if a reduction is a good move?
 Utilize the next input symbol to precisely
 determine whether to call for a reduction.

如何知道归约是否是一个好的决定？利用下一个输入符号精确地确定是否要求归约。

LR(1) Item

- **Idea:** Carry more information in the state to rule out some invalid reductions (**splitting LR(0) states**)
- General form of an LR(1) item: $[A \rightarrow \alpha \cdot \beta, a]$
 - $A \rightarrow \alpha\beta$ is a production and a is a terminal or $\$$
 - “1” refers to the length of the 2nd component: the **lookahead** (向前看字符)*
 - The lookahead symbol has no effect **if β is not ϵ** since it only helps determine whether to reduce (a will be inherited during state transitions)
 - An item of the form $[A \rightarrow \alpha \cdot, a]$ calls for a reduction by $A \rightarrow \alpha$ **only if the next input symbol is a** (the set of such a 's is a **subset** of $\text{FOLLOW}(A)$)

*: LR(0) items do not have lookahead symbols, and hence they are called LR(0)

规范 LR(1)项

在状态中携带更多信息以排除一些无效的归约（通过分裂 LR (0) 状态）

LR(1)项的一般形式: $[A \rightarrow \alpha \cdot \beta, a]$

- $A \rightarrow \alpha\beta$ 是一个产生式, a 是一个终结符或右端结束标记
- 1 指的是第二个分量 a 的长度, 第二个分量 a 叫做向前看字符
- 当 β 不为空时, 向前看字符没有任何作用因为它决定是否归约
- 一个形如 $[A \rightarrow \alpha \cdot, a]$ 的项只有在下一个输入符号等于 a 时, 才会按照 $A \rightarrow \alpha$ 进行归约。这样的 a 的集合总是的 FOLLOW(A) 子集

$A \rightarrow \alpha\beta$ 是一个产生式和 a 是终端还是\$ “1” 是指第二个组成部分的长度: 展望(向前看字符)*如果 β 不是 ϵ 因为它只帮助确定是否减少(a 将在状态转换期间继承) 表单中的一
项 $A \rightarrow \alpha \cdot, a$ 要求减少 $A \rightarrow \alpha$ 仅当下一个输入符号 a (a' s 是 FOLLOW 的子集(A))*:

LR (0) 项没有先行符号, 因此它们被称为 LR (0)

Constructing LR(1) Item Sets (1)

- Constructing the collection of LR(1) item sets is essentially the same as constructing the canonical collection of LR(0) item sets. The only differences lie in the CLOSURE and GOTO functions.

```
SetOfItems CLOSURE(I) {  
    J = I;  
    repeat  
        for ( each item  $[A \rightarrow \alpha \cdot B \beta, a]$  in I )  
            for ( each production  $B \rightarrow \gamma$  in  $G'$  )  
                for ( each terminal  $b$  in FIRST( $\beta a$ ) )  
                    add  $[B \rightarrow \gamma, b]$  to set J;  
    until no more items are added to J;  
    return J;  
}
```

```
SetOfItems CLOSURE(I) {  
    J = I;  
    repeat  
        for ( each item  $A \rightarrow \alpha \cdot B \beta$  in J )  
            for ( each production  $B \rightarrow \gamma$  of G )  
                if (  $B \rightarrow \gamma$  is not in J )  
                    add  $B \rightarrow \gamma$  to J;  
    until no more items are added to J on one round;  
    return J;  
}
```

It only generates the new item $[B \rightarrow \gamma, b]$ from $[A \rightarrow \alpha \cdot B \beta, a]$ if b is in FIRST(βa)

构造 LR (1) 项集族本质上与构造规范 LR (0) 项集族相同。唯一的区别在于 CLOSURE 和 GOTO 函数。

Why b should be in $\text{FIRST}(\beta a)$?

- **An informal analysis:**

- The lookahead b decides when to reduce γ to B
- Suppose the algorithm also generates an item for b that is NOT in $\text{FIRST}(\beta a)$
- At a certain point, when we see γ on stack top and b as next input, we would reduce γ to B ($[B \rightarrow \gamma \cdot, b]$ must come from $[B \rightarrow \cdot \gamma, b]$)
- Since we generate $[B \rightarrow \cdot \gamma, b]$ from $[A \rightarrow \alpha \cdot B \beta, a]$ during closure computation, we hope to see $\alpha B \beta$ at stack top sometime after reducing γ to B and reduce using $A \rightarrow \alpha B \beta$ when a is the next input symbol
- Unfortunately, this is impossible when b is not in $\text{FIRST}(\beta a)$
- **Then why generate $[B \rightarrow \cdot \gamma, b]$ from $[A \rightarrow \alpha \cdot B \beta, a]$???**

Constructing LR(1) Item Sets (2)

- Constructing the collection of LR(1) item sets is essentially the same as constructing the canonical collection of LR(0) item sets. The only differences lie in the CLOSURE and **GOTO** functions.

```
SetOfItems GOTO( $I, X$ ) {  
    initialize  $J$  to be the empty set;  
    for ( each item  $[A \rightarrow \alpha \cdot X \beta, a]$  in  $I$  )  
        add item  $[A \rightarrow \alpha X \cdot \beta, a]$  to set  $J$ ;  
    return CLOSURE( $J$ );  
}
```

GOTO(I, X) in LR(0) item sets:

The closure of the set of all items $[A \rightarrow \alpha X \cdot \beta]$ such that $[A \rightarrow \alpha \cdot X \beta]$ is in I .

The lookahead symbols are passed to new items from existing items

构造 LR (1) 项集族本质上与构造规范 LR (0) 项集族相同。唯一的区别在于 CLOSURE 和 GOTO 函数。

向前看符号从现有项传递到新项

Constructing LR(1) Item Sets (3)

```
void items( $G'$ ) {  
     $C = \{\text{CLOSURE}(\{[S' \rightarrow \cdot S]\})\};$   
    repeat  
        for ( each set of items  $I$  in  $C$  )  
            for ( each grammar symbol  $X$  )  
                if ( GOTO( $I, X$ ) is not empty and not in  $C$  )  
                    add GOTO( $I, X$ ) to  $C$ ;  
    until no new sets of items are added to  $C$  on a round;  
}
```

Constructing the
collection of LR(0)
item sets

```
void items( $G'$ ) {  
    initialize  $C$  to  $\{\text{CLOSURE}(\{[S' \rightarrow \cdot S, \$]\})\};$   
    repeat  
        for ( each set of items  $I$  in  $C$  )  
            for ( each grammar symbol  $X$  )  
                if ( GOTO( $I, X$ ) is not empty and not in  $C$  )  
                    add GOTO( $I, X$ ) to  $C$ ;  
    until no new sets of items are added to  $C$ ;  
}
```

Constructing the
collection of LR(1)
item sets



LR(1) Item Sets Example

- Augmented grammar:

- $S' \rightarrow S \quad S \rightarrow CC \quad C \rightarrow cC \mid d$

- Constructing I_1 item set and GOTO function:

- $I_0 = \text{CLOSURE}([S' \rightarrow \cdot S, \$]) =$

- $\{[S' \rightarrow \cdot S, \$], [S \rightarrow \cdot CC, \$], [C \rightarrow \cdot cC, c/d], [C \rightarrow \cdot d, c/d]\}$

$\text{FIRST}(\$) = \{\$ \}$

$\text{FIRST}(C\$) = \{c, d\}$

- $\text{GOTO}(I_0, S) = \text{CLOSURE}(\{[S' \rightarrow S \cdot, \$]\}) = \{[S' \rightarrow S \cdot, \$]\}$

- $\text{GOTO}(I_0, C) = \text{CLOSURE}(\{[S \rightarrow C \cdot C, \$]\}) =$

- $\{[S \rightarrow C \cdot C, \$], [C \rightarrow \cdot cC, \$], [C \rightarrow \cdot d, \$]\}$

$\text{FIRST}(\$) = \{\$ \}$

- $\text{GOTO}(I_0, c) = \text{CLOSURE}(\{[C \rightarrow c \cdot C, c/d]\}) =$

- $\{[C \rightarrow c \cdot C, c/d], [C \rightarrow \cdot cC, c/d], [C \rightarrow \cdot d, c/d]\}$

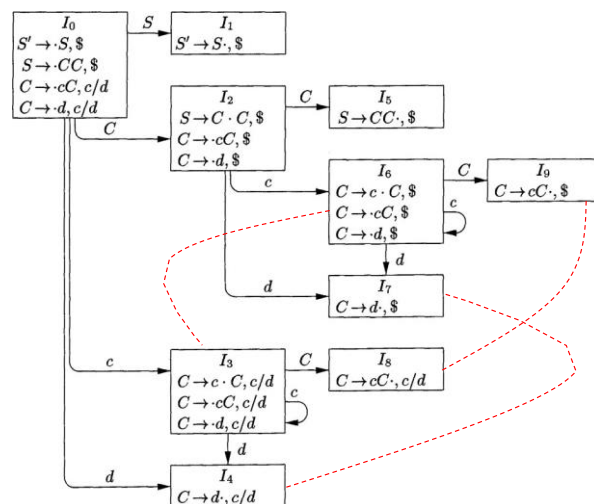
- $\text{GOTO}(I_0, d) = \text{CLOSURE}(\{[C \rightarrow d \cdot, c/d]\}) = \{[C \rightarrow d \cdot, c/d]\}$

The GOTO Graph Example

10 states in total

These states are equivalent if we ignore the lookahead symbols (SLR makes no such distinctions of states):

- I_3 and I_6
- I_4 and I_7
- I_8 and I_9



Constructing Canonical LR(1) Parsing Tables (1)

1. Construct $C' = \{I_0, I_1, \dots, I_n\}$, the collection of LR(1) item sets for the augmented grammar G'
2. State i of the parser is constructed from I_i . Its parsing action is determined as follows:
 - If $[A \rightarrow \alpha \cdot a\beta, b]$ is in I_i and $\text{GOTO}(I_i, a) = I_j$, then set $\text{ACTION}[i, a]$ to “*shift j*.” Here, a must be a terminal.
 - If $[A \rightarrow \alpha \cdot, a]$ is in I_i , $A \neq S'$, then set $\text{ACTION}[i, a]$ to “*reduce $A \rightarrow \alpha$* ”
 - If $[S' \rightarrow S \cdot, \$]$ is in I_i , then set $\text{ACTION}[i, \$]$ to “*accept*”

If any **conflicting actions** result from the above rules, we say the grammar is **not LR(1)**

构造规范 LR(1)语法分析表

1. 构造增广文法 G' 的规范 LR(1)项集族 $\{I_0, I_1, \dots, I_n\}$

2. 根据 I_i 构造得到状态 i , 状态 i 的语法分析动作按照下面的方法决定:

- 如果 $[A \rightarrow \alpha \cdot a\beta, b]$ 在 I_i 中并且 $\text{GOTO}[I_i, a] = I_j$, 那么将 $\text{ACTION}[i, a]$ 设置为 “移入 j ”
- 如果 $[A \rightarrow \alpha \cdot, a]$ 在 I_i 中, 设置 $\text{ACTION}[i, a]$ 为 “归约 $A \rightarrow \alpha$ ”。这里 A 不等于 S'
- 如果 $[S' \rightarrow S \cdot, \$]$ 在 I_i 中, 那么将设置 $\text{ACTION}[i, \$]$ 为 “接受”

如果以上规则导致任何冲突的动作, 我们就说这个文法不是 LR(1)的。

Constructing Canonical LR(1) Parsing Tables (2)

3. The goto transitions for state i are constructed from all nonterminals A using the rule: If $\text{GOTO}(I_i, A) = I_j$, then $\text{GOTO}(i, A) = j$
4. All entries not defined in steps (2) and (3) are made “error”
5. The initial state of the parser is the one constructed from the set of items containing $[S' \rightarrow \cdot S, \$]$

3. 状态 i 对于各个非终结符号 A 的转换使用下面的规则构造得到：如果 $\text{GOTO}[I_i, A] = I_j$ ，那么 $\text{GOTO}[i, A] = j$
4. 规则 2、3 没有定义的所有条目都设置为“报错”
5. 语法分析器的初始状态就是根据 $[S' \rightarrow \cdot S, \$]$ 所在项集构造得到的状态

LR(1) Parsing Table Example

- **Grammar:**

- $S' \rightarrow S$
- $S \rightarrow CC$
- $C \rightarrow cC \mid d$

- These pairs of states can be seen as being split from the corresponding LR(0) states:

- (3, 6)
- (4, 7)
- (8, 9)

STATE	ACTION			GOTO	
	<i>c</i>	<i>d</i>	\$	<i>S</i>	<i>C</i>
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

Error Recovery in LR Parsing

- An LR parser should be able to handle errors:
 - Report the precise location of an error
 - Recover from an error and continue with the parsing
- Two typical error recovery strategies
 - [Panic-mode recovery \(恐慌模式\)](#)
 - [Phrase-level recovery \(短语层次的恢复\)](#)

LR 语法分析器应该能够处理错误：

- 报告错误的精确位置
- 从错误中恢复并继续分析

两种典型的错误

- 恐慌模式
- 短语层次的恢复

Panic-Mode Recovery

- **Basic idea:** Discard zero or more input symbols until a synchronization token (同步词法单元) is found
- **Rationale:**
 - The parser always looks for a prefix of the input that can be derivable from a non-terminal A
 - When there is an error, it means it is impossible to find such a prefix
 - If errors only occur in the part related to A , we can skip the part by looking for a symbol that can legitimately follow A
 - **Example:** If A is *stmt*, then the synchronization symbol can be a semicolon

恐慌模式

基本思想：丢弃零个或多个输入符号，直到同步词法单元已找到

原理：

- 语法分析器总是查找可以从非终结符 A 推导的输入的前缀
- 当出现错误时，意味着无法找到这样的前缀
- 如果错误仅发生在与 A 相关的部分，我们可以通过寻找一个可以合法跟随的符号

来跳过这一部分

- 示例：如果 A 是 $stmt$ ，则同步词法单元可以是分号

Phrase-Level Recovery

- **Basic idea:**

- Examine each error entry in the parsing table and decide the most likely programmer error that would give rise to the error
- Modify the top of the stack or first input symbols and issue messages to programmers

STATE	ACTION						GOTO
	id	+	*	(	)	\$	
0	s3	e1	e1	s2	e2	e1	1
1	e3	s4	s5	e3	e2	acc	
2	s3	e1	e1	s2	e2	e1	6
3	r4	r4	r4	r4	r4	r4	
4	s3	e1	e1	s2	e2	e1	7
5	s3	e1	e1	s2	e2	e1	8
6	e3	s4	s5	e3	s9	e4	
7	r1	r1	s5	r1	r1	r1	
8	r2	r2	r2	r2	r2	r2	
9	r3	r3	r3	r3	r3	r3	

Example phrase-level recovery:

- Remove the right) from the input;
- Issue diagnostic “unbalanced right parenthesis”.

短语层次的恢复

基本思路：

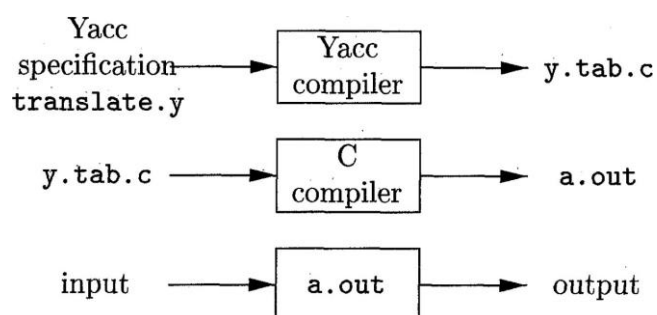
- 检查语法分析表中的每个错误条目，并确定最有可能导致错误的程序员错误
- 修改栈顶部或第一个输入符号，并向程序员发出消息

短语层次的恢复示例：

- 删除右侧的)
- 提示“不匹配的右括号”。

The Parser Generator YACC/BISON

- Yacc: yet another compiler-compiler
- Bison: an extension and improvement of Yacc



本节将介绍如何使用语法分析器生成工具来帮助构造一个编译器的前端，我们将使用 LALR 语法分析生成工具 Yacc 作为讨论的基础。Yacc，又一个编译器的编译器

Yacc 在 UNIX 系统中是以命令的方式出现的，它已经用于实现多个编译器产品。Bison 是 GNU 项目的一部分，是对 Yacc 的扩展和改进。

Structure of YACC Source Programs

- **Declarations (声明)**

- Ordinary C declarations
- Grammar tokens

- **Translation rules (翻译规则)**

- Rule = a grammar production + the associated semantic action

- **Supporting C routines (辅助性C语言例程)**

- Directly copied to `y.tab.c`
- Can be invoked in the semantic actions
- `yylex()` must be provided, which returns tokens
- Other procedures such as error recovery routines may be provided

declarations %% translation rules %% supporting C routines
--

声明

- 通常的 C 声明
- 对词法单元的声明

翻译规则

- 翻译规则 = 产生式 + 语义动作

辅助性 C 语言例程

- 将会被直接拷贝至 `y.tab.c`
- 可以在语义动作中调用
- 必须提供 `yylex()` 这一词法分析器，它将返回一个由词法单元和相关属性组成的

词法单元

- 可以添加错误恢复例程等其他过程

Translation Rules

$$\begin{array}{lcl} \langle \text{head} \rangle & : & \langle \text{body} \rangle_1 \quad \{ \langle \text{semantic action} \rangle_1 \} \\ & | & \langle \text{body} \rangle_2 \quad \{ \langle \text{semantic action} \rangle_2 \} \\ & & \dots \\ & | & \langle \text{body} \rangle_n \quad \{ \langle \text{semantic action} \rangle_n \} \\ & ; & \end{array}$$

- The first head is taken to be the **start symbol**
- A semantic action is a sequence of C statements
 - $\$ \$$ refers to the attribute value associated with the nonterminal of the head
 - $\$ i$ refers to the value associated with i th grammar symbol of the body
- A semantic action is performed when we reduce by the associated production
 - We can compute a value for $\$ \$$ in terms of the $\$ i$'s

翻译规则

第一个产生式头符号被视为开始符号

一个语义动作是一个 C 语句的序列

- $\$ \$$ 表示和相应产生式头部的非终结符关联的属性值
- $\$ i$ 表示和相应产生式体中第 i 个文法符号关联的属性值

当我们按照一个产生式归约时就会执行该产生式相关联的语义动作

- 我们通常根据 $\$ i$ 的值来计算 $\$ \$$ 的值

YACC Source Program Example

```
%{
#include <ctype.h>
%}

%token DIGIT

%%
line : expr '\n'      { printf("%d\n", $1); }
    ;
expr : expr '+' term  { $$ = $1 + $3; }
    | term
    ;
term : term '*' factor { $$ = $1 * $3; }
    | factor
    ;
factor : '(' expr ')' { $$ = $2; }
      | DIGIT
      ;

%%
yylex() {
    int c;
    c = getchar();
    if (isdigit(c)) {
        yylval = c-'0';
        return DIGIT;
    }
    return c;
}
```

Fall 2024

BUPT Compilers

152

Conflicts Resolution in YACC

- **Default strategy:**
 - **Shift/reduce conflicts:** always shift
 - **Reduce/reduce conflicts:** reduce with the production listed first
- **Specifying the precedence and associativity of terminals**
 - **Associativity:** %left, %right, %nonassoc
 - **Shift a /reduce $A \rightarrow a$ conflict:** compare the precedence of a and $A \rightarrow a$ (use associativity when precedence is not enough)
 - The declaration order of terminals determine their precedence
 - The precedence of a production is equal to the precedence its rightmost terminal. It can also be specified using %prec<terminal> which defines the precedence of the production to be the same as the terminal

Fall 2024

BUPT Compilers

153

Yacc 的冲突解决

默认策略：

- 移入/归约冲突：始终移入
- 归约/归约冲突：归约第一个列出的产生式

指定终结符的优先级和结合性：

- 结合性：%left、%right、%nonassoc
- 移入 a /归约 $A \rightarrow a$ 冲突：比较的优先级 a 和 $A \rightarrow a$ （优先级不够时使用结合性）

性）

- 终结符的声明顺序决定了它们的优先级
- 一个产生式的优先级等于它最右边终结符的优先级。也可以使用

用%prec<terminal>指定，它将产生式的优先级定义为与终结符相同

Error Recovery in YACC

- In YACC, error recovery uses a form of error productions
 - General form: $A \rightarrow \text{error } \alpha$
 - The users can decide which nonterminals (e.g., those generating expressions, statements, blocks, etc.) will have error productions
- Example: $\text{stmt} \rightarrow \text{error ;}$
 - When the parser encounters an error, it would skip just beyond the next semicolon and assumes that a statement had been found
 - The semantic action of the error production will be invoked: it would not need manipulate the input, but could simply generate a diagnostic message

Yacc 的错误恢复

在 Yacc 中，错误恢复使用一种错误产生式的形式：

- 一般形式： $A \rightarrow \text{error } \alpha$
- 用户可以决定哪些非终结符（例如，生成表达式、语句、块等的非终结符）将

具有错误产生式

错误示例: ***stmt*** $\rightarrow$ **error**;

- 该错误产生式规定当语法分析器遇到错误时，它要跳到下一个分号之后，并假设已经找到一条语句
- 将调用错误产生式的语义动作不需要处理输入，只需生成诊断消息

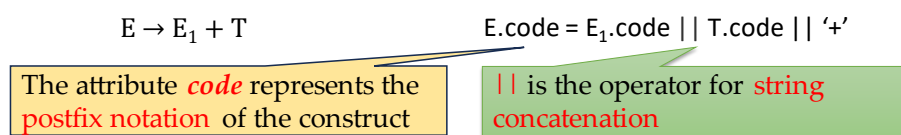
A Brief Introduction

- **Syntax-directed translation (语法制导的翻译)** is the process of language translation guided by context-free grammars
 - Here, “language translation” is in a broad sense
 - Transforming infix expressions (中缀表达式) to postfix expressions (后缀表达式)
 - A language construct is typically made of smaller constructs
 - The semantics of a construct can be **synthesized** from its constituent constructs’ semantics
 - The type of the expression $x + y$ is determined by the type of x and y , and the operator $+$
 - Or **inherited** from other constructs (e.g., siblings in the parse tree)
 - In “int x ”, the type of x is determined by the type specifier to the left of x

- 读取源程序的输入字符，将其归类为词素，并生成词素序列
- 必要时将词目添加到符号表中
- 删除注释和空白（空格、换行符、制表符等）
- 将错误信息与行号关联起来（跟踪换行符）

Syntax-Directed Definitions

- A **syntax-directed definition** (**语法制导定义, SDD**) is a context-free grammar together with **attributes** and **rules**
 - A set of **attributes** (**属性**) is associated with each grammar symbol*
 - Can be anything, e.g., data type of expressions, # instructions in the generated code
 - A **semantic rule** (**语义规则**) is associated with a production and describes how attributes are computed



*Grammar symbols represent language constructs. Nonterminal nodes and subtrees rooted at these nodes correspond to productions.

语法制导定义（SDD）是一种无上下文语法，同时包含属性和规则。

一组属性（attributes）与每个语法符号相关联*。

可以是任何属性，例如表达式的数据类型，# 生成代码中的指令

语义规则（semantic rule）与语法生成相关联，描述如何计算属性

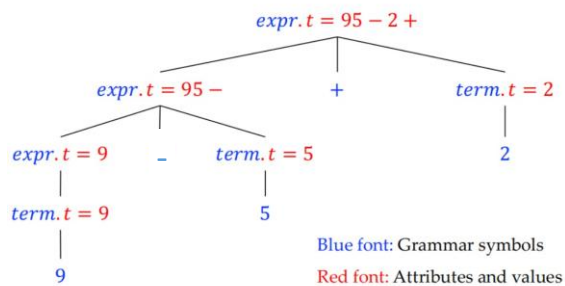
属性 code 表示结构体的后缀符号

|| 是字符串连接操作符

Annotated Parse Tree

- An **annotated parse tree** for **infix expression** 9 -5+2
- The attribute **t** represents postfix notation

expr $\rightarrow$ expr + term
expr $\rightarrow$ expr - term
expr $\rightarrow$ term
term $\rightarrow$ 0
...
term $\rightarrow$ 9



注释解析树

下位表达式 9-5+2 的注释解析树

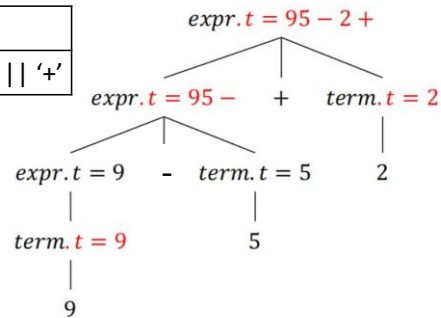
属性 t 代表后缀符号

Synthesized Attributes (综合属性)

- An attribute is said to be **synthesized** if its value at a parse-tree node N is only determined from **attribute values at the children of N** and at N itself (defined by a semantic rule associated with the production at N)

ProProduction	$\text{Expr} \rightarrow \text{expr}_1 + \text{term}$
Rule	$\text{Expr.t} = \text{expr}_1.\text{t} \parallel \text{term.t} \parallel '+'$

ProProduction	$\text{Term} \rightarrow 9$
Rule	$\text{Term.t} = '9'$



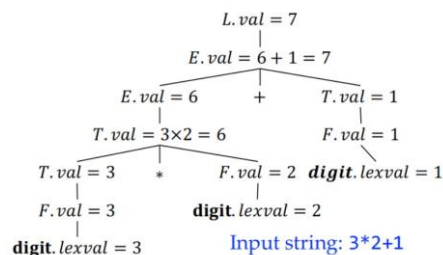
Terminals can also have synthesized attributes (lexical values), but there are not rules for computing the value of an attribute for a terminal.

如果一个属性在解析树节点 N 上的值仅由 N 的子节点和 N 本身的属性值决定，则称该属性为合成属性；

A Complete Example of SDD

- The SDD below helps compute the value of an expression
- SDD's do not specify the evaluation order** of attributes on a parse tree
 - Any order that computes an attribute after all other attributes that depends on is fine
 - Synthesized attributes have a nice property** that they can be evaluated during a single **bottom-up** traversal of a parse tree (it is often unnecessary to explicitly create the tree)

PRODUCTION	SEMANTIC RULES
1) $L \rightarrow E \mathbf{n}$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

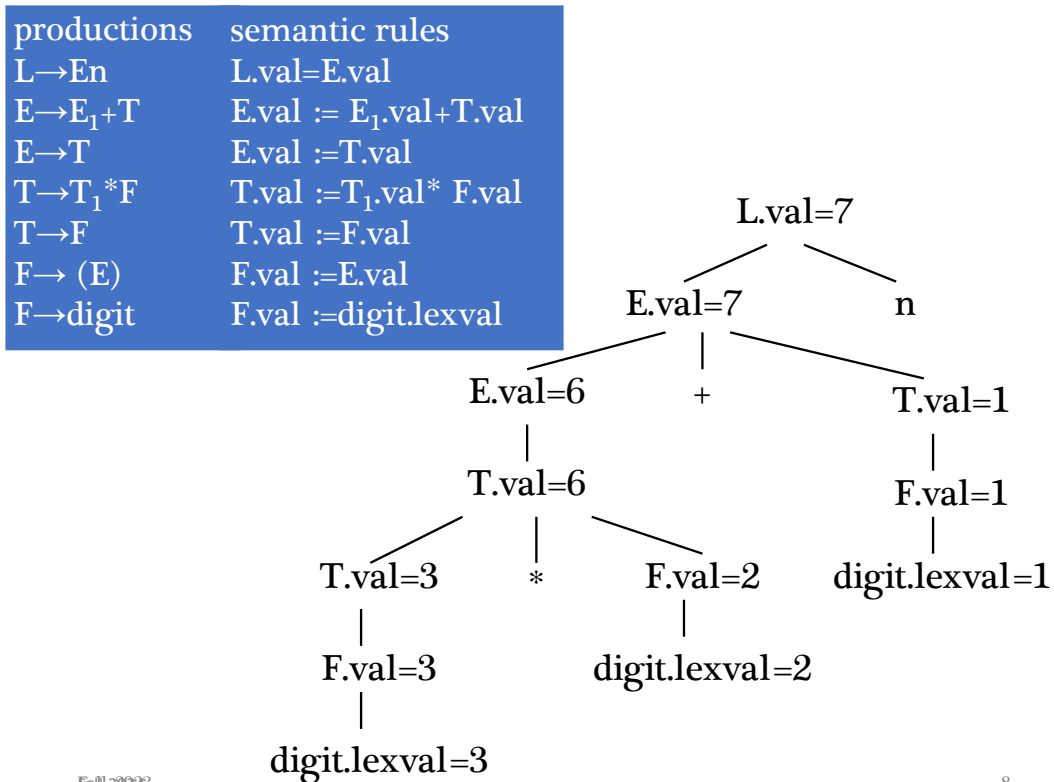


下面的 SDD 可以帮助计算表达式的值

SDD 不指定解析树上属性的计算顺序

任何顺序，只要能在依赖的所有其他属性之后计算一个属性就可以了

合成属性有一个很好的特性，即它们可以在解析树的一次自下而上的遍历中进行评估
(通常不需要显式创建解析树)。



Inherited Attributes (继承属性)

- **Inherited attributes** have their value at a parse-tree node determined from attribute values at **the node itself, its parent, and its siblings** in the parse tree

We already have synthesized attributes. When are inherited attributes useful???

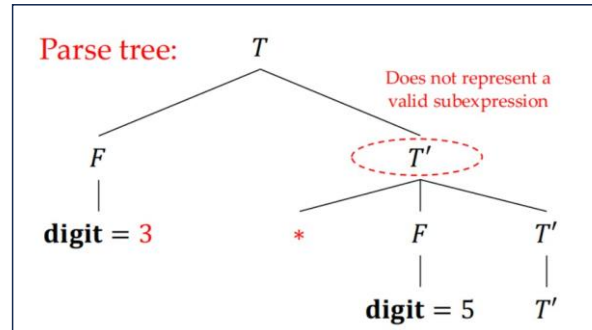
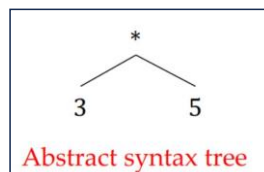


继承属性在解析树节点上的值由该节点本身、其父节点和解析树中同级节点的属性值决定

回答：继承属性在处理与上下文相关的问题时仍然是非常有用的。它为我们提供了一种在语法树中考虑外部信息的方法，从而使得我们可以更为精确和灵活地描述语法制导定义的属性

Top-Down Parse of 3*5

$T \rightarrow FT'$
 $T' \rightarrow *FT'$
 $T' \rightarrow \epsilon$
 $F \rightarrow \text{digit}$



For some grammars, the structure of the parse tree does not **match** the abstract syntax of the code (3 and * are in different subtrees)

Not all non-terminals in a parse tree correspond to language constructs, e.g., T' above.

对于某些语法，解析树的结构与代码的抽象语法不一致（3 和 * 位于不同的子树中）

SDD with Inherited Attributes

PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

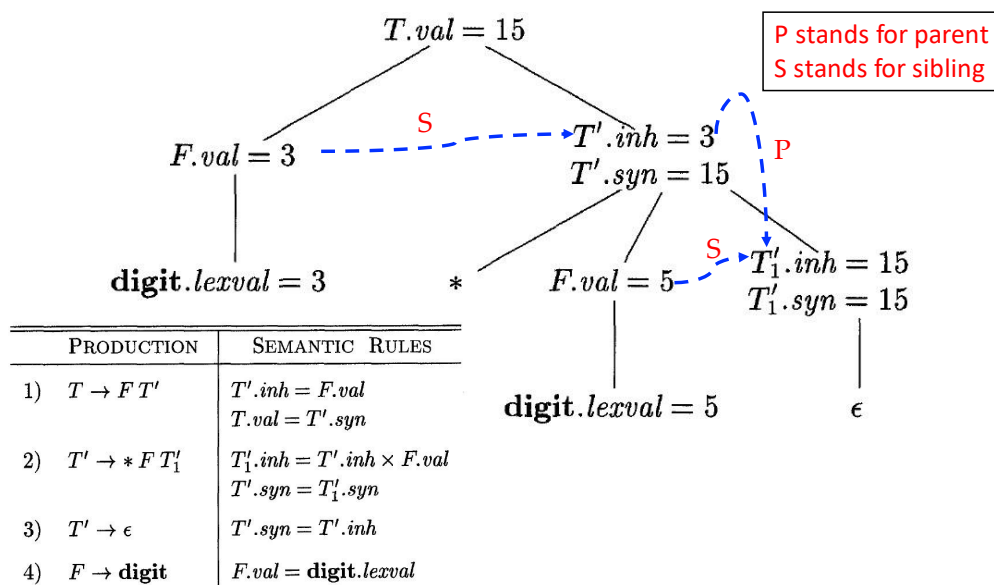
value flow

The left operand of the operator * is inherited by T' and kept for later computation when T' further gets replaced

The inherited attribute of T' is not defined by a rule associated with the production (2) or (3), whose head is T'

运算符 $*$ 的左操作数由 T' 继承, 并在 T' 进一步被替换时保留用于以后的计算。

Annotated Parse Tree for $3*5$



先从具体的输入开始（这是语义计算的起点）结合分析树的结构来分析

根据 4) 产生式及其语义规则, 得到 $F.val$,

根据 2) 产生式及其语义规则, 要想计算 $T'_1.inh = T'.inh$ (未知) $\times F.val$ (5)。要

计算 $T'.syn = T'_1.syn$ (未知)

根据 1) 产生式及其语义规则, $T'.inh = F.val$ (已知) $= 3$, $T.val = T'.syn$ (未知)

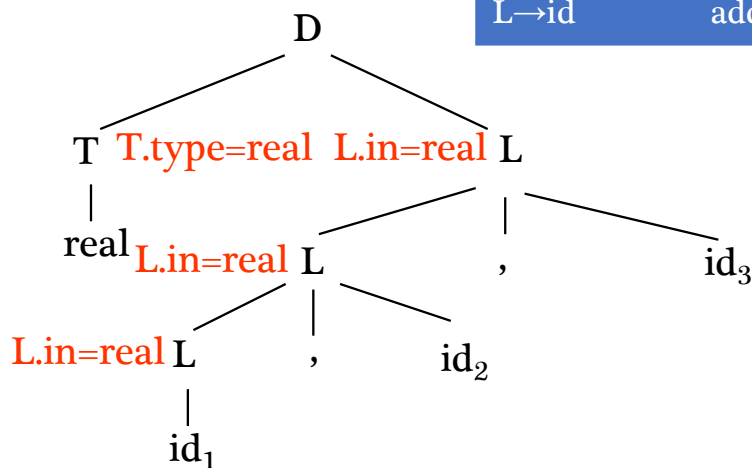
返回 2) 产生式及其语义规则, 计算 $T'_1.inh = T'.inh$ (3) $\times F.val$ (5) $= 15$ 。

根据 3) 产生式及其语义规则, 计算 $T'.syn = T'_1.inh = 15$ 。

返回 2) 产生式及其语义规则, 计算 $T'.syn = T'_1.syn$ (15)

返回 1) 产生式及其语义规则, 计算 $T.val = T'.syn$ (15)

Annotated Parse Tree
for real id_1 , id_2 , id_3



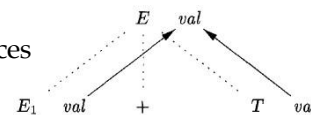
PRODUCTIONS	SEMANTIC RULES
$D \rightarrow TL$	$L.in := T.type$
$T \rightarrow int$	$T.type := integer$
$T \rightarrow real$	$T.type := real$
$L \rightarrow L_1, id$	$L_1.in := L.in$ $addtype(id.entry, L.in)$
$L \rightarrow id$	$addtype(id.entry, L.in)$

name	type
...	...
id_3	real
id_2	real
id_1	real

Symbol table

Evaluation Orders for SDD's

- Given parse tree nodes $N, M_1, M_2, M_3, \dots, M_k$, if the attribute a of N is defined as $N.a = f(M_1.a_1, M_2.a_2, \dots, M_k.a_k)$ then in order to compute $N.a$, we must first compute $M_i.a_i$ ($1 \leq i \leq k$)
- Dependency graphs* (依赖图) are a useful tool for determining evaluation orders
 - Depict the **information flow** among the attribute instances in **a particular parse tree**
 - Model the **partial order** among attribute instances



给定解析树节点 $N, M_1, M_2, M_3, \dots, M_k$, 如果 N 的属性 a 定义为 $N.a = f(M_1.a_1, M_2.a_2, \dots, M_k.a_k)$, 那么为了计算 $N.a$, 我们必须先计算 M_1 。

依赖图 (依赖图) 是确定评估顺序的有用工具

描述特定解析树中属性实例之间的信息流

为属性实例之间的部分顺序建模

Dependency Graph

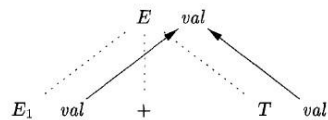
- An **edge** from one **attribute instance** (a_1) to another (a_2) means that the value of a_1 is needed to compute the value of a_2
- If there is any **cycle** in a dependency graph, we cannot find an order to compute the value of all attribute instances

PRODUCTION

$E \rightarrow E_1 + T$

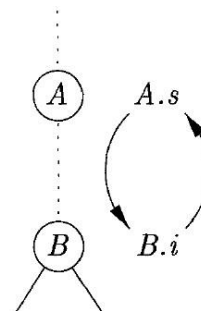
SEMANTIC RULE

$E.val = E_1.val + T.val$



Dotted lines: parse tree edges

Solid lines: dependency graph edges



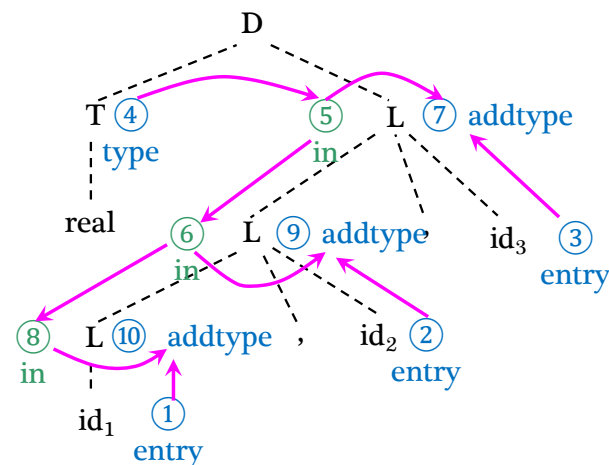
从一个属性实例 (a_1) 到另一个属性实例 (a_2) 的边表示计算 a_2 的值需要 a_1 的值。

如果依赖关系图中存在循环, 我们就无法找到计算所有属性实例值的顺序。

Example1: Parsing $\text{real id}_1, \text{id}_2, \text{id}_3$

Attribute values can be computed according to any *topological sort* (拓扑排序) of the graph, e.g., 1, 2, 3, ..., 10 in the example below

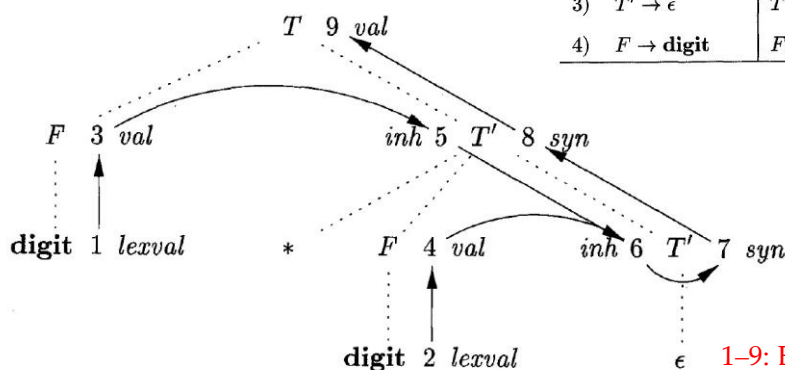
PRODUCTIONS	SEMANTIC RULES
$D \rightarrow TL$	$L.in := T.type$
$T \rightarrow \text{int}$	$T.type := \text{integer}$
$T \rightarrow \text{real}$	$T.type := \text{real}$
$L \rightarrow L_1, \text{id}$	$L_1.in := L.in$ $\text{addtype}(\text{id.entry}, L.in)$
$L \rightarrow \text{id}$	$\text{addtype}(\text{id.entry}, L.in)$



1–10: Evaluation order

Example2: Parsing $3*2$

PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



1–9: Evaluation order

属性值可根据图的任意拓扑排序 (拓扑排序) 计算, 例如下例中的 1、2、3、.....、9

Ordering the Evaluation of Attributes

- Given an SDD, it is very hard to tell whether there exist any parse trees whose dependency graphs have cycles
- In practice, translations can be implemented using classes of SDD's that guarantee an evaluation order*

S-attributed SDD's

L-attributed SDD's

*The dependency graphs for such SDDs are directed acyclic graphs

如果一个属性文法不存在属性之间的循环依赖关系，则称该文法为良定义的
一个依赖图的任何拓扑排序都给出一个语法树中结点的语义规则计算的有效顺序
在给定 SDD 的情况下，很难判断是否存在依赖图具有循环的解析树。

在实践中，可以使用保证求值顺序的 SDD 类来实现翻译*。(这种 SDD 的依赖图是有向无环图)

S 属性 SDD

L 属性 SDD

有向无环图 directed acyclic graphs

S-Attributed Definitions

- An SDD is **S-attributed** if every attribute is synthesized

PRODUCTION	SEMANTIC RULES
1) $L \rightarrow E \mathbf{n}$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit.lexval}$

Intuitively, there cannot be cycles in the dependency graph of any parse tree, since edges always go from children nodes to parent nodes, never the other way around.

S 属性定义

如果每个属性都是合成的，那么语法制导定义就是 S 属性的

直观地说，任何解析树的依赖关系图中都不可能存在循环，因为边总是从子节点到父节点，而不是相反。

S-Attributed Definitions Cont.

- When an SDD is S-attributed, we can evaluate its attributes in any **bottom up order** of the parse-tree nodes
e.g., **postorder traversal (后序遍历)** of the parse tree
- S-attributed definitions can be easily implemented during **bottom-up parsing** (corresponds to a postorder traversal)

```
postorder(N) {  
    for ( each child C of N, from the left ) postorder(C);  
    evaluate the attributes associated with node N;  
}
```

当 SDD 具有 S 属性时，我们可以按照解析树节点的任何自下而上的顺序评估其属性。

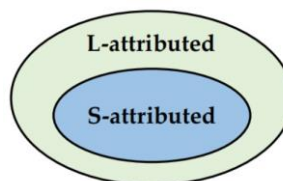
例如，解析树的后序遍历 (postorder traversal

S 属性定义可以在自下而上的解析过程中轻松实现 (相当于后序遍历)

L-Attributed Definitions

- An SDD is *L-attributed* if for each production $A \rightarrow X_1X_2...X_n$, for each $J = 1 \dots n$, each inherited attribute of x_j depends on only:
 - the attributes of $x_1, \dots, x_{j-1}$ (either synthesized or inherited), or
 - the inherited attributes of A

Dependency-graph edges can go from left to right (for a parse tree), but not right to left (hence "L-attributed")



L 属性定义

如果对于每个产品 $A \rightarrow X_1X_2...X_n$ ，对于每个 $J = 1 \dots n$ ， x_j 的每个继承属性仅取决于以下属性，则 SDD 是 L 属性 SDD：

$x_1, \dots, x_{j-1}$ 的属性（合成的或继承的），

或 A 的继承属性

依赖关系图的边可以从左到右（对于解析树而言），但不能从右到左（因此称为 "L-属性"）。

L-Attributed Definition Example

PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ Left sibling's attribute $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$ • Parent's inherited attribute • Left sibling's attribute
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

Synthesized attributes: val, syn, lexval . Inherited attributes: inh

- 左同胞的属性
- 父辈的继承属性 - 左同胞的属性

Attribute Evaluation for L-Attributed Definitions

Input: A node n in a parse tree T

Output: An evaluation order for the attributes of the subtree rooted at n

Procedure depth_first(n)*
Begin

When evaluating the inherited attributes of a node, the attributes of nodes to its left have been evaluated

↑
Guarantee

for every child m of n from left to right do begin
 evaluate the inherited attributes of m ;
 depth_first(m); // here m 's synthesized attributes will be evaluated
 end
 evaluate the synthesized attributes of n ;

end

* The inherited attributes of n are computed before calling depth_first(n), as indicated by the for body

在计算节点的继承属性时，已计算了其左侧节点的属性

前序遍历计算继承属性

输入: 解析树 T 中的节点 n

输出: 以 n 为根的子树属性的评估顺序

程序 $\text{depth_first}(n)^*$

开始

对于 n 的每个子节点 m ，从左到右依次执行 开始

 评估 m 的继承属性;

$\text{depth_first}(m)$; // 这里将评估 m 的合成属性

结束

评估 n 的合成属性;

保证: 在评估节点的继承属性时，已评估了其左侧节点的属性

end

Attribute Evaluation for L-Attributed Definitions

Input: A node n in a parse tree T

Output: An evaluation order for the attributes of the subtree rooted at n

Procedure $\text{depth_first}(n)^*$
Begin

When evaluating the inherited attributes of a node, the inherited attributes of its parent node have been evaluated

for every child m of n from left to right do begin

 evaluate the inherited attributes of m ;

$\text{depth_first}(m)$; // here m 's synthesized attributes will be evaluated

end

 evaluate the synthesized attributes of n ;

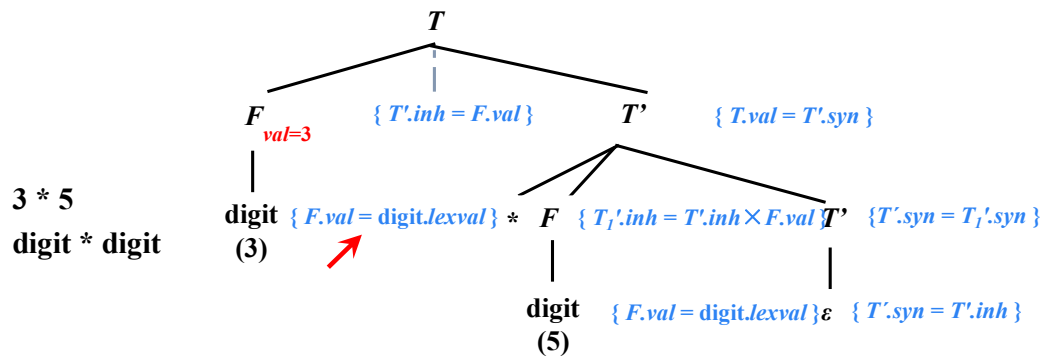
end

Guarantee

* The inherited attributes of n are computed before calling $\text{depth_first}(n)$, as indicated by the for body

保证: 在计算节点的继承属性时，已计算出其父节点的继承属性

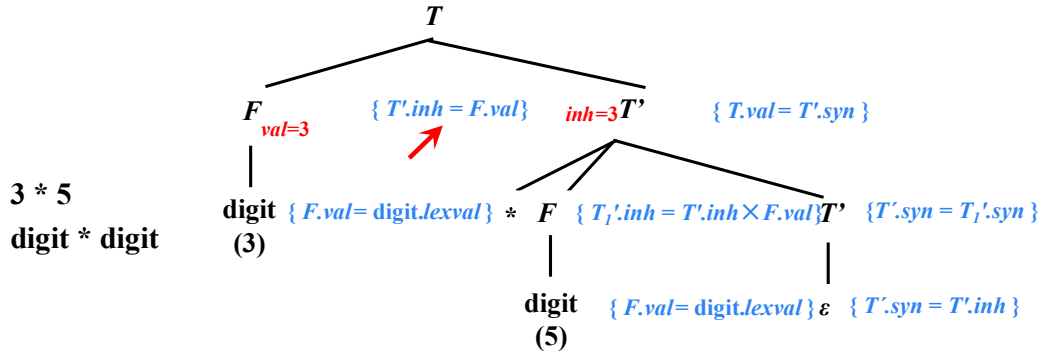
PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



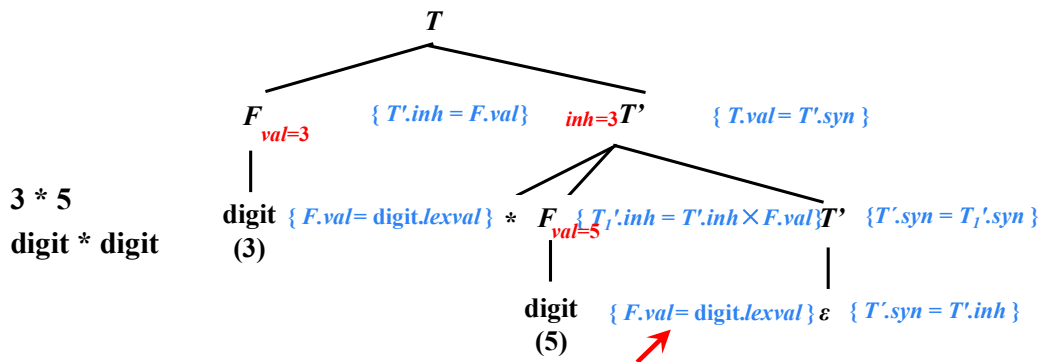
➤ 通用方法

- 首先建立一棵语法分析树，然后按照从左到右的深度优先顺序来执行这些动作，也就是说在一个前序遍历过程中执行

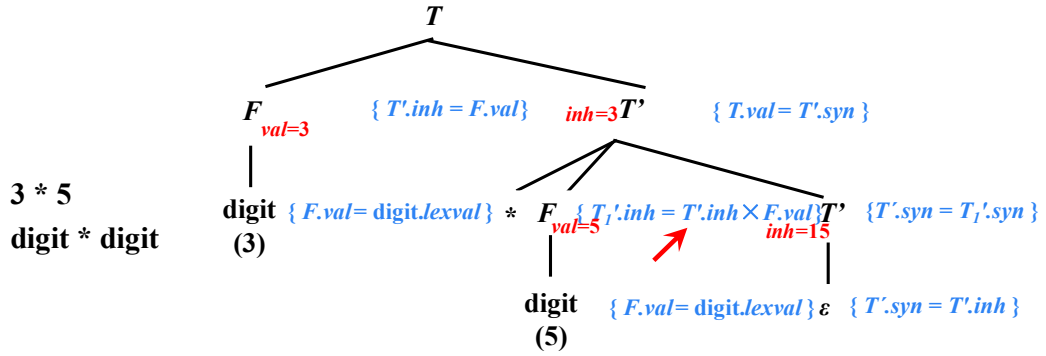
PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



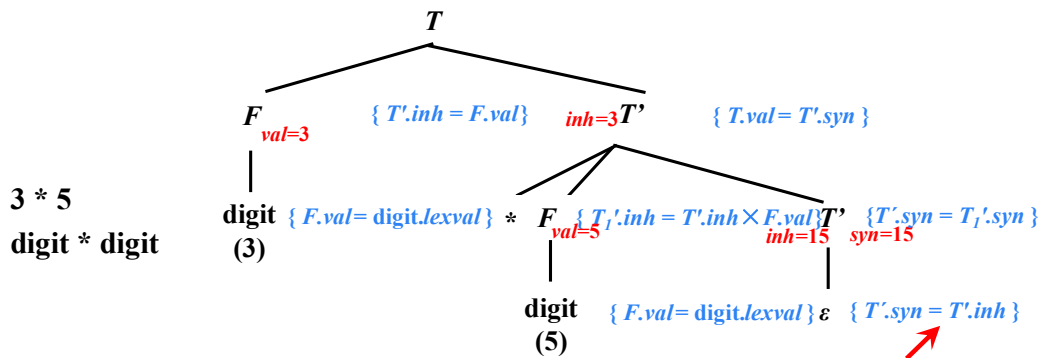
PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



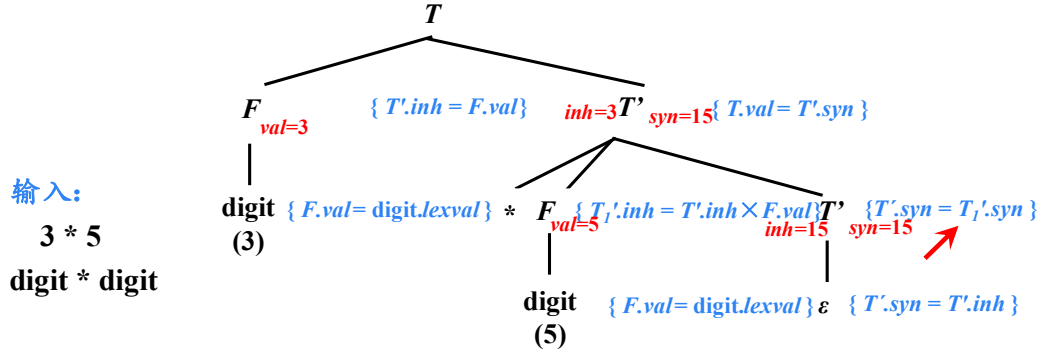
PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



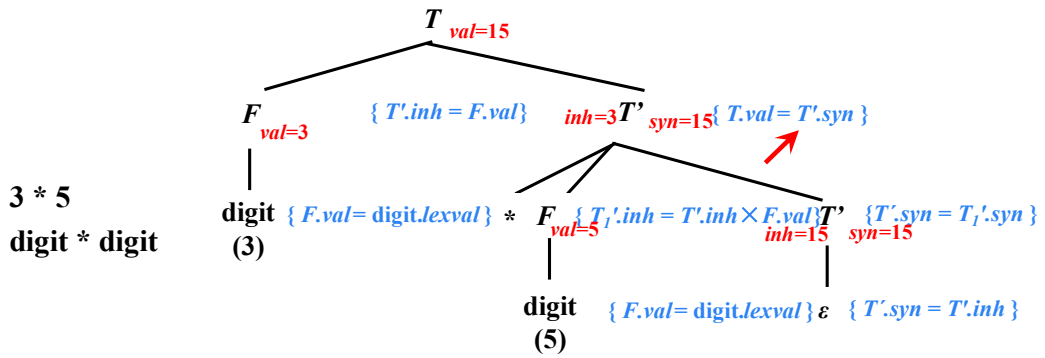
PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



Attribute Evaluation for L-Attributed Definitions

Input: A node n in a parse tree T

Output: An evaluation order for the attributes of the subtree rooted at n

Procedure depth_first(n)*

Begin

for every child m of n from left to right **do begin**

evaluate the inherited attributes of m ;

depth_first(m); // here m 's synthesized attributes will be evaluated

end

evaluate the synthesized attributes of n ;

end

Can be implemented in
top-down parsing

* The inherited attributes of n are computed before calling depth_first(n), as indicated by the for body

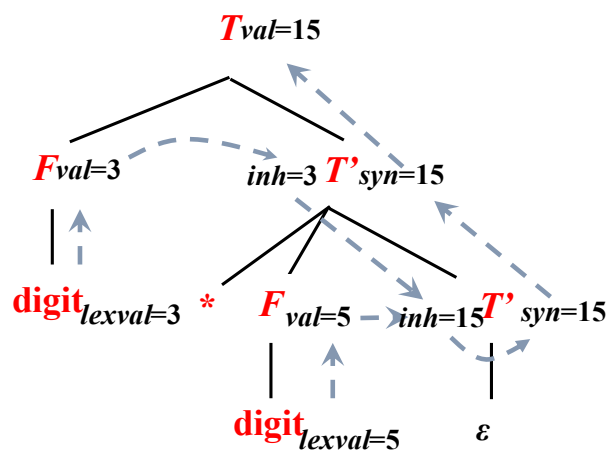
可自顶向下解析

Can be implemented in
top-down parsing

input:

3 * 5

digit * digit

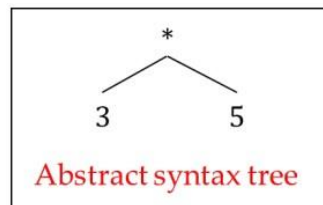


通用方法（两趟）

首先建立一棵语法分析树，然后按照从左到右的深度优先顺序来执行这些动作，也就是说在一个前序遍历过程中执行(继承属性的计算)，然后再执行一个后序遍历完成综合属性的计算。

Construction of Syntax Tree (1)

- **Abstract syntax tree** (revisited)
 - Each interior node N represents a **construct** (corresponding to an **operator**)
 - The children of N represent the meaningful **components of the construct** represented by N (corresponding to **operands**)



抽象语法树（重温）

每个内部节点 N 代表一个结构体（对应一个运算符）

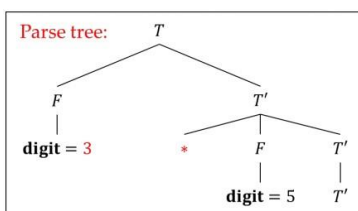
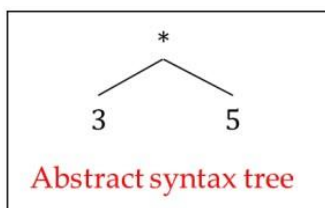
N 的子节点代表 N 所代表结构的有意义成分（相当于操作数）

Construction of Syntax Tree (2)

- Syntax tree vs. parse tree

In a syntax tree, interior nodes represent **programming constructs**, while in a parse tree, interior nodes represent **nonterminals***

A parse tree is also called a **concrete syntax tree**, and the underlying grammar is called a **concrete syntax** for the language



* Not all nonterminals represent programming constructs, e.g., those introduced to eliminate left recursions (in the earlier L-attributed definition example)

语法树与解析树

在语法树中，内部节点代表编程结构，而在解析树中，内部节点代表非终结符*。

解析树也称为具体语法树，其底层语法称为语言的具体语法

Construction of Syntax Tree (3)

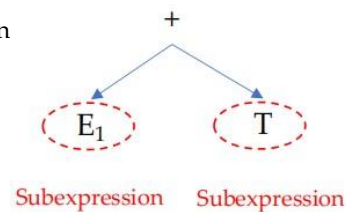
- An S-attributed SDD for building syntax trees for simple expressions

Each node of the syntax tree is implemented as an **object** with a field *op*, representing the label of the node, and some additional fields

Leaf node: one additional field holding the lexical value

Interior node: # additional fields = # of children

PRODUCTION	SEMANTIC RULES
1) $E \rightarrow E_1 + T$	$E.node = \text{new Node}('+', E_1.node, T.node)$
2) $E \rightarrow E_1 - T$	$E.node = \text{new Node}('-', E_1.node, T.node)$
3) $E \rightarrow T$	$E.node = T.node$
4) $T \rightarrow (E)$	$T.node = E.node$
5) $T \rightarrow \text{id}$	$T.node = \text{new Leaf}(\text{id}, \text{id.entry})$
6) $T \rightarrow \text{num}$	$T.node = \text{new Leaf}(\text{num}, \text{num.val})$



Fall 2024

BUPT Compilers

38

为简单表达式构建语法树的 S 属性 SDD

语法树的每个节点都是一个对象，其中有一个字段 *op*（代表节点的标签）和一些附加字段

叶节点：一个包含词法值的附加字段

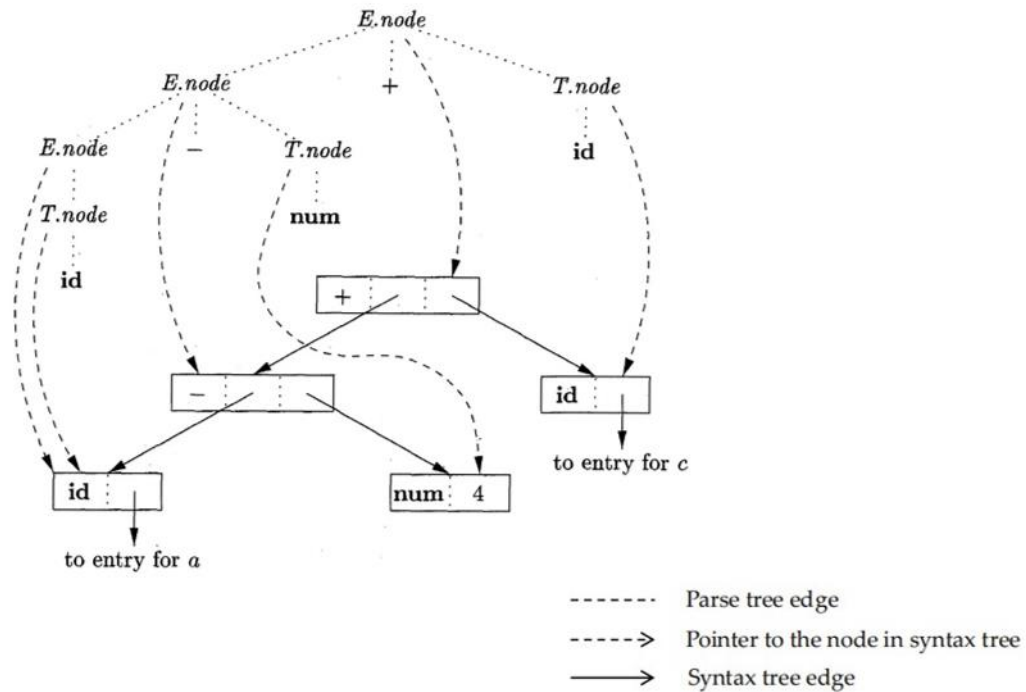
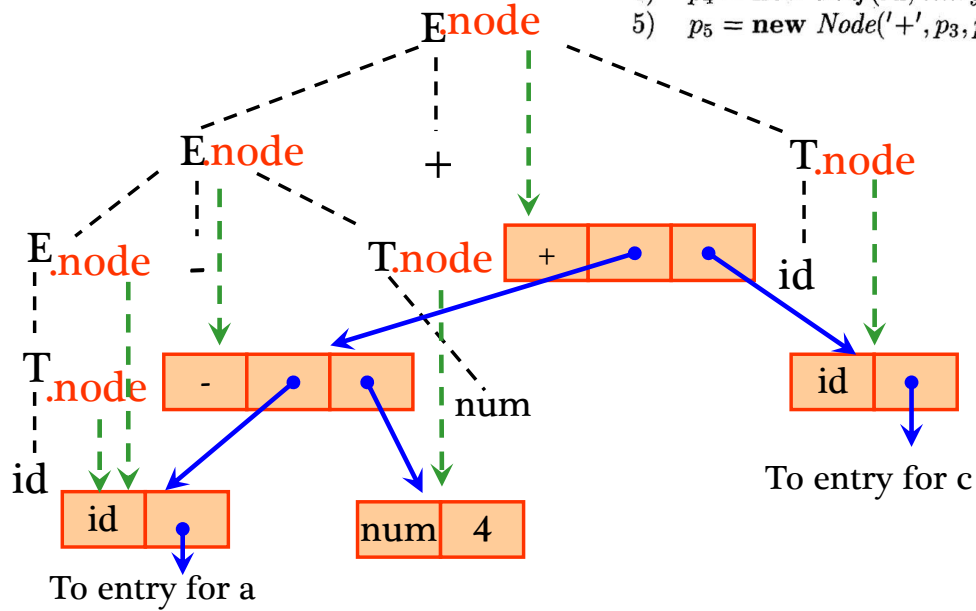
内部节点：额外的字段数量等于子节点的数量

Construction of Syntax Tree (4)

Input expression: $a - 4 + c$

Steps (bottom-up evaluation):

- 1) $p_1 = \text{new Leaf}(\text{id}, \text{entry-a});$
- 2) $p_2 = \text{new Leaf}(\text{num}, 4);$
- 3) $p_3 = \text{new Node}('-', p_1, p_2);$
- 4) $p_4 = \text{new Leaf}(\text{id}, \text{entry-c});$
- 5) $p_5 = \text{new Node}('+', p_3, p_4);$



Computing the Structure of a Type

```
int[2][3] a = ...;
```

What is the type of **a**?



- Leave it to the next chapter...

计算类型的结构

Syntax-Directed Translation Schemes

- SDD's tell us what to do (**high-level specifications**) in the translation, but not how to do
- **Syntax-directed translation schemes** (SDT's, 语法制导的翻译方案) specify more details on how to do the translation
- An SDT is a context-free grammar with semantic actions (program fragments) embedded within production bodies
 - Differ from the semantic rules in SDD's
 - Semantic actions can appear **anywhere** within a production body

SDD 告诉我们在翻译中要做什么（高级规范），但不告诉我们如何做

语法制导的翻译方案（SDT's, Syntax-directed translation schemes）规定了如何进行翻译的更多细节。

SDT 是一种无上下文语法，其语义动作（程序片段）嵌入在生产体中：

与 SDD 的语义规则不同

语义动作可以出现在制作体的任何地方

Implementation of SDT's

- Any SDT can be implemented by:
 - First building a parse tree
 - Treating semantic actions as “**virtual**” parse-tree nodes
 - Performing a left-to-right depth-first (i.e., preorder) tree traversal and triggering the actions when visiting the corresponding nodes

任何 SDT 都可以通过以下方式实现：

首先构建一棵解析树

将语义操作视为 “虚拟” 解析树节点

执行从左到右的深度优先（即预排序）树遍历，并在访问相应节点时触发操作

An Example SDT (1)

- The SDT below implements a simple calculator

SDT	$L \rightarrow E \text{ n}$	$\{ \text{print}(E.val); \}$	Semantic actions: Real code in {}
	$E \rightarrow E_1 + T$	$\{ E.val = E_1.val + T.val; \}$	
	$E \rightarrow T$	$\{ E.val = T.val; \}$	
	$T \rightarrow T_1 * F$	$\{ T.val = T_1.val \times F.val; \}$	
	$T \rightarrow F$	$\{ T.val = F.val; \}$	
	$F \rightarrow (E)$	$\{ F.val = E.val; \}$	
	$F \rightarrow \text{digit}$	$\{ F.val = \text{digit.lexval}; \}$	
SDD	PRODUCTION	SEMANTIC RULES	Semantic rules: Definitions (not code)
	1) $L \rightarrow E \text{ n}$	$L.val = E.val$	
	2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$	
	3) $E \rightarrow T$	$E.val = T.val$	
	4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$	
	5) $T \rightarrow F$	$T.val = F.val$	
	6) $F \rightarrow (E)$	$F.val = E.val$	
	7) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$	

1. SDD (Syntax Directed Definition) :

- SDD 定义了与语法产生式关联的属性和规则。它为每个语法产生式指定一个或多个属性计算或语义动作。
- SDD 关注于如何为语法树中的节点计算属性值，但不一定涉及产生中间代码或目标代码的具体操作。
- 它可以有合成属性和继承属性，因此可以描述为 S 属性的（只有合成属性）或 L 属性的（允许继承属性但有限制）。

2. SDT (Syntax Directed Translation) :

- SDT 是 SDD 的一个特例，它专门用于生成中间代码或目标代码。换句话说，SDT 的目的是翻译，它定义了与语法产生式关联的动作，这些动作通常在解析时执行。
- 当解析器在处理产生式时，与该产生式关联的 SDT 动作被执行，从而生成代码或进行其他翻译工作。
- 动作可以嵌入到产生式中，例如，在 YACC 或 Bison 这样的工具中，可以直接在语法规则中插入 C 代码作为动作。

总结差异:

- SDD 更为通用, 关注于为语法树的节点计算属性。这些属性可以用于多种目的, 如类型检查、变量绑定等。
- SDT 特化于翻译任务, 即将源代码转换为中间代码或目标代码。它通常涉及在解析过程中执行特定的动作。

尽管有这些区别, 但在实际应用中, SDD 和 SDT 的界限可能会模糊, 因为编译器的语义分析和翻译阶段通常是交织在一起的

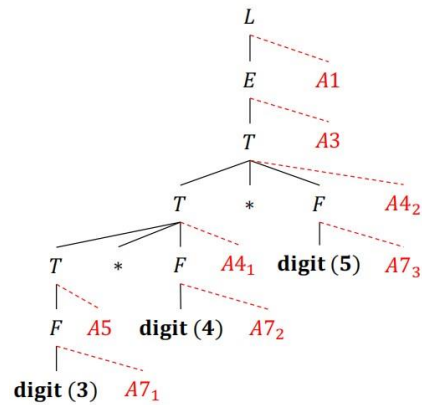
An Example SDT (2)

- Parse and calculate $3*4*5$

L	$\rightarrow$	$E \mathbf{n}$	$\{ \text{print}(E.val); \}$	$A1$
E	$\rightarrow$	$E_1 + T$	$\{ E.val = E_1.val + T.val; \}$	$A2$
E	$\rightarrow$	T	$\{ E.val = T.val; \}$	$A3$
T	$\rightarrow$	$T_1 * F$	$\{ T.val = T_1.val \times F.val; \}$	$A4$
T	$\rightarrow$	F	$\{ T.val = F.val; \}$	$A5$
F	$\rightarrow$	(E)	$\{ F.val = E.val; \}$	$A6$
F	$\rightarrow$	digit	$\{ F.val = \mathbf{digit.lexval}; \}$	$A7$

Order of actions:

$A7_1, A5, A7_2, A4_1, A7_3, A4_2, A3, A1$



SDT's With Actions Inside Productions

$$B \rightarrow X\{a\}Y$$

- The action a should be done after we have recognized X (if X is a terminal) or all the terminals derived from X (if X is a nonterminal)
 - If the parse is **bottom-up**, we perform the **action a** as soon as X appears on the top of the parsing stack
 - If the parse is **top-down**, we perform the **action a** before attempting to expand Y (if Y is a nonterminal) or check for Y on the input (if Y is a terminal)

结合语法分析树理解

动作 a 应在我们识别完 X （如果 X 是终结符）或由 X 派生的所有终结符（如果 X 是非终结符）后执行

如果解析是自下而上的，那么一旦 X 出现在解析栈顶端，我们就执行操作 a （此时，有关 X 的所有信息都已在栈中）

如果解析是自上而下的，我们在尝试扩展 Y （如果 Y 是非终结符）或检查输入中的 Y （如果 Y 是终结符）之前执行操作 a （此时有关 X 的所有信息已分析完毕）

SDT's Implementable During Parsing

- In practice, SDT's are often implemented during parsing
without building a parse tree
- **Not all** SDT's can be implemented during parsing *
- Even if the underlying grammar is parsable by a method (e.g., LL, LR), after introducing semantic actions, the method may become inapplicable
- **Determine if an SDT can be implemented during parsing**
 - Introduce distinct *marker nonterminals* to replace each embedded action; each marker M has only one production $M \rightarrow \epsilon$
 - If the grammar with marker nonterminals can be parsed by a given method, then the SDT can be implemented during parsing

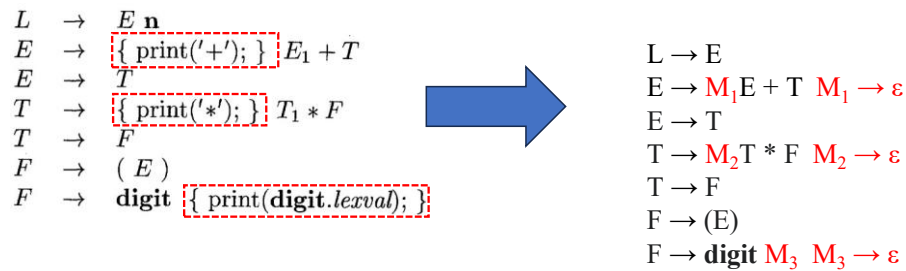
*Note that all SDT's can be implemented after building the parse tree, as earlier mentioned

注意，如前所述，所有 SDT 都可以在构建分析树后实施。

将语义动作标记非终结 M 来代替，对应的的产生式为 $M \rightarrow \epsilon$ ，得到文法可以被 LL 或 LR 分析器所分析，这样的 SDT 才能在语法分析时一并进行。

A Problematic SDT

- This SDT translates infix expression to prefix expressions



It is impossible to build parsing tables without conflicts using top-down or bottom-up parsing methods. This SDT cannot be implemented during parsing.

使用自上而下或自下而上的解析方法不可能建立没有冲突的解析表。

自下而上的解析方法来建立没有冲突的解析表。这种 SDT 无法在解析过程中执行。

Uses of SDT's

- We can use SDT's to implement two important classes of SDD's:

The underlying grammar is LR, and the SDD is S-attributed

The underlying grammar is LL, and the SDD is L-attributed

我们可以使用 SDT 来实现两类重要的 SDD:

基础语法是 LR, SDD 是 S 属性的

基础语法是 LL, SDD 是 L 属性的

底层语法是 LR, 而 SDD 是 S 属性的:

- **LR 语法:** 这是一种可以用自底向上的方式进行解析的语法。LR 解析器从左到右读取输入, 但决策是基于右侧的上下文进行的。它使用一个堆栈来存储已经读取的符号, 并在适当的时候进行规约。

- **S 属性的 SDD:** 这意味着所有属性都是合成属性, 它们只从子节点传递给父节点, 没有从父节点传递给子节点的继承属性。这对于自底向上的 LR 解析是非常合适的, 因为我们从叶子节点开始构建并向上移动, 因此只需要合成属性即可。

底层语法是 LL, 而 SDD 是 L 属性的:

- **LL 语法:** 这是一种可以用自顶向下的方式进行解析的语法。LL 解析器从左到右读取输入, 并基于左侧的上下文来进行决策。它通常使用递归下降的方法, 直接从起始符号开始, 并尝试应用各种产生式。

- **L 属性的 SDD:** 这种 SDD 除了合成属性外, 还包括继承属性, 但这些继承属性只能依赖于它左边的兄弟节点或其父节点的属性。由于 LL 解析是自顶向下的, 所以可以在解析过程中计算这些继承属性。

Postfix Translation Schemes

- If the grammar of an SDD is LR, and the SDD is S-attributed, then we can construct a *postfix SDT* (后缀SDT) to implement the SDD in bottom-up parsing

- Semantic actions always appear at the end of productions (hence “postfix”)

PRODUCTION	SEMANTIC RULES			
1) $L \rightarrow E \mathbf{n}$	$L.val = E.val$ SDT	$L \rightarrow E \mathbf{n}$	{ print($E.val$); }	SDT
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$	$E \rightarrow E_1 + T$	{ $E.val = E_1.val + T.val$; }	
3) $E \rightarrow T$	$E.val = T.val$	$E \rightarrow T$	{ $E.val = T.val$; }	
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$	$T \rightarrow T_1 * F$	{ $T.val = T_1.val \times F.val$; }	
5) $T \rightarrow F$	$T.val = F.val$	$T \rightarrow F$	{ $T.val = F.val$; }	
6) $F \rightarrow (E)$	$F.val = E.val$	$F \rightarrow (E)$	{ $F.val = E.val$; }	
7) $F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit.lexval}$	$F \rightarrow \mathbf{digit}$	{ $F.val = \mathbf{digit.lexval}$; }	

This is possible because in bottom-up parsing, before reducing to a production head, the grammar symbols in the production body have been visited and their synthesized attributes have been computed.

如果 SDD 的语法是 LR，而且 SDD 是 S 属性的，那么我们就可以构造一个后缀 SDT（后缀 SDT），在自下而上的解析中实现 SDD 语义动作总是出现在制作的末尾（因此称为 “后缀”）。

之所以能做到这一点，是因为在自下而上的解析过程中，在还原到生产头之前，已经访问过生产体中的语法符号，并计算了它们的合成属性。

Parser-Stack Implementation of Postfix SDT's

- Postfix SDT's can be implemented during LR parsing by executing the actions when reductions occur
- The synthesized attributes can be placed along with the grammar symbols on the stack

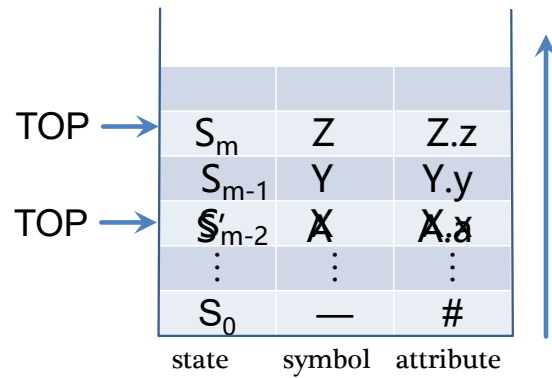
	X	Y	Z	State/grammar symbol
	$X.x$	$Y.y$	$Z.z$	Synthesized attribute(s)

↑
top

If we do reduction using $A \rightarrow XYZ$, then the attributes of A can be calculated based on the attributes of X , Y , and Z , which are already on the stack.

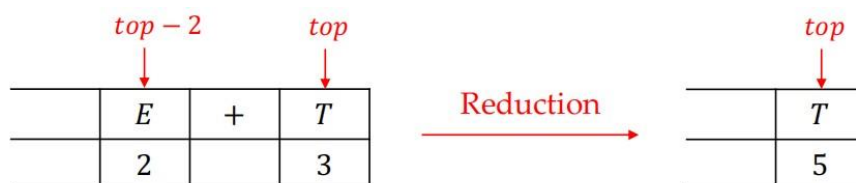
在 LR 解析过程中，可以通过在还原发生时执行以下操作来实现后缀 SDT 合成的属性可与语法符号一起放在堆栈中

如果我们使用 $A \rightarrow XYZ$ 进行还原，那么 A 的属性就可以根据堆栈中已经存在的 X 、 Y 和 Z 计算出来。就可以根据堆栈中已经存在的 X 、 Y 和 Z 的属性计算出 A 的属性。



The Calculator Example

PRODUCTION	ACTIONS
$L \rightarrow E \mathbf{n}$	{ $\text{print}(\text{stack}[\text{top} - 1].\text{val});$ $\text{top} = \text{top} - 1;$ }
$E \rightarrow E_1 + T$	{ $\text{stack}[\text{top} - 2].\text{val} = \text{stack}[\text{top} - 2].\text{val} + \text{stack}[\text{top}].\text{val};$ $\text{top} = \text{top} - 2;$ }
$E \rightarrow T$	
$T \rightarrow T_1 * F$	{ $\text{stack}[\text{top} - 2].\text{val} = \text{stack}[\text{top} - 2].\text{val} \times \text{stack}[\text{top}].\text{val};$ $\text{top} = \text{top} - 2;$ }
$T \rightarrow F$	
$F \rightarrow (E)$	{ $\text{stack}[\text{top} - 2].\text{val} = \text{stack}[\text{top} - 1].\text{val};$ $\text{top} = \text{top} - 2;$ }
$F \rightarrow \text{digit}$	

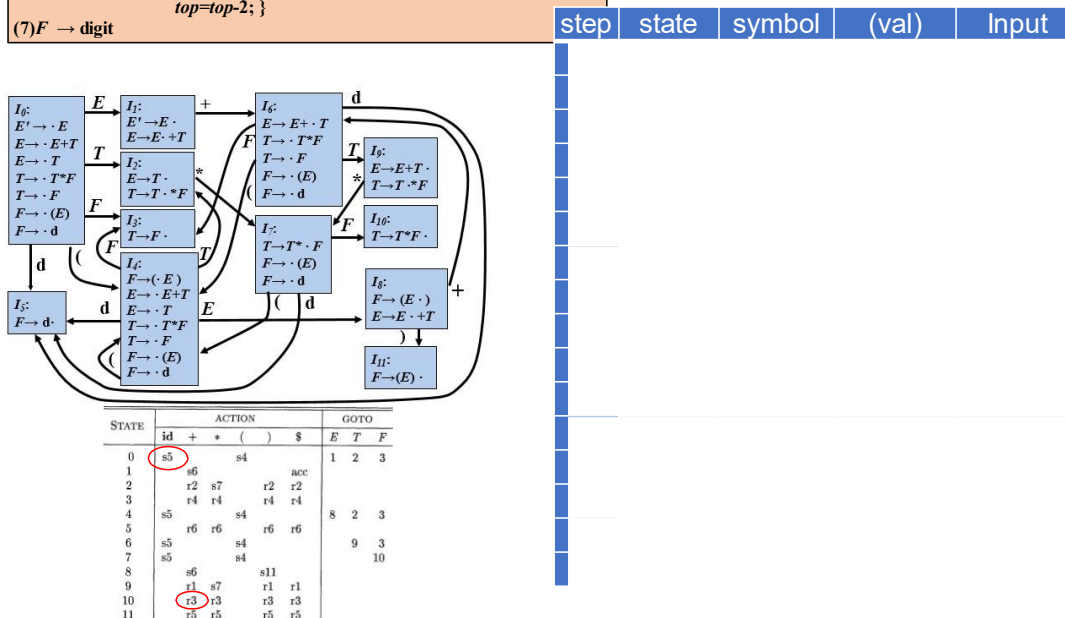


- 在许多语言中，保留字或关键词（如 then）也符合标识符模式

- 问题：搜索标识符的过渡图也能识别保留字

PRODUCTIONS	ACTIONS
(1) $E' \rightarrow E$	{ print (stack[top].val); }
(2) $E \rightarrow E_1 + T$	{ stack[top-2].val = stack[top-2].val + stack[top].val ; top=top-2; }
(3) $E \rightarrow T$	
(4) $T \rightarrow T_1 * F$	{ stack[top-2].val = stack[top-2].val $\times$ stack[top].val ; top=top-2; }
(5) $T \rightarrow F$	
(6) $F \rightarrow (E)$	{ stack[top-2].val = stack[top-1].val ; top=top-2; }
(7) $F \rightarrow \text{digit}$	

$$\bullet 3*5+4n$$



Uses of SDT's

- We can use SDT's to implement two important classes of SDD's:

The underlying grammar is LR, and the SDD is S-attributed

The underlying grammar is LL, and the SDD is L-attributed

SDT's for L-Attributed SDD's

- L-attributed SDD's can be implemented during top-down parsing, if the underlying grammar is LL
- The way of turning an L-attributed SDD into an SDT is to place semantic actions at appropriate positions in the concerned production $A \rightarrow X_1X_2...X_n$
 - Embed the action that computes the **inherited attributes** for a nonterminal X_i immediately **before** X_i in the production body
 - Place the actions that compute a **synthesized attribute** for the production head **at the end** of the production body

如果基础语法是 LL，那么 L 归因 SDD 可以在自上而下的解析过程中实现。

将 L 属性 SDD 转化为 SDT 的方法是将语义动作放在相关语篇 $A \rightarrow X_1X_2...X_n$ 的适当位置上。

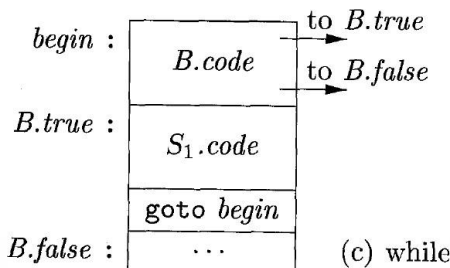
将计算非终端 X_i 的继承属性的动作紧接在生产体中 X_i 的前面

将为语篇头部计算合成属性的操作放在语篇主体的末尾

An L-Attributed SDD

- The SDD generates labels for the *while* loop

$S \rightarrow \text{while} (B) S_1$ $\text{begin} = \text{newlabel}();$
 $B.\text{true} = \text{newlabel}();$
 $B.\text{false} = S.\text{next};$
 $S_1.\text{next} = \text{begin};$
 $S.\text{code} = \text{label}(\text{begin}) \parallel B.\text{code} \parallel \text{label}(B.\text{true}) \parallel S_1.\text{code} \parallel \text{gen}('goto' \text{ begin});$



Inherited attributes: S.next, B.true, B.false

Synthesized attribute: S.code, S1.code

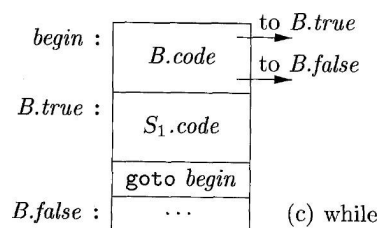
S.Begin while 语句开头

C.True S1 的第一条语句

Turning into an SDT

- Semantic actions:

- $\text{begin} = \text{newlabel}(); B.\text{true} = \text{newlabel}();$
- $B.\text{false} = S.\text{next};$
- $S_1.\text{next} = \text{begin};$
- $S.\text{code} = \dots;$



- According to the rules of action placement:

- b) should be placed before C, c) should be placed before S_1 , and d) should be placed at the end of the production body
- a) can be placed at the very beginning; there is no constraint

$S \rightarrow \text{while} (\{ \text{begin} = \text{newlabel}(); B.\text{true} = \text{newlabel}(); B.\text{false} = S.\text{next}; \}$
 $B) \quad \{ S_1.\text{next} = \text{begin}; \}$
 $S_1 \quad \{ S.\text{code} = \text{label}(\text{begin}) \parallel B.\text{code} \parallel \text{label}(B.\text{true}) \parallel S_1.\text{code} \parallel \text{gen}('goto' \text{ begin}); \}$

- 语义操作:

- a) `L1 = new(); L2 = new();`
- b) `C.false = S.next; C.true = L2;`
- c) `S1.next = L1;`
- d) `S.code = ... ;`

根据动作位置规则,

b) 应放在判断语句 C 之前, c) 应放在 S1 语句之前 (计算 S1 的继承属性), d) 应放在产生式末尾 (产生式左部 S 的综合属性计算应该在产生式右部所有文法符号的属性都计算完才能计算)

- a) 可以放在开头; 没有限制条件

Translation During Recursive-Descent Parsing

- Many translation applications can be addressed using L -attributed SDD's. It is possible to **extend a recursive -descent parser to implement L -attributed SDD's** as follows:
 - A recursive-decent parser has a function A for each nonterminal A
 - Use the arguments of function A to pass A's **inherited attributes**
 - Return the **synthesized attributes** of A when the function A completes
- With the above extension, in the body of the function A, we need to both **parse** and **handle attributes**

许多翻译应用都可以使用 L 归因 SDD 来解决。如下所示, 我们可以扩展递归-下降解析器来实现 L 属性 SDD:

递归-迭代解析器为每个非终端 A 都有一个函数 A

使用函数 A 的参数传递 A 的继承属性

当函数 A 完成时, 返回 A 的合成属性

通过上述扩展，在函数 A 的主体中，我们需要同时解析和处理属性

算术表达式计算的例子

➤ 例

SDT

- 1) $T \rightarrow F \{ T'.inh = F.val \} T'$
 $\{ T.val = T'.syn \}$
- 2) $T' \rightarrow *F \{ T_1'.inh = T'.inh \times F.val \} T_1'$
 $\{ T'.syn = T_1'.syn \}$
- 3) $T' \rightarrow \varepsilon \{ T'.syn = T'.inh \}$
- 4) $F \rightarrow \text{digit} \{ F.val = \text{digit.lexval} \}$

为每个非终结符 A 构造一个函数， A 的每个继承属性对应该函数的一个形参，函数的返回值是 A 的综合属性值

对出现在 A 产生式右部中的每个文法符号的每个属性都设置一个局部变量

对于每个动作，将其代码复制到语法分析器，并把对属性的引用改为对相应变量的引用

```

T'syn T'(token, T'inh)
{
  D: Fval, T1'inh, T1'syn;
  if token="*" then
  {
    Getnext(token);
    Fval=F(token);
    T1'inh= T'inh×Fval;
    Getnext(token);
    T1'syn=T1'(token, T1'inh);
    T'syn=T1'syn;
    return T'syn;
  }
  else if token= "$" then
  {
    T'syn= T'inh;
    return T'syn;
  }
  else Error;
}
  
```

➤ 例

SDT

- 1) $T \rightarrow F \{ T'.inh = F.val \} T'$
 $\{ T.val = T'.syn \}$
- 2) $T' \rightarrow *F \{ T_1'.inh = T'.inh \times F.val \} T_1'$
 $\{ T'.syn = T_1'.syn \}$
- 3) $T' \rightarrow \varepsilon \{ T'.syn = T'.inh \}$
- 4) $F \rightarrow \text{digit} \{ F.val = \text{digit.lexval} \}$

Tval **T**(token)

```

{
  D: Fval, T'inh, T'syn;
  Fval = F(token);
  T'inh = Fval;
  Getnext(token);
  T'syn = T1'(token, T'inh);
  Tval = T'syn;
  return Tval;
}
  
```

➤ 例

SDT

- 1) $T \rightarrow F \{ T'.inh = F.val \} T'$
 $\{ T.val = T'.syn \}$
- 2) $T' \rightarrow *F \{ T_1'.inh = T'.inh \times F.val \} T_1'$
 $\{ T'.syn = T_1'.syn \}$
- 3) $T' \rightarrow \varepsilon \{ T'.syn = T'.inh \}$
- 4) $F \rightarrow \text{digit} \{ F.val = \text{digit.lexval} \}$

Fval F(token)

```
{  
    if token  $\neq$  digit then Error ;  
    Fval=token.lexval ;  
    return Fval;  
}
```

➤ 例

SDT

- 1) $T \rightarrow F \{ T'.inh = F.val \} T'$
 $\{ T.val = T'.syn \}$
- 2) $T' \rightarrow *F \{ T_1'.inh = T'.inh \times F.val \} T_1'$
 $\{ T'.syn = T_1'.syn \}$
- 3) $T' \rightarrow \varepsilon \{ T'.syn = T'.inh \}$
- 4) $F \rightarrow \text{digit} \{ F.val = \text{digit.lexval} \}$

Desent()

```
{  
    D: Tval;  
    Getnext(token);  
    Tval = T(token);  
  
    if token  $\neq$  "$" then Error ;  
    return ;  
}
```

Compiler Front End

- The front end of a compiler **analyzes a source program** and **creates an intermediate representation (IR, 中间表示)**, from which the back end generates target code
- Ideally, details of the source language are confined to the front end, and details of the target machine to the back end

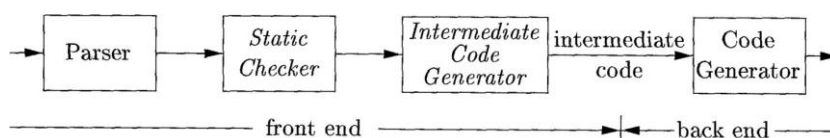


Figure 6.1: Logical structure of a compiler front end

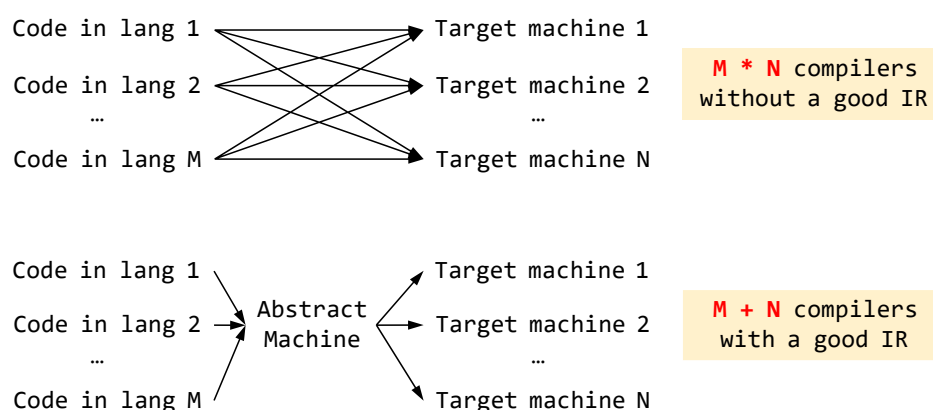
在编译器的分析——综合模型中，前端对源程序进行分析并产生中间表示，后端在此基础上生成目标代码。理想情况下，和源语言相关的细节在前端分析中处理，即关于目标机器器的细节则在后端处理。

本章的内容涉及中间代码表示、静态类型检查和中间代码生成。为简单起见，我们假设一个编译程序的前端处理按照图 6-1 所示方式进行组织，顺序地进行语法分析、静态检查和中间代码生成。有时候这几个过程也可以结合起来，在语法分析中一并完成。我们将使用第 2 章和第 5 章中的语法制导定义来描述类型检查和翻译过程。大部分的翻译方案可以基于第 5 章中给出的自顶向下或自底向上的语法分析技术来实现。所有的方案都可以通过生成并遍历抽象语法树来实现。

静态检查包括类型检查（type checking），类型检查保证运算被应用到兼容的运算分量。静态检查还包括在语法分析之后进行的所有语法检查。例如，静态检查保证了 C 语言中的一条 break 指令必然位于一个 while/for/switch 语句之内。如果不存在这样的语句，静态检查将报告一个错误。

本章讨论中间表示、静态类型检查和中间代码生成。为了简单起见，我们假设编译器前端的组织如图 6.1 所示，其中解析、静态检查和中间代码生成是按顺序完成的；有时它们可以被组合并折叠到解析中。我们将使用“语法分析”和“语法制导翻译”来具体说明检查和翻译。许多翻译方案可以使用“语法制导翻译”中的技术在自下而上或自上而下的解析过程中实现。所有方案都可以通过创建语法树然后遍历树来实现。

The Benefits of A Good IR



基于一个适当定义的中端表示形式，可以把针对源语言 i 的前端和针对目标机器 j 的后端组合起来，构造得到源语言 i 在目标机器 j 上的一个编译器。这种创建编译器组合的方法可以大大减少工作量：只要写出 m 种前端和 n 种后端处理程序，就可以得到 $m \times n$ 种编译程序。

在编译器的分析-综合模型中，前端分析源程序并创建中间表示，后端从中生成目标代码。理想情况下，源语言的详细信息限制在前端，目标机器的详细信息限制在后端。

通过适当定义的中端表示，可以通过将语言 i 的前端与机器 j 的后端相结合来构建语言 i 和机器 j 的编译器。这种创建编译器套件的方法可以节省大量工作：只需编写 m 个前端和 n 个后端即可构建 $m \times n$ 个编译器

More on IR



Figure 6.2: A compiler might use a sequence of intermediate representations

- A compiler may construct a sequence of IR's
 - **High-level IR's** like syntax trees are close to the source language. They depict the structure of the source program and are suitable for tasks like static type checking
 - **Low-level IR's** are close to the target machines and are suitable for machine-dependent tasks like register allocation and instruction selection
 - Three-address code can be high- or low-level, depending on the choice of operators
- **Interesting fact:** C is often used as an intermediate form. The first C++ compiler has a front end that generates C and a C compiler as a backend

本章介绍的方法可以用于多种中间表示，包括抽象语法树和三地址代码。这两种中间表示术语“三地址代码”来自一般形式 $x = y \text{ op } z$ 的指令，具有三个地址：两个用于操作数 y 和 z ，一个用于综合结果 x 。

在将给定源语言的一个程序翻译成特定的目标机器代码的过程中，一个编译器可能构造出一系列中间表示，如图 6-2 所示。高层的表法接近于源语言，而低层的表示接近于目标机器。语法树是高层的中间表示，它刻画了源程序的自然的层次性结构，并且适用于静态类型检查这样的处理。

低层的表示形式适用于机器相关的处理任务，比如寄存器分配、指令选择等。通过选择不同的运算符，三地址代码既可以是高层的表示方式，也可以是低层的表示方式。在 6.2.3 将看到，对表达式而言，语法树和三地址代码只是在表面上有所不同。对于循环语句，语法树表示了语句的各个组成部分，而三地址代码包含标号和跳转指令，用来表示目标语言的控制流。

不同的编译器对中间表示的选择和设计各有不同。中间表示可以是一种真正的语言，也可以由编译器的各个处理阶段共享的多个内部数据结构组成。C 语言是一种程序设计语言。它具有很好的灵活性和通用性，可以很方便地把 C 程序编译成高效的机器代码，并且

有很多 C 的编译器可用，因此 C 语言也常常被用作中间表示。早期的 C++ 编译器的前端生成 C 代码，而把 C 编译器作为其后端。

在将给定源语言的程序翻译成给定目标机器的代码的过程中，编译器可以构造一系列中间表示，如图 6.2 所示。高级表示接近源语言，低级表示接近目标机器。语法树是高级的；它们描述了源程序的自然层次结构，非常适合静态类型检查等任务。

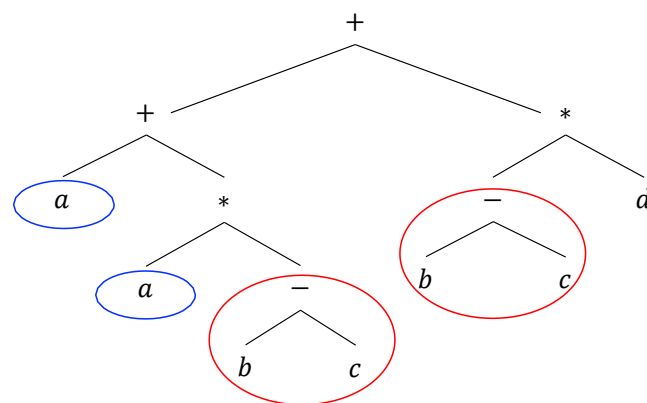
低级表示适用于机器相关的任务，例如寄存器分配和指令选择。三地址代码的范围可以从高级到低级，具体取决于运算符的选择。对于表达式，语法树和三地址代码之间的差异只是表面上的，我们将在第 6.2.3 节中看到。例如，对于循环语句，语法树表示语句的组成部分，而三地址代码包含标签和跳转指令来表示控制流，就像在机器语言中一样。

中间表示的选择或设计因编译器而异。中间表示可以是实际语言，也可以由编译器各阶段共享的内部数据结构组成。C 是一种编程语言，但它经常被用作中间形式，因为它很灵活，可以编译成高效的机器代码，并且它的编译器可以广泛使用。最初的 C++ 编译器由生成 C 的前端组成，将 C 编译器视为后端。

DAG's for Expressions

- In a syntax tree, the tree for a **common subexpression** would be **replicated** as many times as the subexpression appears

▪ Example: $a + a * (b - c) + (b - c) * d$



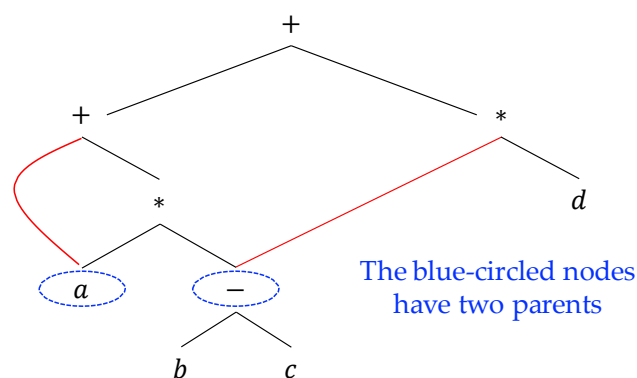
和表达式的语法树类似，一个 DAG 的叶子结点对应于原子运算分量，而内部结点对应于运算符。与语法树不同的是，如果 DAG 中的一个结点 N 表示一个公共子表达式，则

N 可能有多个父结点。在语法树中，公共子表达式每出现一次，代表该公共子表达式的子树就会被复制一次。

DAG's for Expressions Cont.

- A *directed acyclic graph* (DAG, 有向无环图) identifies the common subexpressions and represents expressions succinctly

- Example: $a + a * (b - c) + (b - c) * d$



因此，DAG 不仅更简洁地表示了表达式，而且可以为最终生成表达式的高效代码提供重要的信息。

叶子结点 a 在表达式中出现了两次，因此 a 有两个父结点。值得注意的是，结点 “-” 代表公共子表达式 $b-c$ 的两次出现。该结点同样有两个父结点，表明该子表达式在子表达式 $a*(b-c)$ 和 $(b-c)*d$ 中两次被使用。尽管 b 和 c 在整个表达式中出现了两次，但它们对应的结点只有一个父结点，因为对它们的使用都出现在同样的公共子表达式 $b-c$ 中。

Constructing DAG's

- DAG's can be constructed by the same SDD that constructs syntax trees
- **The difference:** When constructing DAG's, a new node is created if and only if there is no existing identical node

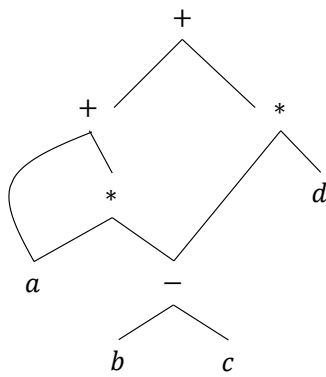
PRODUCTION	SEMANTIC RULES
1) $E \rightarrow E_1 + T$	$E.node = \text{new Node}('+', E_1.node, T.node)$
2) $E \rightarrow E_1 - T$	$E.node = \text{new Node}('-', E_1.node, T.node)$
3) $E \rightarrow T$	$E.node = T.node$
4) $T \rightarrow (E)$	$T.node = E.node$
5) $T \rightarrow \text{id}$	$T.node = \text{new Leaf}(\text{id}, \text{id.entry})$
6) $T \rightarrow \text{num}$	$T.node = \text{new Leaf}(\text{num}, \text{num.val})$

Reuse existing nodes

图 6-4 给出的 SDD（语法制导定义）既可以用来构造语法树，也可以用来构造 DAG。它在例 5.11 中曾用于构造语法树。在那里，函数 Leaf 和 Node 每次被调用都会构造出一个新结点。要构造得到 DAG, 这些函数就要在每次构造新结点之前首先检查是否已存在这样的结点。如果存在一个已被创建的结点，就返回这个已有的结点。例如，在构造一个新结点 Node (op, left, right) 之前，我们首先检查是否已存在一个结点，该结点的标号为 op, 且其两个子结点为 left 和 right。如果存在这样的结点，Node 函数返回这个已存在的结点，否则它创建一个新结点。

Constructing DAG's Cont.

- The construction steps



- 1) $p_1 = \text{Leaf}(\mathbf{id}, \text{entry-}a)$
- 2) $p_2 = \text{Leaf}(\mathbf{id}, \text{entry-}a) = p_1$
- 3) $p_3 = \text{Leaf}(\mathbf{id}, \text{entry-}b)$
- 4) $p_4 = \text{Leaf}(\mathbf{id}, \text{entry-}c)$
- 5) $p_5 = \text{Node}('-', p_3, p_4)$
- 6) $p_6 = \text{Node}('*', p_1, p_5)$
- 7) $p_7 = \text{Node}('+', p_1, p_6)$
- 8) $p_8 = \text{Leaf}(\mathbf{id}, \text{entry-}b) = p_3$
- 9) $p_9 = \text{Leaf}(\mathbf{id}, \text{entry-}c) = p_4$
- 10) $p_{10} = \text{Node}('-', p_3, p_4) = p_5$
- 11) $p_{11} = \text{Leaf}(\mathbf{id}, \text{entry-}d)$
- 12) $p_{12} = \text{Node}('*', p_5, p_{11})$
- 13) $p_{13} = \text{Node}('+', p_7, p_{12})$

图 6.5 所示的步骤序列构建了图 6.3 中的 DAG，前提是 Node 和 Leaf 返回现有的节点（如果可能），如下所示上面讨论过。

我们假设 entry-a 指向 a 的符号表条目，对于其他标识符也是如此。当在步骤 2 中重复调用 Leaf(id;entry-a)时，返回先前调用创建的 node，因此 $p_2 = p_1$ 。类似地，步骤 8 和 9 返回的 nodes 与步骤 3 和 4 返回的相同（即 $p_8 = p_3$ 和 $p_9 = p_4$ ）。因此，第 10 步返回的节点必须与第 5 步返回的节点相同；即 $p_{10} = p_5$

Three-Address Code (三地址代码)

- In three-address code, there is **at most one operator** on the right side of an instruction
 - Instructions are often in the form $x = y \text{ op } z$
- **Operands** (or addresses) can be:
 - **Names** in the source programs
 - **Constants**: a compiler must deal with many types of constants
 - **Temporary names** generated by a compiler

在三地址代码中，一条指令的右侧最多有一个运算符；也就是说，不允许使用组合算术表达式。因此，像 $x+y*z$ 这样的源语言表达式可能会被翻译成三地址指令序列：

其中 $t1$ 和 $t2$ 是编译器生成的临时名称。多运算符算术表达式和嵌套控制流语句的这种分解使得三地址代码成为目标代码生成和优化的理想选择，如第 8 章和第 9 章所述。中间值名称的使用通过程序计算可以轻松地重新排列三地址代码。

地址可以是以下之一：

- 名称：为了方便起见，我们允许源程序名称作为三地址代码中的地址出现。

在实现中，源名称由指向其符号表条目的指针替换，其中保留了该名称的所有信息。

- 常量：实际上，编译器必须处理许多不同类型的常量和变量。第 6.5.2 节考虑了表达式内的类型转换。

- 编译器生成的临时文件：每次需要临时文件时创建一个不同的名称非常有用，特别是在优化编译器时。当寄存器分配给变量时，如果可能的话，可以组合这些临时变量。

Instructions (1)

1. **Assignment instructions:**
 - $x = y \text{ op } z$, where op is a binary arithmetic/logical operation
 - $x = \text{op } y$, where op is a unary operation
2. **Copy instructions:** $x = y$
3. **Unconditional jump instructions:** $\text{goto } L$, where L is a label of the jump target
4. **Conditional jump instructions:**
 - $\text{if } x \text{ goto } L$
 - $\text{if Flase } x \text{ goto } L$
 - $\text{if } x \text{ relop } y \text{ goto } L$

现在我们考虑本书其余部分中使用的常见三地址指令。符号标签将由改变控制流的指令使用。符号标签表示指令序列中三地址指令的索引。实际索引可以通过单独传递或通过“回填”来替换标签，第 6.7 节中讨论了这一点。以下是常见三地址指令形式的列表：

1 赋值指令：

$x = y \text{ op } z$ ，其中 op 是二进制算术/逻辑运算， x 、 y 和 z 是地址。

$x = \text{op } y$ ，其中 op 是一元运算。基本的一元运算包括一元减、逻辑非和转换运算符，例如，将整数转换为浮点数。

2 复制指令： $x = y$ ，其中 x 被赋予 y 的值

3 无条件跳转指令： $\text{goto } L$ ，其中 L 是跳转目标的标记，带有标签 L 的三地址指令是下一个要执行的指令。

4 有条件跳转指令：

$\text{if } x \text{ goto } L$

$\text{if Flase } x \text{ goto } L$

$\text{if } x \text{ relop } y \text{ goto } L$

if x goto L 和 ifFalse x goto L 形式的条件跳转。如果 x 为 true 和 false, 这些指令分别执行接下来带有标号 L 的指令。否则, 照常执行接下来的三地址指令

条件跳转, 例如 if x relop y goto L, 它将关系运算符 (<、==、>= 等) 应用于 x 和 y, 如果 x 与 relop 与 y 存在关系, 则接下来执行标号为 L 的指令。如果不是, 则依次执行 if x relop y goto L 后面的三地址指令

Instructions (2)

5. Procedural calls and returns

- param x_1
- ...
- param x_n
- call p, n (procedure call)
- $y = \text{call } p, n$ (function call)
- return y

6. Indexed copy instructions: $x = y[i]$ $x[i] = y$

- Here, $y[i]$ means the value in the location i memory units beyond location y

7. Address and pointer assignment instructions:

- $x = \&y$ $x = *y$ $*x = y$

5 程序调用和返回:

过程调用和返回是使用以下指令实现的: param x 为参数; call p, n 和 y = call p, n 分别用于过程和函数调用; return y, 其中 y 表示返回值, 是可选的。这些三地址指令的常见用法见下面的三地址指令序列

它是 p($x_1 ; x_2 ; \dots ; x_n$) 调用的一部分。“call p, n” 中的 n 是实际参数的数量。这个 n 并不是多余的, 因为调用可以嵌套。也就是说, 前面的一些 param 语句可能是 p 返回之后才执行的某个函数调用的参数, 而 p 的返回值又成为这个后续函数调用的另一个参数

6 带有下标的复制指令:

$x = y[i]$ 这里, $y[i]$ 表示位置 i 存储单元中超出位置 y 的值。

$x[i] = y$ 这里, $y[i]$ 表示位置 i 存储单元中超出位置 y 的值。

7 地址和指针赋值指令：

$x = \&y$, $x = *y$, $*x = y$

$x = \&y$ 、 $x = *y$ 和 $*x = y$ 形式的地址和指针赋值。指令 $x = \&y$ 将 x 的右值设置为 y 的位置（左值）。这个 y 通常是一个名称，也可能是一个临时变量，它表示一个诸如 `Array` 这样具有左值的表达式。 x 是指针名称或临时指针。在指令 $x = *y$ 中， y 可能是一个指针或一个临时变量，其右值是一个内存位置。这个指令使得 x 的右值等于存储在该位置中的值。最后，指令 $*x = y$ 将 y 的右值 x 赋给由 x 指向的目标的右值。

Example

- Source code: `do i = i + 1; while (a[i] < v);`

```
L:  t1 = i + 1
    i = t1
    t2 = i * 8
    t3 = a [ t2 ]
    if t3 < v goto L
```

(a) Symbolic labels.

```
100: t1 = i + 1
101: i = t1
102: t2 = i * 8
103: t3 = a [ t2 ]
104: if t3 < v goto 100
```

(b) Position numbers.

Assuming each array element takes 8 units of space

该语句的两种可能的翻译如图 6.9 所示。图 6.9(a) 中的翻译使用了附加到第一条指令的符号标签 `L`。(b) 中的翻译显示了每一条指令的位置编号，图中选择以 100 开始。在两个翻译中，最后一条指令都是目标为第一条指令的条件跳转指令。乘法运算 `i*8` 适用于每个元素占用 8 个存储单元的数组。

选择使用哪些运算符是中间表示形式设计的一个重要问题。显然，这个运算符集合中的运算符要足够丰富，以便实现源语言中的所有运算。接近机器指令的运算符可以使在目标机器上实现中间表示形式更加容易。然而，如果前端必须为某些源语言运算生成很长的

指令序列，那么优化器和代码生成器就需要花费更多的时间去重新发现程序的结构，然后才能为这些运算生成高质量的目标代码。

Representation of Instructions

- The description of three-address instructions specifies the **components of each type of instruction**, but does not specify the representation of instructions
- In a compiler, these instructions can be implemented as **objects/records** with fields for the operator and the operands
- Three typical representations:
 - **Quadruples** (四元式表示方法)
 - **Triples** (三元式表示方法)
 - **Indirect triples** (间接三元式表示方法)

三地址指令的描述指定了每种指令的组成部分，但没有指定这些指令在数据结构中的表示。

在编译器中，这些指令可以实现为对象或带有运算符和操作数字段的记录。

三个这样的表示称为“四元组”、“三元组”和“间接三元组”。

四元式表示方法

三元式表示方法

间接三元式表示方法

Quadruples (四元式)

- A *quadruple* (or “*quad*”) has four fields
 - General form: *op arg₁ arg₂ result*
 - *op* contains an *internal code* for the operator
 - *arg₁, arg₂, result* are *addresses* (operands)
 - Example: $x = y + z \rightarrow + \quad y \quad z \quad x$
- **Unary operators** like $x = \text{minus } y$ or $x = y$ do not use *arg₂*
- **param operators** use neither *arg₂* nor *result*
- **Conditional/unconditional jumps** put the target label in *result*

一个四元组（或 “四元组”）有四个字段

一般形式： *op arg1 arg2 result*

字段 *op* 包含一个运算符的内部代码

arg1、*arg2*、*result* 是地址（操作数）

例如，三地址指令 $x = y + z$ 通过将 $+$ 放入 *op*、将 *y* 放入 *arg1*、将 *z* 放入 *arg2* 以及将 *x* 放入 *result* 来表示。

以下是该规则的一些例外情况

一元运算符（如 $x = -y$ 或 $x = y$ ）不使用 *arg2*

参数运算符既不使用 *arg2* 也不使用 *result*

条件/非条件跳转将目标标签放在 *result* 中

Quadruples Example

- Assignment statement: $a = b * -c + b * -c$

```
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a = t5
```

(a) Three-address code

	op	arg ₁	arg ₂	result
0	minus	c		t ₁
1	*	b	t ₁	t ₂
2	minus	c		t ₃
3	*	b	t ₃	t ₄
4	+	t ₂	t ₄	t ₅
5	=	t ₅		a
	...			

(b) Quadruples

Temporaries

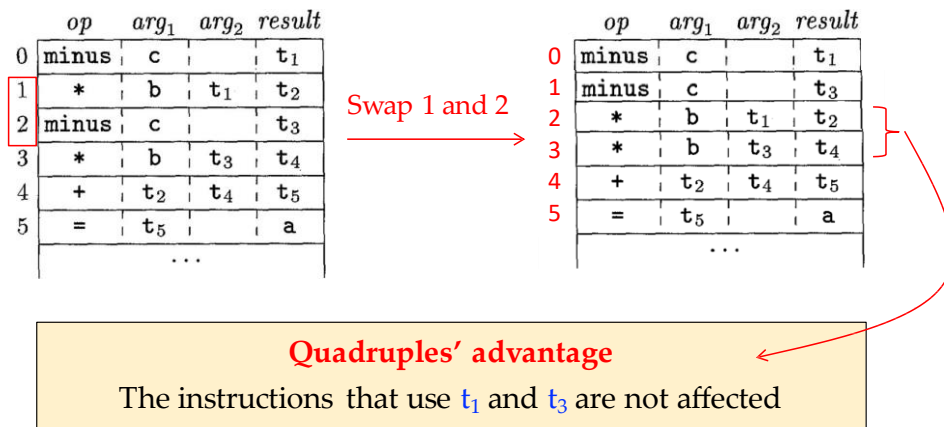
The result field is used primarily for temporary names
Temporary names waste space (symbol table entries)

赋值的三地址代码 $a = b * -c + b * -c$ 的三地址代码如图 6.10(a) 所示。这里我们使用特殊的 minus 运算符来表示“-c”中的单目减运算符“-”，以区别于“b-c”中的双目减运算符“-”。请注意，单目减的三地址语句中只有两个地址，复制语句 $a = t_5$ 也是如此。

为了便于阅读，我们在 arg₁、arg₂ 字段中使用 a、b 和 c 等实际标识符，结果如图 6.10(b) 所示，而不是指向其符号表条目的指针。临时名称可以像程序员定义的名称一样输入到符号表中，也可以实现为 Temp 类的对象，这个 temp 类有自己的方法。

Problem of Triples

- In optimizing compilers, instructions are often moved around

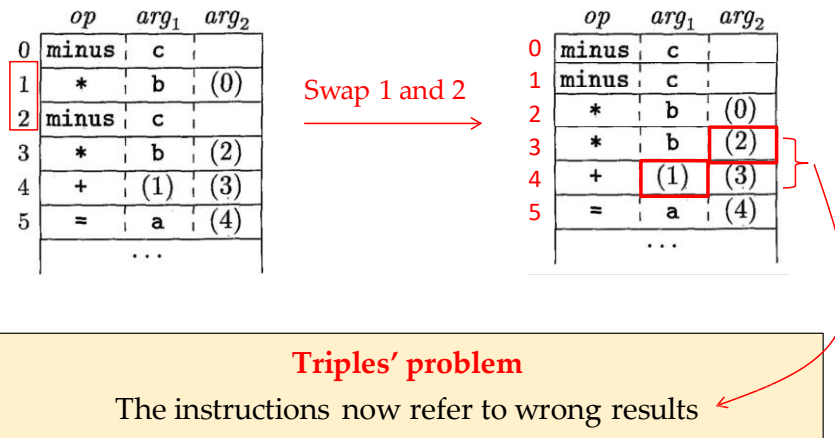


在优化编译器中，由于指令的位置常常会发生变化，四元式相对于三元式的优势就体现出来了。

使用四元式时，如果我们移动了一个计算临时变量 t 的指令，那些使用 t 的指令不需要做任何改变。而使用三元式时，对于运算结果的引用是通过位置完成的，因此如果改变一条指令的位置，则引用该指令的结果的所有指令都要做相应的修改。

Problem of Triples Cont.

- In optimizing compilers, instructions are often moved around

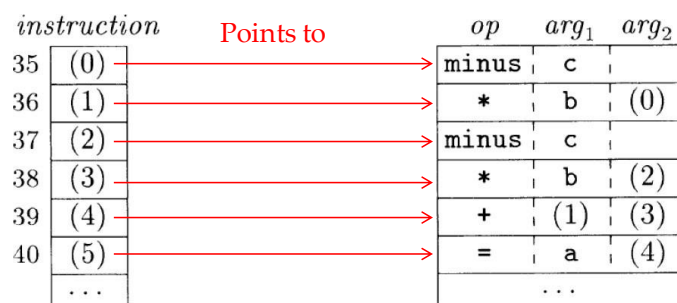


三地址的问题:

说明现在提到了错误的结果，位置变了

Indirect Triples (间接三元式)

- Indirect triples* consist a list of **pointers** to triples (the *instruction* array below)

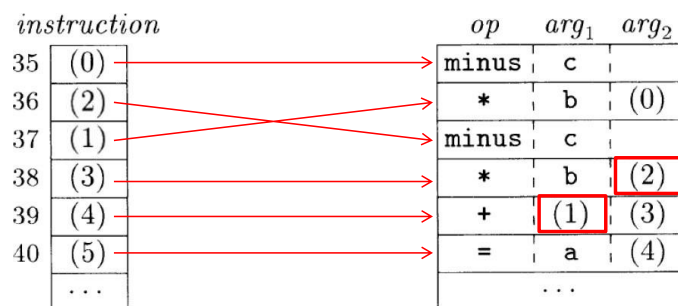


间接三元组由一个指向三元组的指针列表（下面的指令数组）

间接三元式(indirect triple)包含了一个指向三元式的指针的列表，而不是列出三元式序列本身。例如，我们可以使用数组 *instruction* 按照适当的顺序列出指向三元式的指针。这样，图 6-11b 中的三元式序列就可以表示成为图 6-12 所示的形式。

Indirect Triples (间接三元式)

- An optimization can move an instruction by reordering the *instruction* list (without modifying the triples)



Pointers!

After swapping, the 1st and 2nd instructions in the list still point to the right ones

优化可以通过重新排序指令列表来移动指令（无需修改三元组）

交换后，列表中的第一条和第二条指令仍然指向正确的指令

Static Single-Assignment Form (1)

- **Static single-assignment** form (SSA, 静态单赋值形式) is an IR that facilitates certain code optimizations
- In SSA, each name receives **a single assignment**

```
p = a + b
q = p - c
p = q * d
p = e - p
q = p + q
```

```
p1 = a + b
q1 = p1 - c
p2 = q1 * d
p3 = e - p2
q2 = p3 + q1
```

(a) Three-address code. (b) Static single-assignment form.

静态单赋值形式静态单赋值形式（SSA，Static Single-Assignment FormStatic single-assignment form）是一种 IR，有助于某些代码优化。

在 SSA 中，每个名称只接受一次赋值。

SSA 和三地址代码的区别主要体现在两个方面。首先，SSA 中的所有赋值都是针对具有不同名字的变量的，这也是“静态单赋值”这一名字的由来。

图 6-13 给出了分别以三地址代码形式和静态单赋值形式表示的中间程序。注意，SSA 表示中对变量 p 和 q 的每次定值都以不同的下标加以区分。

Static Single-Assignment Form (2)

- The same variable may be defined in two different control-flow paths

```
if ( flag ) x = -1; else x = 1;  
y = x * a;
```



Which name should we use in $y = x * a$?

同一变量可在两个不同的控制流路径中定义

在一个程序中，同一个变量可能在两个不同的控制流路径中被定值。例如，下列源程序

中，x 在两个不同的控制流路径中被定值。如果我们对条件语句

的真分支和假分支中的 x 使用不同的变量名，那么我们应该在赋值运算 $y=x*a$ 中使用哪个名字？

Static Single-Assignment Form (3)

- The same variable may be defined in two different control-flow paths

```
if ( flag ) x = -1; else x = 1;  
y = x * a;
```

- SSA uses a notational convention called ϕ -function to combine the two definitions of x

```
if ( flag )  $x_1$  = -1; else  $x_2$  = 1;  
 $x_3$  =  $\phi(x_1, x_2)$ ; //  $x_1$  if control flow passes through the true part; otherwise  $x_2$   
y =  $x_3$  * a;
```

同一变量可在两个不同的控制流路径中定义:SSA 使用一种称为 ϕ 函数的符号惯例来合并 x 的两个定义

这也是 SSA 的第二个特别之处。SSA 使用一种被称为 ϕ 函数的表示规则将 x 的两处定值合并起来:

如果控制流经过这个条件语句的真分支, $\phi(x_1, x_2)$ 的值为 x_1 ; 否则, 如果控制流经过假分支, ϕ 函数的值为 x_2 。也就是说, 根据到达包含 ϕ 函数的赋值语句的不同控制流路径, ϕ 函数返回不同的参数值。

Types and Type Checking

- *Data type* or simply *type* tells a compiler or interpreter how the programmers intend to use the data
- The usefulness of type information
 - Find faults in the source code
 - Determine the storage needed for a name at runtime
 - Calculate the address of an array element
 - Insert type conversions
 - Choose the right version of an arithmetic operator (e.g., `fadd`, `iadd`)
- *Type checking* (类型检查) uses logical rules to make sure that the types of the operands match the type expected by an operator

数据类型或简单的类型告诉编译器或解释器程序员打算如何使用数据

类型信息的作用

查找源代码中的错误

在运行时确定名称所需的存储空间

计算数组元素的地址

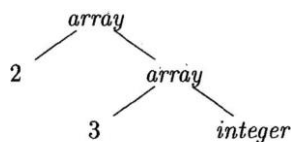
插入类型转换

选择算术运算符的正确版本（如 `fadd`、`iadd`）

类型检查(类型检查) 使用逻辑规则确保操作数的类型与运算符所期望的类型匹配。更明确地讲，类型检查保证运算分量的类型和运算符的预期类型相匹配。例如，Java 要求 `&` 运算符的两个运算分量必须是 `boolean` 型。如果满足这个条件，结果也具有 `boolean` 类型。

Type Expressions (类型表达式)

- **Types have structure**, which can be represented by *type expressions*
 - A type expression is either a basic type, or
 - Formed by applying a *type constructor* (类型构造算子) to a type expression
- **`array(2, array(3, integer))`** is the type expression for `int[2][3]`
 - **`array`** is a type constructor with two arguments: a number, a type expression



类型具有结构，可以用类型表达式来表示：

类型表达式可能是基本类型，

也可能通过把被称为类型构造算子的运算符作用于类型表达式而得到

数组类型 `int[2][3]` 表示 “由两个数组组成的数组，其中的每个数组各包含 3 个整数”。

它的类型表达式可以写成 `array (2, array (3, integer))`。该类型可以用如图 6-14 所示的树来描述。`array` 运算符有两个参数：一个数字和一个类型。

The Definition of Type Expression

- A basic type is a type expression
 - *boolean, char, integer, float, and void, ...*
- A type name (e.g., name of a class) is a type expression
- A type expression can be formed
 - By applying the *array* type constructor to a number and a type expression
 - By applying the *record* type constructor to the field names and their types
 - By applying the $\rightarrow$ type constructor for function types
- If *s* and *t* are type expressions, then their Cartesian product $s \times t$ is a type expression (e.g., represent a tuple of types such as function parameters)

* Type expressions may contain type variables (e.g., those generated by compilers) whose values are type expressions

基本类型是一种类型表达式

类型名称（如类名）是一个类型表达式

类型表达式可以通过以下方式形成

对数字和类型表达式应用 *array* 类型构造函数

对字段名及其类型应用 *record* 类型构造函数

对函数类型应用 $\rightarrow$ 类型构造函数

如果 *s* 和 *t* 是类型表达式，那么它们的笛卡尔乘积 $s \times t$ 就是一个类型表达式（例如，表示函数参数等类型的元组）。

* 类型表达式可能包含类型变量（例如由编译器生成的变量），其值为类型表达式

Type Equivalence

- Type checking rules usually have the following form

If two type expressions are equivalent
then return a given type
else return **type_error**

Code under analysis:
`a + b`

- The key is to define when two type expressions are equivalent
 - **The main difficulty** arises from the fact that most modern languages allow the naming of user-defined types
 - In C/C++, type naming is achieved by the `typedef` statement

类型检查规则通常有以下形式

如果两个类型表达式等价，则返回给定类型，否则返回 `type_error`

关键是定义两个类型表达式何时等价

主要的困难来自于大多数现代语言都允许对用户定义的类型进行命名。

在 C/C++ 中，类型命名是通过 `typedef` 语句实现的

Name Equivalence (名等价)

- Treat named types as basic types; **names in type expressions are not replaced** by the exact type expressions they define
- Two type expressions are name equivalent if and only if **they are identical** (represented by the same syntax tree, with the same labels)

```
typedef struct { int  
    data[100]; int  
    count;  
} Stack;
```

```
typedef struct { int  
    data[100]; int  
    count;  
} Set;
```

Code under analysis:

Stack x, y;

Set r, s;

x = y; ✓

r = s; ✓

x = r; ✗

<http://web.eecs.utk.edu/~bvanderz/teaching/cs365Sp14/notes/types.html>

将命名的类型视为基本类型；类型表达式中的名称不会被其定义的类型表达式所替换
当且仅当两个类型表达式完全相同时，它们的名称才是等价的(由相同的语法树表示，
具有相同的标签)

Structural Equivalence (结构等价)

- For named types, replace the type names by their type expressions and recursively check the substituted trees

```
typedef struct { int  
    data[100]; int  
    count;  
} Stack;
```

```
typedef struct { int  
    data[100]; int  
    count;  
} Set;
```

Code under analysis:

```
Stack x, y;
```

```
Set r, s;
```

```
x = y; ✓
```

```
r = s; ✓
```

```
x = r; ✓
```

对于已命名的类型，用其类型表达式替换类型名，并递归检查替换后的树

Declarations (类型的声明)

- The grammar below deals with basic, array, and record types
 - Nonterminal *D* generates a sequence of declarations
 - T* generates basic, array, or record types, A record type is a sequence of declarations for the fields of the record, surrounded by curly braces
 - B* generates one of the basic types: `int` and `float`
 - C* generates sequences of one or more integers, each surrounded by brackets

$$\begin{aligned} D &\rightarrow T \text{ id } ; D \mid \epsilon \\ T &\rightarrow B C \mid \text{record } \{ D \} \\ B &\rightarrow \text{int} \mid \text{float} \\ C &\rightarrow \epsilon \mid [\text{num}] C \end{aligned}$$

下面的语法涉及基本类型、数组类型和记录类型

非终结符号 D 生成一系列声明。

非终结符号 T 生成基本类型、数组类型或记录类型。

非终结符号 B 生成基本类型 `int` 和 `float` 之一。

非终结符号 C（表示“分量”）产生零个或多个整数，每个整数用方括号括起来。

一个数组类型包含一个由 B 指定的基本类型，后面跟一个由非终结符号 C 指定的数组分。一个记录类型(T 的第二个产生式) 由各个记录字段的声明序列构成，并被花括号括起来。

Storage Layout for Local Names (局部变量的存储布局)

- From the type of a name, we can decide the amount of memory needed for the name at run time
 - The *width (宽度)* of a type: # memory units needed for an object of the type
 - For data of varying lengths, such as strings, or whose size cannot be determined until run time, such as dynamic arrays, we only reserve a fixed amount of memory for a pointer to the data
- For **local names of a function**, we always assign contiguous bytes*
 - For each such name, at compile time, we can compute a **relative address**
 - Type information and relative addresses are stored in **symbol table**

* This follows the principle of proximity and is mainly for performance considerations.

根据名称的类型，我们可以决定运行时名称所需的内存量。运行时名称所需的内存量
类型的宽度（宽度）： 该类型对象所需的 # 内存单位

对于长度不等的的数据（如字符串），或在运行前无法确定大小的数据（如动态数组），
我们只为数据指针预留固定的内存量。

对于函数的局部名称，我们总是分配连续的字节*。

对于每个这样的名称，我们可以在编译时计算出一个相对地址

类型信息和相对地址存储在符号表中

* 这遵循就近原则，主要是出于性能考虑。

An SDT for Computing Types and Their Widths

- **Synthesized attributes**: *type, width*
- Global variables *t* and *w* pass type and width information from a *B* node in a parse tree to the node for the production $C \rightarrow \epsilon$
 - In an SDD, *t* and *w* would be *C*'s **inherited attributes** (the SDD is L-attributed)*

$T \rightarrow B$	$\{ t = B.type; w = B.width; \}$
C	$\{ T.type = C.type; T.width = C.width; \}$
$B \rightarrow \text{int}$	$\{ B.type = \text{integer}; B.width = 4; \}$
$B \rightarrow \text{float}$	$\{ B.type = \text{float}; B.width = 8; \}$
$C \rightarrow \epsilon$	$\{ C.type = t; C.width = w; \}$
$C \rightarrow [\text{num}] C_1$	$\{ C.type = \text{array}(\text{num.value}, C_1.type);$ $C.width = \text{num.value} \times C_1.width; \}$

This SDT can be implemented during recursive-descent parsing

合成属性：类型、宽度

全局变量 *t* 和 *w* 将类型和宽度信息从解析树中的 *B* 节点传递到生成 $C \rightarrow \epsilon$ 的节点。

节点传递类型和宽度信息。在 SDD 中，*t* 和 *w* 是 *C* 的继承属性（SDD 是 L 属性）*。（该 SDT 可以在递归下降分析过程中调用）

T 产生式的产生式体包含一个非终结符号 *B*、一个动作和一个非终结符号 *C*，其中 *C* 显示在下一行上。*B* 和 *C* 之间的动作是将 *t* 设置为 *B.type*，并将 *w* 设置为 *B.width*。

如果 $B \rightarrow \text{int}$ ，则 *B.type* 被设置为 *integer*，*B.width* 被设置为 4，即一个整型数的宽度。

类似的，如果 $B \rightarrow \text{float}$ ，则 *B.type* 和 *B.width* 分别被设置为 *float* 和 8，即宽度为一个浮点数的宽度。

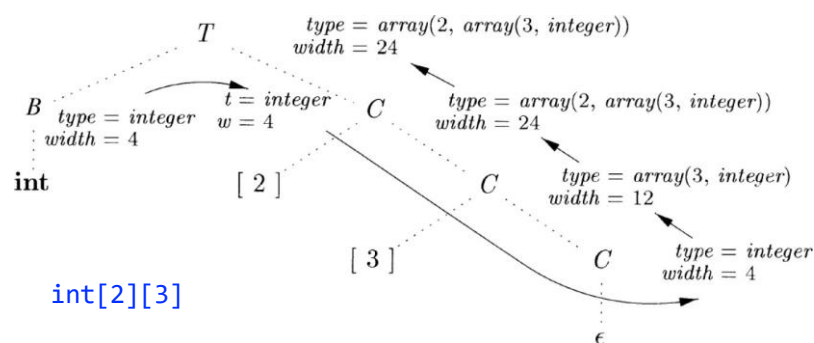
C 的产生式决定了 *T* 生成的是一个基本类型还是一个数组类型。如果 $C \rightarrow \epsilon$ ，则 *t* 变成 *C.type*，且 *w* 变成 *C.width*。

否则, C 就描述了一个数组分量。C $\rightarrow$ [num]C1 的动作将类型构造算子 array 应用于运算分值 num. value 和 C1. type,构造得到 C. type。例如, 应用 array 的结果可能是图 6-14 所示的树形结构。

数组的宽度是将数组元素的个数乘以单个数组元素的宽度而得到。如果连续存放的整数的地址之间的差距为 4,那么一个整数数组的地址计算将包含乘 4 运算。这样的乘法运算为优化提供了机会, 因此让前端程序在其输出中明确描述这些运算将有助于优化。在这一章中, 我们将忽略其他与机器相关特性, 比如数据对象的地址必须和机器字的边界对齐。

Translation Process Example

- Recall the translation during recursive-descent parsing
 - Use the arguments of function A() to pass nonterminal A's **inherited attributes***
 - Evaluate and Return the **synthesized attributes** of A when the A() completes



* In our example, we use global variables *t* and *w*

在递归下降解析过程中调用翻译

使用函数的参数 A() 传递非终结符 A' s 继承属性* (在例子中我们使用全局变量 t 和 w)

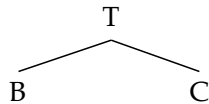
计算并返回的综合属性 A 当 A() 完成

类型 int[2] [3] 的语法分析树用图 6-16 中的虚线描述。图中的实线描述了 type 和 width 是如何从 B 结点开始, 通过变量 t 和 w, 沿着多个 C 组成的链下传, 然后又作为综合属性 type 和 width 沿此链返回的。在访问包含 C 结点的子树之前, 变量 t 和 w 被赋予 B. type

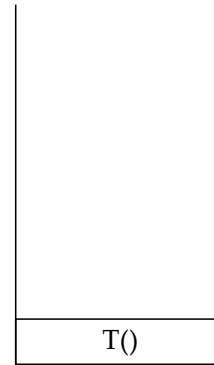
和 B.width 的值。变量 t 和 w 的值在 $C \rightarrow \epsilon$ 对应的结点上使用，然后开始沿着多个 C 结点组成的链向上对综合属性求值。

Translation Process Example

Input string: `int[2][3]`



Step 1: Rewrite T using $T \rightarrow BC$

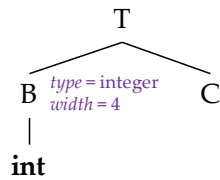


Call stack

步骤 1: 使用 $T \rightarrow BC$ 重写 T

Translation Process Example

Input string: `int[2][3]`



Step 2:

- Rewrite B using $B \rightarrow \mathbf{int}$
- Match input
- Execute semantic action

$B \rightarrow \mathbf{int} \quad \{ B.type = integer; B.width = 4; \}$

B()
T()

Call stack

步骤 2:

使用 $B \rightarrow \mathbf{int}$ 重写 B 。

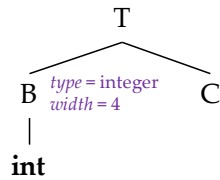
匹配输入

执行语义操作

Translation Process Example

Input string: `int[2][3]`

$t = \text{integer}$
 $w = 4$

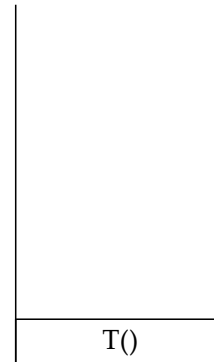


Step 3:

- B() returns
- Execute semantic action

$T \rightarrow B$
 C

$\{ t = B.type; w = B.width; \}$
 $\{ T.type = C.type; T.width = C.width; \}$



Call stack

步骤 3:

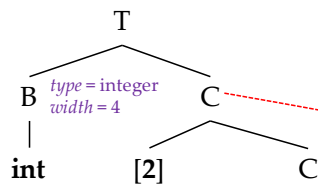
B() 返回

执行语义操作

Translation Process Example

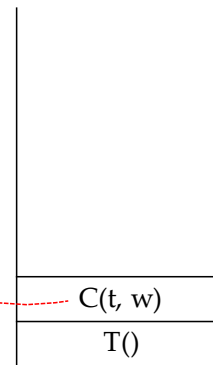
Input string: `int[2][3]`

$t = \text{integer}$
 $w = 4$



Step 4:

- Rewrite C using $C \rightarrow [\text{num}]C$
- Match input



Call stack

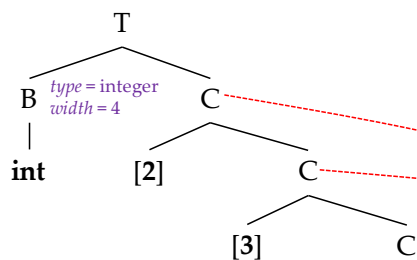
使用 $C \rightarrow [\text{num}]C$ 重写 C

匹配输入

Translation Process Example

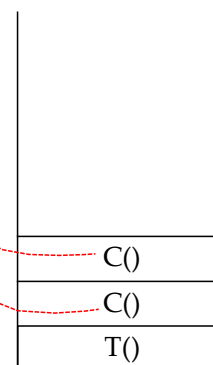
Input string: `int[2][3]`

$t = \text{integer}$
 $w = 4$



Step 5:

- Rewrite C using $C \rightarrow [\text{num}]C$
- Match input



Call stack

匹配输入

Input string: `int[2][3]`

Step 5:

- Rewrite C using $C \rightarrow \epsilon$

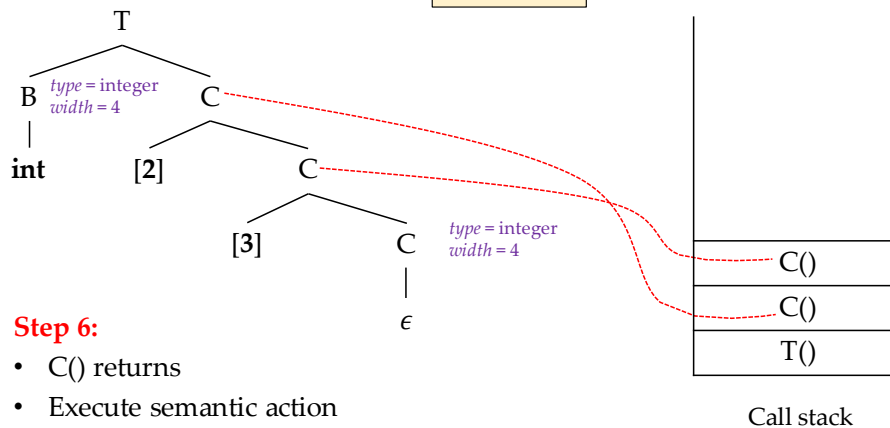
43

使用 $C \rightarrow \epsilon$ 重写

Translation Process Example

Input string: `int[2][3]`

$t = \text{integer}$
 $w = 4$



$C \rightarrow \epsilon$ $\{ C.type = t; C.width = w; \}$

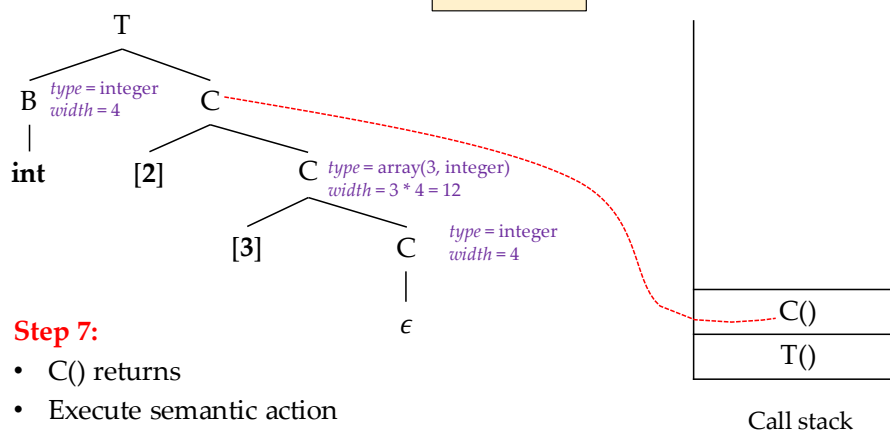
C() 返回

执行语义操作

Translation Process Example

Input string: `int[2][3]`

$t = \text{integer}$
 $w = 4$



$C \rightarrow [\text{num}] C_1$ $\{ C.type = \text{array}(\text{num.value}, C_1.type);$
 $C.width = \text{num.value} \times C_1.width; \}$

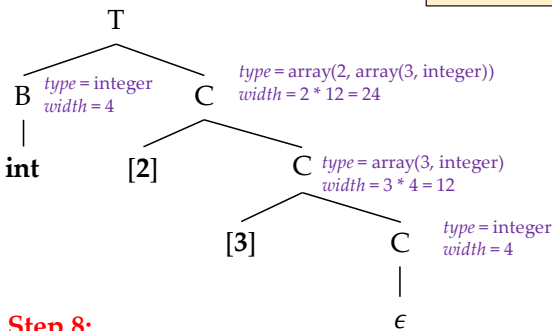
C() 返回

执行语义操作

Translation Process Example

Input string: `int[2][3]`

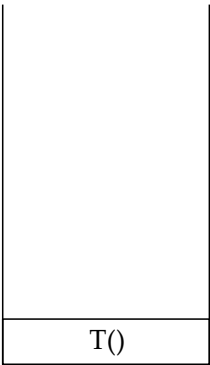
`t = integer`
`w = 4`



Step 8:

- C() returns
- Execute semantic action

$C \rightarrow [\text{num}] C_1$ $\{ \begin{array}{l} C.type = \text{array}(\text{num.value}, C_1.type); \\ C.width = \text{num.value} \times C_1.width; \end{array} \}$



Call stack

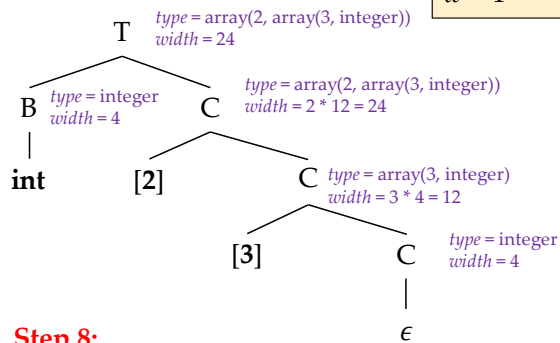
C() 返回

执行语义操作

Translation Process Example

Input string: `int[2][3]`

`t = integer`
`w = 4`



Step 8:

- T() returns
- Execute semantic action

$T \rightarrow B$
 C

$\{ t = B.type; w = B.width; \}$
 $\{ T.type = C.type; T.width = C.width; \}$

Call stack

T() 返回

执行语义操作

Sequences of Declarations

- When dealing with a procedure, local variables should be put in a separate symbol table; their declarations can be processed as a group
 - Name, type, and relative address of each variable should be stored
- The translation scheme below handles a sequence of declarations
 - *offset*: the next available relative address; *top*: the current symbol table

$P \rightarrow D \quad \{ offset = 0; \}$

$D \rightarrow T \text{ id } ; \quad \{ top.put(id.lexeme, T.type, offset);$
 $offset = offset + T.width; \}$

$D \rightarrow D_1$

$D \rightarrow \epsilon$

Computing relative addresses of declared names

像 C 和 Java 这样的语言支持将单个过程中的所有声明作为一个组进行处理。这些声明可能分布在一个 Java 过程中，但是仍然能够在分析该过程时处理它们。因此，我们可以使用一个变量，比如 `offset`，来跟踪下一个可用的相对地址。

图 6-17 中的翻译方案处理形如 `T id` 的声明的序列，其中的 `T` 如图 6-15 所示产生一个类型。

在考虑第一个声明之前，`offset` 被设置为 0。每处理一个变量 `x` 时，`x` 被加入符号表，它的相对地

址被设置为 `offset` 的当前值。随后，`x` 的类型的宽度被加到 `offset` 上。

产生式 $D \rightarrow T \text{ id} ; D1$ 中的语义动作首先执行 `top.put(id.lexeme, T.type, offset)`，创建一个符号表条目。这里的 `top` 指向当前的符号表。方法 `top.put` 为 `id.lexeme` 创建一个符号表条目，该条目的数据区中存放了类型 `T.type` 和相对地址 `offset`。

如果我们把第一个产生式写在同一行中：

$$P \rightarrow \{ \text{offset} = 0 ; \} D \quad (6.1)$$

则图 6-17 中对 `offset` 的初始化处理就变得更加容易理解。生成 ϵ 的非终结符号称为标记非终结符号，其作用是重写产生式，使得所有的语义动作都出现在产生式右部的尾端，具体方法见 5.5.4 节。

Fields in Records and Classes*

- Two assumptions:
 - The field names within a record must be distinct
 - The offset for a field name is relative to the data area (数据区) for that record
- For convenience, we use a symbol table for each record type
 - Store both type and relative address of fields
- A record type has the form *record(t)*
 - *record* is the type constructor
 - *t* is a symbol table object, holding info about the fields of this record type

* Self-study materials

图 6-17 中对声明的翻译方案还可以用于处理记录和类中的字段。要把记录类型加入到图 6-15 所示的文法中，只需要加上下面的产生式：

$$T \rightarrow \text{record}\{ 'D' \}$$

这个记录类型中的字段由 D 生成的声明序列描述。图 6-17 中的方法可以用来确定这些字段的类型和相对地址，当然我们需要小心地处理下面两件事：

- 一个记录中各个字段的名称必须是互不相同的。也就是说，在由 D 生成的声明中，同一个名称最多出现一次。

- 字段名的偏移量，或者说相对地址，是相对于该记录的数据区字段而言的。

为了方便起见，记录类型将使用一个专用的符号表，对它们的各个字段的类型和相对地址进行编码。

记录类型形如 $\text{record}(t)$ ，其中 record 是类型构造算子， t 是一个符号表对象，它保存了有关该记录类型的各个字段的信息。

记录类型的形式为 $\text{record}(t)$

record 是类型构造函数

t 是一个符号表对象，包含该记录类型字段的信息

Fields in Records and Classes

$T \rightarrow \text{record}\{ ' \}$	$\{ \text{Env.push}(top); top = \text{new Env};$ $\text{Stack.push}(offset); offset = 0; \}$
$D ' \}$	$\{ T.type = \text{record}(top); T.width = offset;$ $top = \text{Env.pop}; offset = \text{Stack.pop}; \}$

- The class *Env* implements symbol tables
- *Env.push(top)* and *Stack.push(offset)* save the current symbol table and offset; later, they will be popped to continue with other translation
- The translation scheme can be adapted to deal with classes

为了使翻译方案更加具体，图 6-18 中的动作给出了某个实现的伪代码。

令 Env 类实现符号表。

对 Env.push(top)的调用将 top 所指的当前符号表压入一个栈中。然后，变量 top 被设置为指向一个新的符号表。类似的，offset 被推入名为 Stack 的栈中，offset 变量被重置为 0。

在 D 中的声明被翻译之后，符号表 top 保存了这个记录中所有字段的类型和相对地址。而且，offset 还给出了存放所有字段所需的存储空间。第二个动作将 T.type 设为 record (top)，并将 T.width 设为 offset。然后，变量 top 和 offset 将被恢复为原先被压入栈中的值，以完成这个记录类型的翻译。

有关记录类型存储方式的讨论还可以被推广到类，因为我们无需为类中的方法保留存储空间。

Expressions and Arrays

- An expression with more than one operator: $a + b * c$
 - Translate into multiple instructions with at most one operator per instruction

$$a + b * c \quad \longrightarrow \quad \begin{array}{l} t_1 = b * c \\ t_2 = a + t_1 \end{array}$$

- An array reference $A[i][j]$ will expand into a sequence of three-address instructions that calculate an **address** for the reference

一个带有多个运算符的表达式（比如 $a+b*c$ ）将被翻译成为每条指令最多包含一个运算符的指令序列。

一个数组引用 $A[i][j]$ 将被扩展成一个计算该引用的地址的三地址指令序列。我们将在 6.5 节中考虑表达式的类型检查，并在 6.6 节中介绍如何使用布尔表达式来处理程序的控制流。

SDD for Expressions (1)

PRODUCTION	SEMANTIC RULES
$S \rightarrow \text{id} = E ;$	$S.code = E.code \parallel$ $gen(top.get(id.lexeme) \text{'=' } E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \text{new Temp}()$ $E.code = E_1.code \parallel E_2.code \parallel$ $gen(E.addr \text{'=' } E_1.addr \text{'+' } E_2.addr)$
$ - E_1$	$E.addr = \text{new Temp}()$ $E.code = E_1.code \parallel$ $gen(E.addr \text{'=' 'minus' } E_1.addr)$
$ (E_1)$	$E.addr = E_1.addr$ $E.code = E_1.code$
$ \text{id}$	$E.addr = top.get(id.lexeme)$ $E.code = \text{''}$

S.code and *E.code* denote three-address code

E.addr denotes the address that will hold the value of *E*

top denotes the current symbol table; *get* returns the address of *id* (a variable)

gen generates three-address instructions

All attributes are synthesized. This **S-attributed SDD** can be implemented during bottom-up parsing.

图 6-19 中的语法制导定义使用 *S* 的属性 *code* 以及表达式 *E* 的属性 *addr* 和 *code*, 为一个赋值语句 *S* 生成三地址代码。

属性 *S.code* 和 *E.code* 分别表示 *S* 和 *E* 对应的三地址代码。

属性 *E.addr* 则表示存放 *E* 的值的地址。回忆一下 6.2.1 节，一个地址可以是变量名字、常量或编译器产生的临时量。

SDD for Expressions (2)

PRODUCTION	SEMANTIC RULES
$S \rightarrow \text{id} = E ;$	$S.code = E.code \parallel$ $gen(top.get(\text{id.lexeme}) \neq E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \text{new Temp}()$ $E.code = E_1.code \parallel E_2.code \parallel$ $gen(E.addr \neq E_1.addr + E_2.addr)$
$ - E_1$	$E.addr = \text{new Temp}()$ → Temporary name generated by compiler $E.code = E_1.code \parallel$ $gen(E.addr \neq \text{'minus'} E_1.addr)$
$ (E_1)$	$E.addr = E_1.addr$ $E.code = E_1.code$
$ \text{id}$	$E.addr = top.get(\text{id.lexeme})$ → Check the symbol-table entry for id and save its address in E.addr $E.code = ''$

考虑图 6-19 中语法制导定义的最后产生式 $E \rightarrow \text{id}$ 。若表达式只是一个标识符，比如说 x ，那么 x 本身就保存了这个表达式的值。这个产生式对应的语义规则把 $E.addr$ 定义为指向该 id 的实例对应的符号表条目的指针。

令 top 表示当前的符号表。当函数 $top.get$ 被应用于 id 的这个实例的字符串表示 id.lexeme 时，它返回对应的符号表条目。 $E.code$ 被设置为空串。

当规则为 $E \rightarrow (E_1)$ 时，对 E 的翻译与对子表达式 E_1 的翻译相同。因此， $E.addr$ 等于 $E_1.addr$ ， $E.code$ 等于 $E_1.code$ 。

图 6-19 中的运算符 $+$ 和单目 $-$ 是典型语言中的运算符的代表。 $E \rightarrow E_1 + E_2$ 的语义规则生成了根据 E_1 和 E_2 的值计算 E 的值的代码。计算得到的值存放在新生成的临时变量中。如果 E_1 的值计算后被放入 $E_1.addr$ ， E_2 的值被放到 $E_2.addr$ 中，那么 $E_1 + E_2$ 就可以被翻译为 $t = E_1.addr + E_2.addr$ ，其中 t 是一个新的临时变量。 $E.addr$ 被设为 t 。连续执行 $\text{new Temp}()$ 会产生一系列互不相同的临时变量 $t_1, t_2, \dots$ 。

SDD for Expressions (3)

PRODUCTION	SEMANTIC RULES
$S \rightarrow \text{id} = E ;$	$S.code = E.code \parallel$ $gen(top.get(id.lexeme) \neq E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \text{new Temp}()$ $E.code = E_1.code \parallel E_2.code \parallel$ $gen(E.addr \neq E_1.addr \neq E_2.addr)$
$ - E_1$	$E.addr = \text{new Temp}()$ $E.code = E_1.code \parallel$ $gen(E.addr \neq \text{'minus'} E_1.addr)$
$ (E_1)$	$E.addr = E_1.addr$ $E.code = E_1.code$
$ \text{id}$	$E.addr = top.get(id.lexeme)$ $E.code = \text{'}'$

Concatenate instructions

为方便起见，我们使用记号 $gen(x \neq y \neq z)$ 来表示三地址指令 $x = y + z$ 。当被传递给 gen 时，变量 x 、 y 、 z 的位置上出现的表达式将首先被求值，而像 $\neq$ 这样的引号内的字符串则按照字面值传递。其他的三地址指令的生成方法类似，也是将 gen 作用于表达式和字符串的组合。

当我们翻译产生式 $E \rightarrow E_1 + E_2$ 时，图 6-19 中的语义规则首先将 $E_1.code$ 和 $E_2.code$ 连接起来，然后再加上一条将 E_1 和 E_2 的值相加的指令，从而生成 $E.code$ 。新增加的这条指令将求和的结果放入一个为 E 生成的临时变量中，用 $E.addr$ 表示。

产生式 $E \rightarrow -E_1$ 的翻译过程与此类似。这个规则首先为 E 创建一个新的临时变量，并生成一条指令来执行单目 $-$ 运算。

最终，产生式 $S \rightarrow \text{id} = E;$ 所生成的指令将表达式 E 的值赋给标识符 id 。和规则 $E \rightarrow \text{id}$ 中一样，这个产生式的语义规则使用函数 $top.get$ 来确定 id 所代表的标识符的地址。 $S.code$ 包含的指令首先计算 E 的值并将其保存到由 $E.addr$ 指定的地址中，然后再将这个值赋给这个 id 实例的地址 $top.get(id.lexeme)$ 。

Inefficiency in the SDD

PRODUCTION	SEMANTIC RULES
$S \rightarrow \text{id} = E ;$	$S.code = E.code$ $gen(top.get(id.lexeme) \neq E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \text{new Temp}()$ $E.code = E_1.code E_2.code$ $gen(E.addr \neq E_1.addr + E_2.addr)$
$ - E_1$	$E.addr = \text{new Temp}()$ $E.code = E_1.code$ $gen(E.addr \neq \text{minus } E_1.addr)$
$ (E_1)$	$E.addr = E_1.addr$ $E.code = E_1.code$
$ \text{id}$	$E.addr = top.get(id.lexeme)$ $E.code = ''$

Code attributes can be very long strings (as the expressions can be arbitrarily complex)

code 属性可以是很长的字符串（因为表达式可以任意复杂）

Incremental Translation Scheme

- In the SDT below, *gen* not only *generates* a three-address instruction, but also *appends* it to the sequence of instructions generated so far
- In comparison, in the previous SDD, the *code* attribute can be long strings after concatenations

$S \rightarrow \text{id} = E ;$	$\{ gen(top.get(id.lexeme) \neq E.addr); \}$
$E \rightarrow E_1 + E_2$	$\{ E.addr = \text{new Temp}();$ $gen(E.addr \neq E_1.addr + E_2.addr); \}$
$ - E_1$	$\{ E.addr = \text{new Temp}();$ $gen(E.addr \neq \text{minus } E_1.addr); \}$
$ (E_1)$	$\{ E.addr = E_1.addr; \}$
$ \text{id}$	$\{ E.addr = top.get(id.lexeme); \}$



Why this incremental approach can guarantee the correct order of instructions?

在下面的 SDT 中, *gen* 不仅生成三地址指令, 而且

还将其附加到迄今为止生成的指令序列中

相比之下, 在以前的 SDD 中 *code* 属性可以是串联后的长字符串

为什么这种增量方法可以保证指令的正确顺序?

code 属性可能是很长的字符串, 因此就像 5.5.2 节中讨论的那样, 它们通常是用增量的方式生成的。因此, 我们不会像图 6-19 所示的那样构造 *E.code*, 我们可以设法像图 6-20 中那样只生成新的三地址指令。在这个增量方式中, *gen* 不仅要构造出一个新的三地址指令, 还要将它添加到至今为止已生成的指令序列之后。指令序列可以暂时放在内存中以进一步处理, 也可以增量地输出。

Incremental Translation Scheme

- In the SDT below, *gen* not only *generates* a three-address instruction, but also *appends* it to the sequence of instructions generated so far
- In comparison, in the previous SDD, the *code* attribute can be long strings after concatenations

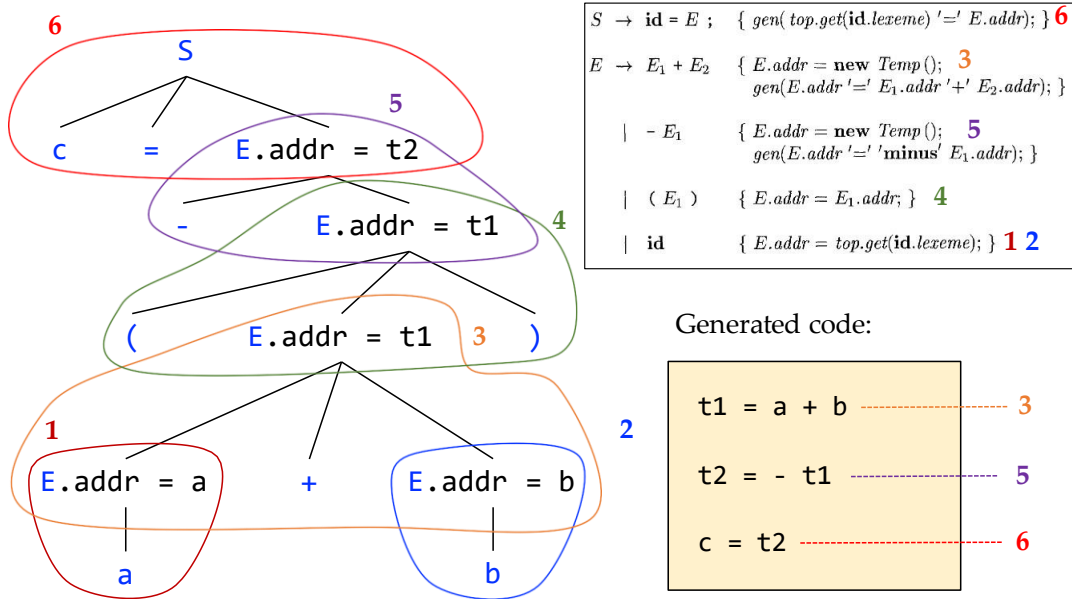
$S \rightarrow \text{id} = E ;$	$\{ \text{gen}(\text{top.get}(\text{id.lexeme}) \neq E.\text{addr}); \}$
$E \rightarrow E_1 + E_2$	$\{ E.\text{addr} = \text{new Temp}();$ $\text{gen}(E.\text{addr} \neq E_1.\text{addr} + E_2.\text{addr}); \}$
$\mid - E_1$	$\{ E.\text{addr} = \text{new Temp}();$ $\text{gen}(E.\text{addr} \neq \text{'minus'} E_1.\text{addr}); \}$
$\mid (E_1)$	$\{ E.\text{addr} = E_1.\text{addr}; \}$
$\mid \text{id}$	$\{ E.\text{addr} = \text{top.get}(\text{id.lexeme}); \}$

This postfix SDT can be implemented in bottom-up parsing where subexpressions are always handled first (e.g., the code of E_1 and E_2 is generated before E)

采用增量方式时不需再用到 *code* 属性, 因为对 *gen* 的连续调用将生成一个指令序列。例如, 图 6-20 中对应于 $E \rightarrow E_1 + E_2$ 的语义规则直接调用 *gen* 产生一条加法指令。在此之前, 翻译方案已经生成了计算 E_1 的值并放入 $E_1.\text{addr}$ 、计算 E_2 的值并放入 $E_2.\text{addr}$ 的指令序列。

这种后缀 SDT 可以在自下而上的解析中实现, 其中总是首先处理子表达式 (例如, E_1 和 E_2 的 *code* 在 E 之前生成)

Example: Translating $c = -(a + b)$



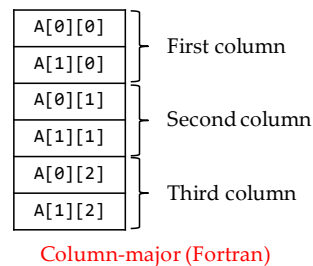
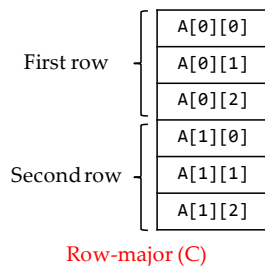
Fall 2024

BUPT Compilers

59

Addressing Array Elements

- Array elements can be accessed quickly if they are stored consecutively
- For an array A with n elements, the **relative address of $A[i]$** is:
 - $base + i * w$ ($base$ is the relative address of $A[0]$, w is the width of an element)
- For a 2D array A (row-major layout), the relative address of $A[i_1][i_2]$ is:
 - $base + i_1 * w_1 + i_2 * w_2$ (w_1 is the width of a row, w_2 is the width of an element)



Fall 2024

BUPT Compilers

60

如果数组元素连续存储，则可以快速访问它们

对于阵列 A 具有 n 元素的相对地址 $A[i]$ 是:

$base + i * w$ ($base$ 是相对地址 $A[0]$, w 是元素的宽度)

对于二维阵列 A (行主布局) $A[i_1][i_2]$ 是:

$base + i_1 * w_1 + i_2 * w_2$ (w_1 是一行的宽度, w_2 是元素的宽度)

Addressing Array Elements

- Array elements can be accessed quickly if they are stored consecutively
- For an array A with n elements, the **relative address of $A[i]$** is:
 - $base + i * w$ ($base$ is the relative address of $A[0]$, w is the width of an element)
- For a 2D array A (row-major layout), the relative address of $A[i_1][i_2]$ is:
 - $base + i_1 * w_1 + i_2 * w_2$ (w_1 is the width of a row, w_2 is the width of an element)
- Further generalize to k -dimensional array A (row-major layout), the relative address of $A[i_1][i_2] \dots [i_k]$ is:
 - $base + i_1 * w_1 + i_2 * w_2 + \dots + i_k * w_k$ (w 's can be generalized as above)

如果数组元素连续存储,则可以快速访问它们

对于阵列 A 具有 n 元素的相对地址 $A[i]$ 是: 一维数组: 假设每个数组元素的宽度是 w , 则数组元素 $a[i]$ 的相对地址是: $base + i \times w$

其中, $base$ 是数组的基地址, $i \times w$ 是偏移地址

$base + i * w$ ($base$ 是相对地址 $A[0]$, w 是元素的宽度)

对于二维阵列 A (行主布局) $A[i_1][i_2]$ 是: 假设一行的宽度是 w_1 , 同一行中每个数组元素的宽度是 w_2 , 则数组元素 $a[i_1][i_2]$ 的相对地址是:

$$base + i_1 \times w_1 + i_2 \times w_2$$

$base + i_1 * w_1 + i_2 * w_2$ (w_1 是一行的宽度, w_2 是元素的宽度)

进一步推广到 k -维度数组 A (行主布局) $A[i_1][i_2] \dots [i_k]$ 是:

$base + i_1 * w_1 + i_2 * w_2 + \dots + i_k * w_k$ (w 's 可以如上所述进行推广)

Translation of Array References

- The main problem in generating code for array references is to **relate the address-calculation formula to the grammar**
 - The relative address of $A[i_1][i_2] \dots [i_k]$ is $base + i_1 * w_1 + i_2 * w_2 + \dots + i_k * w_k$
 - Productions for generating array references: $L \rightarrow L[E] \mid \mathbf{id}[E]$

为数组引用生成代码的主要问题是将地址计算公式与语法联系起来

$A[i_1][i_2][i_k]$ 的相对地址是 $base + i_1 * w_1 + i_2 * w_2 + \dots + i_k * w_k$

用于生成数组引用的产生式: $L \rightarrow L[E] \mid \mathbf{id}[E]$

➤ 假设 $\text{type}(a) = \text{array}(3, \text{array}(5, \text{array}(8, \text{int})))$,
 一个整型变量占用4个字节,

$$\begin{aligned} \text{则 } \text{addr}(a[i_1][i_2][i_3]) &= \text{base} + i_1 * w_1 + i_2 * w_2 + i_3 * w_3 \\ &= \text{base} + i_1 * 160 + i_2 * 32 + i_3 * 4 \end{aligned}$$

$a[i_1]$ 的宽度 ————
 $a[i_1][i_2]$ 的宽度 ————
 $a[i_1][i_2][i_3]$ 的宽度 ————

SDT for Array References (1)

```

L → id [ E ] { L.array = top.get(id.lexeme);
               L.type = L.array.type.elem;
               L.addr = new Temp();
               gen(L.addr '=' E.addr '*' L.type.width); }

| L1 [ E ] { L.array = L1.array;
              L.type = L1.type.elem;
              t = new Temp();
              L.addr = new Temp();
              gen(t '=' E.addr '*' L.type.width);
              gen(L.addr '=' L1.addr '+' t); }
  
```

$L.array$: a pointer to the symbol-table entry for the array name

$L.array.base$: the base address of the array

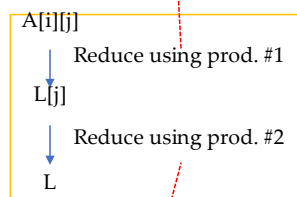
$L.addr$: a temporary for computing the offset for the array reference

$L.type$: the type of the subarray generated by L

$t.elem$: for any array type t , $t.elem$ gives the element type

A is a 2*3 array of integers

$L.type$ is the type of A's element:
 $\text{array}(3, \text{int})$



$L.type$ is the type of A[i]'s element:
 int

与 C 和 Java 中一样，我们假定数组元素的最小编号是 0。我们使用式(6.4)，基于宽度来计算相对地址，而不是像式(6.6)中那样使用元素的数量来计算地址。图 6-22 所示的翻译方案为带有数组引用的表达式生成三地址代码。它包括了图 6-20 中给出的产生式和语义动作，同时还包括了涉及非终结符号 L 的产生式。

非终结符号 L 有三个综合属性：

L.array 是一个指向数组名字对应的符号表条目的指针。在分析了所有的下标表达式之后，该数组的基地址，也就是 L.array.base,被用于确定一个数组引用的实际左值。

L.addr 指示一个临时变量。这个临时变量将被用于累加公式(6.4)中的 $i_j \times w_1$ 项，从而计算数组引用的偏移量。

L.type 是 L 生成的子数组的类型。对于任何类型 t，我们假定其宽度由 t.width 给出。我们把类型（而不是宽度）作为属性，是因为无论如何类型检查总是需要这个类型信息。

对于任何数组类型 t，假设 t.elem 给出了其数组元素的类型。

SDT for Array References (2)

- The semantic actions of L-productions compute offsets
- The address of an array element is $base + offset$

$$\begin{aligned}
 E \rightarrow E_1 + E_2 & \quad \{ E.addr = \text{new Temp}(); \\
 & \quad \quad \quad gen(E.addr := E_1.addr + E_2.addr); \} \\
 | \text{ id} & \quad \{ E.addr = top.get(id.lexeme); \} \\
 | L & \quad \{ E.addr = \text{new Temp}(); \\
 & \quad \quad \quad gen(E.addr := L.array.base + L.addr); \}
 \end{aligned}$$

Instruction of the form $x = a[i]$

Array references can be part of an expression

L-产生式的语义动作计算偏移

数组元素的地址为 $base + offset$

产生式 $E \rightarrow E1 + E2$ 和 $E \rightarrow id$ 与以前相同。新的产生式 $E \rightarrow L$ 的语义动作生成的代码将 L 所指位置上的值复制到一个新的临时变量中。和前面对产生式 $S \rightarrow L = E$ 的讨论一样, L 所指的地址就是 $L.array.base[L.addr]$ 。其中, 属性 $L.array$ 仍然给出了数组名, $L.array.base$ 给出了数组的基地址。属性 $L.addr$ 表示保存偏移量的临时变量。数组引用的代码将存放在由基地址和偏移量给出的位置中的右值放入 $E.addr$ 所指的临时变量中。

SDT for Array References (3)

$$S \rightarrow id = E ; \quad \{ gen(top.get(id.lexeme) \neq E.addr); \}$$

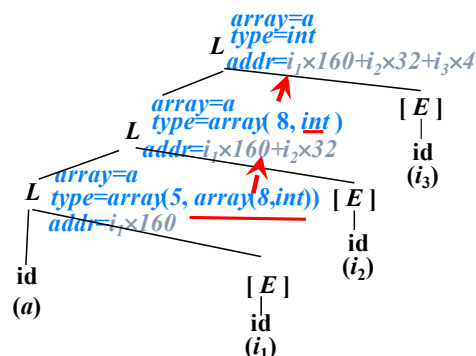
$$| \quad L = E ; \quad \{ gen(L.addr.base'[L.addr]' \neq E.addr); \}$$

Instruction of form $a[i] = x$

Array references can appear at the LHS of an assignment statement

产生式 $S \rightarrow id = E$; 代表一个对非数组变量的赋值语句, 它按照通常的方法进行处理。

$S \rightarrow L = E$; 的语义动作产生了一个带下标的复制指令, 它将表达式 E 的值存放到数组引用 L 所指的内存位置。回顾一下, 属性 $L.array$ 给出了数组的符号表条目。数组的基地址 (即 0 号元素的地址) 由 $L.array.base$ 给出。属性 $L.addr$ 表示一个临时变量, 它保存了 L 生成的数组引用的偏移量。因此, 这个数组引用的位置是 $L.array.base[L.addr]$ 。这个指令将地址 $E.addr$ 中的右值放入 L 的内存位置中。



$$base + i_1 \times w_1 + i_2 \times w_2 + \dots + i_k \times w_k$$

假设 $type(a) = array(3, array(5, array(8, int)))$, 计算 $a[i_1][i_2][i_3]$ 数组引用的偏移地址

➤ L 的综合属性

➤ $L.type$: L 生成的数组元素的类型（数组类型的表达式：二元组（元素数量，元素类型））

➤ $L.addr$: 指示一个临时变量，该临时变量用于累加公式中的 $i_j \times w_j$ 项，从而计算数组引用的偏移量

➤ $L.array$: 数组名在符号表的入口地址

翻译的关键：地址计算公式与数组引用的文法关联起来

简单赋值语句的翻译采用的是自底向上的翻译方法，这里还可以延续该方法。

从左向右扫描，查询符号表得到 a 的类型表达式和它的基地址，base 和 i1 的值就可以确定了，接下来求 w1（a[i1] 的宽度），由 a 的类型表达式即知道其元素 a[i1] 的类型表达式，设置 L.type 来表示，计算出 a[i1] 的宽度，设置 L.offset 偏移地址，接下来 a[i1][i2] 是 a[i1] 的一个元素，可通过子节点 a[i1] 元素的类型表达式获得 a[i1][i2] 的类型表达式，进而得到其宽度 w2，接下来计算累计的偏移地址 L.offset

数组引用的 SDT

假设 $type(a) = array(3, array(5, array(8, int)))$,
翻译语句片段 “ $a[i_1][i_2][i_3]$ ”

$$addr(a[i_1][i_2][i_3]) = base + i_1 \times w_1 + i_2 \times w_2 + i_3 \times w_3$$

```

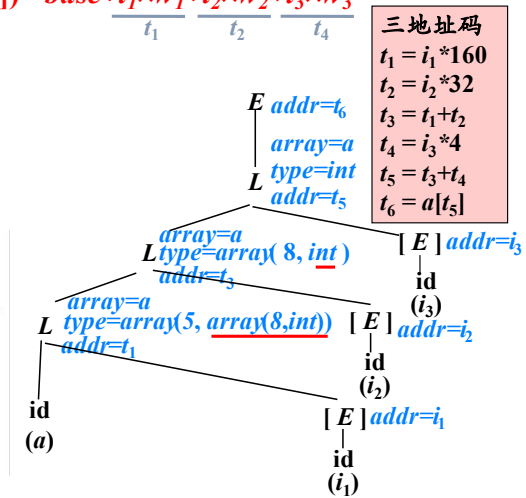
E → E1 + E2  { E.addr = new Temp();
                  gen(E.addr := E1.addr + E2.addr); }

| id             { E.addr = top.get(id.lexeme); }

| L              { E.addr = new Temp();
                  gen(E.addr := L.array.base + L.addr); }

L → id [ E ]     { L.array = top.get(id.lexeme);
                  L.type = L.array.type.elem;
                  L.addr = new Temp();
                  gen(L.addr := E.addr * L.type.width); }

| L1 [ E ]      { L.array = L1.array;
                  L.type = L1.type.elem;
                  t = new Temp();
                  L.addr = new Temp();
                  gen(t := E.addr * L.type.width);
                  gen(L.addr := L1.addr + t); }
    
```



```

E → E1 + E2  { E.addr = new Temp(); 6
                  gen(E.addr := E1.addr + E2.addr); }

| id             { E.addr = top.get(id.lexeme); } 1 3

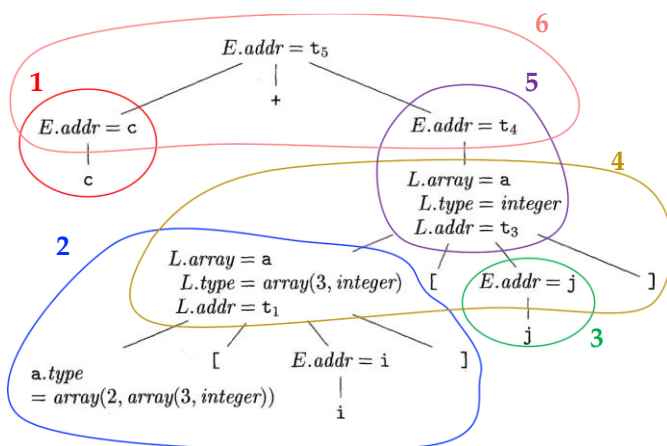
| L              { E.addr = new Temp(); 5
                  gen(E.addr := L.array.base + L.addr); }
    
```

```

L → id [ E ]     { L.array = top.get(id.lexeme); 2
                  L.type = L.array.type.elem;
                  L.addr = new Temp();
                  gen(L.addr := E.addr * L.type.width); }

| L1 [ E ]      { L.array = L1.array;
                  L.type = L1.type.elem; 4
                  t = new Temp();
                  L.addr = new Temp();
                  gen(t := E.addr * L.type.width);
                  gen(L.addr := L1.addr + t); }
    
```

Translating $c + a[i][j]$



Generated code:

```

t1 = i * 12 2
t2 = j * 4 4
t3 = t1 + t2
t4 = a[t3] 5
t5 = c + t4 6
    
```

L 的综合属性:

L.array: 数组名在符号表的入口地址

L.type: L 生成的数组元素的类型

L.Addr: 指示一个临时变量, 该临时变量用于累加公式中的 $ij \times w_j$ 项, 从而计算数组引用的偏移量

Type Checking

- To do type checking, a compiler needs to assign a **type expression** to each component of the source program
- The compiler then determines that the type expressions conform to a **collection of logical rules** (i.e., the **type system**)
 - A **sound** type system allows us to determine statically that type errors cannot occur at run time
- A language is **strongly typed** if the compiler guarantees that the programs it accepts will run without type errors (**sound type system**)
 - **Strongly typed**: Java (`double a; int b = a; //cannot compile`)
 - **Weakly typed**: C/C++ (`double a; int b = a; //implicit conversion`)

要进行类型检查, 编译器需要为源程序的每个组件分配一个类型表达式

编译器然后确定类型表达式符合逻辑规则的集合 (即类型系统)

健全的类型系统允许我们静态地确定类型错误不会在运行时发生

如果编译器保证其接受的程序在运行时不会出现类型错误, 则语言是强类型的 (健全的类型系统)

强类型: Java (`double a; int b=a; //无法编译`)

弱类型: C/C++ (`double a; int b=a; //隐式转换`)

为了进行类型检查(type checking), 编译器需要给源程序的每一个组成部分赋予一个类型表达式。然后, 编译器要确定这些类型表达式是否满足一组逻辑规则。这些规则称为源语言的类型系统(type system)。

类型检查具有发现程序中的错误的潜能。原则上，如果目标代码在保存元素值的同时保存了元素类型的信息，那么任何检查都可以动态地进行。一个健全(sound)的类型系统可以消除对动态类型错误检查的需要，因为它可以帮助我们静态地确定这些错误不会在目标程序运行的时候发生。如果编译器可以保证它接受的程序在运行时刻不会发生类型错误，那么该语言的这个实现就被称为强类型的。

除了用于编译，类型检查的思想还可以用于提高系统的安全性，使得人们安全地导入和执行软件模块。Java 程序被编译成为机器无关的字节码，在字节码中包含了有关字节码中的运算的详细类型信息。导入的代码在被执行之前首先要进行类型检查，以防止因疏忽造成的错误和恶意攻击。

Rules for Type Checking

- **Typesynthesis (类型合成)**

- Build up the type of an expression from the types of subexpressions
 - **Typicalform:** if f has type $s \rightarrow t$ and x has type s , then expression $f(x)$ has type t
 - **Example:** $f(x) = -x$ (can be generalized to multi-argument cases)

- **Typeinference (类型推导)**

- Determine the type of a language construct from the way it is used
 - **Typicalform:** if $f(x)$ is an expression, then for some types α and β , f has type $\alpha \rightarrow \beta$ and x has type α
 - **Example:** let $null$ be a function that tests whether a list is empty, then from the usage $null(x)$, we can tell that x must be a list

类型合成：

根据子表达式的类型建立表达式的类型

典型形式：如果 f 具有类型 $s \rightarrow t$ 和 x 具有类型 s ，那么表达式 $f(x)$ 具有类型 t

示例： $f(x) = -x$ （可以推广到多参数的情况）

类型推导：

根据语言结构的使用方式确定其类型

典型形式：如果 $f(x)$ 是表达式，则对于某些类型 α 和 β , f 具有类型 $\alpha \rightarrow \beta$ 和 x 具有类型 α

示例： `null` 是一个测试列表是否为空的函数，然后从用法 `null(x)`，我们可以看出 x 必须是列表

Type Conversions

- Consider an expression $x * i$, where x is a float and i is an integer
 - The representation (the way of organizing 0/1 bits) of integers and floating-point numbers is different
 - Different machine instructions are used for operations on integers and floats
 - Convert integers to floats: $t_1 = (\text{float}) i$ $t_2 = x \text{ fmul } t_1$
- Type conversion SDT for a simple case (using type synthesis)
 - $E \rightarrow E_1 + E_2$
 - ```
{ if($E_1.type = integer$ and $E_2.type = integer$) $E.type = integer$;
 else if($E_1.type = float$ and $E_2.type = integer$) $E.type = float$;
 ...
}
```

考虑类似于  $x + i$  的表达式，其中  $x$  是浮点数类型而  $i$  是整型。因为整数和浮点数在计算机中有不同的表示形式，而且使用不同的机器指令来完成整数和浮点数运算 C 编译器需要把  $+$  的某个运算分量进行转换，以保证在进行加法运算时两个运算分量具有相同的类型。

假定在必要的时候可以使用一个单目运算符 `(float)` 将整数转换成浮点数 例如，整数 2 在表达式 `2 * 3.14` 对应的代码中被转换成浮点数：

```
t1 = (float) 2
```

```
t2 = t1 * 3.14
```

我们可以扩展这样例子，考虑运算符的整型和浮点型版本。比如， `int *` 表示作用于整型运算分量的运算符，而 `float *` 表示作用于浮点型运算分量的运算符。



我们将扩展 6.4.2 节中的用于表达式翻译的翻译方案，以说明如何进行类型综合。我们引入

另一个属性  $E.type$ , 该属性的值可以是 `integer` 或 `float`。和  $E \rightarrow E1 + E2$  相关的规则可用如下的伪代码给出：

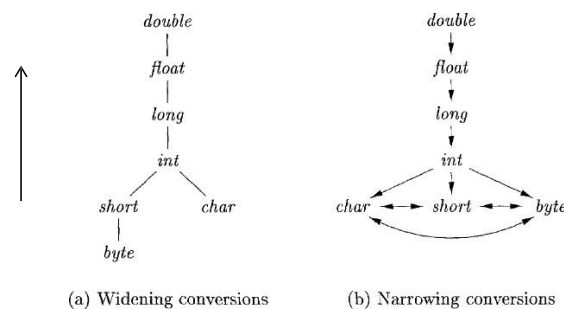
简单情况的类型转换 SDT（使用类型合成）

$E \rightarrow E1 + E2.$

```
{if($E1.type = integer$ and $E2.type = integer$) $E.type = integer$;
else if $E1.type = float$ and $E2.type = integer$ $E.type = float$;
...
}
```

## Widening and Narrowing (1)

- Type conversion rules vary from language to language
- Java distinguishes between *widening* conversions (类型拓宽) and *narrowing* conversions (类型窄化)

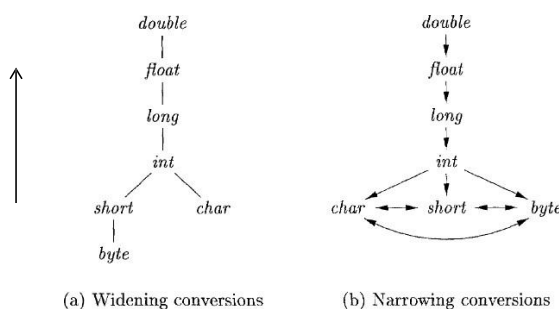


随着需要转换的类型的增多，需要处理的不同情况也急剧增多。因此，在处理大量的类型时，精心组织用于类型转换的语义动作就变得非常重要。

不同语言具有不同的类型转换规则。图 6-25 中的 Java 的转换规则区分了拓宽(widening)转换和窄化(narrowing)转换。

## Widening and Narrowing (2)

- **Widening** conversions **preserve information** and can be done automatically by the compiler (*implicit* type conversions, or *coercions*)
- **Narrowing** conversions **lose information** and require programmers to write code to cause the conversion (*explicit* type conversions, or *casts*)



拓宽转换可以保持原有的信息，而窄化转换则可能丢失信息。

拓宽规则通过图 6-25a 中的层次结构给出：在该层次结构中位于较低层的类型可以被拓宽为较高层的类型。因此，char 类型可以被拓宽为 int 型和 float 型，但是不可以被拓宽为 short 类型。

窄化转换的规则如图 6-25b 所示：如果存在一条从 s 到 t 的路径，则可以将类型 s 窄化为类型 t。可以看出，char、short、byte 之间可以两两相互转换。

如果类型转换由编译器自动完成，那么这样的转换就称为隐式转换。隐式转换也称为自动类型转换(coercion)。在很多语言中，自动类型转换仅仅限于拓宽转换。如果程序员必须写出某些代码来引发类型转换运算，那么这个转换就称为显式的。显式转换也称为强制类型转换(cast)。

# SDT for Type Conversion

- $\text{max}(t_1, t_2)$  takes two types  $t_1$  and  $t_2$  and returns the **maximum** (or **least upper bound**) of the two types in the widening hierarchy
- $\text{widen}(a, t, w)$  generates type conversions if needed to widen an address  $a$  of type  $t$  into a value of type  $w$

```
Addr widen(Addr a, Type t, Type w)
{
 if (t = w) return a;
 else if (t = integer and w = float) {
 temp = new Temp();
 gen(temp := (float) a);
 return temp;
 }
 else error;
}
```

```
E → E1 + E2 { E.type = max(E1.type, E2.type);
 a1 = widen(E1.addr, E1.type, E.type);
 a2 = widen(E2.addr, E2.type, E.type);
 E.addr = new Temp();
 gen(E.addr := a1 + a2); }
```

检查  $E \rightarrow E_1 + E_2$  的语义动作使用了两个函数：

1)  $\text{max}(t_1, t_2)$  接受  $t_1$  和  $t_2$  两个类型的参数并返回拓宽层次结构中这两个类型中的最大者（或者最小上界）。如果  $t_1$  或  $t_2$  之一没有出现在这个

层次结构中，比如有个类型是数组类型或指针类型，那么该函数返回一个错误信息。

2) 如果需要将类型为  $t$  的地址  $a$  中的内容转换成  $w$  类型的值，则函数  $\text{widen}(a, t, w)$  将生成类型转换的代码。如果  $t$  和  $w$  是相同的类型，则该函数返回  $a$  本身。否则，它会生成一条指令来完成转换工作并将转换结果放置到临时变量  $\text{temp}$  中。这个临时变量将作为结果返回。函数  $\text{widen}$  的伪代码如图 6-26 所示，这里假设只有 `integer` 和 `float` 两种类型。

# Example

- $a + b$  (suppose  $a$  is of *int* type and  $b$  is of *float* type)

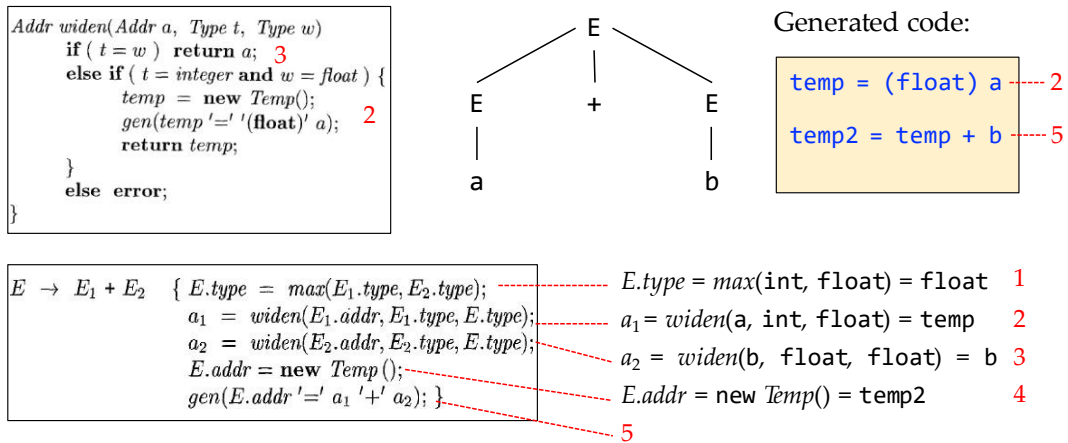


图 6-27 中  $E \rightarrow E_1 + E_2$  的语义动作说明了如何把类型转换加入到图 6-20 所示的翻译表达式的方案中。

在这个语义动作中，如果  $E_1$  的类型不需要被转换成  $E$  的类型，那么临时变量  $a_1$  就是  $E_1.addr$ 。如果需要进行这样的转换，则  $a_1$  就是 `widen` 函数返回的一个新的临时变量。类似地， $a_2$  可能是  $E_2.addr$ ，也可能是一个新临时变量，用于存放转换后的  $E_2$  的值。如果两个变量都是整型或者都是浮点型，就不需要进行任何转换。我们会发现，将两个不同类型的值相加的唯一方法是把它们都转换成为第三种类型。

# Control Flow

- Boolean expressions are often used to [alter the flow of control](#) or [compute logical values](#)
- **Grammar:**  $B \rightarrow B \parallel B \mid B \&\& B \mid !B \mid (B) \mid E \text{ rel } E \mid \text{true} \mid \text{false}$
- Given the expression  $B_1 \parallel B_2$ , if  $B_1$  is true, then the expression is true without having to evaluate  $B_2$ . Thus, neither  $B_1$  nor  $B_2$  is necessarily evaluated fully.\*
- In *short-circuit code*, the boolean operators  $\&\&$ ,  $\parallel$ ,  $!$  translate into jumps. [The operators do not appear in the code.](#)

If  $B_1$  or  $B_2$  has side effect (e.g., changing the value of a global variable), then the effect may not occur

在编程语言中，布尔表达式通常用于

1. 改变控制流。 Boolean 表达式在改变控制流的语句中用作条件表达式。

这种布尔表达式的值由程序到达的某个位置隐含地指出。例如，在  $\text{if}(E) S$  中，如果运行到语句  $S$ ，则表达式  $E$  必须为 `true`。

2. 计算逻辑值。一个布尔表达式的值可以表示为 `true` 或 `false`。这样的布尔表达式也可以像算术表达式一样，使用带有逻辑运算符的三地址指令进行求值。

布尔表达式的使用意图，要根据其语法上下文决定。例如，关键字 `if` 后面的表达式用于更改控制流，而一个赋值语句右侧的表达式用于表示一个逻辑值。这样的语法上下文可以通过多种方式来指定：我们可以使用两个不同的非终结符，也可使用继承的属性，还可以在语法分析过程中设置一个标记。此外，我们还可以构建一个语法树并调用不同的过程来处理布尔表达式的两种不同的使用。

本节将介绍用于改变控制流的布尔表达式，为此引入一个新的非终结符  $B$ 。布尔表达式是将作用布尔变量或关系表达式的布尔运算符构成的。我们使用 C 语言的方法，用  $\&\&$ ， $\parallel$ ， $!$  分别表示 AND，OR，NOT 运算符。关系表达式的形式为  $E \text{ rel } E$ 。其中， $E$  为算术表达式。写出布尔表达式的文法：

$B \rightarrow B \ \&\& \mid B \ \&\& \ B \mid ! \ (B) \mid E \ \text{rel} \ E \mid \text{true} \mid \text{false}$

我们通过属性 `rel.op` 来指明 `rel` 究竟表示 6 种比较运算符 `<`, `<=`, `=`, `!=`, `>` 和 `>=` 中的哪一种。按照惯例, 假设 `||` 和 `&&` 是左结合的, `||` 的优先级最低, 再其次为!

给定表达式  $B1 \ \&\& \ B2$ , 如果  $B1$  为真, 那么无需对  $B2$  求值, 表达式也为真。因此,  $B1$  和  $B2$  都不一定要完全求值\*。

程序设计语言的语义定义决定是否需要对一个布尔表达式的各个部分都进行求值。如果语言定义允许(或要求)不对布尔表达式的某部分求值, 则编译器可以优化布尔表达式的求值过程, 只要已经求值的部分足以确定整个表达式的值就可以了。因此, 在诸如  $B1 \ || \ B2$  之类的表达式中  $B2$ ,  $B1$  和  $B2$  都不必完全地求值。如果  $B1$  或  $B2$  是具有副作用的表达式(例如, 它包含了改变一个全局变量的函数), 那这么做就可能会得到意料之外的结果。

在短路(跳转)代码中, 布尔运算符 `&&`、`||`、`!` 转化为跳转。这些运算符本身不出现在代码中, 布尔表达式的值是通过代码序列中的位置来表示的。

## Short-Circuit Code Example

• `if ( x < 100 || x > 200 && x != y ) x = 0;`

```
 if x < 100 goto L2
 ifFalse x > 200 goto L1
 ifFalse x != y goto L1
L2: x = 0
L1:
```

上述语句可以翻译成图 6.34 的代码。在此翻译中, 如果布尔表达式为 `true`, 则控制到达 `L2`。如果表达式为 `false`, 则控制将跳转过 `L2` 和赋值语句 `x = 0`, 直接转到 `L1`。

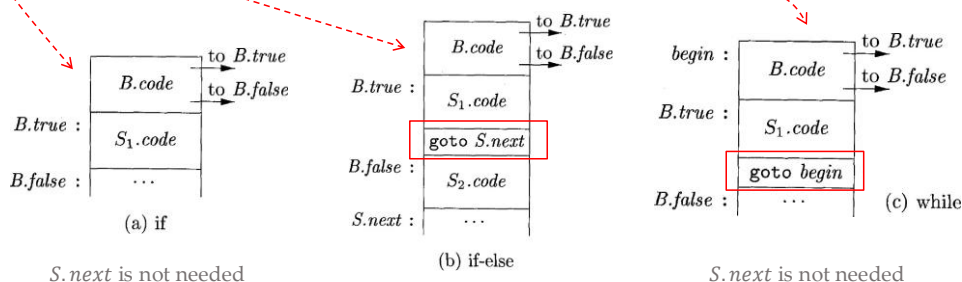
# Flow-of-Control Statements

- Grammar:

- $S \rightarrow \text{if } (B) S_1$
- $S \rightarrow \text{if } (B) S_1 \text{ else } S_2$
- $S \rightarrow \text{while } (B) S_1$

**Inherited attributes:**

- $B.true$ : the label to which control flows if  $B$  is true
- $B.false$ : the label to which control flows if  $B$  is false
- $S.next$ : the label for the instruction immediately after the code for  $S$



Fall 2024

BUPT Compilers

81

现在我们考虑在语句上下文中如何将布尔表达式转换为三地址代码。

在这些产生式中，非终结符  $B$  表示布尔表达式，非终结符  $S$  表示语句。

这个文法将之前我们在语法制导翻译中（例 5.19）所介绍的 `while` 表达式进行了推广。和那个例子一样， $B$  和  $S$  都有一个综合属性 `code`，该属性给出了翻译得到的三地址指令。为简单起见，我们使用语法制导定义来构造得到翻译结果 `B.code` 和 `S.code`，结果值是字符串。定义了 `code` 属性的语义规则可以按照如下的方法来实现：首先构造语法树，然后在树遍历期间产生目标代码。这些规则还可通过第 5.5 节中列出的任何方法来实现。

如图 6.35(a) 所示，对 `if(B) S1` 的翻译结果中包含了 `B:code`，其后是 `S1.code`。

`B:code` 中存在基于  $B$  值的跳转。如果  $B$  为真，则控制流转向 `S1.code` 的第一条指令；如果  $B$  为假，则控制流立即转向紧随在 `S1.code` 之后的指令。`B:code` 和 `S:code` 中跳转的标号使用继承属性来处理。我们将布尔表达式  $B$  和两个标号 `B:true` 和 `B:false` 相关联。如果  $B$  为 `true`，则控制流向 `B:true`；如果  $B$  为 `false`，则控制流转到 `B:false`。对于语句  $S$ ，我们关联一个继承属性 `S.next`，表示紧接在  $S$  的代码之后的指令的标号。在某些情况下，紧跟在 `S:code` 后面的指令是跳转到某个标号  $L$  的跳转指令。使用 `S.next` 可以避免在 `S:code` 中出现这样一条跳转指令，它的目标又是一个以  $L$  为目标的跳转指令。

## SDD for Flow-of-Control Statements (1)

| PRODUCTION                                            | SEMANTIC RULES                                                                                                                                                                                                                            |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $P \rightarrow S$                                     | $S.next = newlabel()$<br>$P.code = S.code \parallel label(S.next)$                                                                                                                                                                        |
| $S \rightarrow \text{assign}$                         | $S.code = \text{assign.code}$                                                                                                                                                                                                             |
| $S \rightarrow \text{if} ( B ) S_1$                   | $B.true = newlabel()$<br>$B.false = S_1.next = S.next$<br>$S.code = B.code \parallel label(B.true) \parallel S_1.code$                                                                                                                    |
| $S \rightarrow \text{if} ( B ) S_1 \text{ else } S_2$ | $B.true = newlabel()$<br>$B.false = newlabel()$<br>$S_1.next = S_2.next = S.next$<br>$S.code = B.code$<br>$\parallel label(B.true) \parallel S_1.code$<br>$\parallel gen('goto' S.next)$<br>$\parallel label(B.false) \parallel S_2.code$ |

This SDD is neither S-attributed nor L-attributed.  
Computing the attributes requires a separate pass after parsing.

图 6.36 和 6.37 给出的语法制导定义可以为 if、if-else 和 while 语句的上下文中为布尔表达式生成三地址代码。

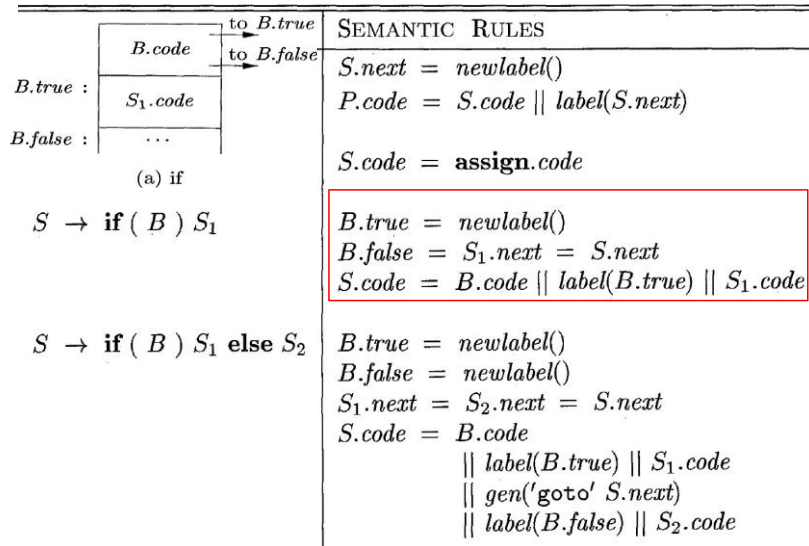
我们假设 new label() 每次被调用时都会创建一个新的标号，并且 label(L) 将标号 L 附加到下一个要生成的三地址指令上。

- 一个程序包含一条由产生式  $P \rightarrow S$  生成的语句组成。与该产生式相关的语法规则将  $S.next$  初始化为一个新的标号。  $P.code$  包含  $S.code$ ， $S.code$  后跟新标签  $S.next$  组成。

- 产生式  $S \rightarrow \text{assign}$  中的词法单元 assign 是一个表示赋值语句的占位符。赋值语句的翻译如第 6.4 节所述；在这里对控制流的讨论中， $S.code$  就是  $\text{assign.code}$ 。

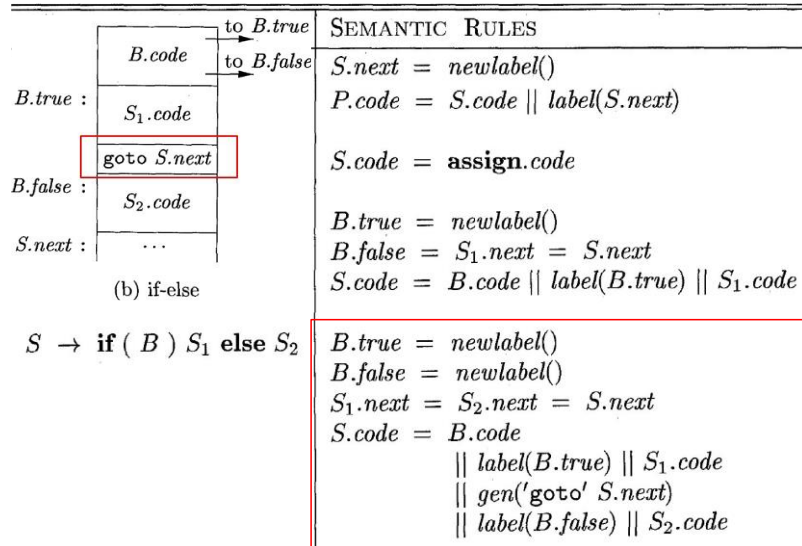


## SDD for Flow-of-Control Statements (1)



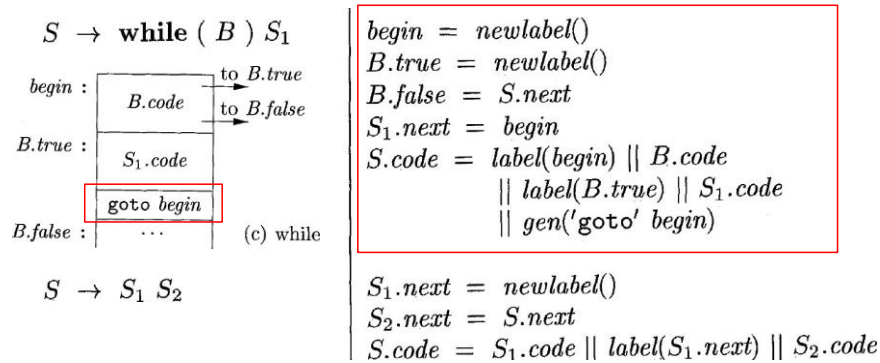
- 在翻译  $S \rightarrow \text{if}(B) S_1$  时，图 6.36 中的语义规则创建一个新的标号  $B.true$ ，并将其关联到为语句  $S_1$  生成的第一个三地址指令中，如图 6.35(a) 所示。因此，在  $B$  的代码中跳转到  $B.true$  的指令跳转到语句  $S_1$  对应的代码。此外，通过将  $B.false$  设置为  $S.next$ ，我们确保当  $B$  的值为  $false$ ，控制将跳过  $S_1$  的代码。

## SDD for Flow-of-Control Statements (1)



在翻译 if-else-语句  $S \rightarrow \text{if}(B) S_1 \text{ else } S_2$  时，如果  $B$  为真，则布尔表达式  $B$  的代码跳至  $S_1$  代码的第一条指令，如果  $B$  为假，则跳至  $S_2$  代码的第一条指令，如图 6.35(b)所示。此外，控制从  $S_1$  或  $S_2$  流向紧跟在  $S$  代码之后的三地址指令——该指令的标号由继承属性  $S.next$  给出。在  $S_1$  的代码之后有一条 goto  $S.next$  指令，使得控制流跳过  $S_2$  的代码。  $S_2$  之后不需要 goto，因为  $S_2.next$  就是  $S.next$ 。

## SDD for Flow-of-Control Statements (2)

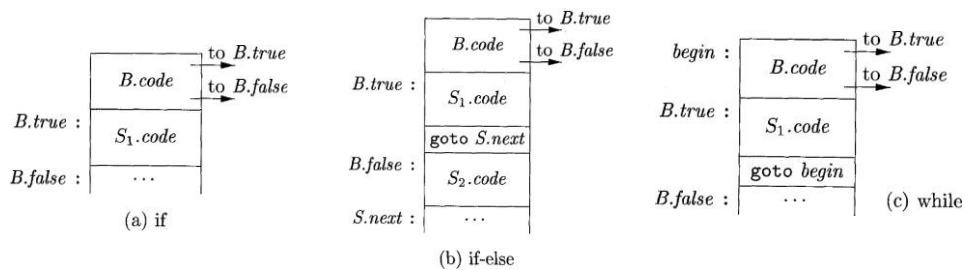


$S \rightarrow \text{while} ( B ) S_1$  的代码由  $B:\text{code}$  和  $S_1:\text{code}$  组成, 如图 6.35(c)所示。我们使用一个局部变量  $\text{begin}$  来保存附加到该  $\text{while}$  语句的第一条指令上的标号。这个  $\text{while}$  语句的第一条指令也是  $B$  的第一条指令。我们这里使用变量而不是属性, 是因为  $\text{begin}$  对于这个产生式来说是局部的语义规则。继承属性  $S:\text{next}$  标记了当  $B$  为  $\text{false}$  时控制必须转向的标号; 因此,  $B:\text{false}$  被设置为  $S:\text{next}$ 。在  $S_1$  的第一条指令上附加了一个新的标号  $B:\text{true}$ 。  $B$  的指令中的跳转指令在  $B$  为真时跳转到这个标号。我们在  $S_1$  的代码之后, 放置了一条  $\text{goto begin}$  指令, 它跳转到布尔表达式的代码的开始处。请注意,  $S_1:\text{next}$  被设置标号  $\text{begin}$ , 因此从  $S_1:\text{code}$  中跳出的指令可以直接跳转  $\text{begin}$ 。

$S \rightarrow S_1 S_2$  的代码由  $S_1$  代码和  $S_2$  组成。相应的语义规则主要处理标号。 $S_1$  代码之后的第一条指令就是  $S_2$  代码的起始指令。紧跟在  $S_2$  代码之后的指令也是跟在  $S$  的代码之后的指令。

# Translating Boolean Expressions in Flow-of-Control Statements

- A boolean expression  $B$  is translated into three-address instructions that evaluate  $B$  using conditional and unconditional jumps to one of two labels:  $B.true$  and  $B.false$ 
  - $B.true$  and  $B.false$  are two inherited attributes. Their value depends on the context of  $B$  (e.g., *if* statement, *if-else* statement, *while* statement)



如图 6.35 的代码布局所示，布尔表达式  $B$  被转换为三地址指令，它使用有条件 and 无条件跳转指令来对  $B$  求值。这些跳转指令的目标是两个标号之一：如果  $B$  为 `true`，则  $B.true$ ；如果  $B$  为 `false`，则  $B.false$ 。

$B.true$  和  $B.false$  是两个继承属性。它们的值取决于的上下文（如 *if* 语句、*if-els* 语句、*wile* 语句）

# Generating Three-Address Code for Booleans (1)

| PRODUCTION                       | SEMANTIC RULES                                                                                                                                                                     |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $B \rightarrow B_1 \    \ B_2$   | $B_1.true = B.true$ // short-circuiting<br>$B_1.false = newlabel()$<br>$B_2.true = B.true$<br>$B_2.false = B.false$<br>$B.code = B_1.code \    \ label(B_1.false) \    \ B_2.code$ |
| $B \rightarrow B_1 \ \&\& \ B_2$ | $B_1.true = newlabel()$<br>$B_1.false = B.false$ // short-circuiting<br>$B_2.true = B.true$<br>$B_2.false = B.false$<br>$B.code = B_1.code \    \ label(B_1.true) \    \ B_2.code$ |
| $B \rightarrow ! \ B_1$          | $B_1.true = B.false$<br>$B_1.false = B.true$<br>$B.code = B_1.code$                                                                                                                |

图 6.37 中针对布尔表达式的语义规则是图 6.36 中语句的语义规则的一个补充。

B 的其余产生式翻译如下：

- 假设 B 的形式为 B1 || B2.如果 B1 为真，那么我们立即知道 B 本身为真，因此 B1:true 与 B:true 相同。如果 B1 为 false，则必须计算 B2，因此我们将 B1:false 设为 B2 代码中第一条指令的标号。 B2 的真出口和假出口分别与 B 的真出口和假出口相同。
- B1 && B2 的翻译类似。
- 不需要为 B →! B1 产生新的代码，只需将 B 的真假出口互换即可得到 B1 的真假出口。

# Generating Three-Address Code for Booleans (2)

|                                      |                                                                                                                                                                             |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $B \rightarrow E_1 \text{ rel } E_2$ | $B.code = E_1.code \parallel E_2.code$<br>$\parallel \text{gen('if' } E_1.addr \text{ rel.op } E_2.addr \text{ 'goto' } B.true)$<br>$\parallel \text{gen('goto' } B.false)$ |
| $B \rightarrow \text{true}$          | $B.code = \text{gen('goto' } B.true)$                                                                                                                                       |
| $B \rightarrow \text{false}$         | $B.code = \text{gen('goto' } B.false)$                                                                                                                                      |

图 6.37 中的第四个产生式， $B \rightarrow E_1 \text{ rel } E_2$ ，直接翻译成三地址比较指令，并跳转到适当的位置。例如， $a < b$  形式的  $B$  转换为：

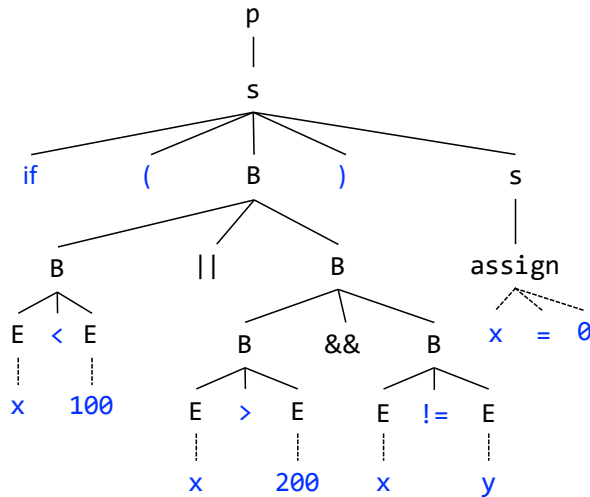
if  $a < b$  goto B:true

goto B:false

将常量 `true` 和 `false` 分别翻译成目标为 B:true 和 B:false 的跳转指令。

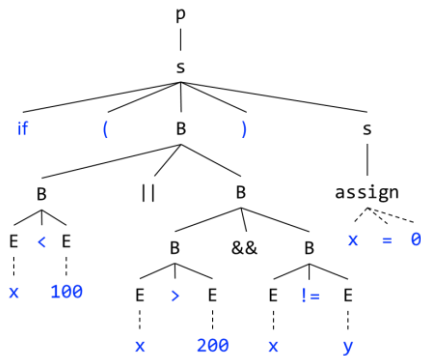
# Example

• `if ( x < 100 || x > 200 && x != y ) x = 0;`



Dashed lines mean that the reduction may consist of multiple steps

# Example

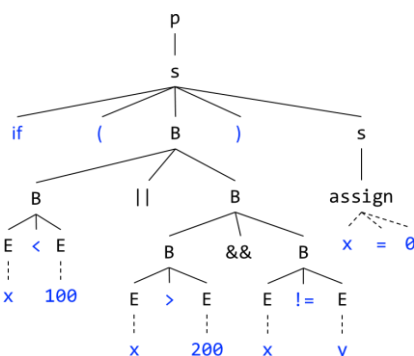


| PRODUCTION                           | SEMANTIC RULES                                                                                                                                                       |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $P \rightarrow S$                    | $S.next = newlabel()$<br>$P.code = S.code \parallel label(S.next)$                                                                                                   |
| $S \rightarrow assign$               | $S.code = assign.code$                                                                                                                                               |
| $S \rightarrow if ( B ) S_1$         | $B.true = newlabel()$<br>$B.false = S_1.next = S.next$<br>$S.code = B.code \parallel label(B.true) \parallel S_1.code$                                               |
| $B \rightarrow B_1 \parallel B_2$    | $B_1.true = B.true$<br>$B_1.false = newlabel()$<br>$B_2.true = B.true$<br>$B_2.false = B.false$<br>$B.code = B_1.code \parallel label(B_1.false) \parallel B_2.code$ |
| $B \rightarrow B_1 \&\& B_2$         | $B_1.true = newlabel()$<br>$B_1.false = B.false$<br>$B_2.true = B.true$<br>$B_2.false = B.false$<br>$B.code = B_1.code \parallel label(B_1.true) \parallel B_2.code$ |
| $B \rightarrow ! B_1$                | $B_1.true = B.false$<br>$B_1.false = B.true$<br>$B.code = B_1.code$                                                                                                  |
| $B \rightarrow E_1 \text{ rel } E_2$ | $B.code = E_1.code \parallel E_2.code$<br>$\parallel gen('if' E_1.addr \text{ rel } op E_2.addr 'goto' B.true)$<br>$\parallel gen('goto' B.false)$                   |

Traversing the parse tree to evaluate the attributes helps generate intermediate code

# Example

- `if ( x < 100 || x > 200 && x != y ) x = 0;`



Generated code:

```

if x < 100 goto L2
goto L3
L3: if x > 200 goto L4
 goto L1
L4: if x != y goto L2
 goto L1
L2: x = 0
L1:

```

使用图 6.36 和 6.37 中的语法制导定义，我们将得到图 6.38 中的代码。

语句 `if ( x < 100 || x > 200 && x != y ) x = 0` 构成了由  $P \rightarrow S$  生成的程序。这个产生式的语义规则生成了 `S` 的代码之后第一个指令的新标号 `L1`。

语句 `S` 的形式为 `if(B) S1`，其中 `S1` 是 `x = 0`；因此，图 6.36 中的规则生成一个新的标号 `L2`，并将其附加到 `S1.code` 中的第一条（也是唯一的，在本例中）指令，即 `x = 0` 处。

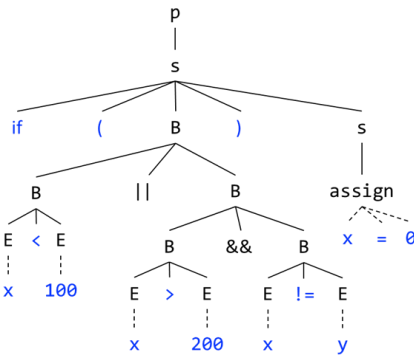
因为 `||` 优先级低于 `&&`，所以式(6.13) 中布尔表达式的形式为 `B1 || B2`，其中 `B1` 是 `x < 100`。遵循图 6.37 中的规则，`B1:true` 是 `L2`，即语句 `x = 0` 的标号；`B1:false` 是一个新的标号 `L3`，附加到 `B2` 代码中的第一条指令上。

请注意，生成的代码不是最佳的，因为翻译结果比例 6.21 中的代码多了 3 条指令（`goto`）。指令 `goto L3` 是多余的，因为 `L3` 是下一条指令的标号。如像例 6.21 所示使用 `ifFalse` 代替 `if` 指令，可以消除两条 `goto L1` 指令，。



## Example (Optimization)

- `if ( x < 100 || x > 200 && x != y ) x = 0;`



Optimization: Using `ifFalse` avoids redundant `gotos`

```

if x < 100 goto L2
ifFalse x > 200 goto L1
ifFalse x != y goto L1
L2: x = 0
L1:

```

这个 `ifFalse` 指令利用了控制流会从一个指令自然流动到下一个指令的性质，因此当 `x > 200` 时，控制就直接“穿越”到标号 `L1`，从而减少了一个跳转指令。

## Boolean Values and Jumping Code

- Besides being used to alter the flow of control in statement, a boolean expression may also be evaluated for its value
  - Example: `x = a < b`
- Grammar:
  - `S → id = E ; | if ( E ) S | while ( E ) S | S S`
  - `E → E || E | E && E | E rel E | E + E | ( E ) | id | true | false`
- A way of handling both roles of boolean expressions is to first build a syntax tree for expressions, and then use separate code-generation functions depending on the context of `E`
  - For example, for `E` in `S → while ( E ) S1`, generate jumping code
  - For `E` in `S → id = E`, generate code to compute boolean values

布尔值和跳转代码

布尔表达式除了用于改变语句中的控制流外，还可以求值

示例:  $x = a < b$

语法:

处理布尔表达式两种角色的方法是，首先为表达式建立语法树，然后根据 $E$ 的上下文使用不同的代码生成函数。

例如，对于  $S \rightarrow \text{while}(E)S1$  中的 $E$ ，生成跳转代码

对于  $S \rightarrow \text{id} = E$ 中的 $E$ ，生成计算布尔值的代码

## Backpatching (回填)\*

- A **key problem** when generating code for boolean expressions and flow-of-control statements is to **match a jump instruction with the jump target**
- **Example:  $\text{if}(B)S$** 
  - According to the short-circuit translation,  $B$ 's code contains a jump to the instruction following the code for  $S$  (executed when  $B$  is false)
  - However,  $B$  must be translated before  $S$ . The jump target is unknown when translating  $B$
  - Earlier, we address the problem by passing labels as inherited attributes ( $S.\text{next}$ ), but this requires another separate pass to bind labels to addresses

How to address the problem in one pass?



\* Will be discussed in lab sessions

在生成布尔表达式和流程控制语句代码时，一个关键问题是将跳转指令与跳转目标匹配起来。

示例:  $\text{if}(B)S$

根据短路翻译，布尔表达式 $B$ 的翻译结果中包含一个跳转指令。当 $B$ 为假时，该指令将跳转到紧跟在 $S$ 的代码之后的指令处。在一趟式的翻译中， $B$ 必须在处理 $S$ 之前就翻译完毕。但是，在翻译 $B$ 时，跳转目标是未知的。

早些时候，我们通过将标号作为继承属性传递到生成相关跳转指令的地方来解决这个问题(*S. next*)，但这需要再进行一趟处理，将标号和具体的地址绑定。

# One-Pass Code Generation Using Backpatching (1)

- **Basic idea of backpatching (基本思想):**
  - When a jump is generated, its target is temporarily left unspecified
  - Incomplete jumps are grouped into lists. All jumps on a list have the same target
  - Fill in the labels for incomplete jumps when the targets become known
- **The technique (技术细节):**
  - For a nonterminal  $B$  that represents a boolean expression, we define two synthesized attributes: *truelist* and *falselist*
  - *truelist*: a list of jump instructions for which we must fill in the label to which control goes if  $B$  is true
  - *falselist*: a list of jump instructions for which we must fill in the label to which control goes if  $B$  is false

本节采用一种称为回填的补充方法，它把一个由跳转指令组成的列表以综合属性的形式进行传递。具体来说，当生成一个跳转指令时，暂时不指定跳转的目标。这样的跳转都被放在一个跳转列表中，当可以确定正确的目标标号时，才去填充这些指令的目标标号。同一列表中所有跳转指令具有相同的目标标号。

技术细节：

对于表示布尔表达式的非终结符 $B$ ，我们定义了两个综合属性：*truelist* 和 *falselist* 用于管理布尔表达式的跳转代码中的标号。

特别是， $B.truelist$  将是一个包含跳转或条件跳转指令列表，我们必须向这些指令中插入适当的标号，也就是当  $B$  为真时控制流应该转向的标号。类似地， $B.falselist$  也是一个包含跳转指令的列表，这些指令最终获得的标号就是当  $B$  为 *false* 时，控制流应该转向的标号。在生成  $B$  的代码时，跳转到 *true* 和 *false* 出口的跳转指令是不完整的，标号字段尚未填写。这些不完整的跳转指令被保存在  $B.truelist$  和  $B.falselist$  指向的列表中。类似

地，语句  $S$  具有综合属性  $S:\text{nextlist}$  也是一个跳转指令列表，这些指令应该跳转到紧跟在  $S$  的代码之后的指令。

## One-Pass Code Generation Using Backpatching (2)

- **The technique (技术细节) Cont.:**
  - *makelist( $i$ )*: create a new list containing only  $i$ , the index of a jump instruction, and return the pointer to the list
  - *merge( $p_1, p_2$ )*: concatenate the lists pointed by  $p_1$  and  $p_2$ , and return a pointer to the concatenated list
  - *backpatch( $p, i$ )*: insert  $i$  as the target label for each of the instructions on the list pointed by  $p$

我们将生成的指令放入一个指令数组中，而标号将是该数组的下标。为了处理跳转指令的列表，我们使用三个函数：

*makelist( $i$ )*

创建一个只包含  $i$  的列表， $i$  是跳转指令的标号，函数返回指向新创建的列表的指针

*merge( $p_1, p_2$ )*

将  $p_1$  和  $p_2$  指向的列表进行合并，返回指向合并后的列表的指针

*backpatch( $p, i$ )*

将  $i$  作为目标标号插入到  $p$  所指列表中的各指令中

# Backpatching for Boolean Expressions (布尔表达式的回填)

- An SDT suitable for generating code for boolean expressions during bottom-up parsing
- Grammar:
  - $B \rightarrow B_1 \parallel MB_2 \mid B_1 \&\& MB_2 \mid !B_1 \mid (B_1) \mid E_1 \text{ rel } E_2 \mid \text{true} \mid \text{false}$
  - $M \rightarrow \epsilon$

我们现在构建一个可以在自下而上语法分析过程中生成布尔表达式目标代码的翻译方案。这个语法中的标记非终结符  $M$ ，它引发的语义动作在适当的时刻获取将要生成的下一条指令的下标。语法如下：

- 1)  $B \rightarrow B_1 \parallel M B_2$  { *backpatch*( $B_1.falselist, M.instr$ );  
 $B.truelist = merge(B_1.truelist, B_2.truelist)$ ;  
 $B.falselist = B_2.falselist$ ; }
- 2)  $B \rightarrow B_1 \&\& M B_2$  { *backpatch*( $B_1.truelist, M.instr$ );  
 $B.truelist = B_2.truelist$ ;  
 $B.falselist = merge(B_1.falselist, B_2.falselist)$ ; }
- 3)  $B \rightarrow ! B_1$  {  $B.truelist = B_1.falselist$ ;  
 $B.falselist = B_1.truelist$ ; }
- 4)  $B \rightarrow ( B_1 )$  {  $B.truelist = B_1.truelist$ ;  
 $B.falselist = B_1.falselist$ ; }
- 5)  $B \rightarrow E_1 \text{ rel } E_2$  {  $B.truelist = makelist(nextinstr)$ ;  
 $B.falselist = makelist(nextinstr + 1)$ ;  
 $gen('if' E_1.addr \text{ rel.op } E_2.addr 'goto -')$ ;  
 $gen('goto -')$ ; }
- 6)  $B \rightarrow \text{true}$  {  $B.truelist = makelist(nextinstr)$ ;  
 $gen('goto -')$ ; }
- 7)  $B \rightarrow \text{false}$  {  $B.falselist = makelist(nextinstr)$ ;  
 $gen('goto -')$ ; }
- 8)  $M \rightarrow \epsilon$  {  $M.instr = nextinstr$ ; }

考虑上述文法中对应于规则 (1)  $B \rightarrow B_1 \parallel M B_2$  的语义动作。如果  $B_1$  为 **true**，则  $B$  也为 **true**，因此  $B_1.truelist$  中的跳转指令就成为  $B.truelist$  的一部分。然而，如果  $B_1$  为假，我们接下来必须测试  $B_2$ 。因此  $B_1.falselist$  中的跳转指令的目标必定  $B_2$  的代码的起始位置。这个位置使用标记非终结符  $M$  获得。在即将生成  $B_2$  代码之前， $M$  生成了下一条指令的标号，存放在综合属性  $M.instr$  中。

为了获得该指令标号，我们将产生式  $M \rightarrow \epsilon$  和语义动作动作  $\{M.instr = nextinstr; \}$  关联起来。变量  $nextinstr$  保存了紧跟着的下一条指令的标号。当我们看到产生式  $B \rightarrow B_1 \parallel M B_2$  的剩余部分时，该值将被回填到  $B_1.falselist$  中的指令上（即列表  $B_1.falselist$  上的每条指令都将接收  $M.instr$  作为其目标标号）。

$B \rightarrow B_1 \parallel M B_2$  的语义动作 (2) 和  $B \rightarrow B_1 \&\& M B_2$  语义动作(1)类似。  $B \rightarrow ! B$  的语义动作 (3) 交换真假列表。动作 (4)只是 忽略括号。

为简单起见，语义动作 (5) 生成两条指令，一条有条件转移指令 **goto** 和一条无条件指令。两者的目标标号都没有填充。这些指令被放入新列表中，分别由  $B.truelist$  和  $B.falselist$  指向。

# Backpatching vs. Non-Backpatching (1)

(1) Non-backpatching SDD:

|                                      |                                                                                                                                                                             |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $B \rightarrow E_1 \text{ rel } E_2$ | $B.code = E_1.code \parallel E_2.code$<br>$\parallel \text{gen('if' } E_1.addr \text{ rel.op } E_2.addr \text{ 'goto' } B.true)$<br>$\parallel \text{gen('goto' } B.false)$ |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

(2) Backpatching scheme:

|                                      |                                                                                                                                                                                                     |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $B \rightarrow E_1 \text{ rel } E_2$ | $\{$ $B.truelist = makelist(nextinstr);$<br>$B.falselist = makelist(nextinstr + 1);$<br>$\text{gen('if' } E_1.addr \text{ rel.op } E_2.addr \text{ 'goto' } \_);$<br>$\text{gen('goto' } \_);$ $\}$ |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Comparison:

- In (2), incomplete instructions (指令坯) are added to corresponding lists
- The instruction jumping to  $B.true$  in (1) is added to  $B.truelist$  in (2)
- The instruction jumping to  $B.false$  in (1) is added to  $B.falselist$  in (2)

比较:

在 (2) 中, 指令坯被添加到相应的列表中。

跳转到  $B$  的指令。(1) 中的  $B.true$  被添加到 (2) 中的  $B.truelist$ 。

跳转到  $B$  的指令。(1) 中的  $B.false$  被添加到 (2) 中的  $B.falselist$

## Backpatching vs. Non-Backpatching (2)

(1) Non-backpatching SDD:

|                                   |                                                                                                                                                                      |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $B \rightarrow B_1 \parallel B_2$ | $B_1.true = B.true$<br>$B_1.false = newlabel()$<br>$B_2.true = B.true$<br>$B_2.false = B.false$<br>$B.code = B_1.code \parallel label(B_1.false) \parallel B_2.code$ |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|

(2) Backpatching scheme:

|                                     |                                                                                                                                     |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| $B \rightarrow B_1 \parallel M B_2$ | $\{$ $backpatch(B_1.falselist, M.instr);$<br>$B.truelist = merge(B_1.truelist, B_2.truelist);$<br>$B.falselist = B_2.falselist; \}$ |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|

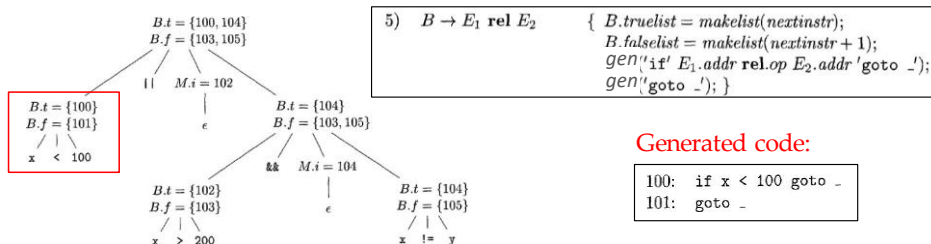
### Comparison:

- The assignments to *true/false* attributes in (1) correspond to the assignments to *truelist/falselist* or *merge* in (2)

(1) 中对 *true/false* 属性的赋值与 (2) 中对 *truelist/falselist* 或 *merge* 属性的赋值相对应。

## Example – Boolean Expressions (1)

- The earlier SDT is a **postfix SDT**. The semantic actions can be performed during a bottom-up parse.
- Boolean expression:  $x < 100 \parallel x > 200 \&\& x \neq y$
- Step 1:** reduce  $x < 100$  to  $B$  by production (5)



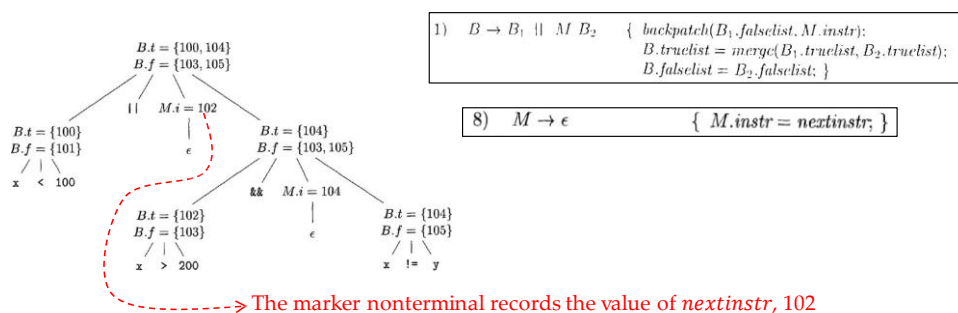


带注释的语法分析树如图 6.44 所示；为了便于阅读，属性 `truelist`、`falselist` 和 `instr` 由其首字母表示。这些动作是在树的深度优先遍历期间执行的。由于所有动作都出现在产生式右侧的末尾，因此它们可以在自下而上的语法分析过程中与归约一起执行。在根据产生式 (5) 将  $x < 100$  归约为  $B$  时，语义动作相应地产生两条指令：

我们可任意从 100 开始为指令编号。

## Example – Boolean Expressions (1)

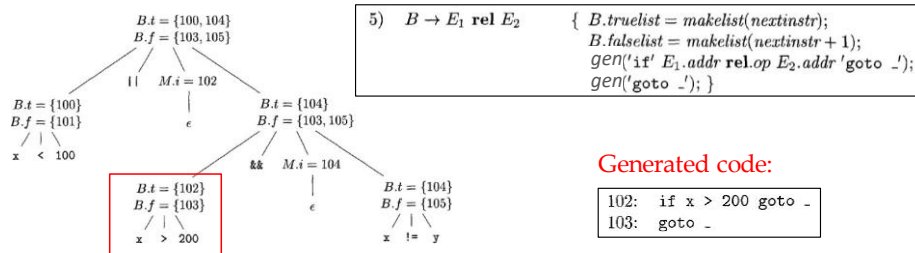
- The earlier SDT is a **postfix SDT**. The semantic actions can be performed during a bottom-up parse.
- Boolean expression:  $x < 100 \parallel x > 200 \&\& x \neq y$
- **Step 2:** reduce  $\epsilon$  to  $M$  by production (8)



产生式 (8) 中的标记非终结符记录下一条指令的标号 102.

## Example – Boolean Expressions (2)

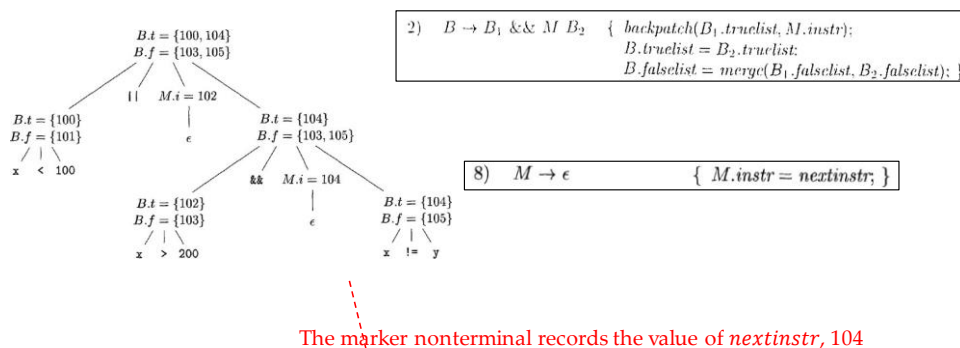
- Boolean expression:  $x < 100 \parallel x > 200 \ \&\& \ x \neq y$
- **Step 3:** reduce  $x > 200$  to  $B$  by production (5)



根据产生式 (5) 将  $x > 200$  归约为 B 时, 产生两条指令:

## Example – Boolean Expressions (2)

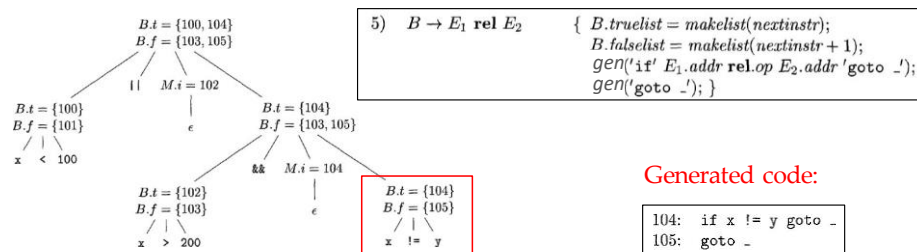
- Boolean expression:  $x < 100 \parallel x > 200 \ \&\& \textcolor{red}{x} \neq \textcolor{red}{y}$
- **Step 4:** reduce  $\epsilon$  to  $M$  by production (8)



子表达式  $x > 200$  对应于产生式 (2) 中的 B1, 标记非终结符 M 记录了下一条指令的标号 104

## Example – Boolean Expressions (3)

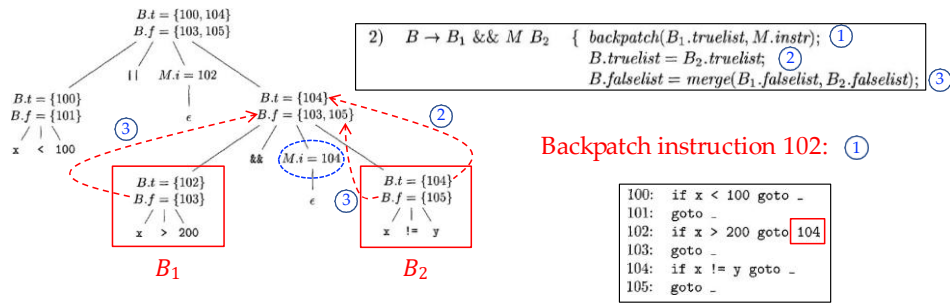
- Boolean expression:  $x < 100 \parallel x > 200 \ \&\& \ x \neq y$
- **Step 5:** reduce  $x \neq y$  to B by production (5)



根据产生式 (5) 将  $x = y$  归约为 B 时, 产生两条指令:

## Example – Boolean Expressions (4)

- Boolean expression:  $x < 100 \parallel x > 200 \ \&\& \ x \neq y$
- **Step 6:** reduce  $B_1 \ \&\& \ M B_2$  to  $B$  by production (2)

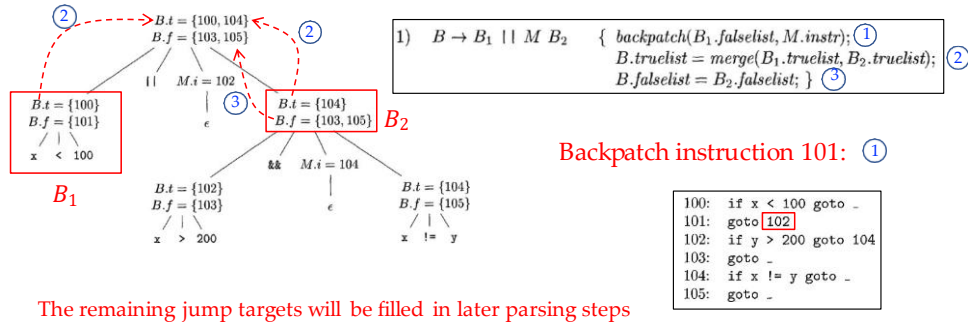


我们现在使用产生式 (2)  $B \rightarrow B_1 \ \&\& \ M \ B_2$  进行归约。相应的语义动作调用  $\text{backpatch}(B_1:\text{truelist}; M:\text{instr})$  将  $B_1$  的真出口绑定到  $B_2$  的第一条指令处。由于  $B_1:\text{truelist}$  为  $\{102\}$ ,  $M:\text{instr}$  为 104, 因此此次对  $\text{backpatch}$  的调用将序号 104 填写到 102 指令中。到目前为止生成的 6 个指令如图 6.45(a) 所示。

1.  $M.\text{instr} = 104 \rightarrow B_1.t\{102\}$
2.  $B.\text{truelist} = B_2.\text{truelist}\{104\}$
3.  $B.\text{falselist} = \text{merge}(B_1.\text{falselist}\{103\}, B_2.\text{falselist}\{105\})$

## Example – Boolean Expressions (5)

- Boolean expression:  $x < 100 \parallel x > 200 \ \&\& \ x \neq y$
- Step 7:** reduce  $B_1 \parallel MB_2$  to  $B$  by production (1)



与产生式(1)  $B \rightarrow B_1 \parallel M B_2$  相关联的语义动作调用  $\text{backpatch}(\{101\}, 102)$ ，得到如图 6.45(b) 所示的指令。整个表达式为 true 当且仅当控制流到达 100 或 104 位置上的跳转指令；整个表达式为 false 当且仅当控制流到达 103 或 105 位置上的跳转指令。在稍后的编译中，当已知表达式为真或假时分别应该做什么的时候，这些指令的目标将会被填写完整

- $\text{backpatch}(B_1.\text{falselist}\{101\}, M.\text{instr}\{102\})$
- $B.\text{truelist} = \text{merge}(B_1.\text{truelist}\{100\}, B_2.\text{truelist}\{104\}) = \{100, 104\}$
- $B.\text{falselist} = B_2.\text{falselist}\{103, 105\} = \{103, 105\}$

## Chapter 6: Run-Time Environments

编译器必须准确地实现源语言定义中嵌入的抽象。这些抽象通常包括我们在 1.6 节中讨论的概念，例如名称、范围、绑定、数据类型、运算符、过程、参数和控制流结构。

编译器必须与操作系统和其他系统软件合作，以支持目标机器上的这些抽象。为此，编译器创建并管理一个运行时环境，并假定其目标程序正在其中执行。该环境处理各种问题，例如源程序中命名的对象的存储位置的布局 and 分配、目标程序访问变量所使用的机制、过程之间的链接、传递机制参数、操作系统接口、输入/输出设备和其他程序。

本章的两个主题是存储位置的分配以及对变量和数据的访问。我们将详细讨论内存管理，包括栈空间分配和堆管理。

# Run-Time Environment

- A compiler must accurately implement the **abstractions** in the source-language definition
  - Names, scopes, bindings, data types, operators, procedures, parameters, flow-of-control constructs...
- To do so, the compiler manages a **run-time environment (运行时刻环境)** and deals with:
  - Layout and allocation of storage locations for data in the source program
  - Mechanisms to access variables
  - Linkages between procedures, the mechanisms for passing parameters
  - Interfaces to the OS, I/O devices, and other programs

• 编译器必须准确实现源语言定义中的抽象概念，包括名称、作用域、绑定、数据类型、运算符、过程、参数、控制流构造等。编译器还必须和操作系统以及其他系统软件协作，在目标机上支持这些抽象概念。

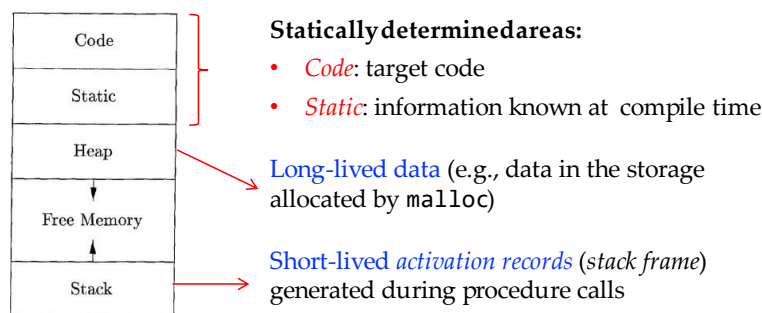
• 为了做到这一点，编译器创建并管理一个运行时环境，它编译得到的目标程序就运行在这个环境中。这个环境处理很多事务，包括内容：

- 为在源程序中命名的对象分配和安排存储位置

- 确定目标程序访问变量时使用的机制
- 过程之间的链接，传递参数的机制
- 与操作系统、I/O 设备和其他程序的接口

## Storage Organization

- Typical subdivision of **run-time memory** into **code** and **data areas** (in the logical address space)



从编译器编写者的角度来看，正在执行的目标程序在其自己的逻辑地址空间中运行，每个程序值在该空间中都有一个地址。对这个逻辑地址空间的管理和组织由编译器、操作系统和目标机共同完成。操作系统将逻辑地址映射到物理地址，而物理地址对整个内存空间编址。

一个目标程序在逻辑地址空间的运行时刻映像包含数据区和代码区，如图 7.1 所示。某个语言的 C++ 在某个操作系统（Linux）上的编译器可能会以这种方式划分存储空间。

在本书中，我们假设运行时存储以连续字节块的形式出现，其中字节是内存的最小编址单元。一个字节包含 8 个二进制位，4 个字节组成一个机器字。多字节数据对象总是存储在一段连续的字节中，并把第一个字节作为它的地址。正如第 6 章所讨论的，一个名字所需的存储空间由其类型决定。基本数据类型（例如字符、整数或浮点数）可以存储在整数个字节中。聚合类型（例如数组或结构）的存储必须足够大以存放这个类型的所有分量。

数据对象的存储布局很大程度上受到目标机器的寻址约束的影响。在许多机器上，整数相加指令可能期望整数对齐，即放置在可被 4 整除的地址处。虽然长度为 10 的字符数组（如 C 语言）只需要足够的字节来容纳 10 个字符，但编译器可以分配 12 个字节以获得正确的对齐方式，留下 2 个字节未使用。因为对齐的考虑而产生的闲置空间称为补白。当空间比较紧张时，编译器可能会压缩数据，以消除补白。但是，在运行时可能需要额外的指令来定位被压缩数据，使得机器在操作这些数据时就好像它们是对齐的。

生成的目标代码的大小在编译时就已经固定下来了，因此编译器可以将可执行目标代码放置在静态确定的区域：代码区。这个区通常位于内存的低端。类似地，程序的某些数据对象的大小可以在编译时刻知道，它们可以被放置在另一个称为静态区的区域中，该区域可被静态确定。放置在这个区域的数据对象包括全局常量和编译器生成的数据。如支持垃圾收集的信息等。之所以要将尽可能多的数据对象进行静态分配，是因为这些对象的地址可以被编译到目标代码中。在 Fortran 的早期版本中，所有数据对象都可以静态分配。

为了在运行时最大限度地利用空间，另外两个区域（栈和堆）位于地址空间剩余部分的相对两端。这些区域是动态的；它们的大小可以随着程序的行而改变。这些区域根据需要向对方增长。栈用于存储称为活动记录的数据结构，这些活动记录在过程调用期间生成。

实际上，栈向低地址方向增长，堆向高地址方向增长。然而，在本章和下一章中，我们将假设栈向较高地址方向增长，以便我们可以在所有示例中使用正偏移量。

正如我们将在下一节中看到的，一个活动记录用于在一个过程调用发生时记录有关机器状态的信息，例如程序计数器和机器寄存器的值。当控制从该次调用返回时，相关寄存器的值被恢复，程序计数器被设置成指向紧跟在这次调用后的点，然后调用过程的活动就可以重新开始。如果一个数据对象的生命周期包含在一次活动的生命周期中，那么该数据对象可以与其他关于该活动的信息一起被分配在堆栈上。

许多编程语言允许程序员通过程序控制人工分配和释放数据对象。例如，C 具有函数 `malloc` 和 `free`，可用于获取和释放任意存储块。堆用于管理这种具有长生命周期的数据。

7.4 节将讨论可用于维护堆的各种内存管理算法。

静态确定的区域包括：

Code（代码）： 目标代码，即在编译时已知的信息，用于生成最终可执行程序指令序列。



Static (静态): 编译时已知的信息, 通常是与全局变量、静态变量以及其他在程序生命周期内保持不变的实体有关的信息。

在这两个静态确定的区域之外, 还有一些涉及动态分配和运行时的区域, 例如:

Long-lived data (长寿命周期的数据): 例如, 由 malloc 分配的存储空间中的数据。这些数据的生命周期可能会跨足过程调用或程序执行的多个阶段。

Short-lived activation records (短寿命周期的激活记录, 也称为栈帧): 在过程调用期间生成的, 用于存储有关该过程执行的信息, 例如局部变量、返回地址等。这些记录通常存储在程序执行的堆栈上, 其生命周期仅限于相应的过程调用。

## Static vs. Dynamic Storage Allocation

- **Static**: the storage-allocation decision can be made by the compiler by looking only at the program text
  - Global constants, global variables
- **Dynamic**: a decision can be made only while the program is running
  - **Stack storage**: The space for names local to a procedure is allocated on a stack. The lifetime of the data is the same as that of the called procedure
  - **Heap storage**: Hold data that may outlive the call to the procedure that created it
    - **Manual memory deallocation** (手工回收内存, e.g., by calling the function free)
    - **Automatic memory deallocation** (a.k.a., garbage collection, 垃圾回收)

运行时环境中数据的布局和内存位置的分配是存储管理中的关键问题。这些问题很棘手, 因为程序文本中的同一个名称在运行时可能指向不同的存储位置。静态和动态这两个形容词分别表示编译时和运行时。

如果编译器可只需通过观察程序的文本而不需要观察该程序在运行时做了什么, 就能做出存储分配决策, 那么我们就说这个存储分配决策是静态的。这包括全局常量和全局变量。

相反，如果只能在程序运行时做出决定，则该决定就是动态的。许多编译器使用以下两种策略的某种组合来进行动态存储分配：

- Stack storage (栈存储)： 用于存储局部于一个过程的名称，这些数据的空间是在一个栈上分配的。数据的生命周期与调用的过程相同。
- Heap storage (堆存储)： 用于保存可能超越创建它的过程调用的数据。这些数据的生命周期可能比创建它的过程更长。堆是虚拟内存的一个区域，它允许对象或其他数据元素在被创建时获得存储空间，并在数据变得无效时释放该存储空间。

## Activation of Procedures

- Procedure calls, or *activation of procedures*, nest in time
  - If a procedure  $P$  calls  $Q$ , the callee  $Q$  returns before its caller  $P$
  - This makes it possible to use stack (*first in/called, last out/return*) to manage the run-time memory used by procedure calls

如果过程调用或过程活动在时间上不是嵌套的，则堆栈分配就不可行。下面的示例说明了过程调用的嵌套情形。

- 如果过程  $P$  调用了过程  $Q$ ，被调用者  $Q$  在其调用者  $P$  之前返回。
- 这使得可以使用栈（先进/调用，后出/返回）来管理过程调用使用的运行时内存。

```

int a[11];
void readArray() { /* Reads 9 integers into a[1], ..., a[9]. */
 int i;
 ...
}
int partition(int m, int n) {
 /* Picks a separator value v, and partitions a[m..n] so that
 a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
 equal to or greater than v. Returns p. */
 ...
}
void quicksort(int m, int n) {
 int i;
 if (n > m) {
 i = partition(m, n);
 quicksort(m, i-1);
 quicksort(i+1, n);
 }
}
main() {
 readArray();
 a[0] = -9999;
 a[10] = 9999;
 quicksort(1,9);
}

```

### Possible activations:

```

enter main()
 enter readArray()
 leave readArray()
 enter quicksort(1,9)
 enter partition(1,9)
 leave partition(1,9)
 enter quicksort(1,3)
 ...
 leave quicksort(1,3)
 enter quicksort(5,9)
 ...
 leave quicksort(5,9)
 leave quicksort(1,9)
leave main()

```

图 7.2 包含一个程序概要，该程序将 9 个整数读入数组 `a` 并使用递归快速排序算法对它们进行排序。

程序主函数有三个任务。它调用 `readArray`，设置上下限值，然后对整个数组调用快速排序 `quicksort`。图 7.3 显示了程序执行可能产生的一系列调用。在此执行中，对 `partition(1; 9)` 的调用返回 4，因此 `a[1]` 到 `a[3]` 保存小于其所选分隔符值 `v` 的元素，而较大的元素位于 `a[5]` 到 `a[9]` 中。

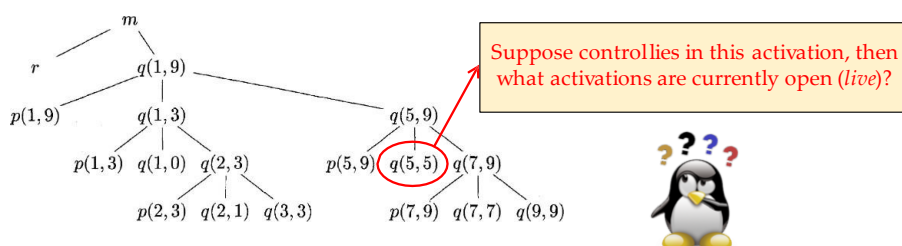
在此示例中，过程活动在时间上是嵌套，在一般情况下也这样。如果过程 `p` 的一个活动调用了过程 `q`，则 `q` 的该次活动必定在 `p` 的活动结束之前结束。常见的情况有以下三种：

1. `q` 的该次活动正常终止。那么，基本上在任何语言中，控制流从 `p` 中调用 `q` 的点之后继续。
2. `q` 的该次活动（或者 `q` 调用的某个过程）直接或间接地中止了，也就是说执行不可能继续下去。在这种情况下，`p` 与 `q` 同时结束。
3. `q` 的该次活动由于 `q` 无法处理的异常而终止。过程 `p` 可能会处理这个异常。此时 `q` 的活动已经终止，而 `p` 的活动继续，尽管 `p` 的活动不一定从调用 `q`

的点开始。如果  $p$  无法处理异常，则  $p$  的活动与  $q$  的活动同时终止。一般来说，某个过程的尚未结束的活动将处理这个异常。

## Activation Trees (活动树)

- We can represent the activations of procedures during the running of an entire program by a tree, called *activation tree*
  - Each **node** corresponds to one activation
  - The **root** is the activation of the “main” procedure
- Two interesting properties:**
  - The sequence of procedure calls corresponds to a preorder traversal
  - The sequence of returns corresponds to a postorder traversal



Fall 2024

Compilers Chapter 6

9

因此，我们可以用一棵树来表示整个程序运行期间过程的活动，称为活动树。

每个节点对应于一个活动，而根是启动程序执行的 “main” 过程的活动。

在过程  $p$  的某个活动的结点上，其子结点对应于被  $p$  的这次活动调用的各个过程的活动。我们按照这些活动被调用的顺序，从左到右显示这些活动。请注意，一个子节点必须在其右侧兄弟节点的活动开始之前结束。

图 7.3 中给出了一个调用和返回序列，而图 7.4 中则显示了一棵完成这个调用/返回序列的可能的活动树。各个函数由其名称的第一个字母表示。请记住，这棵树只是一种可能性，因为后续调用的参数以及沿各个分支的调用次数都受到 `partition` 的返回的值的影

响。

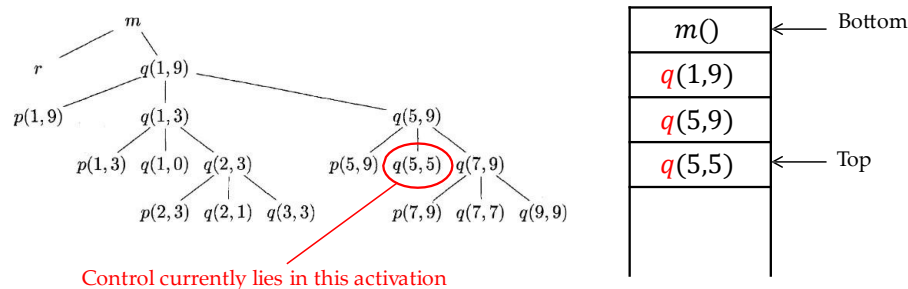
在活动树和程序行为之间存在下列多种有用的对应关系，正因为这些关系使我们可以使用运行时刻栈

- 过程调用的顺序对应于活动树的先序遍历。
- 过程返回的顺序对应于活动树的后序遍历。

3. 假设控制位于某个过程的特定活动中，且该过程活动对应于活动树上的某个结点  $N$ 。那么当前尚未结束的活动（即活跃的）活动就是结点  $N$  及其祖先结点对应的活动。这些活动被调用的顺序是它们从根结点到  $N$  的路径上的出现顺序。这些活动将按照该顺序反序返回。

## Activation Record (活动记录)

- Procedure calls and returns are usually managed by a run-time stack called the *control stack* (or *call stack*)
- Each live activation has an *activation record* (or *stack frame*) on the control stack

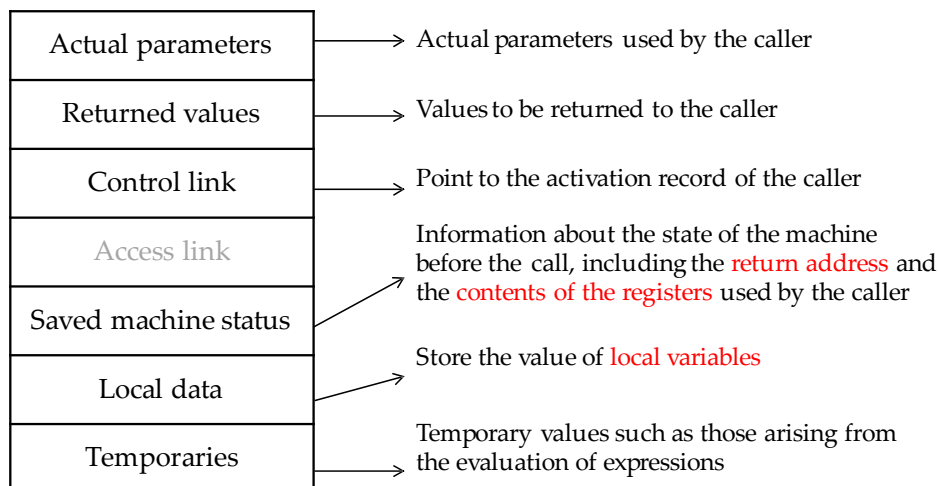


过程调用和返回通常由称为控制栈的运行栈管理。每个活跃的活动在控制栈上都有一个活动记录（有时称为帧）。活动树的根位于栈底部，栈上的全部活动记录的序列对应于活动树中到达当前控制所在活动结点的路径。程序控制所在的活动记录位于栈顶。

如果当前控制位于图 7.4 树的活动  $q(5; 5)$  上中，则  $q(5; 5)$  的活动记录位于控制栈的顶部。紧跟在下面的是  $q(5; 9)$  的活动记录，即树中  $q(5; 5)$  的父结点。再下面是活动记录  $q(1; 9)$ ，栈的底端主函数  $m$  的活动记录，也就是活动树的根。

我们通常会绘制控制栈，使堆栈的底部高于顶部，因此在一个活动记录中出现在页面最下方的元素实际上最靠近栈顶。

# What's in An Activation Record?



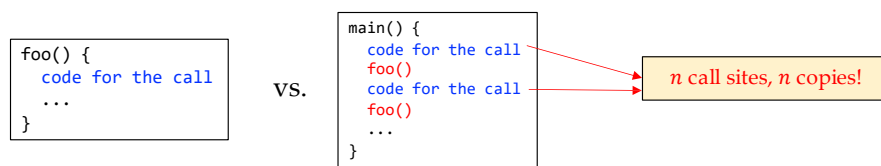
根据所实现的语言的不同，其活动记录的内容也有所不同。以下是活动记录中可能出现的数据类型列表（有关这些元素及它们之间的可能顺序，请参见图 7.5）

1. 临时值，例如由表达式求值过程中产生的中间结果无法保存在寄存器中，就会生成这些临时值。
2. 对应于这个活动记录的过程的局部数据。
3. 保存的机器状态，其中包含对此过程的此次调用之前的机器状态信息。该信息通常包括返回地址（程序计数器的值，被调用的过程必须返回到该值所指的位置）以及一些寄存器中的内容（调用过程会使用这些内容，被调用过程必须在返回时恢复这些寄存器的内容）。
4. 一个“访问链”。当被调用过程需要其他地方（例如，在另一个活动记录中）的某个数据时需要使用访问链进行定位。访问链接在第 7.3.5 节中讨论。
5. 一个控制链，指向调用者的活动记录。
6. 当被调用函数有返回值时，要有一个用于存放这个返回值的的空间。并非所有被调用的过程都会有返回值，即使有，我们也可能更愿意将该值放在一个寄存器中以提高效率。

7. 调用过程使用的实际参数。这些值通常尽可能地放寄存器中，而不是放在活动记录中，因为放在寄存器中会得到更好地效率。然而，我们仍然为它们预留了相应的空间，使得我们的活动记录具有完全的通用性。

## Calling Sequences (方法调用代码序列)

- *Calling sequences*, consisting of code that (1) allocates an activation record on the stack and (2) enters information into its fields
- A *return sequence* (返回代码序列) restores the state of the machine so that the caller can continue its execution after the call
- The code in a calling sequence is **divided between the caller and the callee**
  - There is **no exact division**; the source language, target machine, and the OS may impose requirements that affect the solution
  - A general rule is to **put as much code as possible into the callee**



过程调用是通过所谓的调用序列来实现的。

调用序列：为一个活动记录在栈中分配空间，并在此记录的字段中填写信息。

返回序列：恢复机器状态，使得调用过程能够在调用结束之后继续执行。

即使对于同一种语言，不同实现中的调用序列和活动记录的布局也可能有很大差异。

调用序列中的代码通常被分割到调用过程（调用者）和被调用过程（被调用者）中。在分割运行时刻任务时，调用者和被调用者之间不存在明确界限。源语言、目标机器和操作系统会提出某些要求，使得能够选择出一种较好的解决方案。一般来说，如果一个过程在  $n$  个不同的点上被调用，分配给调用者的那部分调用代码序列会被生成  $n$  次。然而，分配给被调用者的部分仅被生成一次。因此，最好将调用序列中尽可能多的部分放入被调用者中——能够根据被调用者的信息定的部分都应该放到被调用者中。不过，我们将看到被调用者不可能知道所有的事情。

# Calling/Return Sequences' Job

- From the **data** perspective
  - Correctly pass arguments to the callee
  - Correctly pass the return values to the caller
- From the **control** perspective
  - Correctly transfer the control to the first instruction of the callee
  - Correctly transfer the control back to the caller so that it can continue with the instruction immediately after the procedure-call statement

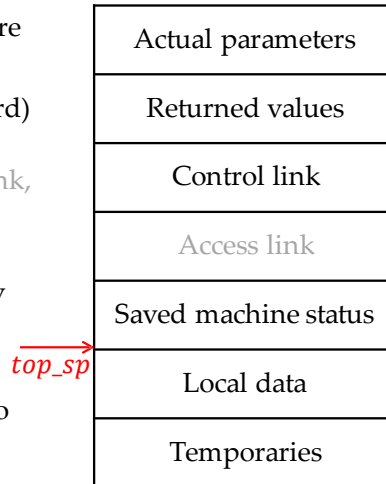
- 从数据角度看
  - 正确地向被调用者传递参数
  - 正确地将返回值传递给调用者
- 从控制角度看
  - 将控制权正确转移到被调用者的第一条指令上
  - 正确地将控制权转回调用程序，以便调用程序在调用语句执行完毕后立即继续执行指令



# Layout of Activation Records

## General Principles

- Values passed between caller and callee are put at the beginning of the callee's activation record (next to the caller's record)
- Fixed-length items (control link, access link, saved machine status) are in the middle
- Items whose size may not be known early enough are placed at the end
- The top-of-stack (*top\_sp*) pointer points to the end of the fixed-length fields



在设计调用序列和活动记录的布局时，可以使用下列原则：

1. 调用者和被调用者之间传递的值通常放在被调用者活动记录的开始位置，因此它们尽可能接近调用者的活动记录。这样做的动机是调用者可以计算该次调用的实在参数的值并将它们放置在自身活动记录的顶部，而不必创建整个被调用者的活动记录，甚至不需要知道该记录的布局。此外，它还使得语言可以使用参数个数或类型可变的过程，例如 C 的 `printf` 函数。被调用者知道应该把返回值放置在相对于其自己的活动记录的哪个位置。同时，不管有多少个参数，它们都将在栈中按顺序地出现在该位置的下方。

2. 固定长度的项被放在中间位置。从图 7.5 中，这些项通常包括控制链、访问链和机器状态字段。如果每次调用中都保存的机器状态的成分相同，那么可以使用同一段代码来保存和恢复每次调用的数据。此外，如果我们机器的状态信息标准化，那么如果发生错误，诸如调试器等这样的程序可以更容易地将栈中的内容解码。

3. 那些在早期无法知道大小的项被放置在活动记录的末尾。大多数局部变量都有固定的长度，编译器通过检查变量的类型就可以确定其长度。然而，有些局部变量的大小只有在程序执行时才能确定；最常见的示例是动态数组，数组大小根据被调用者的某个参数决定。此外，临时量所需的空间的大小通常取决于代码生成阶段能够将多少临时变量放在寄

寄存器中。因此，虽然编译器最终可以知道临时变量所需的空間，但在刚开始生成中间代码时可能并不知道該空間的大小。

4. 我们必须小心确定栈顶指针所指的位置。一种常见的方法是让这个指针指向活动记录中固定长度字段的末尾。这样固定长度的数据就可以通过固定的相对于栈顶指针的偏移量来访问，而中间代码生成器知道这些偏移量。使用这种方法的后果是活动记录中的可变长度字段实际上位于堆栈之上。它们的偏移量需要在运行时计算，但是它们仍然可以基于栈顶指针进行访问，但偏移量为正。

## Steps of A Calling Sequence

1. The caller evaluates the **actual parameters**
2. The caller stores a return address and the old value of ***top\_sp*** into the callee's activation record
3. The caller increments ***top\_sp*** accordingly
4. The callee saves the **register values** and other status info
5. The callee initializes its **local data** and begins execution

图 7.7 给出了调用者和被调用者如何合作管理堆栈的示例。寄存器 `top_sp` 指向当前的顶层活动记录中 机器状态字段的末尾。调用者知道这个位于被调用者的活动记录中的位置。因此，调用者可以负责在控制转向被调用者之前设定 `top sp` 的值。这个调用代码序列以及它在调用者和被调用者之间的划分如下：

1. 调用者计算实在参数的值。
2. 调用者将返回地址和原来的`top_sp`值存放到被调用者的活动记录中。

3. 调用者增加`top_sp`的值（为将来的栈操作做准备），使之指向图 7-7 所示的位置。也就是说，`top_sp`超过了调用者的局部数据和临时变量以及被调用者的参数和机器状态字段。

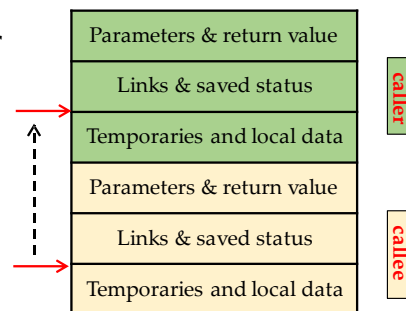
4. 被调用者保存寄存器值和其他状态信息。

5. 被调用者初始化其局部数据并开始执行。

## Steps of A Return Sequence

- The callee places the **return value** next to the actual parameters fields in its activation record
- Using information in the machine-status field, the callee restores **`top_sp`**\* and other **registers**
- Go to the **return address** set by the caller

\* Although `top_sp` has been decremented, the caller knows where to find the return value and use it (recall that caller places the actual parameters)



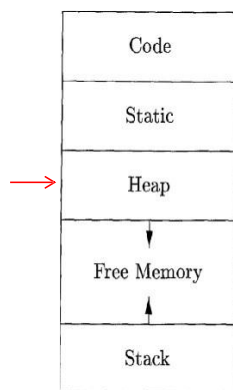
一个与此匹配的返回代码序列：

1. 被调用者将返回值放在参数相邻的位置，如图 7.5 所示。
2. 使用机器状态字段中的信息，被调用者恢复 `top_sp` 和其他寄存器，然后跳转到由调用者放置在状态字段中的返回地址。
3. 尽管 `top_sp` 已经被减小，但调用者知道返回值相对于当前 `top_sp` 值的位置。因此，调用者可以使用那个返回值。

上述调用和返回序列支持使用不同数量的参数来调用同一个被调用程序（例如，如 C 的 `printf` 函数）。请注意，在编译时，调用者的目标代码知道它向被调用中提供的参数的数量和类型。因此调用者知道参数区域的大小。然而，被调用者的目标代码必须还能处理其他调用，因此它要等到被调用时再检查相应的参数字段。使用图 7.7 的组织方法，描述

参数的信息必定放在状态字段的相邻位置，因此被调用者可以找到这个信息。例如，在 C 的 `printf` 函数中，第一个参数描述了其余的参数，因此一旦找到第一个参数，被调用者就可以找到任何其他参数。

## Heap (堆)



- The *heap* is used for data that *lives indefinitely*, or until the program explicitly deletes it
- Many languages enable us to create objects/data whose existence is not tied to the procedure activation that creates them
  - In Java, the objects created with *new* can exist long after the procedure that created them is gone
  - In C, the memory space allocated with *malloc* is accessible until the *free* operation is applied

堆是存储空间的一部分，它被用于存储那些生命周期不确定，或将生存到被程序显式删除为止的数据。虽然局部变量会在它们所属的过程结束时变得不可访问，但许多语言支持创建对象或其他数据，它们的存在与否和创建它们的过程活动无关。

例如，C++ 和 Java 都为程序员提供了 `new` 语句，该语句创建的对象（或指向对象的指针）可以在过程间进行传递。因此，这些对象在创建它们的过程结束后仍然可以存在很长一段时间。此类对象存储在堆上。

在 C 语言中，使用 `malloc` 分配的内存空间在执行 `free` 操作之前都是可以访问的

# The Memory Manager

- Memory manager **allocates and deallocates space within the heap**
  - It serves as an **interface** between application programs and the OS
- The mechanism depends on languages
  - For C/C++, developers need to deallocate memory manually (using **free** or **delete**). The memory manager is responsible for implementing deallocation
  - For Java, garbage collector (a subsystem of memory manager) finds spaces within the heap that are not longer used and reallocates such spaces to hold other data items

在本节中，我们将讨论内存管理器，即在堆内分配和释放空间的子系统；它充当应用程序和操作系统之间的接口。

对于像 C 或 C++ 这样需要手动回收存储块的语言（即通过程序的显式语句，例如 `free` 或 `delete` 进行回收）而言，内存管理器还负责实现空间回收。

在第 7.5 节中，我们讨论垃圾回收，即在堆中查找程序不再使用的空间的过程，因此可以重新分配这些空间来容纳其他数据项。对于像 Java 这样的语言，内存的回收是垃圾收集器完成的。当需要进行垃圾回收时，垃圾收集器是内存管理器的一个重要子系统。

# Basic Functions

- **Allocation (分配内存):** Provide contiguous heap memory when a program requests memory for a variable or object
  - If possible, it satisfies the request [using free space in the heap](#)
  - Otherwise, it seeks to increase the heap storage space by [getting consecutive bytes of virtual memory](#) from the OS
- **Deallocation (回收内存):** Return deallocated space to the pool of free space for reuse
  - Typically, memory managers do not return memory to the OS, even if the program's heap usage drops

内存管理器始终跟踪堆存储中的所有空闲空间。它具有两个基本功能：

- 分配。当程序为变量或对象请求内存时，内存管理器会生成一段连续的具有被请求大小的堆空间。如果有可能，它使用堆中的空闲空间来满足分配请求；如果没有所需大小的空间块可供分配，它试图从操作系统中获取连续的虚拟内存来增加堆的存储空间。如果空间耗尽，内存管理器会将该信息传递回应用程序。
- 回收。内存管理器将释放的空间返回到空闲空间的缓冲池中，以便它可以重用该空间来满足其他分配请求。即使当这个程序的不再需要那么多的堆空间时，内存管理器通常也不会将内存返回给操作系统。

如果下面的 (a)、(b) 两个条件都成立，内存管理会更简单：

- (a) 所有分配请求都是要求相同大小的存储块，
- (b) 存储空间按照可预测地方式释放。例如，先分配先回收。

有一些语言，例如 Lisp，条件 (a) 成立；纯 Lisp 仅使用一种数据元素 —— 一个双指针单元，所有数据结构都是在该元素的基础上构建的。条件 (b) 在某些情况下也成立，最常见的是可以在运行时刻栈上分配的数据。然而，在大多数语言中，(a) 和 (b) 都不普遍成

立。相反，我们需要为不同大小的数据元素分配空间，并且没有好的方法来预测所有已分配对象的生命周期。

因此，内存管理器必须准备好以任何顺序来处理任何大小的空间分配和回收请求，这些请求小到一个字节大到该程序的整个地址空间

## Desirable Properties

- **Space efficiency (空间效率):** Minimize the total heap space needed by a program, i.e., minimizing “fragmentation” (碎片化)
- **Program efficiency (程序效率):** Make better use of the memory hierarchy to allow programs to run faster\*
- **Low overhead (低开销):** Minimize the execution time spent on memory allocation/deallocation

\*The time taken to execute an instruction can vary widely depending on where objects are placed in memory

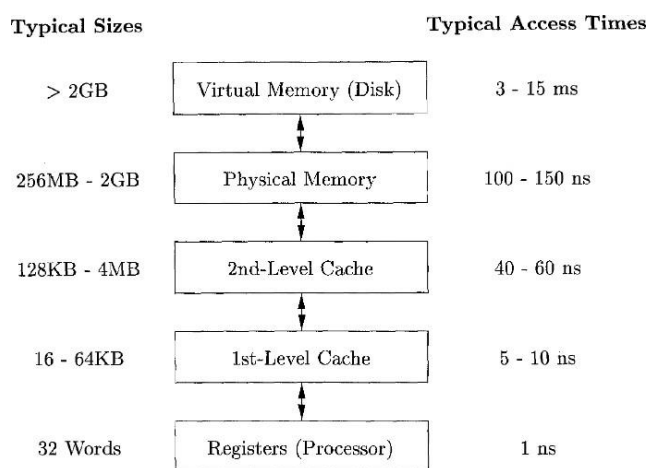
以下是我们希望内存管理器具有的属性：

- 空间效率。内存管理器应该最大限度地减少程序所需的总堆区空间。这样做就可以在一个固定大小的虚拟地址空间中运行更大的程序。空间效率是通过使存储碎片达到最少而得到的。” 在第 7.4.4 节中讨论。

- 程序效率。内存管理器应该充分利用存储子系统，使程序运行得更快。正如我们将在第 7.4.2 节中看到的，根据数据对象在内存中所处的不同位置，执行一条指令所花费的时间可能会有很大差异。幸运的是，程序往往会表现出“局部性”，这是第 7.4.3 节中讨论的一种现象，它指的是通常的程序访问内存时具有的非随机性聚焦的特性。通过关注对象在内存中的放置方法，内存管理器可以更好地利用空间，并有望使程序运行得更快。

- 低开销。由于内存分配和回收是许多程序中的频繁操作，因此这些操作尽可能高效非常重要。也就是说，我们希望最小化开销，即花费在分配和回收上的执行时间在总运行时间所占的比例。请注意，分配开销主要由小型请求决定；管理大型对象的开销相对不太重要，因为它通常会在它上执行大量的计算，这个开销被分摊了。

## Memory Hierarchy



内存管理和编译器优化必须在充分了解存储行为方式的情况下进行。现代机器的设计使程序员无需关心内存子系统的细节就可以编写正确的程序。然而，程序的效率不仅取决于执行的指令数量，还取决于执行每条指令所需的时间。不同情况下执行一条指令所花费的时间可能会有很大差异，因为访问不同的存储区域所花费的时间可能从纳秒到毫秒不等。因此，数据密集型程序可以从充分利用内存子系统的优化中获益匪浅。正如我们将在第 7.4.3 节中看到的，它们可以利用“局部性”现象——典型程序的非随机行为。

内存访问时间的巨大差异是由于硬件技术的根本限制造成的；我们可以构建小而快的存储，或者大而慢的存储，但不能构建既大又快的存储。即使是现在也不可能构建具有纳秒访问时间的千兆字节存储，而纳秒级正是高性能处理器的运行速度。因此，几乎所有现代计算机都将其存储安排为内存层次结构。如图 7.16 所示，内存层次结构由一系列存储



元素组成，较小且较快的存储元素距离处理器“较近”，而较大且较慢的存储元素则距离处理器较远。

通常，处理器具有少量寄存器，寄存器中的内容由软件控制。其次，它具有一级或多级高速缓存，这些高速缓存通常使用静态 RAM 制成，大小为千字节到几兆字节。层次结构的下一层是物理（主）内存，由数百兆字节或千兆字节的动态 RAM 组成。然后，物理内存由下一层的虚拟内存提供支持，虚拟内存是由千兆字节的磁盘实现的。在进行内存访问时，机器首先在最近（最底层）的存储中查找数据，如果数据不存在，则到上一层中寻找，依此类推。

寄存器是稀缺的，因此寄存器的使用是针对特定应用程序定制的，并由编译器生成的代码进行管理。存储层次结构的所有其他层都是自动管理的；这样，不仅简化了编程任务，而且同一个程序可以在具有不同存储配置的机器上高效地运行。每次访问内存时，机器都会从最低层开始逐层搜索每一层的存储，直到找到数据。高速缓存是完全通过硬件进行管理的，以便跟上相对较快的 RAM 访问时间。因为磁盘访问速度相对较慢，虚拟内存是由操作系统进行管理的，辅以称为“转换旁视缓冲”的硬件结构。

数据以连续存储块进行传输。为了分摊访问开销，较大的块用于层次结构中较慢的级别。在主内存和高速缓存之间的数据是按照被称为高速缓存线的块进行传输的，高速缓存线的长度通常为 32 到 256 字节之间。在虚拟内存（磁盘）和主内存之间的数据以被称为页的内存形式进行传输，大小通常在 4K 到 64K 字节之间。

# Program Locality (程序局部性)

- Most programs exhibit a high degree of locality:
  - **Temporal locality (时间局部性):** the memory locations accessed are likely to be accessed again within a short period of time
  - **Spatial locality (空间局部性):** memory locations close to the locations accessed are likely also to be accessed within a short period of time
- Programs spend 90% of their time executing 10% of the code (80/20 or Pareto's Principle in programming)

Locality allows us to take advantage of the memory hierarchy to lower the average memory-access time: place the most common instructions and data in the fast-but-small storage, while leaving the rest in the slow-but-large storage

大多数程序都表现出高度的局部性；也就是说，它们将大部分时间花在执行相对较小的代码部分并仅接触一小部分数据。

如果一个程序访问的内存位置很可能在短时间内被再次访问，我们就说该程序具有**时间局部性**。

如果靠近被访问位置的内存位置也可能在短时间内被访问，我们就说程序具有**空间局部性**。

传统观点认为，程序花费 90% 的时间执行 10% 的代码（80/20 或编程中的帕累托原则）。原因如下：

程序经常包含许多从未执行过的指令。使用组件和库构建的程序仅使用了它们所提供功能的一小部分。此外，随着需求的变化和程序的演化，遗留系统通常包含许多不再使用的指令。

在程序的一次典型运行中，可能被调用的代码中只有一小部分会被实际执行。例如，虽然处理非法输入和异常情况的指令虽然对程序的正确性至关重要，但它们在某次运行中很少被调用。

通常的程序往往大部分时间都花费在执行程序中的最内层循环和最紧凑的递归循环上。

局部性使我们能够利用现代计算机的存储层次结构，如图 7.16 所示。通过将最常用的指令和数据放在快速但较小的存储中，而将其余的留在缓慢但较大的存储中，我们可以显著降低程序的平均内存访问时间。

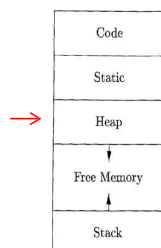
已经发现，许多程序在访问指令和数据的方式上既表现出时间局部性，又表现出空间局部性。然而，数据访问模式通常比指令访问模式表现出更大的多样性。将最近使用的数据放在最快的存储层次中等策略对于常见程序来说效果很好，但对于某些数据密集型程序可能效果不佳。例如，循环遍历非常大的数组。

仅通过查看代码，我们通常无法判断代码的哪些部分将被大量使用，尤其是对于特定输入这一点则更加困难。即使我们知道哪些指令会被频繁执行，最快的高速缓存通常也不足以同时容纳所有指令。因此，我们必须动态调整最快存储的内容，并用它来保存在可能很快会被使用的指令。

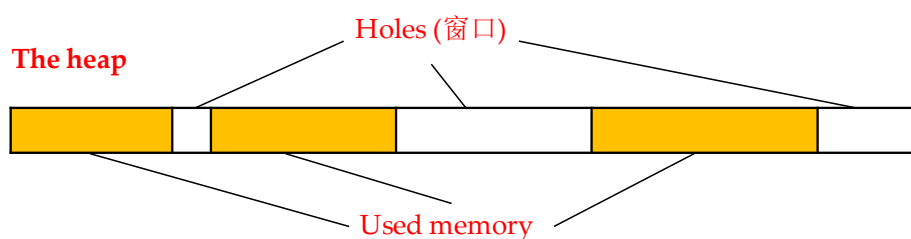
将最近使用的指令保留在高速缓存中的策略往往效果很好；换句话说，过去的情况通常可以很好地预测未来的内存使用情况。当执行一条新指令时，很有可能下一条指令也会被执行。这种现象是一个空间局部性的例子。提高指令空间局部性的一种有效技术是让编译器把很可能连续执行的多个基本块连续存放（始终按顺序执行的指令序列）即放在同一存储页上，甚至在同一高速缓存线上。属于同一循环或同一函数的指令也很有可能被一起执行。

我们还可以通过改变数据布局或计算顺序来提高程序中数据访问的时间和空间局部性。例如，一些程序重复访问大量数据、而每次访问只完成少量计算，这样的程序性能不佳。如果我们能够将一些数据从存储层次结构的慢速层次加载到较快的层次（例如，从磁盘到主内存），并在该数据驻留在较快层中时执行所有针对该数据的计算，那程序的性能就会变得更好了。这个概念可以递归地应用于物理内存、高速缓存和寄存器中的数据的复用。

# The Fragmentation Problem (1)



- At the beginning of program execution, the heap is one contiguous unit of free space
- As the program allocates/deallocates memory, the heap is broken up into free and used chunks



- 在程序执行开始时，堆是一个连续的可用空间单元。
- 随着这个程序分配和回收存储工作的进行，空间被分割成若干空闲存储块和已用存储块，而空闲块不一定位于堆区的某个连续区域中。
- 我们将空闲内存块称为窗口。

# The Fragmentation Problem (2)

- With each allocation request, the memory manager must place the requested memory into a **large-enough hole**
  - Typically, it needs to **split** some hole, creating a yet smaller hole
- With each deallocation request, the freed memory are added back to the pool of free space
  - Should **combine contiguous holes** into larger ones. Otherwise, the holes can only get smaller
- Without a good strategy, the free memory may get *fragmented*, consisting of large numbers of small, noncontiguous holes

对于每个分配请求，内存管理器必须将请求的内存块放入足够大的窗口中。除非找到尺寸完全正确的孔，否则我们需要劈开一些孔，创建一个更小的孔。

对于每个回收请求，被回收的内存块都会添加回空闲空间的缓冲池中。我们将连续的窗口合并成更大的窗口，否则窗口只会变得更小。如果我们不小心，空闲内存最终可能会变得碎片化，由大量不连续的小窗口组成。此时，即使可能有足够的总空闲空间，也可能没有足够大的窗口来满足未来的请求。

# Object Placement Strategies

- **Best-fit algorithm**

- Allocate the requested memory in the smallest available hole that is large enough
- This algorithm spares the large holes to satisfy subsequent, larger requests, which tends to **improve space utilization**

- **First-fit algorithm**

- An object is placed in the first (lowest-address) hole in which it fits
- It **takes less time** to place objects and **improves spatial locality**, but is inferior to best-fit in overall performance

我们通过控制内存管理器如何在堆中放置新对象来减少碎片。

- 根据经验发现，最小化现实程序碎片的一个好策略是将请求的存储分配在满足请求的最小可用窗口中。这种 best-fit 算法倾向于将大的窗口保留下来以满足后续更大的请求。（提升空间利用率）

- 另一种称为 first-fit 的替代方案，将对象放置到第一个（即地址最低的）能够容纳请求对象的窗口中。这种策略在放置对象时所需的时间较少（提升空间局部性），但在总体性能方面不如 best-fit 性能好。

# The Binning Strategy for Best-Fit

- Separate free space chunks into *bins* (容器), according to chunk sizes
- Have more bins for smaller-sized chunks (objects are often small)
- The **Lea memory manager** of the GNU C compiler **gcc** aligns all chunks to 8-byte boundaries
  - There is a bin for every multiple of 8-byte chunks: 16, 24, 32, ..., 512
  - Organizing larger-sized chunks (>512 bytes): the size of the smallest chunk for each bin is twice that of the previous one. Within each bin, the chunks are ordered by size.
  - Wilderness chunk (荒野块): the space can be extended by requesting more pages from the OS. Lea treats this chunk as the largest-sized bin

Learn more about Doug Lea's memory allocator: [https://cw.fel.cvut.cz/old/\\_media/courses/a4m33pal/04\\_dynamic\\_memory\\_v6.pdf](https://cw.fel.cvut.cz/old/_media/courses/a4m33pal/04_dynamic_memory_v6.pdf)

为了更有效地实现 best-fit 放置策略，我们可以根据空闲空间块的大小将它们分在若干个容器中。

一种实用的想法是为较小的尺寸设置较多的容器，因为小对象的个数通常比较多。

例如，GNU C 编译器 gcc 中使用的 Lea 内存管理器将所有块对齐到 8 字节的边界。

- 从 16 字节到 512 字节之间的、每个 8 字节块的整数倍的存储块都设置了一个容器。
- 较大尺寸的容器按照对数间隔（即，每个容器的最小尺寸是前一个 bin 的最小尺寸的两倍）。在每个容器中，块按其大小排序。
- 总有一块可用空间，存储管理器可以向操作系统请求更多页面来扩展这个块。该块称为荒野块，由于其可扩展性，Lea 将其视为最大尺寸存储块的容器。

# The Binning Strategy for Best-Fit

- Binning makes it easy to find the best-fit chunk:
    - For **small sizes** requests, if there is a bin for chunks of that size only, we may take any chunk from the bin (the chunks are of the same size)
    - For **sizes that do not have a private bin**, we find the bin that is allowed to include chunks of the size.
      - Within the bin, where the chunks may have different size, we can use either a first-fit or a best-fit algorithm\*
    - When the above searches fail, we continue to check the bin for the next larger size(s). Eventually, we may reach **wilderness chunk**
- \* When the fit is not exact, the remainder of the chunk will generally need to be placed in a bin with smaller sizes

容器机制使得寻找 best-fit 块变得容易。

- 对于 Lea 内存管理器请求的小尺寸，如果有一个容器仅用于该大小的块，我们可以从该容器中任意取出一个存储块。
- 对于没有专用容器的大小，我们会找到一个允许包含所需大小的块容器。在该容器中，我们可以使用 first-fit 或 best-fit 策略；也就是说，我们要么寻找并选择第一个足够大的块，要么花费更多时间找到足够大的最小块。请注意，如果选择空闲存储块的大小不是正好合适，块的剩余部分通常需要放置在较小尺寸的容器中。
- 然而，目标容器可能是空的，或者该容器中的所有块都太小而无法满足空间请求。在这种情况下，我们只需重复搜索，使用下一个更大尺寸的容器。最终，我们要么找到可以使用的块，要么到达“荒野”块，从这个荒野块中我们肯定可以获得所需的块，但有可能需要请求操作系统为堆区增加更多的内存页。

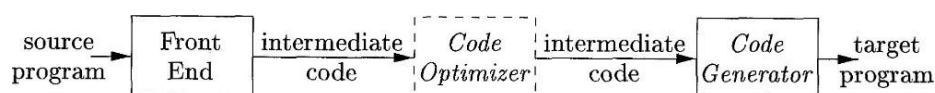
虽然 best-fit 可以提高空间利用率，但就空间局部性而言，它可能并不是最佳的。程序在同一时间分配的块往往具有相似的访问模式并具有类似的生命周期。因此，将它们靠近放置可以改善程序的空间局部性。best-fit 算法的有用的改进之一是在无法找到精确请求



大小的块的情况下使用另一种对象放置方法。在这种情况下，我们使用 `next-fit` 策略，只要刚分割过的存储块中仍有足够的空间容纳这个对象，我们就把这个对象放置在这个存储块中。`next-fit` 策略还可以提高分配操作的速度。

# Code Generator

- **Input:** IR + symbol table; **Output:** target program
- There is often an optimization phase before code generation
- **Three primary tasks of a code generator:**
  - Instruction selection
  - Register allocation and assignment
  - Instruction ordering



- 代码生成器以编译器前端产生的中间表示（IR）以及相关符号表信息作为输入，并产生语义等价的目标程序作为输出，如图 8.1 所示。
- 对代码生成器的要求非常严格。目标程序必须保留源程序的语义并且是高质量的；也就是说，它必须有效地利用目标机器的可用资源。此外，代码生成器本身必须高效运行。
- 挑战在于，从数学上来说，为给定源程序生成最优目标程序的问题是不可判定的；代码生成中遇到的许多子问题（例如寄存器分配）都具有难以处理的计算复杂性。在实践中，我们必使用那些能够产生良好但不一定是最佳代码的启发式技术。幸运的是，启发式方法已经足够成熟，精心设计的代码生成器可以生成比普通代码生成器快几倍的代码。
- 要生成高效目标程序的编译器在代码生成之前包括一个优化阶段。优化器将 IR 映射到另一个可用于产生高效代码的 IR。一般来说，编译器的代码优化和代码生成步骤（通常称为后端）它们可能会在生成目标程序之前对 IR 进行多次处理。

第 9 章详细讨论了代码优化。无论代码生成之前有没有优化步骤，都可以使用本章介绍的技术。

- 代码生成器具有三个主要任务：指令选择、寄存器分配和指派、以及指令排序。第 8.1 节概述了这些任务的重要性。指令选择涉及选择适当的目标机器指令来实现 IR 语句。寄存器分配和指派涉及决定在哪些寄存器中保留哪些值。指令排序涉及决定以什么顺序安排指令的执行。

## Design Issues

- **Design goals:**
  - Correctness (the most important)
  - Ease of implementation, testing, and maintenance
- **Many choices for the input IR:**
  - **Three-address representations:** quadruples, triples, indirect triples
  - **VM representations:** bytecodes and stack-machine code
  - **Graphical representations:** syntax trees and DAG's
- **The target program:**
  - RISC (reduced instruction set computer), CISC (complex instruction...)
  - Absolute machine-language programs; relocatable machine-language programs; assembly-language programs

虽然代码生成器的涉及依赖于中间表示形式、目标语言和运行时系统的特定细节，但指令选择、寄存器分配和指派以及指令排序等任务会在几乎所有的代码生成器设计中碰到。

- ◆ 代码生成器最重要的标准是生成正确的代码。
  - 正确性问题非常突出的原因是代码生成器可能会碰到很多种特殊情况。
  - 在优先考虑正确性的情况下，另一个重要的设计目标是把代码生成器设计得易于实现、测试和维护。

代码生成器的输入是前端生成的源程序的中间表示，以及符号表中的信息。这些信息用于确定 IR 中的名称所指的数据对象的运行时刻地址。

◆ IR 的表示形式有多种选择，包括：

- 三地址表示，例如四元组、三元组、间接三元组；
- 虚拟机表示，例如字节码和堆栈机代码；
- 线性表示，例如后缀表示法；
- 语法树和 DAG 等图形表示。本章中的许多算法都是根据第 6 章中考虑的表示形式来表达的：三地址码、树和 DAG。然而，我们讨论的技术也可以应用于其他中间表示。

在本章中，我们假设前端已经将源程序扫描、解析并翻译成一个相对较低级别的 IR，使得 IR 中出现的名称的值可以用目标机直接处理的量来表示。这些量可以是整数和浮点数等。我们还假设所有语法和静态语义错误都已经被检测出来，必要的类型检查都已经完成，并且类型转换运算已经被插入到必要的地方。因此，代码生成器可以在工作过程中假设它的输入已经排除了这些错误。

◆ 目标程序所依托的体系架构和具体的语言形式

- 目标机器的指令集体系结构对构建产生高质量机器代码的良好代码生成器的难度有重大影响。最常见的目标机架构是 RISC（精简指令集计算机）、CISC（复杂指令集计算机）和基于堆栈的。
- RISC 机器通常具有许多寄存器、三地址指令、简单的寻址模式和相对简单的指令集架构。
- 相比之下，CISC 机器通常具有很少的寄存器、双地址指令、多种寻址模式、多个寄存器类别、可变长度指令以及具有副作用的指令。
- 在基于堆栈的机器中，运算是通过将运算分量压入堆栈，然后对堆栈顶部的运算分量进行运算来完成的。为了实现高性能，栈部元素通常保存在寄存器中。因为人们认为堆栈组织过于限制并且需要大量交换和复制操作，所以基于堆栈的机器几乎消失了。

然而，随着 Java 虚拟机 (JVM) 的引入，基于堆栈的体系结构又复活了。JVM 是 Java 字节码的软件解释器，Java 字节码是 Java 编译器生成的中间语言。这个解释器提供跨多个平台的软件兼容性，这是 Java 成功的一个主要因素。为了克服解释的高性能损失

(大约是 10 倍的数量级), 即时 (JIT) Java 编译器应运而生。这些 JIT 编译器在运行时将字节码转换为目标机器的本机硬件指令集。提高 Java 性能的另一种方法是构建一个编译器, 把 Java 程序直接编译成目标机器的机器指令彻底绕过 Java 字节码。

- 目标程序具体的形式:
- 生成一个使用绝对地址的机器语言程序作为输出的优点是程序可以放置在内存中某个固定的位置上, 并立即执行。程序可以快速编译和执行。
- 输出可重定位的机器语言程序 (通常称为目标模块) 可以使各个子程序能够被分别编译。一组可重定位的目标模块可以被一个链接加载器链接在一起并加载运行。虽然我们要生成可重定位的目标模块, 必须为链接和加载付出代价。但这样做可以使我们获得很多的灵活性。我们可以把子程序分开编译, 并从一个目标模块调用其他先前已编译好的程序。如果目标机没有自动处理重定位, 编译器就必须向加载器提供明确的重定位信息, 以便把分开编译的程序模块链接起来。

生成汇编语言程序作为输出使得代码生成过程更加容易。我们可以生成符号指令并使用汇编器的宏功能来帮助生成代码。这么做的代价是代码生成后还需要增加一个汇编步骤。

在本章中, 我们将使用一个非常简单的类 RISC 计算机作为目标机器。我们添加了一些类似 CISC 的寻址模式, 以便我们还可以讨论 CISC 机器的代码生成技术。为了可读性, 我们使用汇编代码作为目标语言。只要可以根据符号表中存储的偏移量和存放于符号表中的其他信息计算出来, 代码生成器就可以为源程序中的名字生成可重定位地址或绝对地址。这和生成符号地址一样, 都是很简单的事情。

# A Simple Target Machine Model

- A three-address machine with load and store, computation, jump, and conditional jump operations

| Type                | Form                                                              | Effect                                                                                                                                                     |
|---------------------|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Load                | LD <i>dst</i> , <i>addr</i>                                       | load the value in location <i>addr</i> into location <i>dst</i> , where <i>dst</i> is often a register                                                     |
| Store               | ST <i>x</i> , <i>r</i>                                            | store the value in register <i>r</i> into the location <i>x</i>                                                                                            |
| Computation         | OP <i>dst</i> , <i>src</i> <sub>1</sub> , <i>src</i> <sub>2</sub> | apply the operation <i>OP</i> to the values in locations <i>src</i> <sub>1</sub> and <i>src</i> <sub>2</sub> , and place the result in location <i>dst</i> |
| Unconditional jumps | BR <i>L</i>                                                       | jump to the machine instruction with label <i>L</i>                                                                                                        |
| Conditional jumps   | Bcond <i>r</i> , <i>L</i>                                         | jump to label <i>L</i> if the value in register <i>r</i> pass the test <i>Bcond</i> , e.g., less than zero                                                 |

熟悉目标机器及其指令集是设计良好的代码生成器的先决条件。不幸的是，在代码生成的一般讨论中，不可能足够详细地描述任何目标机器来为该机器上的完整语言生成良好的代码。在本章中，我们将使用一个简单计算机的汇编代码作为目标语言。这个计算机是许多寄存器机器的代表。然而，本章介绍的很多代码生成技术也可以用于许多其他类别的机器。

我们的目标计算机是一个三地址机器的模型，具有加载和存储操作、计算操作、跳转操作和条件跳转。这个计算机的内存按照字节寻址，它具有  $n$  个通用寄存器  $R_0; R_1; \dots; R_n$ 。一个完整的汇编语言具有几十到上百个指令。为了避免因为过多的细节而妨碍对概念的理解，我们将使用一个非常有限的指令集，并假设所有的运算分量都是整数。大多数指令由一个运算符、一个目标地址、一个源运算分量的列表组成。指令之前可能有标号。我们假设有以下类型的指令可用：

- 加载操作：指令 LD *dst*,*addr* 将地址 *addr* 中的值加载到位置 *dst* 中。该指令表示赋值  $dst = addr$ 。该指令最常见的形式是 LD *r*,*x*，它将位置 *x* 中的值加载到寄存器 *r* 中。LD *r*<sub>1</sub>,*r*<sub>2</sub> 形式的指令是寄存器到寄存器的复制，其中寄存器 *r*<sub>2</sub> 的内容被复制到寄存器 *r*<sub>1</sub> 中。

- 存储操作：指令  $ST\ x, r$  将寄存器  $r$  中的值存储到位置  $x$  中。该指令表示赋值  $x = r$ 。

- 计算操作：形如  $OP\ dst, src1, src2$ ，其中  $OP$  是  $ADD$  或  $SUB$  等运算符，而  $dst$ 、 $src1$  和  $src2$  是内存位置，这些位置不一定要相互不同。该机器指令的作用是将  $OP$  表示的运算应用于位置  $src1$  和  $src2$  中的值，并将该运算的结果放置在位置  $dst$  中。例如， $SUB\ r1; r2; r3$  计算  $r1 = r2 - r3$ 。先前存储在  $r1$  中的任何值丢失了，但如果  $r1$  等于  $r2$  或  $r3$ ，则计算机首先读取原来的值。只需要一个运算分量的单目运算符没有  $src2$ 。

- 无条件跳转：指令  $BR\ L$  使控制转移到带有标号  $L$  的机器指令。（ $BR$  代表分支。）

- 条件跳转：形式为  $Bcond\ r, L$ ，其中  $r$  是寄存器， $L$  是一个标号，而  $cond$  代表对寄存器  $r$  中的值所做的某个常见测试。例如，如果寄存器  $r$  中的值小于零， $BLTZ\ r, L$  会导致控制跳转到标号  $L$ ，如果不小于零，则允许控制传递到下一条机器指令。

## Addressing Modes (寻址模式)

In instructions, a location can be:

- **Variable name  $x$** : the memory location reserved for  $x$  ( $x$ 's  $l$ -value)
- **$a(r)$** :  $a$  is a variable and  $r$  is a register; the memory location is computed by taking the  $l$ -value of  $a$  and adding to it the value in register  $r$  (this is very useful for accessing arrays)

我们假设我们的目标机器有多种寻址模式：

- 在指令中，一个位置可以是变量名  $x$ ，它指向分配给  $x$  的内存位置（即  $x$  的左值）。
- 位置也可以是一个带有下标的形如  $a(r)$  的地址，其中  $a$  是变量， $r$  是寄存器。 $a(r)$  表示的内存位置是通过获取  $a$  的左值加上存放在寄存器  $r$  中的值来计算的。例如，指令  $LD\ R1, a(R2)$  的效果是  $R1 = \text{contents}(a + \text{contents}(R2))$ ，其中  $\text{contents}(x)$  表示由  $x$  所表示的寄存器或内存位置中存放的内容。这个寻址模式对于访问数组很有用，其中  $a$  是数组的基地址（即第一个元素的地址）， $r$  保存从基地址到数组  $a$  的某个元素所要经过的字节数。

## Addressing Modes (寻址模式)

In instructions, a location can be:

- **\* $r$** : the memory location found in the location represented by the contents of register  $r$ 
  - $LD\ R1, *R2$  has the effect:  $R1 = \text{contents}(\text{contents}(\text{contents}(R2)))$
- **\*constant( $r$ )**: the memory location found in the location obtained by adding the constant to the contents of  $r$  (e.g.,  $*100(r)$ )
- **#constant**: immediate constant addressing mode
  - $LD\ R1, \#100$  loads the integer 100 into register  $R1$

我们还支持另两种间接寻址模式：

- $*r$  表示在寄存器  $r$  的内容所表示的位置上存放的内存位置，
- $*100(r)$  表示在寄存器  $r$  的内容加 100 的和所代表的位置上的内容所代表的位置。

例如， $LD\ R1, *100(R2)$  的作用是设置  $R1 = \text{contents}(\text{contents}(100 + \text{contents}(R2)))$ ，即首先将寄存器  $R2$  中的内容加上 100 的和，取出和值所指向位置中的内容，再把这个内容代表的位置中的值加载到  $R1$  中。



- 最后，我们支持一个直接常数寻址模式。该常数以 # 开头。指令 LD R1, #100 将整数 100 加载到寄存器 R1 中，而 ADD R1, R1, #100 将整数 100 加到寄存器 R1 中。

指令之后的注释由 // 开头

## Examples (1)

- $x = y - z$

```
LD R1, y // R1 = y
LD R2, z // R2 = z
SUB R1, R1, R2 // R1 = R1 - R2
ST x, R1 // x = R1
```

- $b = a[i]$

```
LD R1, i // R1 = i
MUL R1, R1, 8 // R1 = R1 * 8
LD R2, a(R1) // R2 = contents(a + contents(R1))
ST b, R2 // b = R2
```

- 三地址指令  $x=y-z$  可用机器指令序列来实现：

也许我们可以做得更好。良好的代码生成算法的目标之一是尽可能避免使用所有这四个指令。例如， $y$  和/或  $z$  可能已被计算出来并存放在一个寄存器中，如果是这样，我们就可以避免相应的 LD 步骤。同样，如果  $x$  的值被使用时都存放在寄存器中，并且以后不再被用到，我们就不需要把这个值保存回  $x$ 。

- 假设  $a$  是一个数组，其元素是 8（比如实数）字节值。还假设  $a$  的元素下标从 0 开始。我们可以通过下面的指令序列执行三地址指令  $b = a[i]$ ：

也就是说，第二步计算  $8i$ ，第三步将  $a$  的第  $i$  个元素中的值放入寄存器 R2 中，这个元素位于离数组  $a$  的基地址  $8i$  字节的地方。

## Examples (2)

- $a[j] = c$

```
LD R1, c // R1 = c
LD R2, j // R2 = j
MUL R2, R2, 8 // R2 = R2 * 8
ST a(R2), R1 // contents(a + contents(R2)) = R1
```

- $x = *p$

```
LD R1, p // R1 = p
LD R2, 0(R1) // R2 = contents(0 + contents(R1))
ST x, R2 // x = R2
```

- 类似地，对三地址指令  $a[j] = c$  表示的对数组  $a$  的赋值可以实现为：
- 要实现简单的指针间接寻址，例如三地址语句  $x = *p$ ，我们可以使用如下机器指令：

## Examples (3)

- $*p = y$

```
LD R1, p // R1 = p
LD R2, y // R2 = y
ST 0(R1), R2 // contents(0 + contents(R1)) = R2
```

- if  $x < y$  goto L

```
LD R1, x // R1 = x
LD R2, y // R2 = y
SUB R1, R1, R2 // R1 = R1 - R2
BLTZ R1, M // if R1 < 0 jump to M
```

M is a label that represents the first machine instruction generated from the three-address instruction that has label L

- 通过指针的赋值语句  $*p = y$  可以类似地用以下的机器代码实现：
- 最后，考虑一个条件跳转三地址指令，例如 if  $x < y$  goto L

等效的机器代码类似于：

这里，M 是从标号为 L 的三地址指令所产生的机器指令序列中的第一个指令的标号。

对于任何三地址指令，我们希望可以省略这些指令中的某些指令，省略的原因是所需要的运算分量已经在寄存器中，也可能因为结果不需要存放回内存。

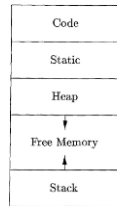
# Addresses in the Target Code

- How to generate code for procedure calls and returns?
  - **Static allocation** (静态分配)
  - **Stack allocation** (栈式分配)
- How to replace names in IR by code to access storage locations?

在第 7.1 节中，我们描述了每个正在执行的程序是如何在其逻辑地址空间上运行，该空间被划分为四个代码及数据区域：

1. 静态确定的区域代码，保存可执行目标代码。目标代码的大小可以在编译时确定。
2. 静态确定的数据区 `Static`，用于保存全局常量和编译器生成的其他数据。全局常量和编译器数据的大小也可以在编译时确定。
3. 动态管理区域堆，用于保存程序执行期间分配和释放的数据对象。堆的大小无法在编译时确定。
4. 动态管理区域栈，用于保存在过程调用和返回期间创建和销毁的活动记录。与堆一样，栈的大小无法在编译时确定。

# Static Allocation (静态分配)



- The size and layout of activation records are determined by the code generator via the information in the symbol table
  - Constant *staticArea* gives the address of the beginning of an activation record
- Target program code for the three-address code *call callee*

```
ST callee.staticArea, #here + 20
BR callee.codeArea
```

Store the literal *return address* (the address of the instruction after BR) at the beginning of the callee's activation record

Constant *codeArea* gives the address of the first instruction of the callee in the Code area of the run-time memory

为了说明简化过程调用和返回的代码生成，我们将重点关注以下三地址语句：

*call callee*

*return*

*halt*

action，这是代表其他三地址语句的占位符

- 活动记录的大小和布局由代码生成器通过符号表中存储的名字信息来确定。

我们将首先说明如何在过程调用时在一个活动记录中存储返回地址，以及在过程调用结束后如何把控制返回到这个地址。为了方便起见，我们假设活动记录中的第一个位置保存返回地址。

- 让我们首先考虑实现最简单情况（静态分配）所需的代码。这里，中间代码中的 *call callee* 语句可以通过两个目标机器指令的序列来实现：

ST 指令把返回地址保存在被调用者活动记录的开始处，BR 将控制传递到被调用过程的目标代码上。属性 *callee.staticArea* 是一个常量，它给出了 *callee* 的活动记录的开始地址，属性 *callee.codeArea* 也是一个常量，指向运行时内存中的 code 区中被调用过程 *callee* 的第一条指令的地址。

ST 指令中的运算分量  $\#here + 20$  是返回地址的文字表示；它是紧跟在 BR 指令之后的指令的地址。我们假设  $\#here$  是当前指令的地址，并且调用序列中的三个常量 ( $callee.staticArea$ ,  $callee.codeArea$ ,  $\#here + 20$ ) 加上两条指令 (ST, BR) 的长度为 5 个字，即 20 个字节。

## Static Allocation (静态分配)

- Code for the *return* statement in a *callee*

BR    *\*callee.staticArea*

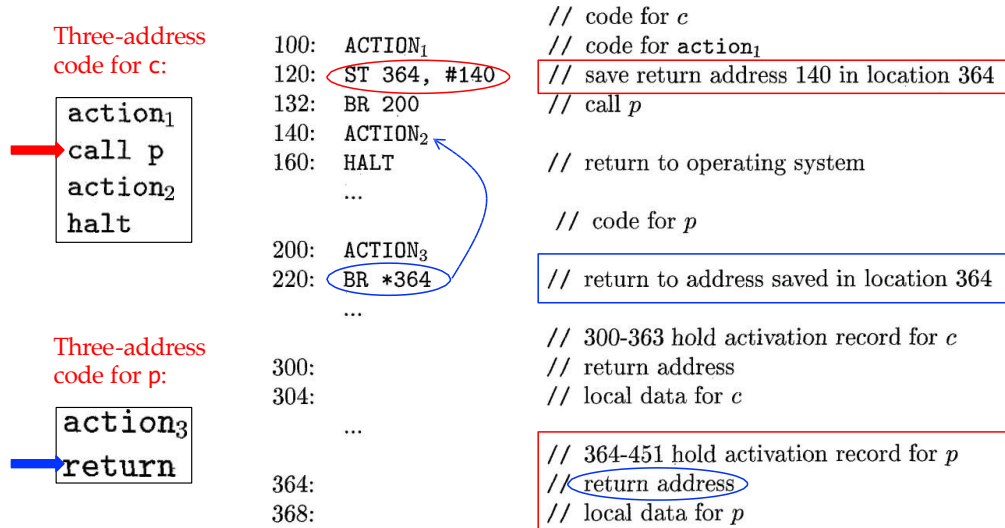


Transfer control to the address saved at the beginning of the activation record for *callee*

过程代码的结尾处是一个返回到调用过程的指令，但是没有调用者的第一个过程例外，它的最后一个指令是 HALT，它将控制权返回给操作系统。 `return` 语句可以通过简单的跳转指令 `BR *callee.staticArea` 来实现，该指令将控制转移到被调用者活动记录开头位置的地址上。

# Example

Note: Here the return address is stored at the beginning of the callee's activation record, which is different from Chapter 6. This is fine since the order among actual parameters, returned values, and saved machine status does not matter.



• 图 8.4 显示了该三地址代码的目标程序。我们使用伪指令 ACTION 来表示执行语句 action 的机器指令序列，它表示与本讨论无关的三地址代码。我们假定过程 *c* 的代码从地址 100 处开始，过程 *p* 的代码从地址 200 处开始。我们假设每个 ACTION 伪指令占用 20 个字节。我们进一步假设这些过程的活动记录以静态方式分配，其位置分别是 300 和 364。

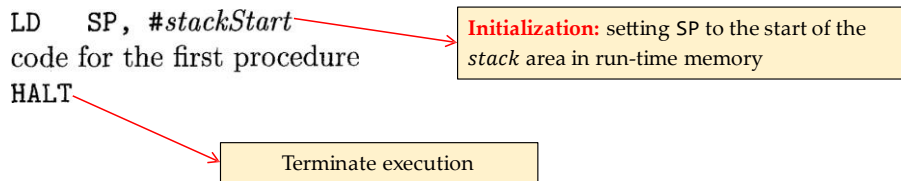
• 从地址 100 开始的指令实现了过程 *c* 的语句。

• 因此，程序的运行从地址 100 上的指令 ACTION<sub>1</sub> 开始。地址 120 处的 ST 指令将返回地址 140 保存在机器状态字段中，这是 *p* 的活动记录中的第一个字。地址 132 处的 BR 指令把控制转移到被调用过程 *p* 的目标代码中的第一条指令。

执行 ACTION<sub>3</sub> 后，位于地址 220 的跳转指令被执行。由于上述调用序列将位置 140 保存在地址 364 处，因此当执行地址 220 处的 BR 语句时，\*364 代表 140。因此，当过程 *p* 终止时，控制返回到地址 140，过程 *c* 继续执行。

# Stack Allocation (栈式分配)

- Static allocation uses **absolute addresses**
- Static allocation can become stack allocation by using **relative addresses** for storage in activation records
  - Maintain in **a register SP** a pointer to **the beginning of the activation record on top of the stack**
- The code for the first procedure (**main**)



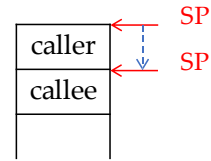
- 如果在保存活动记录时使用相对地址，静态分配就可以变栈分配。
- 然而，在栈分配中，只有等到运行时才能知道一个过程的活动记录的位置。该位置通常存储在寄存器中，因此活动记录中的字可以通过相对于寄存器中值的偏移量来访问。目标机器的下标地址模式可以方便地完成这种访问。

- 活动记录的相对地址可以用相对于活动记录中的任一已知位置的偏移量来表示，正如我们在第 7 章中看到的那样。那么，我们将在寄存器 SP 中维护一个指向栈顶活动记录开始处的指针，这样就可以使所有的偏移量都是正偏移量。当过程调用发生时，调用过程增加 SP 的值，并将控制权转移给被调用过程。当控制权返回给调用者后，我们减少 SP 的值，从而释放被调用过程的活动记录。

- 第一个过程的代码通过将 SP 设置为内存中栈区的开始位置，完成对栈的初始化：



# Stack Allocation (栈式分配)



- A procedure **calling sequence** Additional work comparing to static allocation

|                          |                                                               |                                         |
|--------------------------|---------------------------------------------------------------|-----------------------------------------|
| Each<br>takes 4<br>bytes | <u>ADD</u> <u>SP</u> , <u>SP</u> , # <i>caller.recordSize</i> | <code>// increment stack pointer</code> |
|                          | <u>ST</u> <u>*SP</u> , # <i>here</i> + 16                     | <code>// save return address *</code>   |
|                          | <u>BR</u> <i>callee.codeArea</i>                              | <code>// return to caller</code>        |

- The **return sequence**

|                                                    |                                                          |
|----------------------------------------------------|----------------------------------------------------------|
| <code>BR *0(SP)</code>                             | <code>// return to caller (done in callee)</code>        |
| <code>SUB SP, SP, #<i>caller.recordSize</i></code> | <code>// decrement stack pointer</code> (done in caller) |

Additional work comparing to static allocation

\* Return address is at the beginning of the activation record

- 一个过程调用序列递增 SP 的值，保存返回地址，并将控制权转移到被调用的过程：

运算分量#*caller.recordSize* 表示一条活动记录的大小，因此 ADD 指令使 SP 指向下一条活动记录。

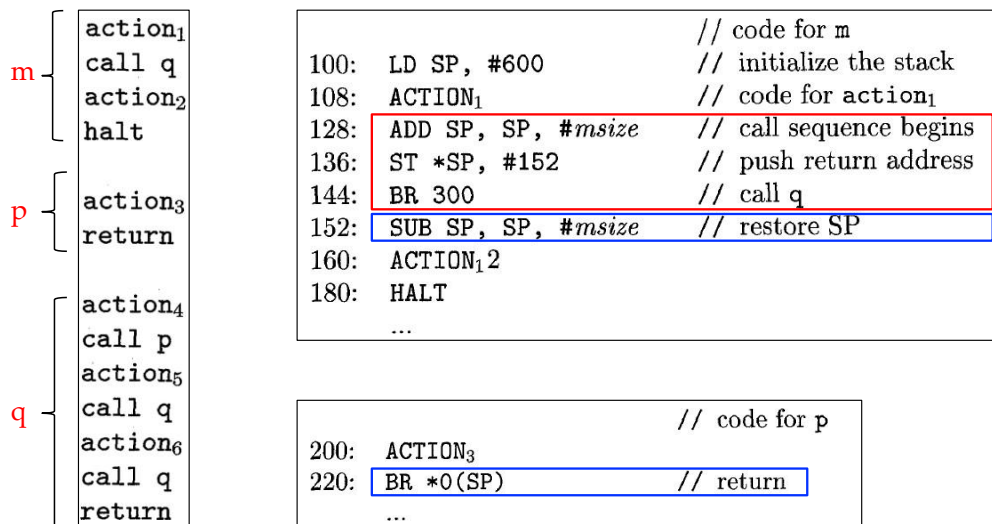
ST 指令中的运算分量#*here*+16 是跟随 BR 之后的指令的地址；它保存在 SP 指向的地址中。

- 返回序列由两部分组成。
  - 被调用的过程使用以下命令将控制权转移到返回地址：在 BR 指令中使用\*0(SP)的原因是我们需要两级间接寻址：0(SP)是活动记录中第一个字所在的位置，而\*0(SP)是保存在那里的返回地址。
  - 返回序列的第二部分位于调用者中，这个序列减少 SP 的值，从而将 SP 恢复为之前的值。也就是说，减法运算后 SP 指向调用者活动记录的开头

# Example

Call sequence

Return sequence



• 例 8.4: 图 8.5 中的程序是上一章中的快速排序程序的抽象。过程 q 是递归的，因此 q 的同一时刻可能有多个活跃的 q 的活动记录。

• 假设过程 m、p 和 q 的活动记录的大小已分别确定为 msize、psize 和 qsize。每个活动记录中的第一个字将保存一个返回地址。我们任意假设这些过程的代码分别从地址 100、200 和 300 开始，栈从地址 600 开始。目标程序如图 8.6 所示。

• 我们假设 ACTION<sub>4</sub> 包含了一个条件跳转指令，跳转到 q 的返回代码序列开始地址 456；否则，递归过程 q 注定会永远调用自己。

• 令 msize、psize 和 qsize 分别为 20、40 和 60。地址 100 处的第一条指令将 SP 初始化为 600，即栈的起始地址。在控制权从 m 转移到 q 的前一刻，SP 中的值是 620（因为 msize 为 20）。随后，当 q 调用 p 时，在地址 320 处的指令将 SP 递增到 680，p 的活动记录从这里开始；当控制返回到 q 的时候，SP 恢复到 620。如果接下来的两个对 q 的递归调用立即返回，则此执行期间 SP 的最大值为 680。但是请注意，栈区中最后被使用的位置是 739，因为从位置 680 开始的 q 的活动记录总共有 60 字节。？

# Example

|   |                     |   |                          |                                       |
|---|---------------------|---|--------------------------|---------------------------------------|
| m | action <sub>1</sub> | p | 300: ACTION <sub>4</sub> | // code for q                         |
|   | call q              |   | 320: ADD SP, SP, #qsize  | // contains a conditional jump to 456 |
|   | action <sub>2</sub> |   | 328: ST *SP, #344        | // push return address                |
|   | halt                |   | 336: BR 200              | // call p                             |
| p | action <sub>3</sub> | q | 344: SUB SP, SP, #qsize  |                                       |
|   | return              |   | 352: ACTION <sub>5</sub> |                                       |
|   |                     |   | 372: ADD SP, SP, #qsize  |                                       |
|   |                     |   | 380: BR *SP, #396        | // push return address                |
| q | action <sub>4</sub> |   | 388: BR 300              | // call q                             |
|   | call p              |   | 396: SUB SP, SP, #qsize  |                                       |
|   | action <sub>5</sub> |   | 404: ACTION <sub>6</sub> |                                       |
|   | call q              |   | 424: ADD SP, SP, #qsize  |                                       |
|   | action <sub>6</sub> |   | 432: ST *SP, #440        | // push return address                |
|   | call q              |   | 440: BR 300              | // call q                             |
|   | return              |   | 448: SUB SP, SP, #qsize  |                                       |
|   |                     |   | 456: BR *0(SP)           | // return                             |
|   |                     |   | ...                      |                                       |
|   |                     |   | 600: ...                 | // stack starts here                  |

## Addresses in the Target Code

- How to generate code for procedure calls and returns?
  - Static allocation (静态分配)
  - Stack allocation (栈式分配)
- How to replace names in IR by code to access storage locations?

# Run-Time Addresses for Names

- A **name** in a three-address statement corresponds to a symbol-table entry
- Statement  $x = 0$ 
  - Suppose the symbol-table entry for  $x$  contains a relative address 12
  - If  $x$  is in a **statically allocated area** (i.e., **static**):
    - The effect of  $x = 0$ : `static[12] = 0`
    - Target code: `LD 112, #0` (suppose static area starts at address 100)
  - If  $x$  is in **stack** (in an activation record):
    - `LD 12(SP), #0`

• 存储分配策略和过程的活动记录中局部数据的布局决定了如何访问名字对应的内存位置。在第六章中，我们假设三地址语句中的名字实际上是指向该名字的符号表条目的指针。这种方法有一个显着的优点：它使编译器更加易于移植，因为即使将编译器移植到使用不同运行时组织的其他机器上，也不需要更改前端。另一方面，在生成中间代码的同时生成特定的访问步骤序列对于一个优化编译器也有极大的好处，因为它可以让优化器利用原本在简单的三地址语句中看不到的细节。

• 无论哪种情况，名字最终都必须被替换为访问存储位置的代码。因此，我们考虑简单三地址复制语句  $x = 0$  的一些细节。

- 在处理过程中的声明之后，假设  $x$  的符号表条目包含  $x$  的相对地址 12。

- 例如，考虑  $x$  位于从地址 `static` 开始的静态分配区域中的情况。那么  $x$  的实际运行时地址是 `static + 12`。虽然编译器最终可以在编译时确定 `static + 12` 的值，但是在生成访问该名字的中间代码时，可能还不知道静态区域的位置。在这种情况下，生成“计算” `static + 12` 的三地址代码是有意义的。当然我们能要理解，该计算在程序运行之前就会完成：它或者在代

码生成阶段完成，或者由加载器完成。那么，赋值语句  $x = 0$  被翻译成 `static[12] = 0`；如果静态区域从地址 100 开始，则该语句的目标代码是 `LD 112, #0`

- 如果  $x$  位于栈中（活动记录中），目标代码是 `LD 12(SP), #0`

## Basic Block and Flow Graph

- Graph representation of intermediate code
  - Partition the intermediate code into **basic blocks**
    - The flow of control can only enter the basic block through its first instruction
    - Control will not leave the block, except possibly at the last instruction in the block (**no halting/branching in the middle**)
  - **Flow graphs**: basic blocks are **nodes** and the **edges** indicate which block can follow which other blocks
- Flow graphs form **the basis of code optimization**
  - They describe how control flows among basic blocks
  - We can know how values are defined and used

本节介绍中间代码的图形表示，即使图形不是由代码生成算法显式构造的，它也有助于讨论代码生成。代码生成受益于上下文。如果我们知道如何定义和使用值，我们就可以更好地进行寄存器分配，正如我们将在第 8.8 节中看到的那样。我们可以通过查看三地址语句序列来更好地进行指令选择，正如我们将在第 8.9 节中看到的那样。该表示的构造如下：

1. 将中间代码划分为基本块，这些基本块是连续三地址指令的最大序列，具有以下属性：
  - (a) 控制流只能通过基本块中的第一个指令进入该块。也就是说，没有跳转到块中间的转移指令。
  - (b) 除了基本块的最后一条指令，控制流在离开基本块之前不会停机或者跳转。
2. 基本块成为流图的节点，流图的边指示哪些块可以跟随哪些其他块。

从第 9 章开始，我们讨论流图上的转换，将原始中间代码转换为“优化后”的中间代码，而从中可以生成更好的目标代码。使用“优化后”的中间代码将其转换为机器代码将使用本章中的代码生成技术。

流程图是代码优化的基础，它们描述了控制如何在基本模块之间流动，我们可以知道值是如何定义和使用的。

## Partitioning Three-Address Instructions into Basic Blocks

- **Input:** A sequence of three-address instructions
- **Output:** A list of basic blocks (each inst. is assigned to one block)
- **Method:**
  - Rules for finding *leader instructions* (首指令, the 1<sup>st</sup> instruction of a basic block)
    1. The *first instruction* in the entire intermediate code is a leader
    2. Any instruction that is the *target of a conditional/unconditional jump* is a leader
    3. Any instruction that *immediately follows a conditional/unconditional jump* is a leader
  - Then, for each leader, its basic block consists of *itself and all instructions up to but not including the next leader* or the end of the intermediate program

我们的第一项工作是将一系列三地址指令划分为基本块。我们从第一条指令开始一个新的基本块，并不断添加指令，直到我们遇到下一条指令的跳转、条件跳转或下一个指令前的标号为止。当没有跳转和标号的情况下，控制按顺序从一条指令进行到下一条指令。这个想法在以下算法中形式化地表示出来。

算法 8.5：将三地址指令划分为基本块。

输入：三地址指令序列。

输出：该序列的基本块列表，其中每条指令恰好被分配给一个基本块。

方法：首先，我们确定中间代码中的哪些指令是首指令，即某个基本块中的第一个指令。跟在中间程序末尾之后的指令不包含在首指令集合中。选择首指令的规则如下：

1. 中间代码中的第一条三地址指令是首指令。
2. 作为条件或无条件跳转指令的目标指令是一个首指令。
3. 紧跟在一个条件或无条件跳转之后的指令是一个首指令。

然后，每个首指令对应的基本块包括从它自己开始，直到下一个首指令（不含）或者中间程序的结尾指令之间的所有指令。

## Basic Block Example

- **Leader instructions:**

- The first instruction (rule #1): 1
- Targets of jumps (rule #2): 3, 2, 13
- Instructions immediately following jumps (rule #3): 10, 12

- **Basic blocks**

- 1-1            2-2
- 3-9            10-11
- 12-12        13-17

```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

```

```

for i from 1 to 10 do
 for j from 1 to 10 do
 a[i, j] = 0.0;
 for i from 1 to 10 do
 a[i, i] = 1.0;

```

图 8.7 中的中间代码将 10X10 矩阵 a 转换为单位矩阵。尽管该代码来自何处并不重要，但它可能是图 8.8 中伪代码的翻译。在生成中间代码时，我们假设每个实数值的数组元素占用 8 个字节，并且矩阵 a 以行优先形式存储。

首先，根据算法 8.5 的规则 (1)，指令 1 是一个首指令。为了找到其他的首指令，我们首先要找到跳转指令。在这个例子中，指令 9、11 和 17 处有 3 个跳转，都是有条件跳转指令。根据规则 (2)，这些跳转指令的目标是首指令；它们分别是指令 3、2 和 13。然后，根据规则 (3)，跟在一个跳转指令后的每条指令都是首指令；它们是指令 10 和 12。请注意，在此代码中，17 后面没有指令，但如果后面有代码，则第 18 条指令也将是首指令。

我们得出结论，指令 1、2、3、10、12 和 13 是首指令。每个首指令对应的基本块包含了从其自身到下一个首指令之前的所有指令。因此，指令 1 的基本块就是指令 1，指令 2 的基本块是指令 2。然而，首指令 3 的基本块包含了从指令 3 到 9 的所有指令。指令 10 的基本块是 10 和 11；指令 12 的基本块只包含指令 12，而指令 13 的基本块是指令 13 到 17。

## Next-Use Information (后续使用信息)

- The *use* of a name in a three-address statement
  - Suppose a statement  $i$  assigns a value to  $x$ . If statement  $j$  has  $x$  as an operand, and control can flow from  $i$  to  $j$  along a path that has no intervening assignments to  $x$ , then we say  $j$  *uses the value of  $x$  computed at  $i$*
  - We further say that  $x$  is *live* (活跃) at statement  $i$ 
    - *Not live*: the *value* computed will not be used, not the variable will not be used
- **Code generation can benefit from such information**
  - If the value of a variable that is currently in a register will never be referenced subsequently, we can assign the register to another variable

在一个三地址语句中对一个名字的使用定义如下。

- 假设三地址语句  $i$  给  $x$  赋了一个值。如果语句  $j$  将  $x$  作为运算分量，并且控制可以沿着没有中间对  $x$  赋值的路径从语句  $i$  流向  $j$ ，则我们说语句  $j$  使用在语句  $i$  计算的  $x$  值。
- 我们进一步说  $x$  在语句  $i$  处处于活动状态。

知道一个变量的值接下来会在何时使用对于生成良好的代码至关重要。如果当前位于寄存器中的变量的值随后永远不会被引用，则可以将该寄存器分配给另一个变量。



# Determine Liveness and Next-Use Information for Each Statement

- **Input:** A basic block  $B$ . Assume that the symbol table initially shows all nontemporary variables in  $B$  as being **live on exit**
- **Output:** At each statement  $i: x = y \text{ op } z$ , we attach to  $i$  the **liveness and next-use information** of  $x$ ,  $y$ , and  $z$ . Here, **we do not consider uses outside the basic block**
- **Method:**
  - **Scan backwards from the last statement in  $B$ .** At each statement  $i: x = y \text{ op } z$ , do the following (the order matters as  $x$  maybe  $y$  or  $z$ ):
    1. Attach to  $i$  the information currently in the symbol table regarding the next use and liveness of  $x$ ,  $y$ , and  $z$
    2. In the symbol table, set  $x$  to “not live” and “no next use”
    3. In the symbol table, set  $y$  and  $z$  to “live” and their next uses to  $i$

对于每个类似于  $x = y + z$  的三地址语句，我们希望确定对  $x$ 、 $y$  和  $z$  的下一使用是什么。我们确定活跃性和后续使用信息的算法对每个基本块进行一次反向遍历。我们将得到的信息存储在符号表中。（在扫描过程中通过为每个名字建立相应的下次引用信息链何额活跃变量信息链，收集各名字在基本块内的下次引用信息和活跃信息）。活跃信息用于代码优化时全局数据流分析。）如算法 8.5 所示，我们可以轻松地扫描三地址语句流来找到各个基本块的结尾。由于过程可以有任意的副作用（比如修改某个外部变量的值），为了方便起见，我们假设每个过程调用指令是一个新的基本块的入口。

对于一个基本块中的每一个语句确定活跃性与后续使用信息。

输入：一个三地址语句的基本块  $B$ 。我们假设最初符号表显示  $B$  中的所有非临时变量都是活跃的。（如果产生中间代码的算法允许某些临时变量在基本块之外引用，也假定这些临时变量在出口处是活跃的。）

输出：对于  $B$  中的每个语句  $i: x = y + z$  中，我们将  $x$ 、 $y$  和  $z$  的活跃信息以及后续使用信息关联到  $i$ 。这里，我们不关心考虑包含本三地址语句的基本块之外的使用。

为了计算名字的下次引用信息，需要对符号表进行扩充，增加记录下次引用信息和活跃信息的域，同时对保存三地址语句的结构进行扩充，使之可以记录语句中出现的每个名字的下次引用信息和活跃信息

方法：我们从 B 中的最后一个语句开始，反向扫描到 B 的开始处。对于 B 中的每个语句  $i: x = y + z$ ，我们执行以下操作：

1. 把在当前符号表中找到的有关  $x$ 、 $y$  和  $z$  的当前后续使用和活跃信息与语句  $i$  关联起来。

2. 在符号表中，将  $x$  的活跃信息和下次引用信息分别设置为“不活跃”和“无后续使用”。（因为  $x$  被重新计算了，之前的值被更改）

3. 在符号表中，将  $y$  和  $z$  设置为“活跃”，并把它们的下一次使用设置为语句  $i$ 。

这里，我们使用  $+$  作为代表任意运算符的符号。如果三地址语句  $i$  形如  $x = +y$  或  $x = y$ ，那么处理步骤依然和上面相同，只是忽略了对  $z$  的处理。请注意，步骤 (2) 和 (3) 的顺序不能颠倒，因为  $x$  可能是  $y$  或  $z$ 。

通过应用此算法，基本块中所有名字的下次引用信息都被收集到，并且保存于符号表中和三地址语句上。如果一个变量在基本块中被引用，则由该变量在符号表中的下次引用信息，以及附加在各三地址语句  $i$  上的信息，可以从前到后依次指示出该名字在基本块中的各个引用所在的位置。

# Running Example

```
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
```

- $i, j, a$  are not temporary names generated by compilers. Initially, only they are live on exit

| Name | Liveness | Next-use |
|------|----------|----------|
| $i$  | Live     |          |
| $j$  | Live     |          |
| $a$  | Live     |          |
| $t1$ | Not live |          |
| $t2$ | Not live |          |
| $t3$ | Not live |          |
| $t4$ | Not live |          |

为了计算名字的下次引用信息，需要对符号表进行扩充，增加记录下次引用信息和活跃信息的域。

我们假设最初符号表显示 B 中的所有非临时变量都是活跃的。

# Running Example

```

3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
→ 8) j = j + 1

```

- **Analyzing line 8):** attach the following information to 8)

| Name     | Liveness | Next-use |
|----------|----------|----------|
| <i>j</i> | Live*    |          |

\* **Live at statement 8 (block exit):** after executing statement 8, the value of *j* will be used by subsequent statements

1. 把在当前符号表中找到的有关 *x*、*y* 和 *z* 的当前后续使用和活跃信息与语句 8 关联起来。(j)

# Running Example

```

3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
→ 8) j = j + 1

```

- **Analyzing line 8):** update symbol table\*

| Name      | Liveness | Next-use |
|-----------|----------|----------|
| <i>i</i>  | Live     |          |
| <i>j</i>  | Live     | 8        |
| <i>a</i>  | Live     |          |
| <i>t1</i> | Not live |          |
| <i>t2</i> | Not live |          |
| <i>t3</i> | Not live |          |
| <i>t4</i> | Not live |          |

\* The updated information will be passed and attached to the previous statements

2. 在符号表中，将  $j$  的活跃信息和下次引用信息分别设置为“不活跃”和“无后续使用”。（因为  $x$  被重新计算了，之前的值被更改）
3. 在符号表中，将  $j$  设置为“活跃”，并把它们的下一次使用设置为语句 8。

## Running Example

```
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
→ 7) a[t4] = 0.0
8) j = j + 1
```

- **Analyzing line 7):** attach the following information to 7)

| Name | Liveness | Next-use |
|------|----------|----------|
| $a$  | Live     |          |
| $t4$ | Not live |          |

- Fall 2024 BUPT Compilers 32
1. 把在当前符号表中找到的有关  $x$ 、 $y$  和  $z$  的当前后续使用和活跃信息与语句 7 关联起来。

# Running Example

```
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
→ 7) a[t4] = 0.0
8) j = j + 1
```

- **Analyzing line 7):** update symbol table

| Name      | Liveness | Next-use |
|-----------|----------|----------|
| <i>i</i>  | Live     |          |
| <i>j</i>  | Live     | 8        |
| <i>a</i>  | Live     | 7        |
| <i>t1</i> | Not live |          |
| <i>t2</i> | Not live |          |
| <i>t3</i> | Not live |          |
| <i>t4</i> | Live     | 7        |

2. 在符号表中，将 *a* 和 *t* 的活跃信息和下次引用信息分别设置为“不活跃”和“无后续使用”。（因为? 被重新计算了，之前的值被更改）
3. 在符号表中，将 *a* 和 *t4* 设置为“活跃”，并把它们的下一次使用设置为语句 7。

# Running Example

```

3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
→ 6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1

```

- **Analyzing line 6):** attach the following information to 6)

| Name      | Liveness | Next-use |
|-----------|----------|----------|
| <i>t3</i> | Not live |          |
| <i>t4</i> | Live     | 7        |

1. 把在当前符号表中找到的有关 *t4*、*t3* 的当前后续使用和活跃信息与语句 6 关联起来。

# Running Example

```

3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
→ 6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1

```

- **Analyzing line 6):** update symbol table

| Name      | Liveness | Next-use |
|-----------|----------|----------|
| <i>i</i>  | Live     |          |
| <i>j</i>  | Live     | 8        |
| <i>a</i>  | Live     | 7        |
| <i>t1</i> | Not live |          |
| <i>t2</i> | Not live |          |
| <i>t3</i> | Live     | 6        |
| <i>t4</i> | Not live |          |

2. 在符号表中，将  $t_4$  的活跃信息和下次引用信息分别设置为“不活跃”和“无后续使用”。（因为? 被重新计算了，之前的值被更改）
3. 在符号表中，将  $t_3$  设置为“活跃”，并把它们的下一次使用设置为语句 6。

## Running Example

```
3) t1 = 10 * i
4) t2 = t1 + j
→ 5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
```

- **Analyzing line 5):** attach the following information to 5)

| Name  | Liveness | Next-use |
|-------|----------|----------|
| $t_2$ | Not live |          |
| $t_3$ | Live     | 6        |

1. 把在当前符号表中找到的有关  $t_3$ 、 $t_2$  的当前后续使用和活跃信息与语句 5 关联起来。



# Running Example

```
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
```

- **Analyzing line 5):** update symbol table

| Name      | Liveness | Next-use |
|-----------|----------|----------|
| <i>i</i>  | Live     |          |
| <i>j</i>  | Live     | 8        |
| <i>a</i>  | Live     | 7        |
| <i>t1</i> | Not live |          |
| <i>t2</i> | Live     | 5        |
| <i>t3</i> | Not live |          |
| <i>t4</i> | Not live |          |

2. 在符号表中，将 *t3* 的活跃信息和下次引用信息分别设置为“不活跃”和“无后续使用”。（因为 *t3* 被重新计算了，之前的值被更改）
3. 在符号表中，将 *t2* 设置为“活跃”，并把它们的下一次使用设置为语句 5。

# Running Example

```
3) t1 = 10 * i
→ 4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
```

- **Analyzing line 4):** attach the following information to 4)

| Name      | Liveness | Next-use |
|-----------|----------|----------|
| <i>j</i>  | Live     | 8        |
| <i>t1</i> | Not live |          |
| <i>t2</i> | Live     | 5        |

1. 把在当前符号表中找到的有关 *t2*、*t1* 和 *j* 的当前后续使用和活跃信息与语句 4 关联起来。

# Running Example

```
3) t1 = 10 * i
→ 4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
```

- **Analyzing line 4):** update symbol table

| Name      | Liveness | Next-use |
|-----------|----------|----------|
| <i>i</i>  | Live     |          |
| <i>j</i>  | Live     | 4        |
| <i>a</i>  | Live     | 7        |
| <i>t1</i> | Live     | 4        |
| <i>t2</i> | Not live |          |
| <i>t3</i> | Not live |          |
| <i>t4</i> | Not live |          |

2. 在符号表中，将  $t_2$  的活跃信息和下次引用信息分别设置为“不活跃”和“无后续使用”。（因为? 被重新计算了，之前的值被更改）
3. 在符号表中，将  $t_1$  和  $j$  设置为“活跃”，并把它们的下一次使用设置为语句 4。

## Running Example

→ 3)  $t_1 = 10 * i$   
4)  $t_2 = t_1 + j$   
5)  $t_3 = 8 * t_2$   
6)  $t_4 = t_3 - 88$   
7)  $a[t_4] = 0.0$   
8)  $j = j + 1$

- **Analyzing line 3):** attach the following information to 3)

| Name  | Liveness | Next-use |
|-------|----------|----------|
| $i$   | Live     |          |
| $t_1$ | Live     | 4        |

1. 把在当前符号表中找到的有关  $t_1$ 、 $i$  和  $j$  的当前后续使用和活跃信息与语句 3 关联起来。

# Running Example

→ 3)  $t1 = 10 * i$   
4)  $t2 = t1 + j$   
5)  $t3 = 8 * t2$   
6)  $t4 = t3 - 88$   
7)  $a[t4] = 0.0$   
8)  $j = j + 1$

- **Analyzing line 3):** update symbol table

| Name      | Liveness | Next-use |
|-----------|----------|----------|
| <i>i</i>  | Live     | 3        |
| <i>j</i>  | Live     | 4        |
| <i>a</i>  | Live     | 7        |
| <i>t1</i> | Not live |          |
| <i>t2</i> | Not live |          |
| <i>t3</i> | Not live |          |
| <i>t4</i> | Not live |          |

2. 在符号表中，将 *t1* 的活跃信息和下次引用信息分别设置为“不活跃”和“无后续使用”。（因为 *t1* 被重新计算了，之前的值被更改）
3. 在符号表中，将 *i* 设置为“活跃”，并把它们的下一次使用设置为语句 3。

# Flow Graphs

- The nodes of the flow graphs are the basic blocks
  - There is an edge from block  $B$  to block  $C$  **iff** it is possible for the first instruction in  $C$  to immediately follow the last instruction in  $B$
- Possible reasons for the introduction of edges ( $B \rightarrow C$ )
  - **Case 1:** There is a (un)conditional jump from the end of  $B$  to the beginning of  $C$
  - **Case 2:**  $C$  immediately follows  $B$  in the original three-address code, and  $B$  does not end in an unconditional jump
  - We say that  $B$  is the *predecessor* (前驱) of  $C$  and  $C$  is the *successor* (后继) of  $B$
- Two special nodes: *entry* (入口结点) and *exit* (出口结点)
  - They do not correspond to executable intermediate instructions
  - There is an edge from entry node to the first executable node of the flow graph; there is an edge to the exit from any basic block that could be executed last

一旦中间代码程序被划分为基本块，我们就可以通过流图来表示它们之间的控制流。流图的节点是基本块。当且仅当块  $C$  中的第一条指令可能紧跟在块  $B$  中的最后一条指令之后时，才存在从块  $B$  到块  $C$  的边沿。存在这样的一条边的原因有两种：

Case1:有一个从  $B$  的末尾到  $C$  的开头的有条件或无条件跳转。

Case2:按照原来的三地址指令序列中的顺序， $C$  紧跟在  $B$  之后，且  $B$  的结尾不存在无条件跳转语句。

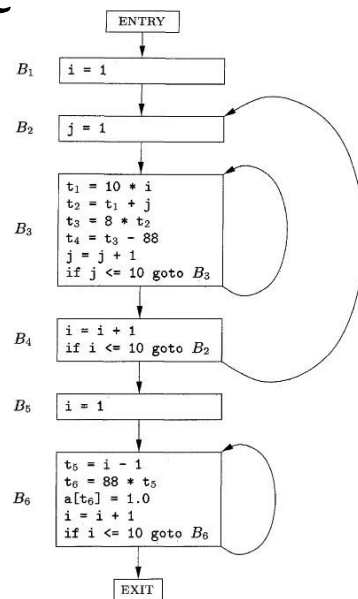
- 我们说  $B$  是  $C$  的前驱，而  $C$  是  $B$  的后继。

通常会添加两个分别称为入口和出口的结点

- 它们不和任何可执行的中间指令对应。
- 从入口到流图的第一个可执行节点（即包含了中间代码的第一条指令的基本块）有一条边。从任何包含了可能是程序的最后执行指令的基本块到出口有一条边。如果程序的最后指令不是一个无条件跳转，则包含了程序最后指令的块是出口结点的一个前驱。但任何包含了跳转到程序之外的跳转指令的基本块也是出口结点的前驱。

# Flow Graph Example

- Edges introduced because of jumps (Case 1)
  - $B_3 \rightarrow B_3$
  - $B_4 \rightarrow B_2$
  - $B_6 \rightarrow B_6$
- The other edges belong to “Case 2”



例 8.8: 例 8.6 (10X10 矩阵转成单位阵) 中构造的一组基本块可以生成图 8.9 的流图。入口点指向基本块 B1, 因为 B1 包含程序的第一条指令。B1 的唯一后继者是 B2, 因为 B1 的结尾不是一个无条件跳指令, 且 B2 的首指令紧跟在 B1 的结尾指令之后。

基本块 B3 有两个后继。其中的一个是它本身, 因为 B3 的首指令 (指令 3) 是 B3 结尾处的条件跳转指令 (指令 9) 的目标。另一个后继是 B4, 因为控制可以穿越 B3 结尾处的条件跳转指令而到达 B4 的首指令。

只有 B6 指向流图的出口结点, 因为到达紧跟在流图对应的程序之后的代码的唯一方法是穿越 B6 结尾处的条件跳转指令。

首先, 从图 8.9 中注意到, 在流程图中, 把到达指令的序号或标号的跳转指令替换为到达基本块的跳转, 这么做是很正常的。回想一下, 每个条件或无条件跳转都是到某个基本块的首指令, 而现在这些跳转指令指向了相应的基本块。这么做的原因是, 在构建了流图之后, 经常会对多个基本块中的指令进行实质性更改。如果跳转的目标是指令, 我们将不得不在每次改变了某个目标指令之后修正跳转指令的目标。

流图就是通常的图, 它可以用任何表示图的数据结构来表示。结点 (基本块) 的内容需要有它们自己的表示方式。我们可能会用一个指向该基本块在三地址指令组中的首指令

的指针，以及基本块的指令数量或一个指向结尾指令的指针来表示结点的内容。然而，由于我们可能经常更改基本块中的指令数量，所以为每个基本块创建一个指令链接列表可能会更有效。

## Loops (循环)

- Programs often spend most of the time in executing loops. It is important for compilers to generate efficient code for loops.
- **Definition of loops:**
  - A loop  $L$  is a set of nodes in the flow graph
  - $L$  contains a node  $e$  called the *loop entry* (循环入口)
  - No node in  $L$  except  $e$  has a predecessor outside  $L$ . That is, every path from the entry of the entire flow graph to any node in  $L$  goes through  $e$ .\*
  - Every node in  $L$  has a nonempty path, completely within  $L$ , to  $e$

\* We say  $e$  dominates the other nodes in  $L$

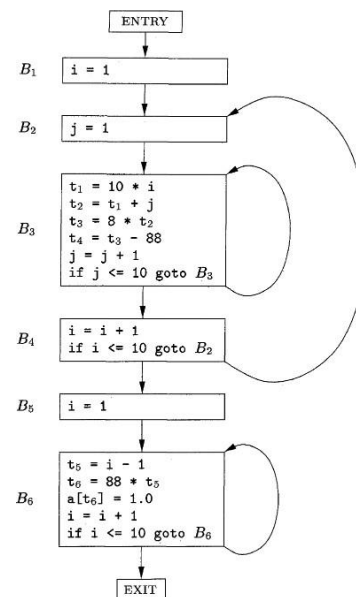
像 while 语句、do-while 语句和 for 语句这样的程序语言结构自然地把循环引入到程序中。由于实际上每个程序的大部分时间都在执行其循环操作，因此编译器为循环操作生成良好的代码尤为重要。许多代码转换最终取决于流图中“循环”的识别。如果下列条件成立，我们就是流图中的一个结点集合  $L$  是一个循环，

1) 在  $L$  中有一个被称为循环入口的结点，它是唯一的其前驱可能在  $L$  之外的结点。也就是说，从整个流图的入口结点开始到  $L$  中的任何结点的路径都必然经过循环入口结点，并且这循环入口结点不是整个流图的入口结点本身。

2)  $L$  中的每个结点都有一个到达  $L$  的入口结点的非空路径，并且该路径全部都在  $L$  中。

# Loop Examples

- $\{B_3\}$
- $\{B_2, B_3, B_4\}$ 
  - $B_2$  is the loop entry
  - $B_1, B_5, B_6$  are not in the loop
    - There is a path from ENTRY to  $B_1$  that does not go through  $B_2$
    - $B_5$  and  $B_6$  have no nonempty paths to  $B_2$
- $\{B_6\}$



示例 8.9：图 8.9 的流图具有三个循环：

1.  $B_3$  本身。
2.  $\{B_2; B_3; B_4\}$ 。

$L = \{B_2; B_3; B_4\}$  的循环入口结点是  $B_2$ 。请注意，这三个节点中，只有  $B_2$  有一个不在  $L$  中的前驱  $B_1$ 。此外，这三个结点中都有在  $L$  中且到达  $B_2$  的非空路径。比如，从  $B_2$  开始就有路径  $B_2 \rightarrow B_3 \rightarrow B_4 \rightarrow B_2$ 。

3.  $B_6$  本身。

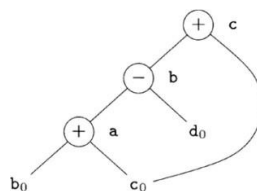
$B_3$  和  $B_6$  是单个节点，这些结点都有到其自身的边。例如， $B_3$  形成一个以  $B_3$  本身为入口的循环。请注意，循环的最后一个要求是从  $B_3$  到其自身有一条非空路径。因此，像  $B_2$  这样的单个节点（它没有一条  $B_2 \rightarrow B_2$  的边）不是循环，因为没有从  $B_2$  到其自身，且在集合  $\{B_2\}$  中的非空路径。



# The DAG Representation of Basic Blocks

- The **DAG (directed acyclic graph)** of a basic block depicts the relationships among the values of all variables in a basic block when it executes (**data dependence**)
- **DAG enables several code-improving transformations:**
  - Eliminate local common subexpressions (**局部公共子表达式**)
  - Eliminate dead code (**死代码**)
  - Apply algebraic laws (**代数恒等式**) to reorder operands of instructions

$a = b + c$   
 $b = a - d$   
 $c = b + c$



Fall 2024

BUPT Compilers

47

许多重要的局部优化技术都是从将基本块转换为 DAG（有向无环图）开始的。在第 6.1.1 节中，我们介绍了 用于表示简单表达式的 DAG。这一想法自然地延伸到在一个基本块内创建的表达式的集合。

基本块的 DAG 表示允许我们对块表示的代码执行一些转换，以改进代码的质量。

- 我们可以消除局部公共子表达式，所谓公共子表达式就是重复计算一个已经计算过的值的指令。
- 我们可以消除死代码，即计算得到的值不会被使用的指令。
- 我们可以对相互独立的语句重新排序；这种重新排序可以减少临时值需要保存在寄存器中的时间。
- 我们可以应用代数定律对三地址指令的运算分量重新排序，这么做有时可以简化计算过程。

# Constructing DAG's

$a = b + c$   
 $b = a - d$   
 $c = b + c$

$a_0$     $b_0$     $c_0$     $d_0$

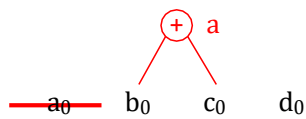
**Step 1:** Create a node for each of the initial values of the variables in the basic block

我们为基本块构造一个 DAG，如下所示：

1. 对于基本块中出现的每个变量有一个对应的 DAG 的结点表示其初始值。

# Constructing DAG's

$a = b + c$   
 $b = a - d$   
 $c = b + c$



The variable  $a$  get a new value.  
Old value  $a_0$  is never used (killed)

## Step 2: Process $a = b + c$

- Create a node  $N$  for the statement
- Label it with  $+$
- Attach  $a$  to  $N$
- The children of  $N$  are those nodes corresponding to the **last definitions** of  $b$  and  $c$

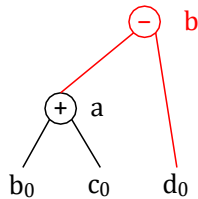
2. 基本块内的每个语句  $s$  都有一个相关联的结点  $N$ 。  $N$  的子结点是基本块中的其他语句的对应结点。这些语句是在  $s$  之前、最后一个对所使用的某个运算分量进行定值的语句。

3. 结点  $N$  的标号是  $s$  中的运算符；同时还有一组变量被关联到  $N$ ，表示  $s$  是在此基本块内最组晚对这些变量进行定值的语句。

4. 某些结点被指明为输出结点。这些结点的变量在基本块的出口处活跃。也就是说，这些变量的值可以稍后在流图的另一个块中使用。这些“活跃变量”的计算是全局数据流分析的问题，在第 9.2.5 节中讨论。

# Constructing DAG's

$a = b + c$   
 $b = a - d$   
 $c = b + c$



## Step 3: Process $b = a - d$

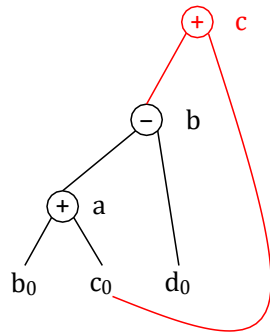
- Create a node  $N$  for the statement
- Label it with  $-$
- Attach  $b$  to  $N$
- The children of  $N$  are those nodes corresponding to the **last definitions** of  $a$  and  $d$

我们为基本块构造一个 DAG，如下所示：

1. 对于基本块中出现的每个变量有一个对应的 DAG 的结点表示其初始值。
2. 基本块内的每个语句  $s$  都有一个相关联的结点  $N$ 。  $N$  的子结点是基本块中的其他语句的对应结点。这些语句是在  $s$  之前、最后一个对所使用的某个运算分量进行定值的语句。
3. 结点  $N$  的标号是  $s$  中的运算符；同时还有一组变量被关联到  $N$ ，表示  $s$  是在此基本块内最组晚对这些变量进行定值的语句。
4. 某些结点被指明为输出结点。这些结点的变量在基本块的出口处活跃。也就是说，这些变量的值可以稍后在流图的另一个块中使用。这些“活跃变量”的计算是全局数据流分析的问题，在第 9.2.5 节中讨论。

# Constructing DAG's

$a = b + c$   
 $b = a - d$   
 $c = b + c$



## Step 4: Process $c = b + c$

- Create a node  $N$  for the statement
- Label it with  $+$
- Attach  $c$  to  $N$
- The children of  $N$  are those nodes corresponding to the **last definitions** of  $b$  and  $c$

2. 基本块内的每个语句  $s$  都有一个相关联的结点  $N$ 。  $N$  的子结点是基本块中的其他语句的对应结点。这些语句是在  $s$  之前、最后一个对所使用的某个运算分量进行定值的语句。

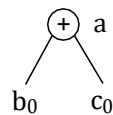
3. 结点  $N$  的标号是  $s$  中的运算符；同时还有一组变量被关联到  $N$ ，表示  $s$  是在此基本块内最组晚对这些变量进行定值的语句。

4. 某些结点被指明为输出结点。这些结点的变量在基本块的出口处活跃。也就是说，这些变量的值可以稍后在流图的另一个块中使用。这些“活跃变量”的计算是全局数据流分析的问题，在第 9.2.5 节中讨论。

# Finding Local Common Subexpression

- When creating a node  $M$ , check if there exists a node  $N$ , which has the same operator and children nodes (order matters) with  $M$
- If such a node  $N$  exists, we do not create the node  $M$  but simply use  $N$  to represent  $M$

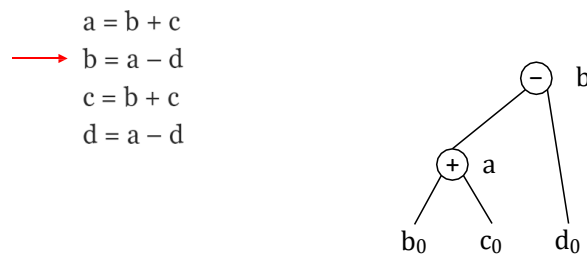
→  $a = b + c$   
 $b = a - d$   
 $c = b + c$   
 $d = a - d$



检测公共子表达式的方法是这样的，当一个新的节点  $M$  被加入到 DAG 时，检测是否存在一个结点  $N$ ，它和  $M$  具有相同运算符和子节点，且子节点的顺序相同。如果是这样， $N$  计算的值与  $M$  计算的的值是一样的，因此，可以用  $N$  替换  $M$ 。该技术在第 6.1.1 节中作为检测公共子表达式的“值编码”方法引入。

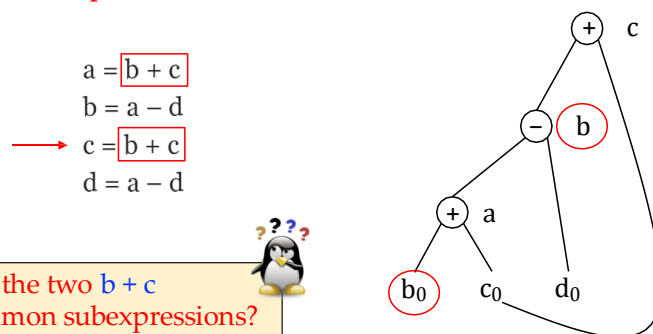
## Finding Local Common Subexpression

- When creating a node  $M$ , check if there exists a node  $N$ , which has the same operator and children nodes (order matters) with  $M$
- If such a node  $N$  exists, we do not create the node  $M$  but simply use  $N$  to represent  $M$



## Finding Local Common Subexpression

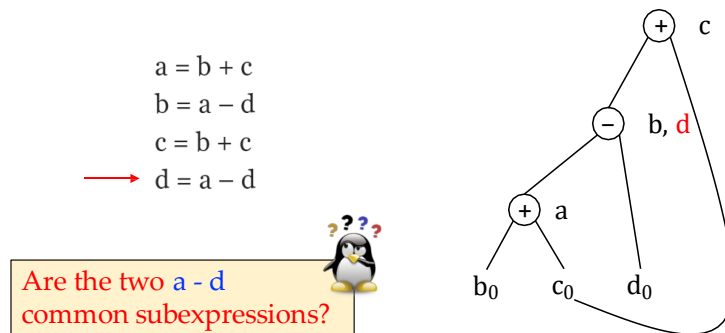
- When creating a node  $M$ , check if there exists a node  $N$ , which has the same operator and children nodes (order matters) with  $M$
- If such a node  $N$  exists, we do not create the node  $M$  but simply use  $N$  to represent  $M$



当我们为第三条语句  $c = b + c$  构造结点时，我们知道  $b + c$  中使用的  $b$  指的是图 8.12 中标号为  $-$  的结点。因为这个结点是  $b$  的最近的定值。因此，我们不会混淆语句 1 和语句 3 所计算的值。

## Finding Local Common Subexpression

- When creating a node  $M$ , **check if there exists a node  $N$** , which has the same operator and children nodes (order matters) with  $M$
- If such a node  $N$  exists, we do not create the node  $M$  but simply **use  $N$  to represent  $M$**



然而，对应于第四条语句  $d = a - d$  的结点的运算符是  $-$ ，且它的子结点是标记有变量  $a$  和  $d_0$  的结点。由于运算符和子结点都与语句 2 对应的结点相同，我们不需要创建此结点，而是将  $d$  添加到这个标记为  $-$  的结点的定值变量表中。

由于图 8.12 的 DAG 中只有三个非叶节点，看起来示例 8.10 中的基本块可以替换为一个只有三个语句的基本块。事实上，如果  $b$  在退出块时不活跃，那么我们不需要计算该变量  $b$ ，并且可以使用  $d$  来存放由图 8.12 中的标号为  $-$  的结点所代表的值。这个基本块就变成了

$a = b + c$

$d = a - d$

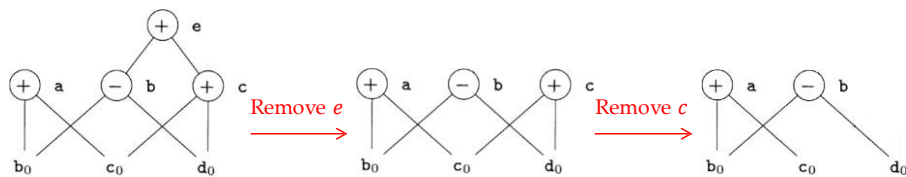
$c = d + c$

然而，如果  $b$  和  $d$  在出口处活跃，则必须使用第四条语句将值从一个复制到另一个。



# Dead Code Elimination

- We can delete from a DAG any **root** (node without ancestors) that has **no live variables** attached
- **Repeatedly applying the above transformation** will remove all nodes corresponding to dead code
- Suppose in the example below, ***c* and *e* are not live** (they have no next uses), but ***a* and *b* are live**



对应于死码消除的 DAG 操作可以按如下方式实现。

我们从一个 DAG 上删除所有没有附加活跃变量的根结点（没有祖先的节点）。

重复应用此处理过程，就可从 DAG 中删除与死代码对应的所有节点。

例 8.12: 如果在图 8.13 中, *a* 和 *b* 是活跃变量, 而 *c* 和 *e* 不是, 我们可以立即删除标记为 *e* 的根结点。然后, 标记为 *c* 的结点就变成根节点, 也可以被删除。标有 *a* 和 *b* 的根结点被保留下来, 因为它们各自都附加了活跃变量。

# The Use of Algebraic Identities

- **Eliminate computations (消除计算步骤)** from a basic block
  - $x + 0 = 0 + x = x$      $x - 0 = x$
  - $x \times 1 = 1 \times x = x$      $x / 1 = x$
- **Reduction in strength (降低计算强度)**, replacing a more **expensive** operator by a **cheaper** one
  - $2 \times x = x + x$      $x / 2 = x \times 0.5$
- **Constant folding (常量合并)**
  - $2 \times 3.14 = 6.28$

We can implement such optimizations by looking for patterns in a DAG

代数恒等式代表了基本块上另一类重要的优化。例如，我们可以应用算术恒等式，例如消除基本块中的计算步骤。

另一类代数优化包括局部强度削弱，即用代价低的运算替代代价高的运算，如下所示：

第三类相关优化是常量合并。使用这种方法时，我们在编译时计算常量表达式，并把此常量表达式替换为求出的值。因此，表达式  $2 \times 3.14$  将被  $6.28$  替换。实际中由于程序中频繁使用符号常量，所有会出现许多常量表达式。

# Code Generator

- **Generate machine instructions from three-address code**
  - Assume there is exactly one machine instruction for each operator
  - Assume some registers are available to hold the values used in a basic block
- **Primary goal:** avoid generating unnecessary loads and stores, i.e., making the best use of registers
- **Four principal uses of registers:**
  1. Hold the operands to perform operations
  2. Hold temporaries
  3. Hold global values
  4. Help with run-time storage management (e.g., holding stack pointers)

在本节中，我们将考虑一种为单个基本块生成代码的算法。它依次考虑每个三地址指令，并跟踪哪些寄存器中有哪些值

基本目标是：避免生成不必要的加载和存储指令。

代码生成过程中的主要问题之一是决定如何最大程度地使用寄存器。寄存器有四种主要用途：

- 在大多数机器体系结构中，执行一个运算时该运算的部分或全部运算分量必须位于寄存器中。
- 寄存器很适合做临时存储器，即在计算较大表达式时保存其子表达式结果，或者更一般地说，寄存器适合用于保存仅在单个基本块中使用的变量的值。
- 寄存器用于保存在一个基本块中计算并在其他块中使用的（全局）值，例如，循环下标的值，每次循环都对该值做增量运算，并在循环体内多次被使用。
- 寄存器通常用于帮助运行时存储管理，例如，管理运行时堆栈，包括堆栈指针的维护以及堆栈的顶部元素也可能被存放在寄存器中。

这些都是相互竞争的需求，因为可用的寄存器数量是有限的。

# Code Generation Algorithm Overview

- The basic process:
  1. **Load:** Considers each three-address instruction in turn and decides what loads are necessary to get the needed operands into registers
  2. **Computation:** After generating the loads, it generates the operation
  3. **Store:** Then, if there is a need to store the result into a memory location, it also generates the store instruction

Only generates loads when necessary  
Try not to overwrite the register whose value is still of use

本节中的算法假设有一组寄存器可用于保存块内使用的值。通常，这组寄存器不包括机器的所有寄存器，因为一些寄存器是专门为全局变量和管理堆栈保留的。我们假设基本块已经通过诸如公共子表达式合并之类的转换而被转换成我们希望的三地址指令序列。我们进一步假设对于每个运算符，有且只有一个对应的机器指令。这个指令对存放在寄存器中的所需的运算分量进行运算，并将结果保留存在寄存器中。机器指令的形式为

LD reg, mem

ST mem, reg

OP reg,reg,reg

我们的代码生成算法依次考虑每个三地址指令，

- 并决定需要哪些加载指令来把所需运算加载进寄存器。
- 在生成加载指令后，它会生成运算代码。
- 然后，如果需要将结果存储到内存位置，它还会生成相应的保存指令。

# Code Generation Algorithm Overview

- Two important data structures

1. **Register descriptor (寄存器描述符)**: For each available register, keeping track of the variable names whose current value is in that register
2. **Address descriptor (地址描述符)**: For each program variable, keeping track of the locations where the current value of that variable can be found
  - A **location** may be a register, a memory address, a stack location

为了做出这些必要的决策，我们需要一个数据结构来说明哪些程序变量的值当前被存放在哪个或哪些寄存器里面。我们还需要知道给定变量的内存位置上的值是否就是该变量的正确值，因为该变量的新值可能已在寄存器中计算出来但尚未存放到内存中。这个数据结构具有以下描述符：

1. 对于每个可用寄存器，寄存器描述符跟踪有哪些变量的当前值存放在此寄存器中。由于我们将考虑那些用于存放在一个基本块内的局部值的寄存器，因此我们假设最初所有寄存器描述符都是空的。随着代码生成的进行，每个寄存器将保存零个或多个变量名字的值。

2. 对于每个程序变量，地址描述符会跟踪记录在哪个或哪些位置上可以找到该变量的当前值。该位置可能是寄存器、内存地址、堆栈位置或其中多个位置的组成的一个集合。该信息可以存储在该变量名对应的符号表条目中。

# Algorithm Details (1)

- An essential part of the algorithm: *getReg(I)*
  - Select registers for each memory location associated with the three-address instruction *I*, according to the register and address descriptors, and data-flow info (e.g., variables live on block exit)

Obviously, how *getReg()* selects registers affects the quality of the generated machine code

该算法的一个重要部分是函数 `getReg(I)`，它为每个与三地址指令 *I* 有关的内存位置选择寄存器。函数 `getReg` 可以访问基本块的所有变量对应的寄存器和地址描述符，并且还可以访问某些有用的数据流信息，例如哪些变量在基本块出口处活跃。我们将在介绍基本算法之后讨论 `getReg`。虽然我们不知道总共有多少寄存器可用于存放基本块的局部数据，因此假设有足够的寄存器，使得在把值存放回内存，释放了所有可用寄存器之后，空闲的寄存器足以完成任何三地址运算。

## Algorithm Details (2)

- For a three-address instruction  $x = y + z$  (here,  $+$  is a **generic operator**), do the following:
  1. Use `getReg()` to **select registers**  $R_x$ ,  $R_y$ , and  $R_z$  for  $x$ ,  $y$ , and  $z$
  2. If  $y$  is not in  $R_y$ , according to the **register descriptor** for  $R_y$ , then generate an instruction
    - `LD  $R_y$ ,  $y'$`  //  $y'$  is a **mem loc** for  $y$  according to  $y'$ 's **address descriptor**
  3. Similar to step 2, generate `LD  $R_z$ ,  $z'$`  if necessary
  4. Generate instruction `ADD  $R_x$ ,  $R_y$ ,  $R_z$`

在一个形如  $x = y + z$  的三地址指令中，我们将 $+$ 视为一般的运算符，将 `ADD` 视为等价的机器指令。因此，我们没利用 $+$ 的交换性。这样，当我们实现该运算时， $y$  的值必须位于 `ADD` 指令中提到的第二个寄存器中，而不是第三个寄存器中。可以按照下面的方式来改造算法：只要  $+$  是满足交换律的运算符，算法同时为  $x = y + z$  和  $x = z + y$  生成代码，然后再选择一个比较好的代码序列。

运算的机器指令

对于三地址指令（例如  $x = y + z$  ），请执行以下操作：

1. 使用 `getReg( $x = y + z$ )` 选择  $x$ 、 $y$  和  $z$  的寄存器。将这些称为  $R_x$ 、 $R_y$  和  $R_z$ 。
2. 如果  $y$  不在  $R_y$  中（根据  $R_y$  的寄存器描述符），则生成指令 `LD  $R_y$ ;  $y'$` ，其中  $y'$  是存放  $y$  的内存位置之一（ $y'$  可根据  $y$  的地址描述符得到）。
3. 类似地，如果  $z$  不在  $R_z$  中，则生成指令 `LD  $R_z$ ;  $z'$` ，其中  $z'$  是存放  $z$  的位置。
4. 生成指令 `ADD  $R_x$ ;  $R_y$ ;  $R_z$` 。

## Algorithm Details (3)

- For a copy instruction  $x = y$ , do the following:
  - We assume *getReg()* will always select the same registers for  $x$  and  $y$
  - If  $y$  is not in that register  $R_y$ , then generate an instruction  $LD\ R_y, y$
- Ending a basic block
  - For temporary variables used within the block, when the block ends, we can forget about their value and assume their register is empty
  - If a variable is live on block exit (or if we don't know the liveness), generate  $ST\ x, R_x$  for each  $x$  whose address descriptor does not say that its current value is in the memory location for  $x$  (the value in register is newer than that in memory)

### 复制语句的机器指令

$x = y$  形式的三地址复制语句是一个重要的特例。我们假设 *getReg* 总是为  $x$  和  $y$  选择相同的寄存器。如果  $y$  尚未在该寄存器  $R_y$  中，则生成机器指令  $LD\ R_y ; y$ 。如果  $y$  已经在  $R_y$  中，我们什么也不做。我们只需调整  $R_y$  的寄存器描述符，表明  $R_y$  中也存放了  $x$  的值。

### 基本块的收尾工作

我们描述算法时表明，在代码结束的时候，基本块中使用的变量可能仅存放在某个寄存器中。如果这个变量是一个仅在块内使用的临时变量，那就没问题；当块结束时，我们可以忘记这些临时变量的值并假设其寄存器为空。但是，如果变量在块出口处活跃，或者如果我们不知道哪些变量在出口处活跃，那么我们需要假设稍后还会用到该变量的值。在这种情况下，对于每个变量  $x$ ，如它的地址描述符表明它的值没有存放在  $x$  的内存位置上，我们必须生成指令  $ST\ x; R$ ，其中  $R$  是在基本块的结尾处存放  $x$  值的寄存器。



# Algorithm Details (4)

Update register and address descriptors with four rules:

1. For the instruction `LD R, x`:
  - a) Change the **register descriptor** for *R* so it holds only *x*
  - b) Change the **address descriptor** for *x* by adding *R* as an additional location
2. For the instruction `ST x, R`:
  - a) Change the **address descriptor** for *x* to include its own memory location\*

\* Values in registers cannot be older than those in memory.

管理寄存器和地址描述符

当代码生成算法生成加载、存储和其他机器指令时，它必须更新寄存器和地址描述符。规则如下：

1. 对于指令 `LD R; x`
  - (a) 更改寄存器 *R* 的寄存器描述符，使其仅包含 *x*。
  - (b) 更改 *x* 的地址描述符，把寄存器 *R* 作为新增位置加入到 *x* 的位置集合中。
  - (c) 从任何不同于 *x* 的变量的地址描述符中删除 *R*。
2. 对于指令 `ST x; R`，更改 *x* 的地址描述符使之包含其自己的内存位置。

# Algorithm Details (5)

Update register and address descriptors with four rules:

3. For an operation such as `ADD  $R_x$ ,  $R_y$ ,  $R_z$`  for implementing a three-address instruction  $x = y + z$ :
  - a) Change the **register descriptor** for  $R_x$  so it holds only  $x$
  - b) Change the **address descriptor** for  $x$  so that its only location is  $R_x$
  - c) Remove  $R_x$  from the **address descriptor** of any variable other than  $x$
4. When processing a copy statement  $x = y$ :
  - a) If `LD  $R_y$ ,  $y$`  is generated, manage descriptors using rule 1
  - b) Add  $x$  to the **register descriptor** for  $R_y$
  - c) Change the **address descriptor** for  $x$  so that its only location is  $R_y$

3. 对于实现三地址指令  $x = y + z$  的 `ADD  $R_x$ ;  $R_y$ ;  $R_z$`  这样的运算而言
    - (a) 更改  $R_x$  的寄存器描述符，使其仅包含  $x$ 。
    - (b) 更改  $x$  的地址描述符，使其只包含位置  $R_x$ 。请注意， $x$  的地址描述符不包含  $x$  的内存位置。
    - (c) 从任何不同于  $x$  的变量的地址描述符中删除  $R_x$ 。
  4. 当我们处理复制语句  $x = y$  时，
    - (a) 如果有必要生成把  $y$  加载入  $R_y$  的加载指令，那么在生成加载指令并（按照规则
- 1) 像处理所有的加载指令那样处理完各个描述符之后，再进行下面的处理：
- (1) 将  $x$  添加到寄存器  $R_y$  的描述符。
  - (2) 更改  $x$  的地址描述符，使其只包含唯一的位置  $R_y$ 。

# Example

- $a, b, c, d$  are live at the block exit
- $t, u, v$  are temporaries, local to the block

$t = a - b$

$u = a - c$

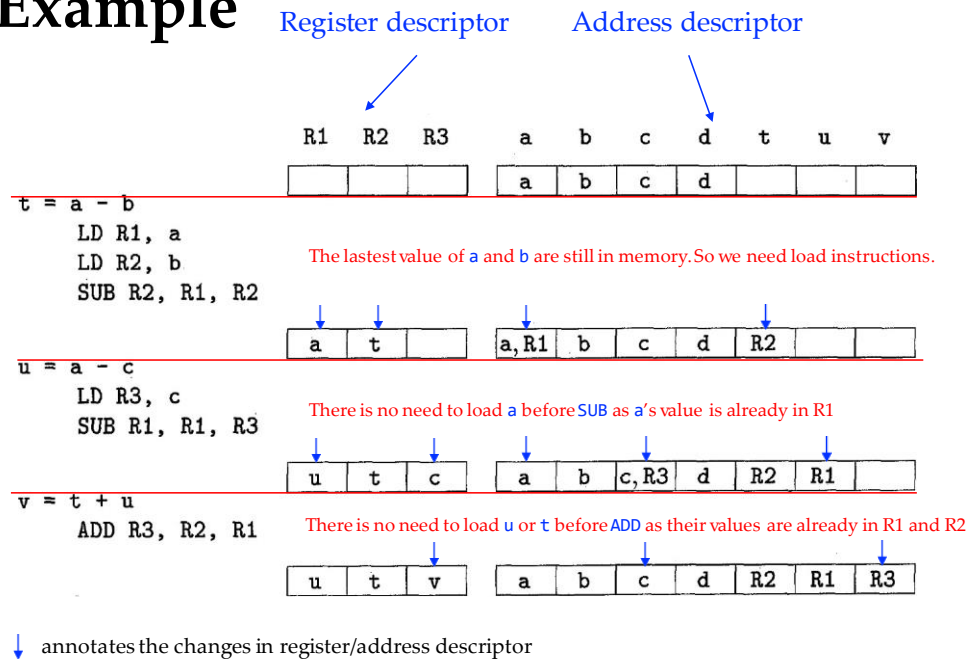
$v = t + u$

$a = d$

$d = v + u$

这里我们假设  $t$ 、 $u$  和  $v$  是块的局部临时变量，而变量  $a$ 、 $b$ 、 $c$  和  $d$  是在块出口处活跃。由于我们还没有讨论 `getReg` 函数如何工作，我们将简单地假设有我们需要的寄存器，但是当不再需要寄存器的值时（例如，它只保存一个临时变量的值，且对这个临时变量的所有使用都已经处理完了），我们就重用这个寄存器。

# Example



生成的所有机器代码指令的摘要如图 8.16 所示。该图还显示了每个三地址指令转换之前和之后的寄存器和地址描述符。

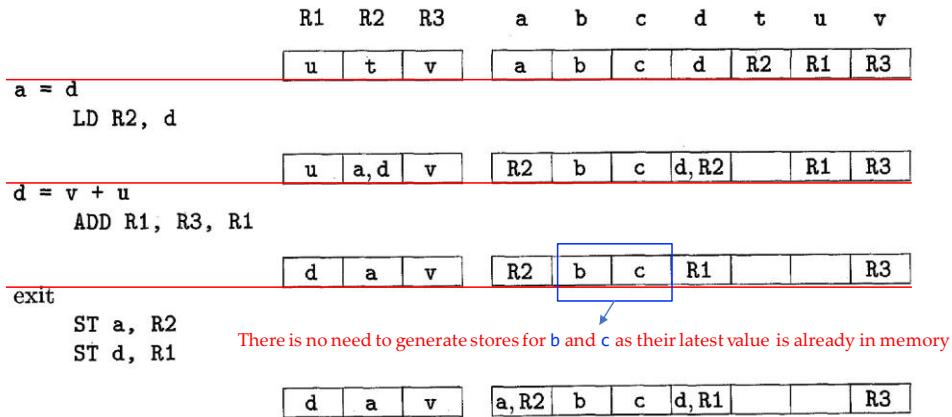
1.因为寄存器中最初没有任何内容，我们需要为第一个三地址指令  $t = a - b$ ，生成三个指令。因此，我们看到 a 和 b 被加载存入寄存器 R1 和 R2，并且 t 的值生成后存放在寄存器 R2 中。请注意，我们可以将 R2 用于 t 是因为块内不需要 R2 中先前的值 b。由于 b 可能在块退出时处于活跃状态，如果它不在自己的内存位置（如其地址描述符所示），我们就必须首先将 R2 存储到 b 中。如果我们需要 R2，那么生成指令  $ST\ b\ R2$  的决定将由 `getReg` 做出。

2.第二条指令  $u = a - c$  不需要加载 a 的指令，因为 a 已经在寄存器 R1 中。我们可以重用 R1 作为结果 u，因为块内不再需要先前在该寄存器中的 a 值，并且如果块外需要 a，则其值位于其自己的内存位置中。请注意，我们更改了 a 的地址描述符，以表明它不再位于 R1 中，而是位于名为 a 的内存位置中。

3.第三条指令  $v = t + u$  仅需要一个加法指令。我们可以使用 R3 来存放结果 v，因为块内不再需要该寄存器中的 c 值，并且 c 在其自己的内存位置中有其值。

# Example

Why do we generate store instruction only at block exit? Why only for a and d?



4. 复制指令  $a=d$  需要一个指令来加载  $d$ ，因为  $d$  不在寄存器中。我们展示了寄存器  $R2$  的描述符，其中包含  $a$  和  $d$ 。寄存器描述符中添加  $a$  是我们处理复制语句的结果，而不是任何机器指令的结果。

5. 第五条指令  $d = v + u$  使用两个存放在寄存器中的值。由于  $u$  是一个临时变量，其值不再需要，因此我们选择重用其寄存器  $R1$  来存放  $d$  的新值。请注意， $d$  现在仅位于  $R1$  中，而不位于其自己的内存位置中。这同样适用于  $a$ ，它在  $R2$  中而不是在称为  $a$  的内存位置中。因此，我们需要为基本块的机器代码增加一个“尾声”，它把出口处活跃的变量  $a$  和  $d$  的值保存到它们的内存位置。这就是图中的最后两条指令。

# Design of the Function *getReg*

- **Goal:** Avoid too much data exchange with memory (by LD and ST)
- **Task:** Pick registers for operands and result of each three-addr inst
- **For the generic example  $x = y + z$ ,** pick register  $R_y$  for operand  $y$ :
  - **Case 1:** If  $y$  is currently in a register, pick a register already containing  $y$  as  $R_y$ . Nothing else needs to be done.
  - **Case 2:** If  $y$  is not in a register, but there is a register that is currently empty, pick one such register as  $R_y$
  - **Case 3:** What if  $y$  is not in any register and there is no register that is currently empty?

最后，让我们考虑如何实现三地址指令  $I$  的  $getReg(I)$ 。有很多选择，但也有一些绝对禁止的选择，这些选择会因丢失一个或多个活跃变量的值而导致错误的代码。我们从操作步骤的情况开始进行研究，对此我们再次使用  $x = y + z$  作为通用示例。首先，我们必须为  $y$  选择一个寄存器，为  $z$  选择一个寄存器。这两次选择所面临的问题是相同的，因此我们将集中精力为  $y$  选择寄存器  $R_y$ 。规则如下：

1. 如果  $y$  当前在寄存器中，则选择已经包含  $y$  的寄存器作为  $R_y$ 。不需要生成一个机器指令来把  $y$  加载该寄存器。
2. 如果  $y$  不在寄存器中，但有一个寄存器当前为空，则选择一个这样的寄存器作为  $R_y$ 。
3. 当  $y$  不在寄存器中并且没有当前为空的寄存器时，就会出现困难的情况。

# Design of the Function *getReg*

*y* is not in register and there is no register that is currently empty:

- **Basic idea:** pick one allowable register and make it “safe” to reuse
- Let  $R$  be a candidate, and suppose  $v$  is a variable in  $R$ 's register descriptor
  - If  $v$ 's address descriptor says **we can find  $v$  somewhere besides  $R$** , it is safe to reuse  $R$
  - If  $v$  is  $x$  and  $x$  is not the other operand  $z$ , it is safe to reuse  $R$ \*
  - If  $v$  is not used later (not live after the instruction), it is safe to reuse  $R$
  - Otherwise, generate  $ST\ v, R$  to place a copy of  $v$  in its own memory location (*spill*, 溢出)
- If  $R$  holds multiple variables, repeat the process for each such variable  $v$ 
  - At the end,  $R$ 's score is the number of generated store instructions. We can pick the candidate with the lowest score

\* If  $v$  is  $x$ , since  $x$  is going to be redefined, it is ok to discard its old value. Why requiring  $x$  is not  $z$ ? Because as  $R$  is chosen to be reused,  $y$ 's value will be loaded into  $R$ . If  $x$  is  $z$ , that means the current value of  $z$  is in  $R$ , then it needs to be stored to avoid data loss.

当  $y$  不在寄存器中并且没有当前为空的寄存器时，无论如何，我们需要选择一个可行的寄存器，并且我们需要确保它可以安全地重用。

设  $R$  为候选者寄存器，并且假设  $v$  是  $R$  的寄存器描述符表明的位于  $R$  中的变量。我们需要确保  $v$  的值不是真正需要的，或者我们可以从其他地方获取  $v$  的值。可能的情况是：

- 如果  $v$  的地址描述符表明  $v$  位于  $R$  之外的某个位置，那么我们就完成了任务。
- 如果  $v$  是  $x$ ，即由指令  $I$  计算的变量，并且  $x$  也不是指令  $I$  的其他运算分量之一（本例中为  $z$ ），那么我们就完成了任务。原因是，在这种情况下，我们知道  $x$  的这个值永远不会再被使用，所以我们可以忽略它。
- 否则，如果稍后不使用  $v$ （即，在指令  $I$  之后，不再使用  $v$ ，并且如果  $v$  在块出口处活跃，那么  $v$  的值必然在基本块中被重新计算），那么我们就完成了任务。
- 如果前三种情况都不满足，则需要生成存储指令  $ST\ v; R$  将  $v$  的副本放置在其自己的内存位置中。这种操作称为溢出。

由于  $R$  此时可能包含多个变量，因此我们对每个这样的变量  $v$  重复上述步骤。最后， $R$  的得分是我们需要生成的保存指令的个数。选择得分最低的寄存器（或者之一）。

# Design of the Function *getReg*

- Picking register  $R_x$  for result  $x$  in  $x = y + z$ :
  - Since a new value of  $x$  is being computed, a register that holds only  $x$  is always an acceptable choice for  $x$
  - If  $y$  is not used after the instruction, and  $R_y$  holds only  $y$ , then  $R_y$  can also be used as  $R_x$  (same for  $R_z$ )
- Pick registers  $R_x$  and  $R_y$  for  $x = y$ :
  - Pick  $R_y$  first (as above)
  - Then choose  $R_x = R_y$

现在，考虑寄存器  $R_x$  的选择。其中的难点和可选项几乎与选择  $R_y$  时的一样，所以我们只提及其中的区别。

1. 由于正在计算  $x$  的新值，因此仅保存  $x$  的寄存器始终是  $R_x$  可接受的选择。即使  $x$  就是  $y$  或  $z$  之一，该语句仍然成立，因为我们的机器指令允许在一条指令中的两个寄存器相同。

2. 如果（像上面对变量  $v$  的描述那样） $y$  在指令  $I$  之后不再使用，且（在必要时加载  $y$  之后） $R_y$  仅保存了  $y$  的值，则  $R_y$  同时也可以用作  $R_x$ 。对于  $z$  和  $R_z$  也有类似的选项。

最后要特别考虑的问题是当  $I$  是复制指令  $x = y$  时的情况。我们用上面描述的方法选择寄存器  $R_y$ ，然后是让  $R_x = R_y$ 。



# Peephole Optimization (窥孔优化)

- A simple but effective technique for **locally improving target code** by examining a **sliding window** of target instructions (the *peephole*, 窥孔) and replacing the instruction sequences within the peephole with *shorter* or *faster* sequences\*
  - Redundant-instruction elimination (冗余指令消除)
  - Flow-of-control optimizations (控制流优化)
  - Algebraic simplifications (代数简化)
  - Use of machine idioms (机器特有指令的使用)

\* Can also be applied directly after intermediate code generation to improve the IR

虽然大多数生产编译器通过仔细的指令选择和寄存器分配来生成良好的代码，但也有一些编译器使用另一种策略：它们生成简单的代码，然后通过对目标代码应用“优化”转换来提高目标代码的质量。术语“优化”有些误导，因为不能保证生成的代码在任何数学测量下都是最优的。然而，许多简单的转换可以显着提高目标程序的运行时间或空间要求。

用于局部改进目标代码的一种简单但有效的技术是窥孔优化，它是检查目标指令的滑动窗口（称为窥孔）并且只要有可能就在窥孔内用更短或更快的指令替换窥孔内的指令序列。也可以在中间代码生成之后直接应用窥孔优化，以改进中间表示。

窥孔是程序上的一个小的滑动窗口。窥孔中的代码不需要是连续的，尽管某些实现确实需要这样做。窥孔优化的特点是每次改进又可能产生出新的优化机会。一般来说，为了获得最大利益，需要重复多次扫描目标代码。在本节中，我们将给出以下具有窥孔优化特征的程序转换示例：

冗余指令消除

控制流优化

代数简化

机器特有指令的使用。

## Redundant Loads and Stores

- A naïve algorithm (not the one we introduced) may generate the following code:
  - `LD R0, a`
  - `ST a, R0`
- **We can delete the store instruction if there is no jump to it** (that is, the two instructions are in the same basic block)
  - If there is a jump to the store instruction, we could not be sure that the LD instruction is always executed before the store instruction (there might be other instructions putting a new value of `a` into `R0`)

如果我们在目标程序中看到指令序列 `LD R0, a`

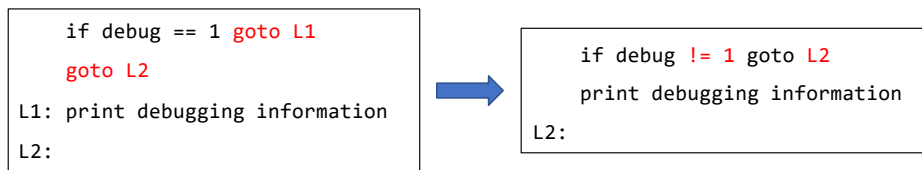
`ST a, R0`

我们可以删除保存指令，因为无论何时执行它，第一条指令都会确保 `a` 的值已经加载到寄存器 `R0` 中。

请注意，如果存储指令有标号，我们无法确定第一条指令总是在第二条指令之前执行，因此我们无法删除存储指令。换句话说，两条指令必须位于同一个基本块中，这样转换才能安全。这种性质的冗余加载和存储不会由上一节的简单代码生成算法生成。然而，像第 8.1.3 节中的原始的代码生成器可能会生成类似的冗余代码序列。

# Unreachable Code

- **Example 1:** An unlabeled instruction immediately following an unconditional jump can be removed
- **Example 2:** Jumps over jumps (级联跳转)



Assuming there is no other jump to L1

How to optimize the code if we know debug is a constant 0?



另一个窥孔优化的机会是删除无法到达的指令。

紧随无条件跳转之后的不带标号的指令可以被删除。可以重复该操作以消除指令序列。例如，出于调试目的，大型程序中可能包含某些代码片段，仅当变量 `debug` 等于 1 时才执行这些代码片段。在中间表示中，此代码可能看起来像

```
if debug == 1 goto L1

goto L2

L1: print debugging information

L2:
```

一个明显的窥孔优化是消除级联跳转指令。因此，无论 `debug` 的值是什么，上面的代码序列都可以替换为

```
if debug != 1 goto L2

print debugging information

L2:
```

How to optimize the code if we know debug is a constant 0?

如果在程序开始时将 `debug` 设置为 0，则常量传播会将此序列转换为

```

if 0 != 1 goto L2

print debugging information

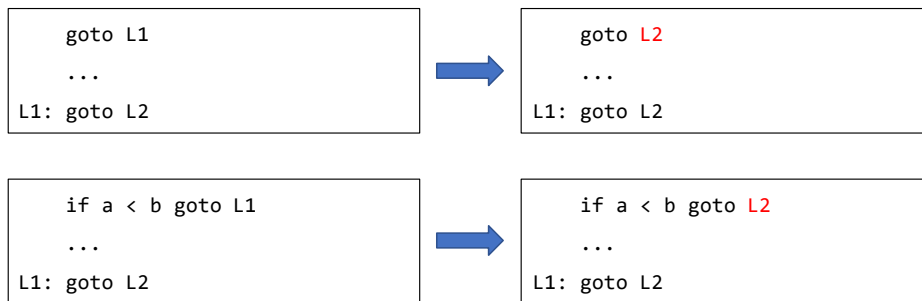
```

L2:

现在，第一条语句的计算结果始终为 `true`，因此该语句可以用 `goto L2` 替换。那么所有打印调试信息的语句都是不可达的，可以被逐一删除。

## Flow-of-Control Optimizations

- Unnecessary jumps



Can we further eliminate L1: goto L2?



简单的中间代码生成算法经常产生目标为无条件跳转指令的无条件跳转指令，到达条件跳转的无条件跳转指令，或者达到无条件跳转指令的条件跳转指令。这些不必要的跳转指令可以通过以下类型的窥孔优化在中间代码或目标代码中消除。我们可以替换顺序

```

goto L1
...
L1: goto L2

```

替换为

```

goto L2
...
L1: goto L2

```

如果现在没有跳转到 L1 的指令，并且语句

L1: goto L2 之前是一个无条件跳转指令，则可以消除这个语句。

类似地，序列

```
 if a < b goto L1
```

```
 ...
```

```
L1: goto L2
```

可以替换为序列

```
 if a < b goto L2
```

```
 ...
```

```
L1: goto L2
```

Can we further eliminate L1: goto L2?

最后，假设只有一个达到 L1 的跳转指令，并且 L1 之前有一个无条件跳转指令，那么序列

```
 goto L1
```

```
 ...
```

```
L1: if a < b goto L2
```

```
L3:
```

可以被替换为序列

```
 if a < b goto L2
```

```
 goto L3
```

```
 ...
```

```
L3:
```

虽然两个序列中的指令数量相同，但在第二个序列我们有时可以跳过无条件跳转指令，但在第一个序列中的却不能。因此，第二个序列的执行时间优于第一个序列的执行时间。

# Register Allocation and Assignment

- Instructions involving only register operands are faster than those involving memory operands
  - Efficient utilization of registers is vitally important in generating good code
- **Register allocation (寄存器分配)**
  - Decide at each point of a program what values should reside in registers
- **Register assignment (寄存器指派)**
  - Decide at each point of a program in which register each value should reside
- **There exist many strategies** for register allocation and assignment

仅涉及寄存器运算分量的指令比涉及存储器运算分量的指令更快。在现代机器上，处理器速度通常比内存速度快一个数量级或更高。因此，寄存器的有效利用对于生成良好的代码至关重要。本节介绍了各种策略，用于在程序中的每个点决定哪些值应驻留在寄存器中（寄存器分配）以及各个值应驻留在哪个寄存器中（寄存器指派）。

# The Simple Strategy

- **Assign specific values to certain registers**
  - Assign array base addresses to a group of registers
  - Assign arithmetic computational values to another group
  - Assign the top of the stack to a fixed register ...
- **Advantage:** Simplifies the design and implementation of a compiler
- **Disadvantage:** Inefficient uses of registers
  - Certain registers may go unused, while many loads and stores are generated for the other registers
- It is reasonable to reserve a few registers for base addresses, stack pointers, and the like

寄存器分配和赋值的一种方法是将目标程序中的特定值分配给特定的寄存器。例如，我们可以决定将基地址分配给一组寄存器，将算术计算分配给另一组，栈顶指针分配给一个固定寄存器，等等。

这种方法的优点是简化了代码生成器的设计。但因为它的应用有太多限制，所以寄存器的使用效率低下；某些被占用的寄存器可能在大部分代码运行中未使用，同时却不得不生成很多不必要的而其他寄存器的加载和保存运算指令。然而，在大多数计算环境中还是要保留一些寄存器，这些被保留的寄存器可以被用作基址寄存器、堆栈指针寄存器或其他类似的用途，其他寄存器则由代码生成器在它认为适当的时候使用。

# Global Register Allocation

- The earlier algorithm uses registers to hold values for the duration of a single basic block
  - All live variables were stored (if necessary) at the end of each block
- To save stores and loads, we can **assign registers to frequently used variables** and keep these registers consistent across block boundaries (**globally**)
  - E.g., assign certain registers to hold the most active values in a loop
- Another strategy is to estimate the benefits of putting a variable in register (via static analysis or profiling) and allocate registers according to the estimations

第 8.6 节中的代码生成算法在单个基本块的运行期间使用寄存器来存放值。然而，在每个基本块的结尾处，所有活跃变量的值都被保存到内存中。为了省略一部分这样的保存及相应的存储指令，我们可以把一些寄存器指派给频繁使用的变量，并使得这些寄存器在不同基本块中的（即全局的）指派保持一致。

由于程序大部分时间都花在它的内部循环上，因此一个自然的全局寄存器指派方法是尝试在整个循环中将频繁使用的值保留在固定寄存器中。暂时假设我们知道一个流图的循环结构，并且我们知道在基本块中计算的哪些值会在该块之外使用。下一章将介绍计算这些信息的技术。

全局寄存器分配的一种策略是分配一些固定数量的寄存器来保存每个内部循环中最活跃的值。不同循环中选择的值可能不同。尚未分配的寄存器可用于保存一个块的局部值，如第 8.6 节所示。这种方法的缺点是固定数量的寄存器并不总是恰好等于用于全局寄存器分配的最佳数量。然而，该方法实现起来很简单，并在 Fortran H 中使用，Fortran H 是 IBM 在 20 世纪 60 年代末为 360 系列机器开发的优化 Fortran 编译器。

使用早期的 C 编译器，程序员可以通过使用寄存器声明显式地进行一些寄存器分配，以便在过程运行期间将某些值保留在寄存器中。明智地使用寄存器声明确实加快了许



多程序的速度，但应鼓励程序员在分配寄存器分配之前首先获取程序运行时刻特征并确定程序的热点代码。

# Introduction

- Code optimization (or code improvement)
  - Eliminating **unnecessary** instructions in object code
  - Replacing one sequence of instructions by a **faster** sequence of instructions that does the same thing
- Global optimizations
  - Most global optimizations are based on **data-flow analyses**
  - Data-flow analyses: algorithms to gather info about a program
- A compiler knows only how to apply relatively **low-level semantic transformations** to optimize code

代码优化（或代码改进）

消除目标代码中不必要的指令

用做同样事情的更快指令序列替代一个指令序列

全局优化

大多数全局优化都基于数据流分析

数据流分析：收集程序信息的算法

编译器只知道如何应用相对低级的语义转换来优化代码

# Two Major Causes of Redundancy

1. Redundant operations at the **source level** (e.g., introduced by programmers for convenience)
2. **Side effect** of writing programs in a high-level language
  - In most languages (other than C/C++, which allows pointer arithmetic), programmers can only refer to **elements of an array** or **fields in a structure** through accesses like  $A[i][j]$  or  $X \rightarrow f1$
  - During compilation, each high-level data-structure access expands into **a sequence of low-level arithmetic operations**
  - Access to the same data structure often create many common low-level operations

典型的程序中存在很多冗余操作。

1.有时，在源代码中会用到冗余。例如，程序员可能会发现重新计算某些结果更加直接和方便，而让编译器去发现实际上只需要进行一次这样的计算。

2.但更多的时候，冗余是使用高级语言编写程序的副产品。

- 在大多数语言中（不包含 C 或 C++，它们允许对指针进行算术运算），程序员别无选择，只能通过  $A[i][j]$  或  $X \rightarrow f1$  之类的访问来引用数组的元素或结构中的字段。

- 当程序被编译时，每一个这样的高级数据结构访问都会被扩展为多个低层次的算术运算，例如计算一个矩阵 A 的第 (i; j) 个元素的位置的运算。

- 对于同一个数据结构的访问通常共享许多公共的低级运算。程序员不知道这些低级运算，也无法自行消除这些冗余。事实上，从软件工程的角度来看，程序员最好只通过数据元素的高级名字来访问它们。这样程序更容易编写，更重要的是，更容易理解和演化。通过让编译器消除冗余，我们可以两全其美：程序既高效又易于维护。

# A Running Example: Quicksort

```
void quicksort(int m, int n)
/* recursively sorts a[m] through a[n] */
{
 int i, j;
 int v, x;
 if (n <= m) return;
 /* fragment begins here */
 i = m-1; j = n; v = a[n];
 while (1) {
 do i = i+1; while (a[i] < v);
 do j = j-1; while (a[j] > v);
 if (i >= j) break;
 x = a[i]; a[i] = a[j]; a[j] = x; /* swap a[i], a[j] */
 }
 x = a[i]; a[i] = a[n]; a[n] = x; /* swap a[i], a[n] */
 /* fragment ends here */
 quicksort(m,j); quicksort(i+1,n);
}
```

|      |                   |      |              |
|------|-------------------|------|--------------|
| (1)  | i = m-1           | (16) | t7 = 4*i     |
| (2)  | j = n             | (17) | t8 = 4*j     |
| (3)  | t1 = 4*n          | (18) | t9 = a[t8]   |
| (4)  | v = a[t1]         | (19) | a[t7] = t9   |
| (5)  | i = i+1           | (20) | t10 = 4*j    |
| (6)  | t2 = 4*i          | (21) | a[t10] = x   |
| (7)  | t3 = a[t2]        | (22) | goto (5)     |
| (8)  | if t3<v goto (5)  | (23) | t11 = 4*i    |
| (9)  | j = j-1           | (24) | x = a[t11]   |
| (10) | t4 = 4*j          | (25) | t12 = 4*i    |
| (11) | t5 = a[t4]        | (26) | t13 = 4*n    |
| (12) | if t5>v goto (9)  | (27) | t14 = a[t13] |
| (13) | if i>=j goto (23) | (28) | a[t12] = t14 |
| (14) | t6 = 4*i          | (29) | t15 = 4*n    |
| (15) | x = a[t6]         | (30) | a[t15] = x   |

Three-address code

接下来，我们将使用名为快速排序的排序程序片段来说明几个重要的可以改进代码的转换。

图 9.1 中的 C 程序源自 Sedgwick，它讨论了如何对这样一个程序进行手工优化。我们不会在这里讨论这个程序在算法方面的所有微妙的细节，例如，a[0] 必然存放着已排序的元素的最小者，而 a[max]则存放最大的元素。

在我们可以优化掉地址计算中的冗余之前，必须首先将程序中的地址运算分解为低级算术运算，这样才能暴露处冗余之处。在本章的其余部分中，我们假设中间表示形式由三地址语句组成，其中所有中间表达式的结果都由临时变量来保存。图 9.1 中标记的程序片段的中间代码如图 9.2 所示。

在这个例子中，我们假设整数占用四个字节。

# Array Accesses

|      |                   |      |              |
|------|-------------------|------|--------------|
| (1)  | i = m-1           | (16) | t7 = 4*i     |
| (2)  | j = n             | (17) | t8 = 4*j     |
| (3)  | t1 = 4*n          | (18) | t9 = a[t8]   |
| (4)  | v = a[t1]         | (19) | a[t7] = t9   |
| (5)  | i = i+1           | (20) | t10 = 4*j    |
| (6)  | t2 = 4*i          | (21) | a[t10] = x   |
| (7)  | t3 = a[t2]        | (22) | goto (5)     |
| (8)  | if t3<v goto (5)  | (23) | t11 = 4*i    |
| (9)  | j = j-1           | (24) | x = a[t11]   |
| (10) | t4 = 4*j          | (25) | t12 = 4*i    |
| (11) | t5 = a[t4]        | (26) | t13 = 4*n    |
| (12) | if t5>v goto (9)  | (27) | t14 = a[t13] |
| (13) | if i>=j goto (23) | (28) | a[t12] = t14 |
| (14) | t6 = 4*i          | (29) | t15 = 4*n    |
| (15) | x = a[t6]         | (30) | a[t15] = x   |

$x = a[i] \rightarrow$  (14), (15)

$a[j] = x \rightarrow$  (20), (21)

Each array access translates into a multiplication and an array-subscripting operation

赋值运算  $x = a[i]$  按照第 6.4.4 节翻译为两个三地址语句 (14)、(15)

类似的,  $a[j] = x$  变成了 (20)、(21)

请注意, 原始程序中的每个数组访问都会转换为一对语句, 包括一个乘法和一个数组下标运算。结果, 这个短短的程序片段被翻译成一个相当长的三地址运算序列。

# Flow Graph

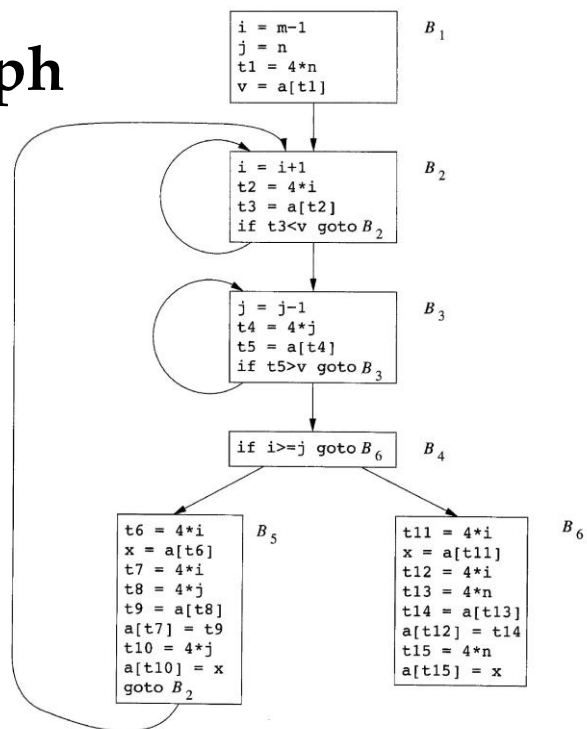


图 9.3 是图 9.2 中程序的流图。

$B_1$  块是其入口结点。

在 8.4 节介绍过，图 9.2 中所有条件和无条件跳转语句的目标在图 9.3 中都被替换为以它们的为目标语句为首语句的基本块。

在图 9.3 中有 3 个循环。块  $B_2$  和  $B_3$  本身就是循环。块  $B_2$ 、 $B_3$ 、 $B_4$  和  $B_5$  一起形成一个循环，其中  $B_2$  是唯一的入口点。

# Semantic-Preserving Transformations

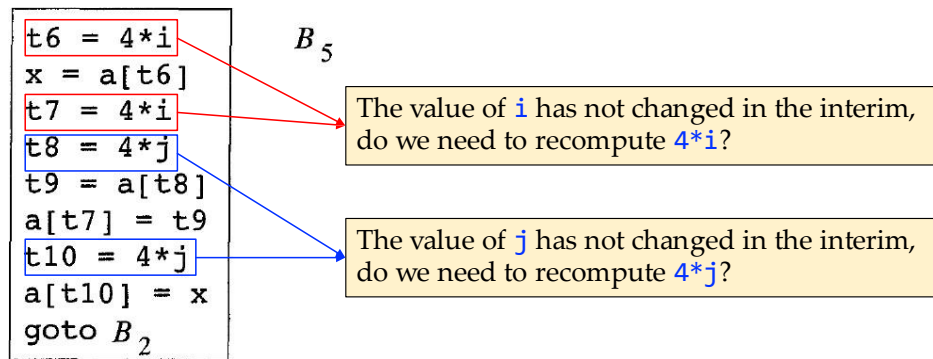
- Common-subexpression elimination
- Copy propagation
- Dead-code elimination
- Code motion

## 9.1.3 保持语义不变的转换

编译器可以使用多种方法改进一个程序，但不改变程序所计算的函数。

公共子表达式消除、复制传播、死代码消除和常量折叠都是这样的函数不变（或者说语义不变）转换的常见例子。我们将逐一介绍这些方法。

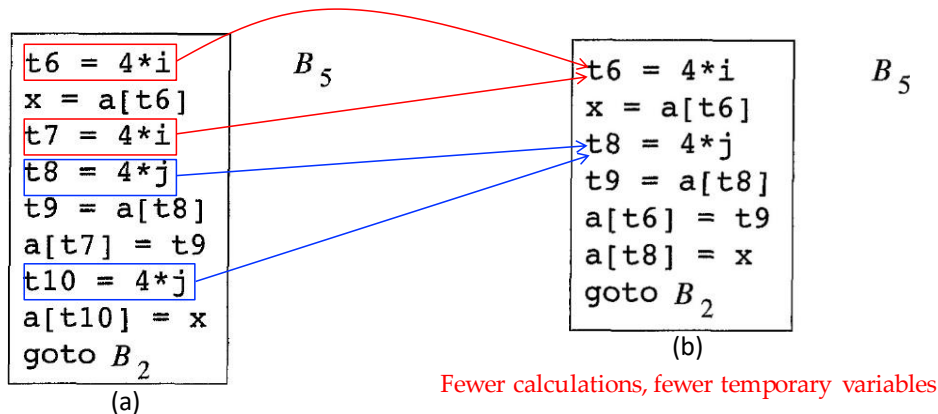
## Local Common-Subexpression Elimination



局部公共子表达删除：通常，程序会包含对相同值的多次计算，例如计算数组中的偏移量。例如，图 9.4(a) 中所示的块  $B_5$  中对  $4*i$  和  $4*j$  进行了重复计算，尽管这些计算全都不是程序员显式要求的。



## Local Common-Subexpression Elimination

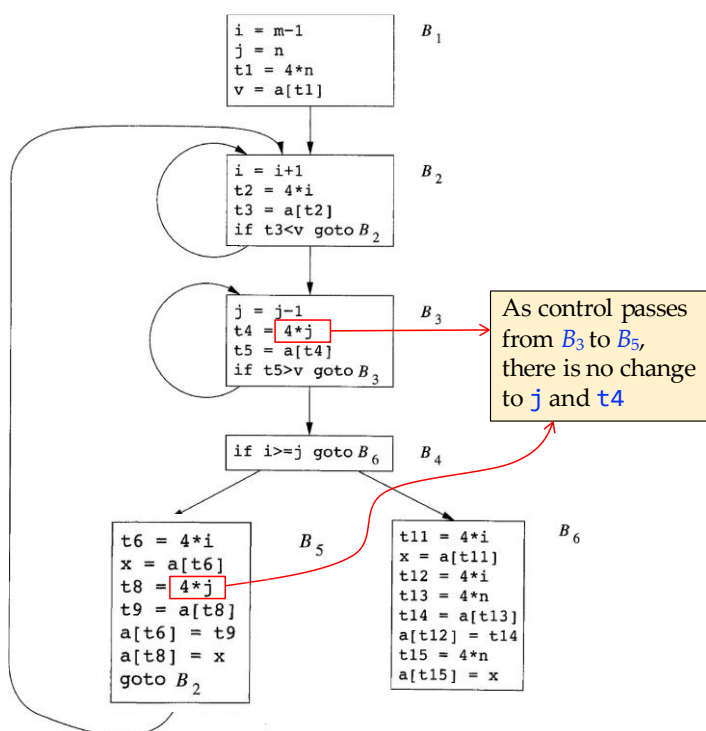


Such duplicate calculations cannot be avoided by programmers because they lie below the level of detail accessible within the source language

正如第 9.1.2 节中提到的，某些这样的重复计算，不可能由程序员来避免，因为这些计算过程处于可在源语言中处理的细节的更下层。如果表达式  $E$  是先前计算过的，并且  $E$  中变量的值自上次计算以来没有改变，则表达式  $E$  的该次出现就称为公共子表达式。如果将  $E$  的上一次计算结果赋予变量  $x$ ，且  $x$  的值在此期间没有改变，我们就可以使用前面计算得到的值，从而避免重新计算  $E$ 。

例 9.1：图 9.4(a) 中对  $t_7$  和  $t_{10}$  的赋值，分别计算了公共子表达式  $4*i$  和  $4*j$ 。这些步骤已经在图 9.4(b) 中被消除了。消除后的代码使用  $t_6$  代替  $t_7$ ，使用  $t_8$  代替  $t_{10}$ 。

## Global Common Subexpression Elimination



全局公共子表达式删除：我们首先讨论  $B_5$  的转换，然后再讨论一些涉及数组相关的精妙之处。

消除局部公共子表达式后， $B_5$  仍对  $4*i$  和  $4*j$  进行求值，如图 9.4 (b) 所示。两者都是公共子表达式；更明确的讲，使用块  $B_3$  中计算得到的  $t_4$  的值， $B_5$  中的三个语句：

$t_8 = 4*j$

$t_9 = a[t_8]$

$A[t_8] = x$

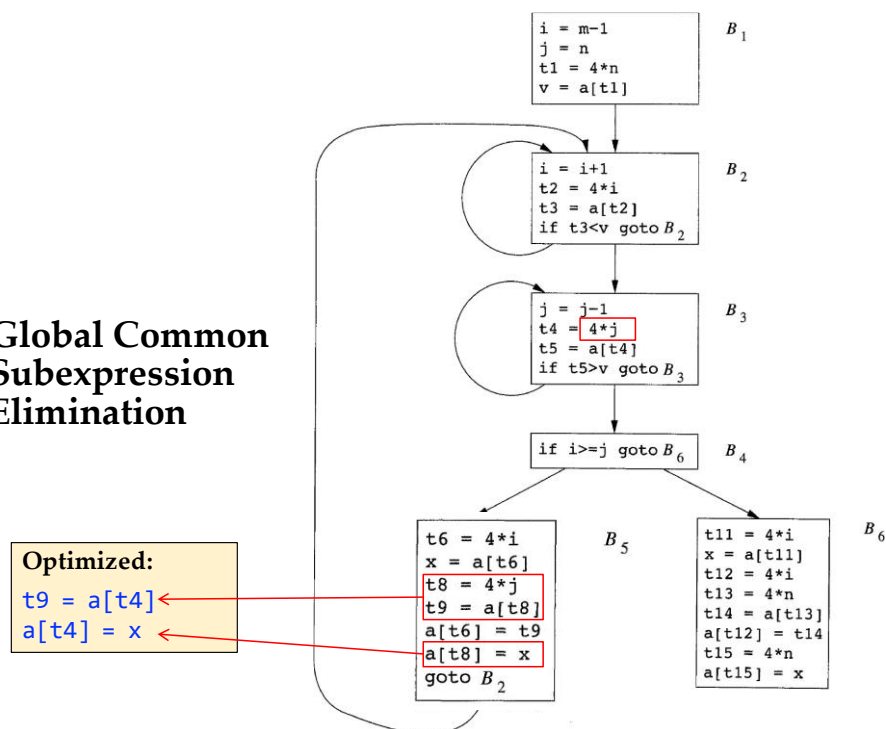
可以替换为

$t_9 = a[t_4]$

$a[t_4] = x$

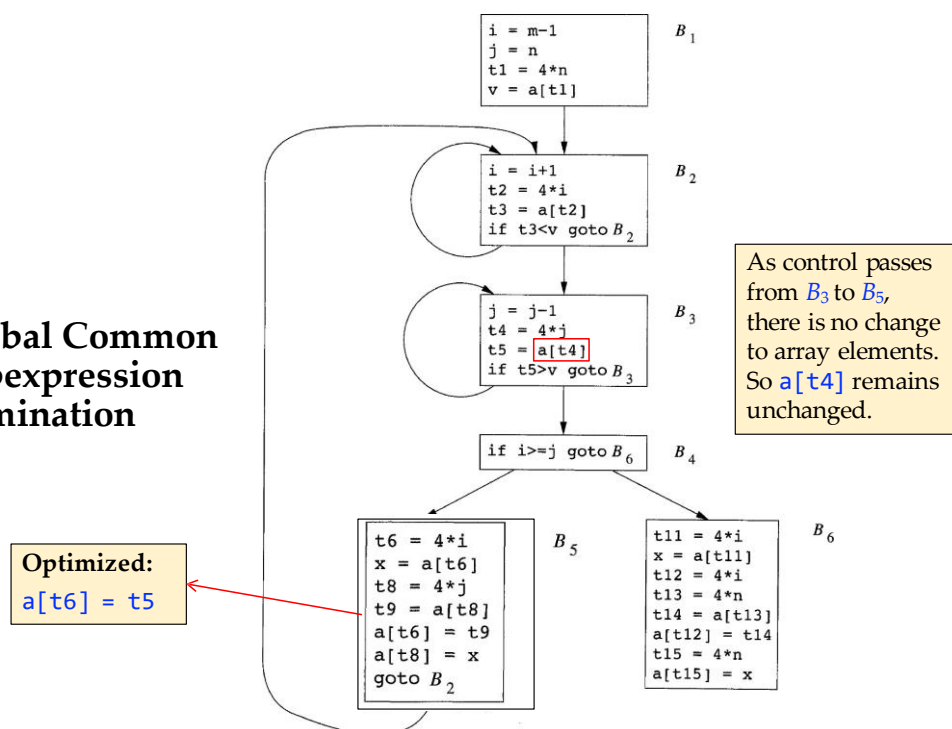
在图 9.5 中，观察到当控制从  $B_3$  中计算  $4*j$  的点传递到  $B_5$  中时， $j$  没有变化， $t_4$  也没有变化，因此如果需要  $4*j$  时，可以使用  $t_4$  来替代。

## Global Common Subexpression Elimination



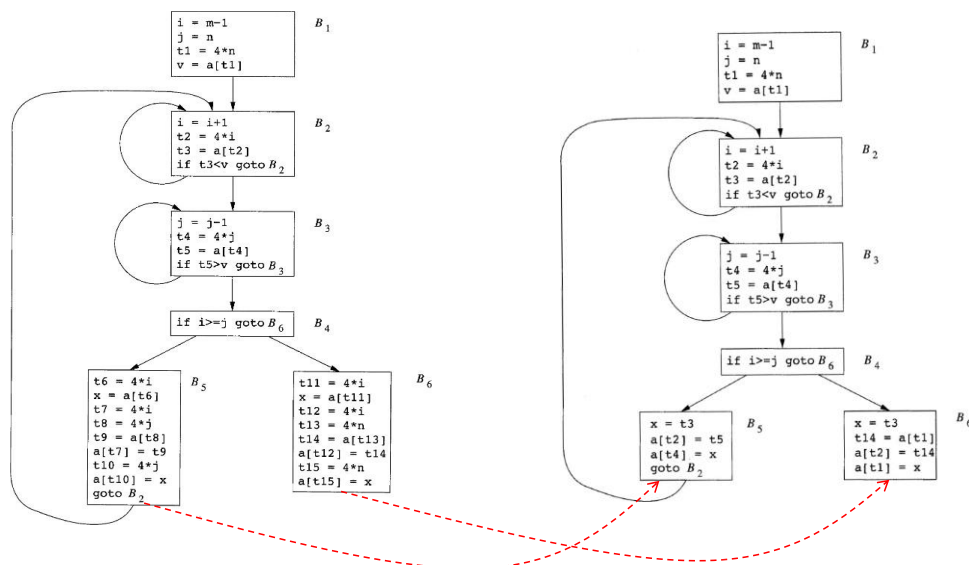
在 用 `t4` 替换 `t8` 后, `B5` 中出现了另一个公共的子表达式 `a[t4]` 。

## Global Common Subexpression Elimination



新的子表达式  $a[t4]$  对应于源代码层次上的值  $a[j]$ 。当控制离开  $B3$  然后进入  $B5$  时，不仅仅  $j$  保留了它的值，而且  $a[j]$  也保留了原来的值。这个值在计算出来之后保存到临时变量  $t5$  中。因为在此期间没有对数组  $a$  的元素进行赋值。因此， $a[j]$  的值不变。 $B5$  中的语句  $a[t6] = t9$  可以被替换为  $a[t6] = t5$

## After Optimizations



类似地，在图 9.4(b) 的块  $B5$  中赋给  $x$  的值 ( $x=a[t6]$ ) 和  $B2$  中赋给  $t3$  的值 ( $t3=a[t2]$ ) 相同。图 9.5 中的块  $B5$  是从图 9.4(b) 中的  $B5$  中消除了与源代码级表达式  $a[i]$  和  $a[j]$  的值对应的公共子表达式之后的结果。图 9.5 中的  $B6$  也进行了一系列类似的变换。

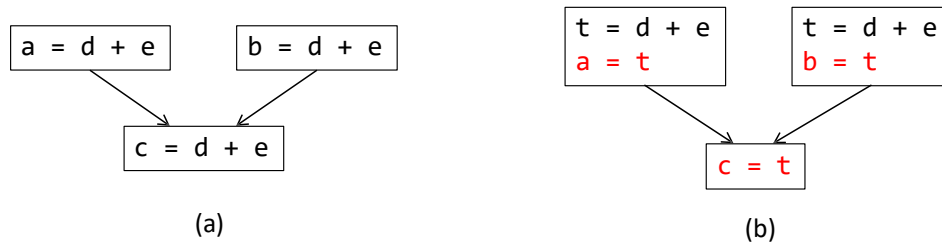
图 9.5 的块  $B1$  和  $B6$  中的表达式  $a[t1]$  不被视为公共子表达式，尽管  $t1$  可以在两个地方使用。在控制离开  $B1$  后、到达  $B6$  之前，它还可能经过  $B5$ ，而  $B5$  中有对  $a$  的赋值。因此， $a[t1]$  在到达  $B6$  时可能和它离开  $B1$  时的值不同。将  $a[t1]$  视为公共子表达式是不安全的。

图 9.5 中的块  $B5$  可以通过使用两个新的变换消除  $x$ ，从而得到进一步改进。其中一个转化考虑形如  $u = v$  的赋值表达式，这种表达式被称为复制语句，简称复制。如果我们在例 9.2 中更详细地讨论，很快就会发现一些复制语句。因为消除公共子表达式的常用算法会引入复制语句，其他几种优化算法也会引入这样的语句。

# Copy Propagation

- Optimization algorithms often introduce *copy statements*

- Form:  $u = v$



Eliminating the common subexpression  $d+e$  introduces the copy statement  $c = t$

去掉公共子表达式  $d+e$  后，就引入了复制语句  $c = t$  (复制语句，即形如  $x = y$  的赋值语句)

常用的公共子表达式消除算法和其它一些优化算法会引入一些复制语句

例 9.3: 为了消除图 9.6(a) 中的公共子表达式语句  $c = d+e$ ，我们必须使用一个新变量  $t$  来保存  $d+e$  的值。在图 9.6(b) 中，变量  $t$  的值而不是表达式  $d+e$  的值被赋给  $c$ 。由于控制可能经过对  $a$  的赋值到达  $c = d+e$  处，也可能经过对  $b$  的赋值到达  $c = d+e$  处，因此用  $c = a$  或  $c = b$  替换  $c = d+e$  是不正确的。

# Copy Propagation

- Copy propagation is to **use v for u**, wherever possible after the copy statement



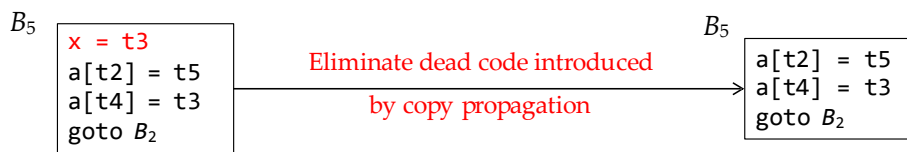
This change may not appear to be an improvement, but it gives us the opportunity to eliminate the assignment to  $x$

复制传播变换背后的思想是在复制语句  $u = v$  之后尽可能使用  $v$  代替  $u$ 。

例如，图 9.5 的块  $B_5$  中的赋值语句  $x = t3$  是一个复制语句。应用于  $B_5$  的复制传播产生图 9.7 中的代码。这一变化可能看起来不是一种改进，但是，正如我们将在第 9.1.6 节中看到的，它为我们提供了消除对  $x$  赋值的语句的机会。

# Dead-Code Elimination

- A variable is *live* at a program point if its value can be used subsequently; otherwise, it is *dead* at that point
- *Dead code*: Statements computing values that are never used
- Dead code appears often because of optimizations



如果一个变量在某一程序点上的值可能会在以后使用，那么我们就说这个变量在该点是活跃的。否则，它在该点就是死的。与此相关的一个想法就是死（或无用）代码。

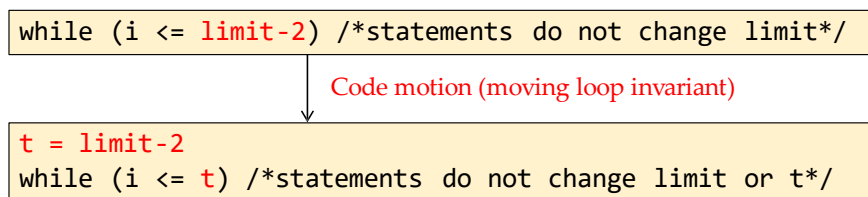
所谓死代码就是其计算结果永远都不会被使用的语句。

虽然程序员不太可能故意引入死代码，死代码多半是因为前面执行过的某些转换而造成的。

复制传播的优点之一是它经常将复制语句变成死代码。例如，先进行复制传播再进行死代码消除，就可以去掉图 9.7 (左图) 中的代码对 `x` 的赋值，并将转换为 (右图)

# Code Motion

- Loops are a very important place for optimizations
- **Loop invariant**: an expression that yields the same result regardless of the number of times a loop is executed
- Moving loop invariants and evaluate them before loops can decrease the running time of a program



循环是优化的一个非常重要的地方，特别是程序往往花费大量时间的内部循环。如果我们减少内部循环中的指令数量，即使我们增加该循环外部的代码量，程序的运行时间也可能会得到改善。

减少循环内部代码数量一个重要改动是代码移动。此转换处理的是那些不管循环执行多少次都得到相同结果的表达式（循环不变计算），在进入循环之前就对它们求值。

请注意，概念“在循环之前”的说法假定存在一个循环入口。所谓循环入口就是一个基本块（请参见第 8.4.5 节）所有循环外部到循环的跳转指令都以它为目标。

示例 9.5：在下面的 while 语句中，对  $\text{limit} - 2$  的求值是一个循环不变计算：

现在， $\text{limit} - 2$  的计算只在进入循环之前被执行一次。之前，如果我们重复循环体  $n$  次，将会对  $\text{limit} - 2$  计算  $n + 1$  次。

循环是进行优化的重要场所

循环不变式：无论循环执行多少次，结果都相同的表达式

移动循环不变式并在循环前对其进行评估可减少程序的运行时间



# What Is Data-Flow Analysis?

- A body of techniques that derive information about the flow of data along program execution paths
  - **Example 1:** Whether two textually identical expressions evaluate to the same value along any execution path? (Identify common subexpressions)
  - **Example 2:** Whether the result of an assignment is used on subsequent execution paths? (Eliminate dead code)
- Data-flow analysis is the foundation of many optimizations

“数据流分析”是指一组用来获取有关数据如何沿着程序执行路径流动的相关信息的技术。

例如，实现全局公共子表达式消除的一种方法要求我们确定在程序的任何可能的执行路径上，两个字面相同的表达式是否会给出相同的值。(找出共同的子表达式)

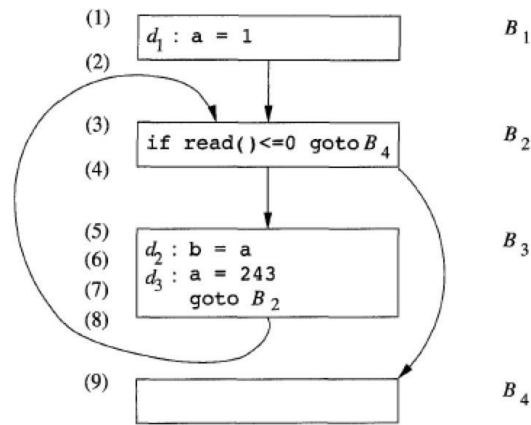
另一个例子是，如果某一个赋值语句的结果在任何后续执行路径中都没有被使用，那么我们可以将赋值作为死代码消除。(消除死代码)

这些和许多其他重要问题都可以通过数据流分析来回答。数据流分析是许多优化的基础。

# The Data-Flow Abstraction

- **Program points**: points before and after each statement

- Within one block, the program point after a statement is the same as the program point before the next statement (e.g., 6)
- If there is an edge from block  $B$  to block  $C$ , then the program point after the last statement of  $B$  **may be followed immediately by** the program point before the first statement of  $C$  (e.g., 8 and 3)



## 数据流抽象

程序的执行可以看作是对程序状态的一系列转换。程序状态由程序中的所有变量的值组成，同时包括运行时刻的栈顶之下各个帧的相关值。一个中间代码语句的每次执行都会把一个输入状态转换成一个新的输出状态。这个输出状态和处于该语句之前的程序点相关联，而输出状态和该语句之后的程序点相关联。

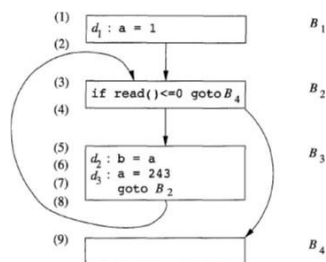
当我们分析程序的行为时，我们必须考虑程序执行时可能采取的各种通过程序的流图的程序点序列（“路径”）。然后，我们从各个程序点上可能的程序状态中提取需要的信息，解决特定的数据流分析问题。在更复杂的分析中，我们必须考虑在调用和返回执行时会形成在各种不同过程的流图之间跳转的路径。但是，要刚开始我们的分析研究时，我们将关注穿越单个过程的单个流图的路径。

让我们看看流图会告诉哪些关于可能执行路径的信息。

- 在一个基本块内，一条语句之后的程序点是与它的下一条语句之前的程序点相同
- 如果从块  $B_1$  到块  $B_2$  之间有一条边，则  $B_2$  的第一个语句之前的程序点可能紧跟在  $B_1$  的最后一条语句的程序点之后。

# The Data-Flow Abstraction

- An **execution path** from point  $p_1$  to point  $p_n$  is a sequence of points  $p_1, p_2, \dots, p_n$  such that for each  $i = 1, 2, \dots, n-1$ , **either**
  - $p_i$  is the point immediately preceding a statement and  $p_{i+1}$  is the point immediately follow that statement, **or**
  - $p_i$  is the end of a block and  $p_{i+1}$  is the beginning of a successor block



The shortest complete execution path: (1, 2, 3, 4, 9)

The next shortest path executing one iteration of the loop: (1, 2, 3, 4, 5, 6, 7, 8, 3, 4, 9)

因此，我们可以把从点  $p_1$  到点  $p_n$  的一个执行路径（简称路径）定义为满足下列条件的点的序列点  $p_1; p_2; \dots; p_n$ ：对于每个  $i = 1; 2; \dots; n-1$ ,

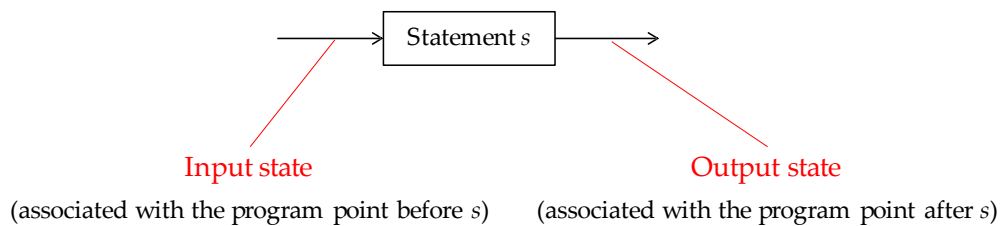
- 要么  $p_i$  是紧接在一条语句之前的点，并且  $p_{i+1}$  是紧接在该语句之后的点，
- 要么  $p_i$  是某个块的末尾，并且  $p_{i+1}$  是该基本块的一个后继块的开始。

一般来说，程序中可能存在无穷多条可能的执行路径，并且执行路径的长度并没有上限。程序分析把可能出现在某个程序点的所有程序状态总结为有穷的特性集合。不同的分析可以选择抽象掉不同的信息，并且一般来说，没有哪个分析会给出状态的完全表示。

例 9.8：即使是图 9.12 中的简单程序也描述了无限多个执行路径。最短的完整执行路径由程序点 (1; 2; 3; 4; 9) 组成，它不进入任何循环。次短的路径执行一次循环，它由程序点 (1; 2; 3; 4; 5; 6; 7; 8; 3; 4; 9) 组成。在这个例子中，我们知道在第一次执行程序点(5)时，由于  $d_1$  的定值， $a$  的值必然为 1。我们说  $d_1$  在第一次迭代的时候到达了点 (5)。在后续迭代中， $d_3$  到达了点 (5)， $a$  的值为 243。

# The Data-Flow Abstraction

- **Program execution** can be viewed as *a series of transformations of the program state* (the values of all variables)
  - Each execution of an intermediate-code statement transforms an **input state** to a new **output state**



程序执行可视为程序状态（所有变量的值）的一系列转换

中间代码语句的每次执行都会将输入状态转换为新的输出状态

# The Data-Flow Abstraction

- When analyzing a program, we must consider all the possible execution paths through a flow graph
  - In general, there is **an infinite number** of possible paths due to the existence of loops (and recursions)
- Data-flow analyses **summarize all the possible program states** that can occur at a program point with **a finite set of facts**
  - Different analyses may choose to abstract out different information
  - In general, no analysis is necessarily a perfect representation of the state

分析程序时，我们必须考虑流图中所有可能的执行路径

一般来说，由于循环（和递归）的存在，可能存在无限多的路径

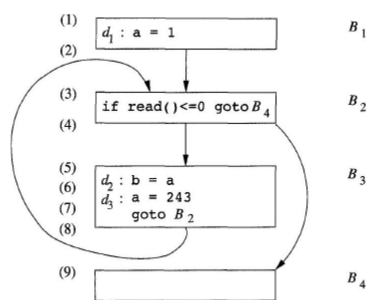
数据流分析用一组有限的事实概括了程序点上可能出现的所有程序状态

不同的分析可能会选择抽取不同的信息

一般来说，任何分析都不一定能完美地反映现状

一般来说，跟踪所有可能路径的所有程序状态是不可能。在数据流分析中，我们不区分到达一个程序点的路径之间的差异。此外，我们并不跟踪整个状态；相反，我们抽象出某些细节，只保留分析所需的数据。两个例子将说明一个程序点上的同一个状态可以导出不同的抽象信息。

## Example (1)



### Reaching definitions:

- The first time point (5) is executed, the value of  $a$  is 1 (i.e., definition  $d_1$  reaches (5) in the first iteration)
- In subsequent iterations,  $d_3$  reaches point (5) and the value of  $a$  is 243
- So, we may summarize all the program states at (5) by saying that the value of  $a$  is one of  $\{1, 243\}$  and defined by one of  $\{d_1, d_3\}$

到达定值：可能沿着某些路径到达某个程序点的定值称为到达定值。

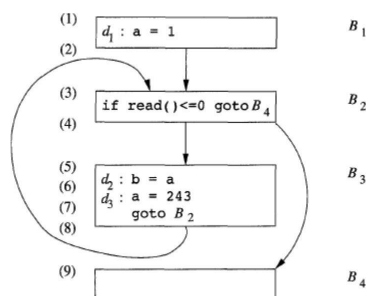
为了帮助用户调试程序，我们可能希望找出变量在某个程序点上可能具有哪些值，以及这些值可能在哪里定值。例如，

在执行第一个时间点 (5) 时， $a$  的值为 1（即定值  $d_1$  在第一次迭代中达到 (5)

在随后的迭代中， $d_3$  到达点 (5)， $a$  值为 243

因此，我们可以对在程序点（5）上的所有程序状态进行如下总结：a 的值总是 {1, 243} 中的一个，并且它由 {d1;d3} 中的一个定值。

## Example (2)



### Constant folding:

- For constant folding, we wish to find those definitions that are the unique definition of their variable to reach a given program point, regardless of the execution path
- For this task, we may simply describe some variables as “constant” or “not a constant” instead of collecting their possible values
  - *a* is not a constant at point (5) and thus cannot be replaced by a constant

常量折叠。

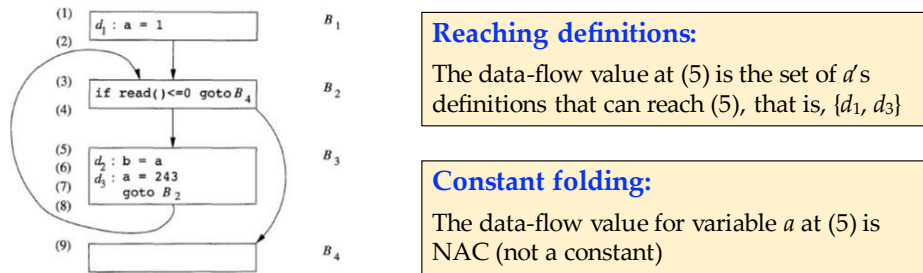
• 如果对变量 *x* 的某次使用只有一个定值可以到达，并且该定值把一个常量赋值给 *x*，那么我们可以简单地把 *x* 替换为该常量。另一方面，如果有多个对 *x* 的定值可以到达某个程序点，那么我们就无法对 *x* 执行常量折叠转换。因此，为了进行常量折叠，我们希望找到这样的定值：对于某个给定的程序点，不管执行哪条路径，它们都是唯一到达该点的对应变量的定值。

• 对于图 9.12 的点 (5)，没有哪个定值是到达该点的对 *a* 的唯一定值，因此对于点 (5) 上的 *a* 来说，该集合是空的。即使一个变量在某个点上被唯一定值，该定值必须把为一个常量赋值给该变量，才可能进行常量折叠。这样，我们可以简单地把某些变量描述为“非常量 NAC”，而不是记录它们所有可能的取值或所有可能的定值。

因此，我们看到，根据分析目的，相同的信息可以通过不同的方式进行概括。

# The Data-Flow Analysis Schema

- We associate with every program point a *data-flow value* that represents an abstraction of the set of all possible program states observed for that point\*



\* In each application of data-flow analysis, the set of possible data-flow values is the domain for the application.

在所有数据流分析应用中，我们都会把每个程序点与一个数据流值相关联，这个值是在该点可能观察到的所有程序状态的集合的抽象表示。所有可能的数据流值集合称为这个数据流应用的域。

例如，用于达到定值的数据流值的域是程序的定值集合的所有子集的集合。某个数据流值是一个定值的集合，我们希望将程序中的每个点与可能到达该点的定值的精确集合关联起来。

达到定值：

常量折叠：

如上所述，对于抽象方式的选择取决于分析的目标。为了提高效率，我们只跟踪相关的信息。

# The Data-Flow Analysis Schema

- $IN[s]$ : The data-flow value **before** the statement  $s$
- $OUT[s]$ : The data-flow value **after** the statement  $s$
- The *data-flow problem* is to find a solution to a set of constraints on the  $IN[s]$ 's and  $OUT[s]$ 's for all statements  $s$ 
  1. Constraints based on the semantics of the statements ("*transfer functions*")
  2. Constraints based on the flow of control

$IN[s]$ : 语句  $s$  之前的数据流值

$OUT[s]$ : 语句  $s$  之后的数据流值

数据流问题是为所有语句  $s$  的  $IN[s]$ 's 和  $OUT[s]$ 's 找到一组约束条件的解决方案。

基于语句语义的限制（“传递函数”）

基于控制流的限制



# Transfer Functions ( 传递函数 )

- The data-flow values before and after a statement are constrained by the semantics of the statement
- **Example:** consider the constant folding case (analyzing whether a variable is a constant at a program point)
  - Statement  $s: x = 3$
  - Three variables:  $x, y, z$
  - If  $\text{IN}[s]$  is  $x: \text{NAC}; y: 7; z: 3$ , then  $\text{OUT}[s]$  is  $x: 3; y: 7; z: 3$
  - What if the statement is  $x = y + z$  or  $x = x + y$ ?

语句前后的数据流值受语句语义的限制。例如，假设我们的数据流分析涉及确定各个程序点处各变量的常量值。如果变量  $a$  在执行语句  $b = a$  之前的值为  $v$ ，那么在该语句之后  $a$  和  $b$  的值都为  $v$ 。一个赋值语句之前和之后的数据流值的关系被称为传递函数。

示例：考虑常量折叠情况（分析变量在程序点是否为常量）

语句  $s: x = 3$

三个变量：  $x, y, z$

如果  $\text{IN}[s]$  是  $x: \text{NAC}; y: 7; z: 3$ ，则  $\text{OUT}[s]$  为  $x: 3; y: 7; z: 3$

如果语句是  $x = y + z$  或  $x = x + y$  呢？

# Transfer Functions ( 传递函数 )

- The relationship between the data-flow values before and after each statement is known as a *transfer function*
- In a **forward-flow problem** (information propagate forward along execution paths), the transfer function  $f_s$  takes the data-flow value before the statement  $s$  and produces a new data-flow value after  $s$ 
  - $OUT[s] = f_s(IN[s])$
- In a **backward-flow problem** (information flow backwards up the execution paths), the transfer function  $f_s$  takes the data-flow value after the statement  $s$  and produces a new data-flow value before  $s$ 
  - $IN[s] = f_s(OUT[s])$

每条语句前后的数据流值之间的关系称为传递函数

在前向数据流问题（信息沿执行路径向前传播）中，一个语句  $s$  的传递函数  $f_s$  以语句  $s$  之前的数据流值作为输入，并产生语句  $s$  之后的新的数据流值。

$$OUT[s] = f_s(IN[s])$$

在逆向数据流问题（信息流沿着执行路径逆向流动）中，一个语句  $s$  的传递函数  $f_s$  把一个语句  $s$  之后的数据流值转变成为语句之前的新的数据流值。

$$IN[s] = f_s(OUT[s])$$

# Control-Flow Constraints

1. **Within a basic block** of statements  $s_1, s_2, \dots, s_n$ , the data-flow value out of  $s_i$  is the same as that into  $s_{i+1}$ 
  - $IN[s_{i+1}] = OUT[s_i]$ , for all  $i = 1, 2, \dots, n-1$
2. **Control-flow edges between basic blocks may create more complex constraints**
  - For example, in reaching definitions analysis, the set of definitions reaching the leader statement of a block should be the union of the definitions after the last statements of each of the predecessor blocks

关于数据流值的约束是从控制流中得到的。

在基本块内控制流很简单。如果块 B 按该顺序由语句  $s_1; s_2; \dots; s_n$  组成，则  $s_i$  输出的控制流值和输入  $s_{i+1}$  的控制流值相同。即， $IN[s_{i+1}] = OUT[s_i]$ ，对于所有  $i = 1; 2; \dots; n$

1.

然而，基本块之间的控制流边会生成一个基本块的最后一个语句和后继基本块的第一个语句之间的约束，这些约束更加复杂。例如，如果对可能达到一个程序点的所有定值感兴趣，那么到达基本块的首语句的定值的集合就是到达它的各个前驱基本块的最后一个语句之后的定值的集合。下一节详细介绍数据如何在块之间流动。

# Data-Flow Schemas on Basic Blocks (The Intra-Block Case)

- Within a basic block, control flows from the beginning to the end without interruption or branching
  - This allows us to restate the data-flow schema in terms of data-flow values entering and leaving the blocks (instead of before/after each statement)
- For a block  $B$  consisting of statements  $s_1, s_2, \dots, s_n$ , we have\*
  - $IN[B] = IN[s_1]$
  - $OUT[B] = OUT[s_n]$
  - $OUT[B] = f_B(IN[B])$ , where  $f_B = f_{s_n} \circ \dots \circ f_{s_2} \circ f_{s_1}$  (composing transfer functions)

\* For backward-flow problem,  $IN[B] = f_B(OUT[B])$ , where  $f_B = f_{s_1} \circ f_{s_2} \circ \dots \circ f_{s_n}$

从技术上讲，数据流模式涉及程序中每个点的数据流值，但我们如果认识到块内的数据流处理通常很简单，就可以节约数据流分析所需的时间和空间。在一个基本程序块内，控制从开始流向结束，没有中断或分支。

这样，我们就可以根据数据流值进出数据块（而不是每条语句之前/之后）来重述数据流模式。

假设块  $B$  按顺序由语句  $s_1 ;::; s_n$  组成。

如果  $s_1$  是基本块  $B$  的第一条语句，则  $IN[B] = IN[s_1]$ ，

同理，如果  $s_n$  是基本块  $B$  的最后一条语句，则  $OUT[B] = OUT[s_n]$ 。

基本块  $B$  的传递函数（用  $f_B$  表示）可以通过组合块中语句的传递函数来导出。也就是说，令  $f_{s_i}$  为语句  $s_i$  的传递函数，那么  $f_B = f_{s_n} ;::; f_{s_2} f_{s_1}$ 。块的开始和结束之间的数据流值关系为  $OUT[B] = f_B(IN[B])$ ：

对于由语句  $s_1, s_2, \dots, s_n$  组成的基本块  $B$ ，我们有\*

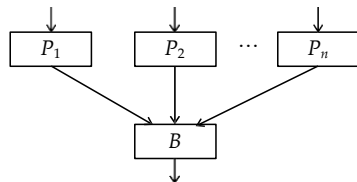
$$IN[B] = IN[s_1]$$

$$OUT[B] = OUT[s_n]$$

$$OUT[B] = f_B(IN[B]), \text{ 其中 } f_B = f_{s_n} \circ \dots \circ f_{s_2} \circ f_{s_1} \text{ (组合传递函数)}$$

\* 对于后向流动问题,  $IN[B] = fb(OUT[B])$ , 其中  $fb = fs_1 \circ fs_2 \circ \dots \circ f_{ns}$

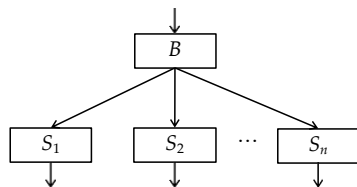
## Data-Flow Schemas on Basic Blocks (The Inter-Block Case)



### Forward-flow problem:

$$IN[B] = U \qquad OUT[P]$$

$P$  a predecessor of  $B$



### Backward-flow problem:

$$OUT[B] = U \qquad IN[S]$$

$S$  a successor of  $B$

The operator  $U$  depends on specific problems.

因基本块之间的控制流而产生的约束可以很容易地通过重写得到, 把原来约束中的  $IN[B]$  和  $OUT[B]$  用  $IN[s_1]$  和  $OUT[s_n]$  代替即可。例如, 如果一个数据流值表示的是可能被赋予某个变量的常量集合, 那么我们就得到一个前向流问题, 其中

$$IN[B] = U \qquad OUT[P]$$

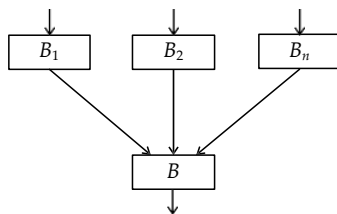
$P$  a predecessor of  $B$

我们将在活跃变量分析时看到逆向数据流问题, 逆向数据流问题的方程是相似的, 但  $IN$  和  $OUT$  的角色被调换了。即

$$OUT[B] = U \qquad IN[S]$$

$S$  a successor of  $B$

## Example (Constant Folding Problem)



**If:**

$\text{OUT}[B_1]: x: 3; y: 4; z: \text{NAC}$

$\text{OUT}[B_2]: x: 3; y: 5; z: 7$

$\text{OUT}[B_3]: x: 3; y: 4; z: 7$

**then:**

$\text{IN}[B]: x: 3; y: \text{NAC}; z: \text{NAC}$

常量折叠问题 NAC 非常量 not a constant

## Solutions to Data-Flow Equations

- Data-flow equations usually do not have a unique solution
  - All data-flow schemas compute **approximations** to the ground truth
- Our goal is to find the most “precise” solution that satisfies both **control-flow** and **transfer** constraints
  - Being precise enables valid code improvements (in general, a higher precision leads to a better improvement)
  - Satisfying constraints guarantees the safety of transformations (does not change program semantics)

与线性算术方程不同，数据流方程通常没有唯一解。所有数据流模式都计算基本事实的近似值。

我们的目标是找到一个最 “精确的 ” 的解决方案，同时满足控制流和传输约束条件。也就是说，我们需要一个解，它能够支持有效的代码改进，但是又不会导致不安全的转换。这些不安全的转换改变了程序计算的内容。

精确的代码可实现有效的代码改进（一般来说，精确度越高，改进效果越好）

满足约束条件可保证转换的安全性（不改变程序语义）

## Reaching Definitions

- A definition  $d$  of some variable  $x$  *reaches* a point  $p$  if **there is a path** from the point immediately following  $d$  to  $p$ , such that  **$d$  is not “killed”** along that path
  - $d$  is *killed* if there is any other definition of  $x$  along the path
  - Intuitively, if  $d$  reaches the point  $p$ , then  $d$  might be the place at which the value of  $x$  used at  $p$  was **last defined**

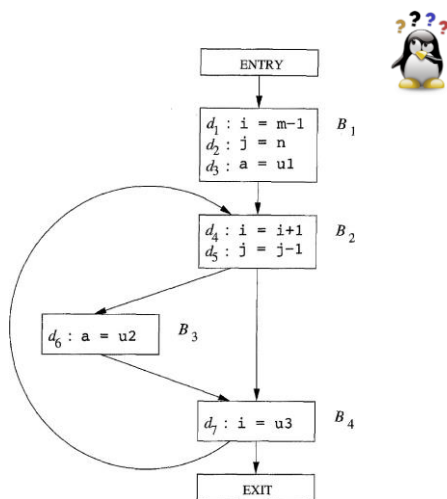
“到达定值”是最常见和最有用的数据流模式之一。通过了解当控制到达每个点时，每个变量  $x$  可能在程序中的何处定值，我们可以确定许多有关  $x$  的性质。

如果有一条从紧随定值  $d$  之后的程序点到达某一程序点  $p$  的路径，并且在这条路径上  $d$  没有被 “杀死”，我们说定值  $d$  到达程序点  $p$ 。如果在这条路径上有对变量  $x$  的其他定值，我们就说变量  $x$  的这个定值被杀死了。

直观地说，如果某个变量  $x$  的一个定值  $d$  到达点  $p$ ，则  $p$  处使用的  $x$  的值可能就是由  $d$  最后定值的。

变量  $x$  的一个定值（可能）将一个值赋给  $x$  的语句。过程参数、数组访问和间接引用都可能另有别名，因此指出一个语句是否指向特定程序变量  $x$  赋值并不是件容易的事情。程序分析必须保守；如果我们不知道一个语句  $s$  是否给  $x$  赋了一个值，我们必须假设它可能对  $x$  赋值；也就是说，语句  $s$  之后，变量  $x$  的值可能还是  $s$  执行之前的原值，但也可能变成了  $s$  所产生的新值。为了简单起见，本章的其余部分假设我们只处理没有别名的变量。此类变量包括大多数语言中的所有局部标量变量；对于 C 和 C++，有些局部变量的地址会被计算出来，这种局部变量不属于这类变量。

## Example



Which definitions reach the block  $B_2$ ?

- $d_1, d_2, d_3$  reach  $B_2$
- $d_5$  reaches  $B_2$  since there is no other definition to  $j$  in the loop
- $d_4$  does not reach  $B_2$  since  $i$  is always redefined by  $d_7$  (i.e.,  $d_7$  reaches  $B_2$ )
- $d_6$  reaches  $B_2$

图 9.13 所示是一个有 7 个定值的流图。

让我们关注所有到达块  $B_2$  的定值。所有在块  $B_1$  中的定值都到达了块  $B_2$  的开头。

因为在转回基本块  $B_2$  的循环中找不到其他对  $j$  的定值，块  $B_2$  中的定值  $d_5: j=j-1$  也可以到达基本块  $B_2$  的开头。但，这个定值杀死了定值  $d_2: j=n$ ，使得  $d_2$  不能到达  $B_3$  或  $B_4$ 。

$B_2$  中的语句  $d_4: i=i+1$  并未到达  $B_2$  的开头，这是因为变量  $i$  总是被  $d_7: i=u_3$  重新定值。

最后，定值  $d_6: a=u_2$  也能够到达块  $B_2$  的开头



# Reaching Definitions

- For reaching definitions analysis, we **allow inaccuracies**. However, the decisions should be *"safe"*.
  - It is ok that some inferred reaching defs cannot actually reach a point
  - However, those defs that can reach a point should be identified
  - In other words, over-approximations are acceptable
- ***Reason of inaccuracies:*** In general, to decide whether each path in a flow graph can be taken is an undecidable problem
  - We often simply assume that every path in the flow graph can be followed in some execution of the program

我们在前面定义的到达定值，有时会允许一定的不精确。但是，它们都是在“安全”或“保守”的方向上不精确。例如，请注意我们假设一个流图的所有边都可以通过。在实践中这一假设可能并不成立。

一般来说，决定流图中的每条路径是否可以被执行是一个不可判定的问题。因此，我们简单地假设流图中的每条路径都可能在程序的某些执行时通过。在大部分到达定值的应用中，在一个定值不可能到达某个点的情况下，假设其能够到达是保守的。因此，我们可以允许在程序实际执行中根本不会被遍历的路径，我们也可以安全地允许定值穿越某个对同一变量的不明确定值。

# Transfer Equations

- Consider a statement  $d: u = v + w$ 
  - It *generates* a definition  $d$  of variable  $u$  and *kills* all other definitions of  $u$
- Transfer function of a statement is in *gen-kill form*:
  - $f_d(x) = gen_d \cup (x - kill_d)$
  - $x$  is the data-flow value (reaching definitions) before the statement
  - $gen_d$  is the set of generated definitions, i.e.,  $\{d\}$
  - $kill_d$  is the set of killed definitions, i.e., all other definitions of  $u$

达到定值的传递方程 我们现在将为达到定值问题设置约束。我们首先检查单个语句的细节。考虑一个定值  $d: u = v + w$  在这里，以及在下面的内容中， $+$  经常代表了一个一般性的二元运算符。以后我们经常这样做。

该语句“生成”一个变量  $u$  的定值并“杀死”程序中其他对  $u$  的定值，而进入这个语句的其他定值不受影响。因此，定值  $d$  的传递函数可以表示为  $f_d(x) = gen_d \cup (x - kill_d)$

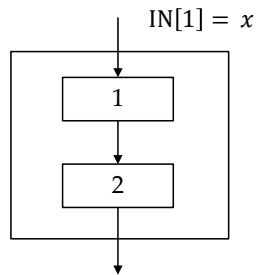
$x$  是该语句前的数据流值

$gen_d = \{d\}$ ，即由该语句生成的定值集合，

$kill_d$  是程序中所有其他对  $u$  的定值

# Transfer Equations

- Transfer function of a basic block is also in *gen-kill form*
- Consider a block of only two statements:



$$OUT[1] = f_1(x) = gen_1 \cup (x - kill_1)$$

$$OUT[2] = f_2(IN[2]) = f_2(OUT[1])$$

$$= gen_2 \cup (OUT[1] - kill_2)$$

$$= gen_2 \cup ((gen_1 \cup (x - kill_1)) - kill_2)$$

$$= gen_2 \cup (gen_1 - kill_2) \cup (x - (kill_1 \cup kill_2))$$

基本块的传递函数可以通过把它包含的所有语句的传递函数组合起来而构造得到。

(9.1) 形式的函数组合，我们将其称为“生成-杀死”形式，

# Transfer Equations

- Generalize to a block  $B$  with  $n$  statements, we have:

$$f_B(x) = gen_B \cup (x - kill_B)$$

where

$$kill_B = kill_1 \cup kill_2 \cup \dots \cup kill_n$$

and

$$gen_B = gen_n \cup (gen_{n-1} - kill_n) \cup \\ (gen_{n-2} - kill_{n-1} - kill_n) \cup \\ \dots \cup (gen_1 - kill_2 - kill_3 - \dots - kill_n)$$

***kill<sub>B</sub>***

The *kill set* is the union of all the defs killed by the individual statements

***gen<sub>B</sub>***

The *gen set* contains all the defs inside the block that are *downward exposed* (visible immediately after the block)

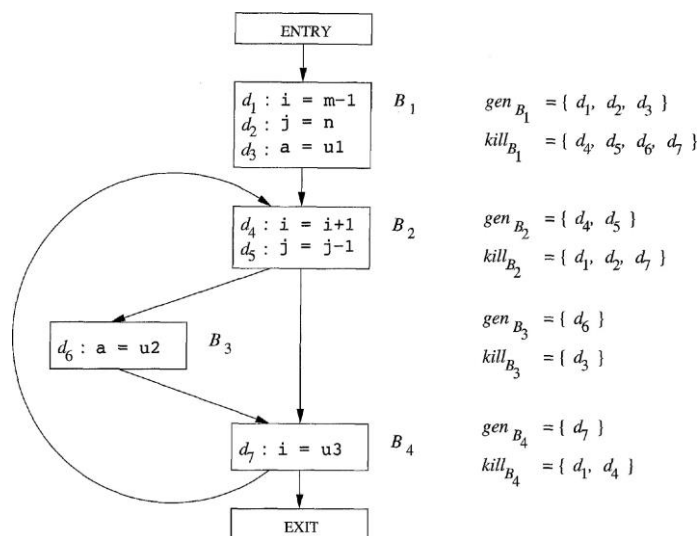
该规则可以扩展到由任意多个语句组成的块。假设块  $B$  有  $n$  个语句，而第  $i$  个语句的传递函数为  $f_i(x) = \text{gen}_i \cup (x - \text{Kill}_i)$  for  $i = 1; 2; \dots; n$ 。那么块  $B$  的传递函数可以写为：

因此，和单个语句一样，一个基本块也会生成一个定值集并杀死一个定值集合。

集合  $\text{gen}$  中包含了所有在紧靠块之后的点上“可见”的该基本块中的定值——我们将它们称为向下可见。在一个基本块中，一个定值是向下可见的，仅当它没有被同一个基本块中较后的对同一变量的定值“杀死”。

一个基本块的  $\text{kill}$  集就是所有被块中各个语句杀死的定值的集合。请注意，一个定值可能会同时出现在基本块的  $\text{gen}$  集和  $\text{kill}$  集中。在这种情况下，该定值会被这个基本块生成，即优先考虑该定值是否在  $\text{gen}$  集中。这是因为在  $\text{gen-kill}$  形式中， $\text{kill}$  集在  $\text{gen}$  集之前被应用。

## Example (1)



## Example (2)

- Consider the basic block:

|              |
|--------------|
| $d_1: a = 3$ |
| $d_2: a = 4$ |

- The *gen* set:  $\{d_2\}$
- The *kill* set contains  $d_1, d_2$  and all other definitions to  $a$

$d_2$  appears in both the *gen* and *kill* set of the block. Nonetheless, since the subtraction of the kill set precedes the union operation, the result of the transfer function for this block always includes  $d_2$ .

基本块  $d_1: a = 3$   $d_2: a = 4$  的 *gen* 集是  $\{d_2\}$ ，因为  $d_1$  不是向下可见的。

基本块 *kill* 集包含了  $d_1$  和  $d_2$ ，因为  $d_1$  杀死了  $d_2$ ， $d_2$  杀死了  $d_1$ 。尽管如此，由于减去 *kill* 集的运算先于和 *gen* 集的并集运算，因此该块的传递函数的结果中总是包含定值  $d_2$

# Control-Flow Equations

$$IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P]$$

The *meet* operator

- **Rationale:**

- A definition reaches a program point as long as there exists at least one path along which the definition reaches. This explains why  $OUT[P] \subseteq IN[B]$ .
- Since a definition cannot reach a point unless there is a path along which it reaches,  $IN[B]$  needs to be no larger than the union of the reaching definitions of all the predecessor blocks

控制流方程 接下来，我们考虑从基本块之间的控制流得到的约束的集合。因为只有一个定值能够沿着至少一条路径到达某个程序点，那么这个定值就到达该程序点。

- 因此只要存在一条从  $P$  到  $B$  的控制边，就会出现  $OUT[P] \subseteq IN[B]$ 。
- 然而，一个定值到达某个程序点的必要条件是它能够沿着某条路径到达这个程序点，因此  $IN[B]$  不应该大于  $B$  的所有前驱基本块出口点的到达定值的并集。也就是说，可以安全地假设如下的方程式成立

我们将并集运算称为达到定值的交汇 *meet* 运算。在任何数据流模式中，我们使用交汇 *meet* 运算符来汇总各条路径会合点上不同路径所作的贡献。

# Problem Formulation

- The reaching definitions problem is defined by the following equations:
  - For the ENTRY block:  $OUT[ENTRY] = \emptyset$ 
    - Since no definitions reach the beginning of a program
  - For all basic blocks  $B$  other than ENTRY:
    - $OUT[B] = gen_B \cup (IN[B] - kill_B)$
    - $IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P]$

我们假设每个控制流图都有两个空的基本块，一个入口节点（代表图的起始点）和一个出口节点（所有离开图的控制流都流向它）。由于没有定值到达图的开头，因此基本块 ENTRY 的传递函数是一个简单的常函数，它返回空集，即  $OUT[entry] = \emptyset$ 。

到达定值问题使用下面的方程定义：

# The Iterative Algorithm

- **Input:** A flow graph with  $kill_B$  and  $gen_B$  computed for each block  $B$
- **Output:**  $IN[B]$  and  $OUT[B]$  for each block  $B$

```
1) OUT[ENTRY] = $\emptyset$;
2) for (each basic block B other than ENTRY) OUT[B] = $\emptyset$;
3) while (changes to any OUT occur)
4) for (each basic block B other than ENTRY) {
5) IN[B] = $\bigcup_{P \text{ a predecessor of } B} OUT[P]$;
6) OUT[B] = $gen_B \cup (IN[B] - kill_B)$;
 }
```

算法的结果是方程的最小不动点，即，对于各个  $IN$  和  $OUT$ ，这个解给出的值总是此方程组的其他解所给出的值的子集。下面的算法的结果是可接受的，因为在某个集合  $IN$  或集合  $OUT$  中的定值确实可以到达该  $IN$  或  $OUT$  所描述的程序点。这个解也是我们所期望的，因为它不包含任何我们确定不会到达的定值。

算法 9.11：达到定值。

输入：一个流图，其中每个块  $B$  的  $kill_B$  集和  $gen_B$  集都已计算出来。

输出：到达流图中各个基本块  $B$  的入口点和出口点的定值的集合，即  $IN[B]$  和  $OUT[B]$

方法：我们使用迭代方法求解。一开始，我们估计对于所有  $B$  都有  $OUT[B] = \text{空}$ ，并逐步逼近想要的  $IN$  和  $OUT$  的值。因为我们必须不停迭代直到各个  $IN$  值（以及各个  $OUT$  值）收敛，我们可以使用一个布尔变量  $change$  来记录每次扫描各基本块时，是否有  $OUT$  值发生更改。但是，在这个以及后面描述的类似算法中，我们假设用来跟踪更改情况的确切机制是可理解的，因此我们删除了这些细节。



该算法如图 9.14 所示。前两行初始化某些数据流值。第 (3) 行开始是一个循环，在循环中我们不停地迭代直至各个值收敛，第 (4) 行到第 (6) 行组成的内循环对入口结点之外的所有基本块应用数据流方程。

## Algorithm Analysis

- *Why is the solution correct?*
  - The algorithm propagates definitions as far as they are not killed, thus simulating all possible executions of the program
- *Why will the algorithm eventually halt?*
  - For every  $B$ ,  $OUT[B]$  never shrinks; once a definition is added, it stays there forever
  - Since the set of all definitions is finite, eventually there will be an iteration during which nothing is added to any  $OUT$ , and the algorithm then terminates.

直观地说，为什么解决方案是正确的？

算法 9.11 会尽可能地传播定值，直到该定值被杀死，这样模拟了程序的所有可能的执行情况。

为什么算法最终会停止？

算法 9.11 最终会停止，因为对于每个  $B$ ， $OUT[B]$  决不会变小；一旦某个定值被加入到  $OUT$  值中，它就会一直待在那里。

由于所有定值的集合是有限的，因此最终必须有一次 `while` 循环的执行没有向任何  $OUT$  添加任何内容。此时算法终止了。在此时终止迭代是安全的，因为如果各个  $OUT$  值没有改变，下一趟中各  $IN$  值也不会改变。而如果各个  $IN$  值没有改变， $OUT$  值也不会改变，如此下去，所有后续的迭代都不会发生改变  $IN$  和  $OUT$  的值。

流图中的结点个数是 `while` 循环的迭代次数的上届。其理由是如果一个定值能够到达某个程序点，它必然可以通过无环的路径到达该点，而一个流图中的结点数量是无环路径中

节点数量的上限。在 while 循环的每次迭代中，每个定值至少沿着问题中的路径前进一个节点，并且根据结点在内层循环中被访问的顺序，它经常一次前进多个节点。

事实上，如果我们适当地安排第 (4) 行的 for 循环访问的块顺序，经验证据表明 while 循环的平均迭代次数低于 5（参见第 9.6.7 节）。由于定值集可以用位向量来表示，并且对这些集合的运算可以使用位向量上的逻辑运算来实现，所以算法 9.11 在实践中出奇地高效。

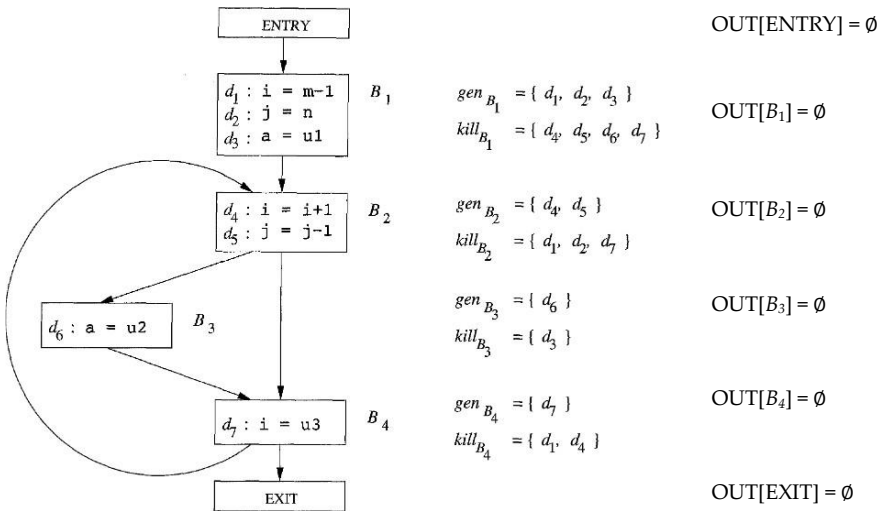
# Example

```

1) OUT[ENTRY] = ∅;
2) for (each basic block B other than ENTRY) OUT[B] = ∅;
3) while (changes to any OUT occur)
4) for (each basic block B other than ENTRY) {
5) IN[B] = $\bigcup_{P \text{ a predecessor of } B} \text{OUT}[P]$;
6) OUT[B] = $\text{gen}_B \cup (\text{IN}[B] - \text{kill}_B)$;
 }

```

Initialization:



我们将用位向量表示图 9.13 的流图中的七个定值 d1;d2;...;d7，其中从左边开始的位 i 代表定值 di。集合的并集是通过相应位向量的逻辑 OR 来计算的。两个集合的差 S-T 通过首先对 T 的位向量求补，然后再将该补码与 S 的位向量进行逻辑 AND 来计算的。

图 9.15 的表中显示的是算法 9.11 中 IN 和 OUT 集取值。其初始值用上标 0 表示（如 OUT[B]0 中所示）它们由图 9.14 的第 (2) 行的循环赋值。它们都是空集，由位向量 000 0000 表示。

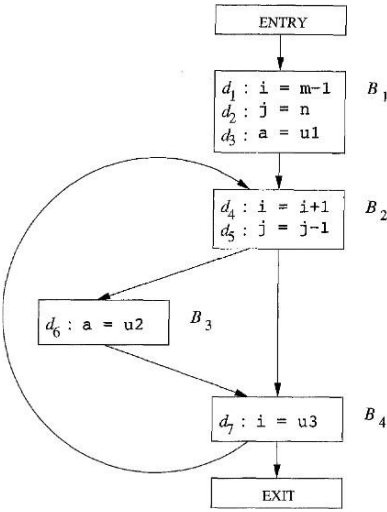
算法的后续迭代中的取值也使用上标表示，第一趟迭代的值标记为 IN[B]1 和 OUT[B]1，第二个趟迭代的值标记为 IN[B]2 和 OUT[B]2。

假设执行第 (4) 行至第 (6) 行的 for 循环，B 依次取值 B1 ;B2 ;B3 ;B4 ;EXIT。

当  $B = B_1$  时, 由于  $OUT[entry] = \emptyset$ ,  $IN[B_1]$  是空集, 并且  $OUT[B_1]$  是  $gen_{B_1}$ 。该值与之前的值  $OUT[B_1]$  不同, 因此我们知道在第一轮中有些值发生了变化 (因此会继续进行第二次循环)。

然后我们考虑  $B = B_2$  并计算

# Example



$gen_{B_1} = \{ d_1, d_2, d_3 \}$   
 $kill_{B_1} = \{ d_4, d_5, d_6, d_7 \}$   
 $gen_{B_2} = \{ d_4, d_5 \}$   
 $kill_{B_2} = \{ d_1, d_2, d_7 \}$   
 $gen_{B_3} = \{ d_6 \}$   
 $kill_{B_3} = \{ d_3 \}$   
 $gen_{B_4} = \{ d_7 \}$   
 $kill_{B_4} = \{ d_1, d_4 \}$

```

1) OUT[ENTRY] = ∅;
2) for (each basic block B other than ENTRY) OUT[B] = ∅;
3) while (changes to any OUT occur)
4) for (each basic block B other than ENTRY) {
5) IN[B] = ∪P: a predecessor of B OUT[P];
6) OUT[B] = genB ∪ (IN[B] - killB);
 }

```

## Iteration #1:

$OUT[ENTRY] = \emptyset$

$IN[B_1] = \emptyset$

$OUT[B_1] = \{d_1, d_2, d_3\}$

$IN[B_2] = \{d_1, d_2, d_3\}$

$OUT[B_2] = \{d_3, d_4, d_5\}$

$IN[B_3] = \{d_3, d_4, d_5\}$

$OUT[B_3] = \{d_4, d_5, d_6\}$

$IN[B_4] = \{d_3, d_4, d_5, d_6\}$

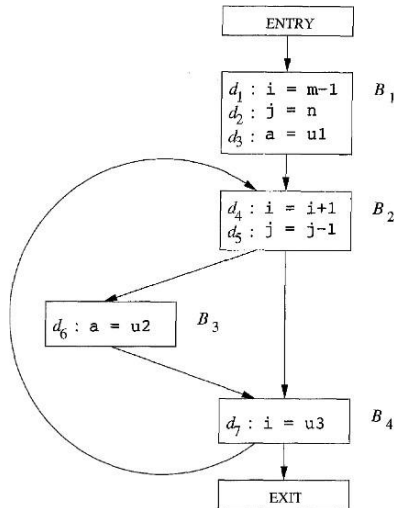
$OUT[B_4] = \{d_3, d_5, d_6, d_7\}$

$IN[EXIT] = \{d_3, d_5, d_6, d_7\}$

$OUT[EXIT] = \{d_3, d_5, d_6, d_7\}$

在第一趟循环的最后,  $OUT[B_2] = 001\ 1100$ , 反映  $B_2$  中生成了  $d_4$  和  $d_5$  的事实, 而  $d_3$  到达了  $B_2$  的开头并且没有在  $B_2$  中被杀死。

# Example



$gen_{B_1} = \{ d_1, d_2, d_3 \}$   
 $kill_{B_1} = \{ d_4, d_5, d_6, d_7 \}$   
 $gen_{B_2} = \{ d_4, d_5 \}$   
 $kill_{B_2} = \{ d_1, d_2, d_7 \}$   
 $gen_{B_3} = \{ d_6 \}$   
 $kill_{B_3} = \{ d_3 \}$   
 $gen_{B_4} = \{ d_7 \}$   
 $kill_{B_4} = \{ d_1, d_4 \}$

```

1) OUT[ENTRY] = ∅;
2) for (each basic block B other than ENTRY) OUT[B] = ∅;
3) while (changes to any OUT occur)
4) for (each basic block B other than ENTRY) {
5) IN[B] = ∪P a predecessor of B OUT[P];
6) OUT[B] = genB ∪ (IN[B] - killB);
 }

```

Iteration #2:

OUT[ENTRY] = ∅

IN[B<sub>1</sub>] = ∅

OUT[B<sub>1</sub>] = {d<sub>1</sub>, d<sub>2</sub>, d<sub>3</sub>}

IN[B<sub>2</sub>] = {d<sub>1</sub>, d<sub>2</sub>, d<sub>3</sub>, d<sub>5</sub>, d<sub>6</sub>, d<sub>7</sub>}

OUT[B<sub>2</sub>] = {d<sub>3</sub>, d<sub>4</sub>, d<sub>5</sub>, d<sub>6</sub>}

IN[B<sub>3</sub>] = {d<sub>3</sub>, d<sub>4</sub>, d<sub>5</sub>, d<sub>6</sub>}

OUT[B<sub>3</sub>] = {d<sub>4</sub>, d<sub>5</sub>, d<sub>6</sub>}

IN[B<sub>4</sub>] = {d<sub>3</sub>, d<sub>4</sub>, d<sub>5</sub>, d<sub>6</sub>}

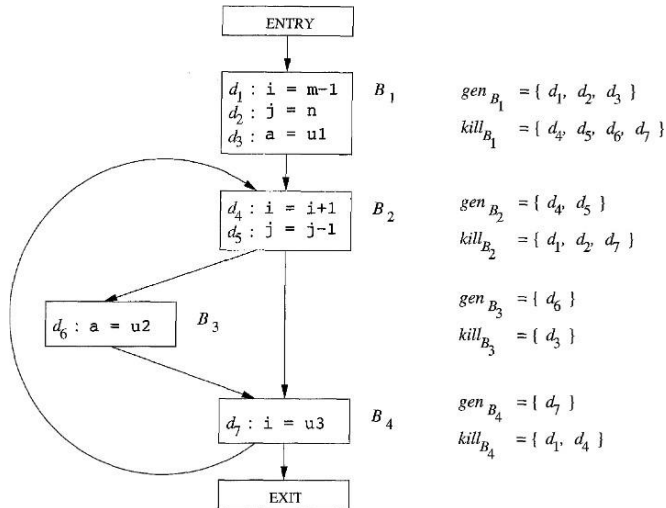
OUT[B<sub>4</sub>] = {d<sub>3</sub>, d<sub>5</sub>, d<sub>6</sub>, d<sub>7</sub>}

IN[EXIT] = {d<sub>3</sub>, d<sub>5</sub>, d<sub>6</sub>, d<sub>7</sub>}

OUT[EXIT] = {d<sub>3</sub>, d<sub>5</sub>, d<sub>6</sub>, d<sub>7</sub>}

在第二轮之后，OUT[B<sub>2</sub>] 已发生更改，反映了 d<sub>6</sub> 也到达 B<sub>2</sub> 的开头并且没有被 B<sub>2</sub> 杀死的事实。我们在第一趟中没有了解到这个事实，因为从 d<sub>6</sub> 到 B<sub>2</sub> 末尾的路径，即 B<sub>3</sub> → B<sub>4</sub> → B<sub>2</sub> 没有在一趟中被顺序经过。也就是说，当我们得知 d<sub>6</sub> 到达 B<sub>4</sub> 的末尾时，我们已经在第一趟中计算了 IN[B<sub>2</sub>] 和 OUT[B<sub>2</sub>]。

# Example



```

1) OUT[ENTRY] = ∅;
2) for (each basic block B other than ENTRY) OUT[B] = ∅;
3) while (changes to any OUT occur)
4) for (each basic block B other than ENTRY) {
5) IN[B] = ∪P a predecessor of B OUT[P];
6) OUT[B] = genB ∪ (IN[B] - killB);
 }

```

Iteration #3: fixed point!

OUT[ENTRY] = ∅

IN[B<sub>1</sub>] = ∅

OUT[B<sub>1</sub>] = {d<sub>1</sub>, d<sub>2</sub>, d<sub>3</sub>}

IN[B<sub>2</sub>] = {d<sub>1</sub>, d<sub>2</sub>, d<sub>3</sub>, d<sub>5</sub>, d<sub>6</sub>, d<sub>7</sub>}

OUT[B<sub>2</sub>] = {d<sub>3</sub>, d<sub>4</sub>, d<sub>5</sub>, d<sub>6</sub>}

IN[B<sub>3</sub>] = {d<sub>3</sub>, d<sub>4</sub>, d<sub>5</sub>, d<sub>6</sub>}

OUT[B<sub>3</sub>] = {d<sub>4</sub>, d<sub>5</sub>, d<sub>6</sub>}

IN[B<sub>4</sub>] = {d<sub>3</sub>, d<sub>4</sub>, d<sub>5</sub>, d<sub>6</sub>}

OUT[B<sub>4</sub>] = {d<sub>3</sub>, d<sub>5</sub>, d<sub>6</sub>, d<sub>7</sub>}

IN[EXIT] = {d<sub>3</sub>, d<sub>5</sub>, d<sub>6</sub>, d<sub>7</sub>}

OUT[EXIT] = {d<sub>3</sub>, d<sub>5</sub>, d<sub>6</sub>, d<sub>7</sub>}

第二趟之后，OUT 集中的所有值都没有变化。因此，算法在第三趟之后终止，此时各个 IN 和 OUT 的值 如图 9.15 的最后两列所示。

## Live-Variable Analysis

- Aim to know for variable  $x$  and point  $p$  whether the value of  $x$  at  $p$  could be used on some path starting at  $p$ 
  - If yes, we say  $x$  is *live* at  $p$ ; otherwise,  $x$  is *dead* at  $p$ .
- An important use of live-variable information is register allocation for basic blocks
  - It is not necessary to store a value computed in a register to memory at the end of a basic block
  - If all registers are used and we need another register, we should favor using a register with a dead value
- Another important use is to eliminate dead code
  - If a value is dead at a point, there is no need to compute it

- 有些代码改进转化所依赖的信息是按照程序中的控制流相反的方向计算的。

现在我们将研究一个这样的例子。在活跃变量分析中，我们希望知道对于变量  $x$  和程序点  $p$ ， $x$  在点  $p$  上的值是否会在流图中的某条从点  $p$  出发的路径中是使用。如果是，我们就说  $x$  在  $p$  上活跃；否则，就说  $x$  在  $p$  处死亡。

- 活跃变量信息的一个重要用途是基本块的寄存器分配。第 8.6 节和第 8.8 节介绍了该问题的各个方面。

在一个值被计算并保存到一个寄存器中后，它很可能会在块中使用该值。如果它在基本块的结尾处是死的，就不必在结尾处保存这个值。

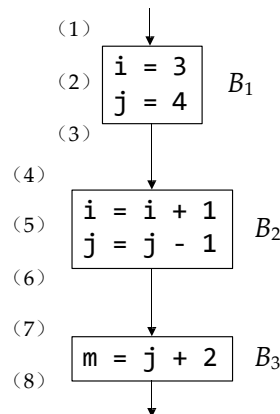
另外，如果所有寄存器都被占用时，如果我们还需要申请另一个寄存器，我们应该倾向于使用一个存放了已死亡的值的寄存器，因为这个值不需要保存到内存。

- 另一个重要的用途就是删除死代码

一个值在某个程序点是死的，就没必要计算它。

## Example

**Live:** value could be used on subsequent path  
**Dead:** value not referenced on subsequent path

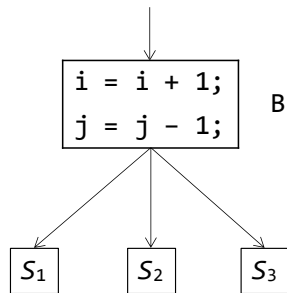


For the partial flow graph, we have:

| Point | i    | j    | m    |
|-------|------|------|------|
| (1)   | Dead | Dead | Dead |
| (2)   | Live | Dead | Dead |
| (3)   | Live | Live | Dead |
| (4)   | Live | Live | Dead |
| (5)   | Dead | Live | Dead |
| (6)   | Dead | Live | Dead |
| (7)   | Dead | Live | Dead |
| (8)   | Dead | Dead | Dead |

**Obsevation:** Live-variable analysis a **backward data-flow problem** since the subsequent defs/uses determine the liveness of a variable at a point

# Data-Flow Equations



## Data-flow values:

$IN[B]$ : variables live at the entry of the block  $B$

$OUT[B]$ : variables live at the exit of block  $B$

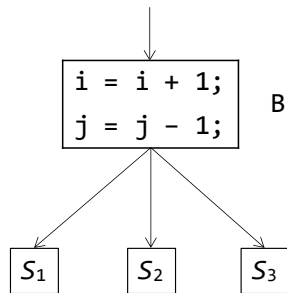
$$OUT[B] = \bigcup_{S \text{ is a successor of } B} IN[S]$$

**Rationale:** For safety, as long as a variable is live at the entry of any successor block of  $B$ , we consider the variable live at the exit of  $B$

这里，我们直接以  $IN[B]$  和  $OUT[B]$  的方式定义数据流方程， $IN[B]$  和  $OUT[B]$  分别表示在紧靠块  $B$  之前和紧随  $B$  之后的点上的活跃变量集合。

为了安全起见，只要一个变量在  $B$  的任何后继块的入口处是活跃的，我们就认为该变量在  $B$  的出口处是活跃的。

# Data-Flow Equations



$$IN[B] = use_B \cup (OUT[B] - def_B)$$

***use<sub>B</sub>***: the set of variables whose values are used in *B* **prior to any definition** of the variable (*i*, *j*)

***def<sub>B</sub>***: the set of variables defined in *B* **prior to any use** of that variable in *B* (i.e., the first time the variable appears, it appears as a definition; *def<sub>B</sub>* is  $\emptyset$  for the example)

这些方程可以通过以下的方法得到，首先定义各个语句的传递函数，然后再把它们组合起来得到一个基本块的传递函数。

我们给出下面的定义

1. *def B* 是指如下变量的集合，这些变量在 *B* 中的定值（即被明确地赋值）先于任何对它们的使用。
2. *useB* 是指如下变量的集合，它们的值可能在 *B* 中先于任何对它们的定值被使用。



# Computing $use_B$ and $def_B$

- **Transfer function for a statement:**
  - $s: x = y + z$
  - $use_s = \{y, z\}$
  - $def_s = \{x\} - \{y, z\}$  //  $y, z$  might be  $x$
- **Transfer function for a block of statements  $s_1, s_2, \dots, s_n$ :**
  - $use_B = use_1 \cup (use_2 - def_1) \cup (use_3 - def_2 - def_1) \cup \dots \cup (use_n - def_1 - def_2 - \dots - def_{n-1})$
  - $def_B = def_1 \cup (def_1 - use_0) \cup (def_2 - use_0 - use_1) \cup \dots \cup (def_n - use_1 - use_2 - \dots - use_{n-1})$

## Problem Formulation

- All variables are dead at the exit of a program
  - $IN[EXIT] = \emptyset$
- For all basic blocks  $B$  other than EXIT:
  - $IN[B] = use_B \cup (OUT[B] - def_B)$
  - $OUT[B] = \bigcup_{S \text{ is a successor of } B} IN[S]$
- Comparing to reaching definitions analysis:
  - Both sets of equations have union as the meet operator
  - Information flow for liveness travels “backward”

根据这些定义， $useB$  中的任何变量都必然被认为在块  $B$  的入口处活跃，而  $def B$  中变量在  $B$  的开头肯定是死的。实际上， $def B$  中的成员 “杀死” 了某个变量可能因从  $B$  开始的某条路径而成为活跃变量的任何机会。

这样，将  $def$  和  $use$  与未知的  $IN$  和  $OUT$  值联系起来的方程定义如下： $IN[exit]=$ ；对于除  $exit$  之外的所有基本块  $B$  来说， $IN[B]=useB[(OUT[B] def B) OUT[B]]= [S B IN[S]$

第一个方程描述了边界条件，即程序的出口处没有变量是活跃的。第二个方程说明一个变量要在进入一个基本块时活跃，必须满足下面两个条件中的一个：要么它在基本块中被重新定值之前就被使用，要么它在离开基本块时活跃且在基本块中没有对它重新定值。第三个方程表示，一个变量在离开一个基本块时活跃当且仅当它在进入该基本块的某个后继时活跃。

应该注意活跃性方程和到达定值方程之间的关系：

两组方程都以并集作为交汇运算。原因是，在各个数据流模式中，我们都沿着路径传播信息，并且我们只关心是否存在任何路径具有我们想要的性质，而不关心某些结论是否在所有的路径上都成立。

然而，活跃性信息流 “向后传播”，与控制流的方向相反。其中的原因时在这个问题中，我们想要确保在一个程序点  $p$  上对变量  $x$  的使用可以被传递到某个执行路径中  $p$  之前的所有程序点，这样我们才知道在前面的这些点上  $x$  的值会被使用。

# The Iterative Algorithm

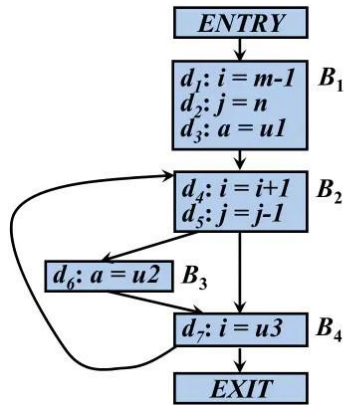
- **Input:** A flow graph with  $def_B$  and  $use_B$  computed for each block  $B$
- **Output:**  $IN[B]$  and  $OUT[B]$  for each block  $B$

```
IN[EXIT] = $\emptyset$;
for (each basic block B other than EXIT) $IN[B] = \emptyset$;
while (changes to any IN occur)
 for (each basic block B other than EXIT) {
 $OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$;
 $IN[B] = use_B \cup (OUT[B] - def_B)$;
 }
```

Similar to the iterative algorithm for reaching definitions analysis,  $IN[B]$  never shrinks and its size is bounded, so the algorithm will eventually halt.

为了解决一个逆向传播的数据流问题，我们初始化  $IN[exit]$  而不是  $OUT[entry]$ 。集合  $IN$  和  $OUT$  的角色互换，并且  $use$  和  $def$  分别替代  $gen$  和  $kill$ 。和到达定值问题一样，活跃性方程的解不必是唯一的，且我们希望得到具有最小活跃变量集合的解。解方程时使用的算法本质上是算法的逆向传播版本 9.11。

# Example



$use_{B1} = \{ m, n, u1 \}$

$def_{B1} = \{ i, j, a \}$

$use_{B2} = \{ i, j \}$

$def_{B2} = \Phi$

$use_{B3} = \{ u2 \}$

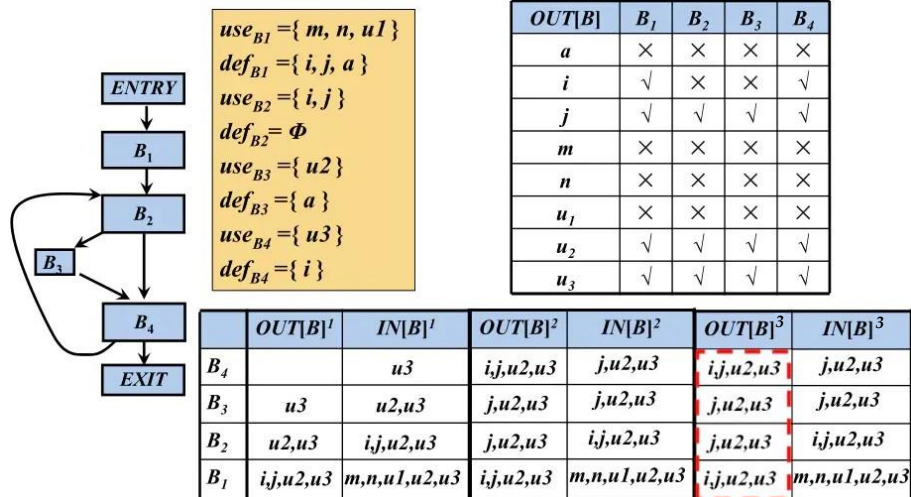
$def_{B3} = \{ a \}$

$use_{B4} = \{ u3 \}$

$def_{B4} = \{ i \}$

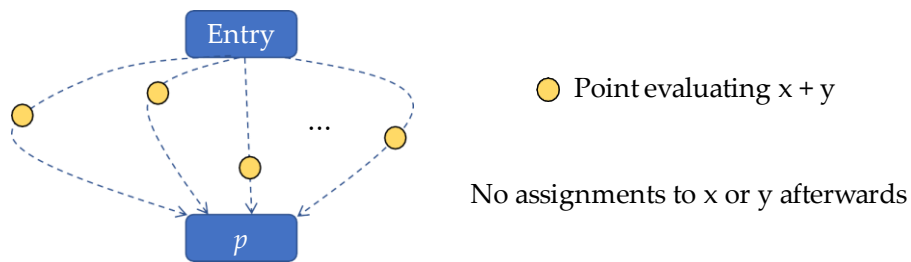
<https://www.jianshu.com/p/ebc1c72b881c>

# Example



# Available Expressions

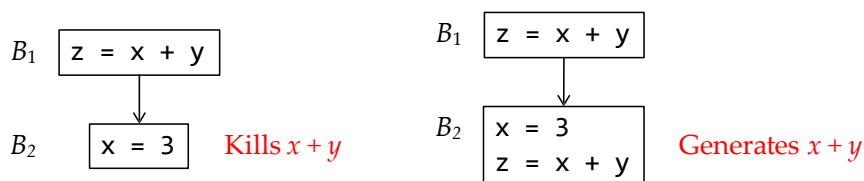
- An expression  $x + y$  is available at a point  $p$  if **every path** from the entry node to  $p$  evaluates  $x + y$ , and after the last such evaluation prior to reaching  $p$ , there are **no subsequent assignments to  $x$  or  $y$**



如果从入口节点 到达程序点  $p$  的每条路径都对表达式  $x + y$  求值，且从最后一个这样的求值之后到  $p$  点的路径上没有再次对  $x$  和  $y$  赋值，那么  $x + y$  在点  $p$  处可用。

# Killing / Generating Expressions

- A block **kills** expression  $x + y$  if:
  - It assigns  $x$  or  $y$  and does not subsequently recompute  $x + y$
- A block **generates** expression  $x + y$  if:
  - It evaluates  $x + y$  and does not subsequently define  $x$  or  $y$



对于可用表达式数据流模式，

我们说一个块对  $x$  或  $y$  赋值（或者可能对它们赋值）并且之后没有再重新计算  $x + y$ ，我们就说该基本块杀死了表达式  $x + y$ 。

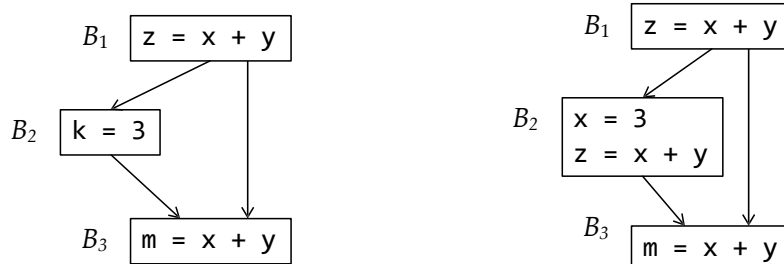
如果一个基本块确实对  $x + y$  求值，并且之后没有再对  $x$  或  $y$  定值，那么这个基本块生成表达式  $x + y$ 。

请注意，“杀死”或“生成”一个可用表达式的概念与达到定值的概念并不完全相同。然而，这些“杀死”和“生成”的概念在行为上和到达定值中相应的概念在本质上是

一致的。

## The Primary Use of Available Expressions

- Detecting global common expressions



**Case 1:**  $B_2$  does not redefine  $x$  or  $y$

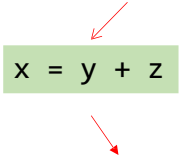
**Case 2:**  $B_2$  redefines  $x$  and recomputes  $x + y$

**In both cases,  $x + y$  needs not to be recomputed in  $B_3$**

可用表达式信息的主要用途是检测全局公共子表达式。

# Computing Generated Expressions

$S$ : the set of expressions available before the statement



$x = y + z$

**Compute the expressions available after the statement:**

- Add to  $S$  the expression of  $y + z$
- Delete from  $S$  any expression involving variable  $x$

If we repeat the steps for each statement in a block  $B$ , we will get the set of **generated expressions** for  $B$  after we process the last statement.

我们可以从头到尾地处理基本块内的各个语句，计算一个基本块内各个点上生成的表达式集合。在该块之前的点上没有生成任何表达式。如果在点  $p$  处可用有表达式的集合是  $S$ ，且  $q$  是  $p$  之后的点，它们之间有语句  $x = y + z$ ，那么我们通过以下两个步骤可用得到点  $q$  上的可用的表达式集合。

1. 将表达式  $y + z$  添加到  $S$  中。
2. 从  $S$  中删除任何涉及变量  $x$  的表达式。

请注意，因为  $x$  可能与  $y$  或  $z$  相同，所有上面这些步骤必须按正确的顺序完成。当我们到达块的末尾后， $S$  就是该块生成的表达式集合。

# Computing Killed Expressions

- For a basic block  $B$ , the set of **killed expressions** is all expressions, say  $y + z$ , such that:
  - $y$  or  $z$  is defined in  $B$
  - $y + z$  is not generated by  $B$

被杀死的表达式的集合是所有类似于  $y + z$  的表达式，其中

- $y$  或  $z$  在块中被定值，
- 并且这个基本块没有生成  $y + z$ 。



# Example

| Statement   | Available Expressions |
|-------------|-----------------------|
|             | $\emptyset$           |
| $a = b + c$ |                       |
|             | $\{b + c\}$           |
| $b = a - d$ |                       |
|             | $\{a - d\}$           |
| $c = b + c$ |                       |
|             | $\{a - d\}$           |
| $d = a - d$ |                       |
|             | $\emptyset$           |

考虑图 9.18 的四个语句。

第一个语句之后  $b + c$  可用。

第二条语句之后， $a - d$  变得可用，但  $b + c$  不再可用，因为  $b$  已被重新定值。

第三条语句不会使  $b + c$  再次可用，因为  $c$  的值立即就被更改了。

最后一条语句之后， $a - d$  不再可用，因为  $d$  已更改。

因此，这个基本块不会生成任何可用表达式，并且所有涉及  $a$ 、 $b$ 、 $c$  或  $d$  的表达式都会被杀死。

# Data-Flow Equations

- At the exit of the entry node, there are expressions available:
  - $OUT[ENTRY] = \emptyset$
- For all basic blocks  $B$  other than ENTRY:
  - $OUT[B] = e\_genB \cup (IN[B] - e\_killB)$
  - $IN[B] = \bigcap_{P \text{ is a predecessor of } B} OUT[P]$

The equations are similar to those for reaching definition analysis except that the meet operator is intersection rather than union.\*

\* This is for safety reasons. Although producing a subset of the exact set of available expressions prevents certain optimizations, this does not lead to the change of program semantics

我们可以通过类似于计算到达定值的方式来找到可用的表达式。假设  $U$  是出现在程序中的一个或多个语句的右部的表达式的全集。对于每个块  $B$ ，令  $IN[B]$  表示在  $B$  的开始处可用的  $U$  中的表达式的集合。 $OUT[B]$  表示在  $B$  的结尾处可用的表达式的集合。定义  $e\_genB$  为  $B$  生成的表达式集合，而  $e\_KillB$  为被  $B$  杀死的  $U$  中的表达式的集合。请注意， $IN$ 、 $OUT$ 、 $e\_gen$  和  $e\_Kill$  都可以用位向量表示。

以下方程给出了未知的  $IN$  和  $OUT$  值之间，以及它们和已知量  $e\_gen$  和  $e\_Kill$  之间的关系：

$OUT[entry] = \text{空};$

并且 对于除 entry 之外的所有基本块，有  $OUT[B] = e\_genB \cup (IN[B] - e\_KillB)$

$IN[B] = \bigcap_{P \in \text{pred}(B)} OUT[P];$

上述方程看起来与达到定值的方程几乎相同。与到达定值一样，这个方程的边界条件也是  $OUT[entry] = \text{空};$ ，这是因为在 ENTRY 的出口处没有任何可用的表达式。最重要的区别在于这个方程组的交汇运算是交集运算而不是并集运算。因为只有当一个表达式在一个基本块的所有前驱的结尾处都可用，它才会在该基本块的开头可用，因此使用交集运算是正

确的。相反，只要一个定值到达了一个基本块的任何一个前驱的结尾处，它就到达该块的开头，所以在到达定值方程中使用并集运算作为交汇运算。

## The Iterative Algorithm

- **Input:** A flow graph with  $e\_kill_B$  and  $e\_gen_B$  computed for each block  $B$
- **Output:**  $IN[B]$  and  $OUT[B]$ , the expressions available at the entry and exit of each block  $B$

```
OUT[ENTRY] = $\emptyset$;
for (each basic block B other than ENTRY) $OUT[B] = U$;
while (changes to any OUT occur)
 for (each basic block B other than ENTRY) {
 $IN[B] = \bigcap_{P \text{ a predecessor of } B} OUT[P]$;
 $OUT[B] = e_gen_B \cup (IN[B] - e_kill_B)$;
 }
```

$U$ : the universal set of all expressions

使用“交”而不是“并”使得可用表达式方程的表现与达到定值的方程组的表现不同。虽然这两个集合都没有唯一的解，但到达定值方程组的解是符合“到达”的定义的最小集合。在求解到达定值方程的过程中，我们首先假设任何地方都没有定值到达，然后逐渐增大到达定值的集合，最终构建得到该解。在这个方法里，除非找到一条能把某个定值  $d$  传播到某个点  $p$  的实际路径，否则我们从来不假设定值  $d$  可以到达点  $p$ 。相反，对于可用的表达式方程组，我们希望得到具有最大可用表达式集合的解。因此，我们首先给出较大的近似值，然后逐步消减。

首先，我们假设在除了入口块的末尾之外的所有地方，所有表达式（即集合  $U$ ）都是可用的。只有当我们发现有一条路径使得某个表达式不可用时，我们才删除这个表达式。这种方法看起来不是那么显而易见，但是我们可以得到了一组真正可用的表达式集合。在处理可用表达式时，保守的做法是生成可用表达式的精确集合的子集。所以说子集是保守的，是因为我们将这个信息用于把一个可用表达式的计算替换为之前计算得到的

值。不知道一个表达式可用只会阻碍我们改进代码，而把一个不可用的表达式认为可用则可能导致我们更改程序的计算结果。

## Summary of The Problems

|                          | Reaching Definitions                                           | Live Variables                                                 | Available Expressions                                          |
|--------------------------|----------------------------------------------------------------|----------------------------------------------------------------|----------------------------------------------------------------|
| <b>Domain</b>            | Sets of definitions                                            | Sets of variables                                              | Sets of expressions                                            |
| <b>Direction</b>         | Forwards                                                       | Backwards                                                      | Forwards                                                       |
| <b>Transfer function</b> | $gen_B \cup (x - kill_B)$                                      | $use_B \cup (x - def_B)$                                       | $e\_gen_B \cup (x - e\_kill_B)$                                |
| <b>Boundary</b>          | $OUT[ENTRY] = \emptyset$                                       | $IN[EXIT] = \emptyset$                                         | $OUT[ENTRY] = \emptyset$                                       |
| <b>Meet (A)</b>          | $\cup$                                                         | $\cup$                                                         | $\cap$                                                         |
| <b>Equations</b>         | $OUT[B] = f_B(IN[B])$<br>$IN[B] = \bigvee_{P, pred(B)} OUT[P]$ | $IN[B] = f_B(OUT[B])$<br>$OUT[B] = \bigvee_{S, succ(B)} IN[S]$ | $OUT[B] = f_B(IN[B])$<br>$IN[B] = \bigvee_{P, pred(B)} OUT[P]$ |
| <b>Initialize</b>        | $OUT[B] = \emptyset$                                           | $IN[B] = \emptyset$                                            | $OUT[B] = U$                                                   |
| <b>Application</b>       | Constant folding                                               | Dead code elimination                                          | Global common subexpression elimination                        |